

Nullexit: Unified Project Document

Omar Alaaeldein

July 14, 2026

Contents

1	Project Directory Tree	3
2	README.md	5
3	agent.md	11
4	devref.md	17
5	diagrams.md	63
6	toggle.sh	67
7	recover.sh	94
8	setup.sh	104
9	scripts/setup-linux.sh	105
10	docker-compose.yml	106
11	scripts/common.sh	109
12	scripts/watcher.sh	121
13	scripts/crypto.sh	125
14	.aiignore	126
15	.claude/settings.local.json	126
16	.gitignore	126
17	Recover-Gateway.applescript	127
18	Toggle-Gateway.applescript	127
19	benchmarks/benchmark.sh	127
20	benchmarks/test_load.py	129
21	black_list.txt	130
22	cloud-init.yaml	131

23	docker/tor/Dockerfile	131
24	docker/tor/entrypoint.sh	131
25	launchd/com.nullexit.wake-recovery.plist	132
26	post-rules.txt	133
27	scripts/Dockerfile.routing-fix	134
28	scripts/Dockerfile.rule-compiler	134
29	scripts/Toggle-Gateway.bat	134
30	scripts/check_circuits.py	137
31	scripts/diagnose-host-leak.sh	138
32	scripts/dns-proxy.py	147
33	scripts/fix-docker-bridge-collision.sh	148
34	scripts/fix-ssh-delay.sh	153
35	scripts/generate_tex.py	154
36	scripts/logger/go.mod	156
37	scripts/logger/main.go	156
38	scripts/pf.conf	160
39	scripts/reinstall-host-tailscale.sh	160
40	scripts/routing-fix.sh	170
41	scripts/rule-compiler/go.mod	174
42	scripts/rule-compiler/main.go	174
43	scripts/setup-common.sh	184
44	scripts/unlock-files.sh	190
45	tor-bridges.txt	191
46	white_list.txt	191

1 Project Directory Tree

```
nullexit/
|-- .aiignore
|-- .claude
|   |-- settings.local.json
|-- .gitignore
|-- LICENSE
|-- README.md
|-- Recover-Gateway.applescript
|-- Toggle-Gateway.applescript
|-- agent.md
|-- benchmarks
|   |-- benchmark.sh
|   |-- test_load.py
|-- black_list.txt
|-- cloud-init.yaml
|-- devref.md
|-- diagrams.md
|-- docker
|   |-- tor
|       |-- Dockerfile
|       |-- entrypoint.sh
|-- docker-compose.yml
|-- launchd
|   |-- com.nullexit.wake-recovery.plist
|-- post-rules.txt
|-- recover.sh
|-- scripts
|   |-- Dockerfile.routing-fix
|   |-- Dockerfile.rule-compiler
|   |-- Toggle-Gateway.bat
|   |-- check_circuits.py
|   |-- common.sh
|   |-- crypto.sh
|   |-- diagnose-host-leak.sh
|   |-- dns-proxy.py
|   |-- fix-docker-bridge-collision.sh
|   |-- fix-ssh-delay.sh
|   |-- generate_tex.py
|   |-- logger
|       |-- go.mod
|       |-- main.go
|   |-- pf.conf
|   |-- reinstall-host-tailscale.sh
|   |-- routing-fix.sh
|   |-- rule-compiler
|       |-- go.mod
|       |-- main.go
|   |-- setup-common.sh
|   |-- setup-linux.sh
|   |-- unlock-files.sh
|   |-- watcher.sh
|-- setup.sh
```

```
|-- toggle.sh  
|-- tor-bridges.txt  
`-- white_list.txt
```

2 README.md

```
1 # nullexit: Tailscale + Cloudflare WARP Docker Gateway
2
3 > **Last updated:** July 12, 2026 [Architecture & Flow Diagrams ](./diagrams.md) [Full Dev Reference ](./
  devref.md)
4
5 **nullexit** is a chained network gateway that routes all Tailscale exit-node traffic through a Cloudflare WARP
  VPN tunnel double-encrypting every packet, hiding your ISP metadata, and providing network-wide DNS ad-
  blocking (AdGuard Home) and kernel-level IP threat blocking (`ipset`/`iptables`) for every device on your
  mesh.
6
7 ---
8
9 ## 1. Prerequisites
10 - Docker and Docker Compose installed.
11 - A Tailscale account.
12 - **macOS:** Install the standalone Tailscale CLI via Homebrew (`brew install tailscale`) and register `
  tailscaled` as a system service (`brew services start tailscale`). No `.app` GUI install required.
13 - **OS & Python:** Developed and tested against **macOS 26.5.2**. The project has zero external Python
  dependencies (no `requirements.txt`) and explicitly targets the default Apple-provided **Python 3.9.6** (`/
  usr/bin/python3`). While the scripts could function on newer versions, `nullexit` explicitly avoids
  experimenting with or modifying the built-in system Python environment to prevent unexpected OS-level side
  effects.
14
15 ## 2. Installation & Setup
16
17 Run the interactive setup script for your OS. It downloads `wgcf`, generates fresh Cloudflare WARP WireGuard
  keys, prompts for a Tailscale Auth Key, and writes your `.env`.
18
19 ```bash
20 ./setup.sh # macOS
21 ./scripts/setup-linux.sh # Linux
22 ```
23
24 ### Reference `.env`
25 This is a quick reference of the common variables. For the **complete, authoritative list** of every `.env`
  variable (including all optional/advanced settings), see `devref.md` 7`.
26
27 ```env
28 WIREGUARD_PRIVATE_KEY=<Generated>
29 WIREGUARD_PUBLIC_KEY=<Generated>
30 WIREGUARD_ADDRESSES=<Generated>
31 TS_AUTHKEY=tskey-auth-...
32 GATEWAY_RULE_PROFILE=medium # light | medium | heavy
33 # Safe MSS for double-tunneled traffic (Tailscale + WARP). Change to 1180 if you experience slow speeds and
  have a healthy path.
34 GATEWAY_MSS=1120
35 # Set to false to run as a 'Headless Server' (Docker acts as an exit node, but the host Mac/Linux's own
  internet is NOT hijacked).
36 GATEWAY_HIJACK_HOST=true
37 GATEWAY_USE_EXIT_NODE=true
38 WARP_FAIL_THRESHOLD=6 # consecutive warp=off polls before auto-shutdown (default 6 = 30s)
39 KILL_SWITCH=false # enforce strict PF lock that breaks SSH if VPN fails
40 # On campus/enterprise networks (WPA2-Enterprise / 802.1x), AP Client Isolation blocks direct
41 # LAN traffic between devices. Tailscale attempts a local P2P upgrade anyway, which causes
42 # macOS gvproxy to SNAT-mangle the reply's source port poisoning the phone's WireGuard
43 # endpoint and blackholing 100% of its traffic. Setting this to false drops those local
44 # hole-punch packets so the phone safely falls back to Tailscale DERP relays.
45 # watcher.sh auto-detects 802.1x and AP isolation on every Wi-Fi change and overrides
46 # this setting automatically you only need to set this manually if auto-detection fails.
47 TAILSCALE_ALLOW_LAN_P2P=false # auto-managed; true only on trusted hotspots/home networks
48 ADGUARD_USER=admin # Username for AdGuard Home (defaults to admin if not set)
49 ADGUARD_PASSWORD=nullexit # Shared password for AdGuard Home and Tor ControlPort (defaults to nullexit
  if not set)
50 BLOCKED_COUNTRIES="kp il" # dynamically block country IP ranges via ipdeny.com
51 TOR_EXCLUDE_EXIT_NODES="" # comma-separated country codes to exclude as Tor exit nodes (e.g. {us},{gb
  })
52 DEBUG_TRACE=false # true = mirror a full command-by-command trace of the toggle to output.log
  (see 9)
53
54 ```
55
56 ## 3. Deploy the Gateway
57
58 **macOS (Recommended):**
59
60 ```bash
61 ./toggle.sh
62 ```
63
64 Running it again stops the gateway and restores your network. Handles the full lifecycle: Colima VM, containers
  , DNS hijacking, Tailscale exit-node routing, rule compilation, and sleep prevention.
```

```

62 **Linux / Manual:**
63 ```bash
64 docker compose up -d
65 ```
66
67 ## 4. Post-Deployment
68
69 ### Approve the Exit Node
70 1. Go to [Tailscale Admin Machines](https://login.tailscale.com/admin/machines).
71 2. Find the `tailscale` device `...` **Edit route settings** enable as Exit Node.
72
73 ### Verify Traffic Flow
74 On any Tailscale client device, select this gateway as the exit node, then visit [api.ipify.org](https://api.ipify.org) or run:
75 ```bash
76 curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)='
77 # expect: warp=on
78 ```
79
80 ### AdGuard DNS Setup
81
82 To ensure client devices correctly route DNS through the AdGuard container (and do not bypass it via `tailscaled`'s internal resolver), you **must** configure your Tailscale Admin Console:
83 1. Go to the **DNS** tab in the [Tailscale Admin Console](https://login.tailscale.com/admin/dns).
84 2. Enable **MagicDNS**.
85 3. Under **Nameservers**, add a **Custom** nameserver and enter the gateway's Tailscale IPv4 address (run `tailscale ip -4` on the gateway machine to find it, e.g., `100.x.x.x`).
86 4. Click the **three dots (...)** next to the nameserver you just added and enable **"Use with exit node"**.
87 5. Delete any other Global Nameservers (like Google or Cloudflare).
88 6. Toggle ON **Override local DNS**.
89
90 Traffic will now be correctly forced into the AdGuard sinkhole, and mobile devices will be blocked from bypassing it via encrypted DNS (DoH/DoT).
91
92 > [!WARNING]
93 > **Disable "Private DNS" on mobile devices.** Android's Private DNS (DoT/DoH) bypasses port 53 interception entirely. Go to `Settings Connections More connection settings Private DNS Off`.
94
95 > [!WARNING]
96 > **Bandwidth doubling:** Streaming 1GB of video on a remote device consumes 1GB download + 1GB upload on the host's network. Be mindful on metered connections.
97
98 ### AdGuard Dashboard
99 Navigate to `http://<gateway-tailscale-ip>:3000` credentials: **admin` / `nullexit`**.
100
101 ### Access Any Mesh Device From Anywhere (SFTP/SSH)
102
103 Once the gateway exit node is online, **any device on your Tailscale mesh can reach any other mesh device, anywhere in the world** as long as both devices are on the mesh and the target device has an SSH server running. No port forwarding, no public IPs, no firewall holes. You can SSH into any device using its MagicDNS hostname (e.g. `ssh username@my-macbook`) or its assigned Tailscale IP (`100.x.x.x`).
104
105 > **Security Tip (Zero Local Attack Surface):** It is highly recommended to disable macOS's native "Remote Login" and "Screen Sharing" in System Settings to prevent exposing those ports (22, 5900) to physical Wi-Fi networks (which scanners can fingerprint). The gateway enforces `tailscale up --ssh=true` and the `pf` Kill-Switch automatically blocks local Wi-Fi access to these ports. Since both MagicDNS and Tailscale IPs route through the exact same encrypted tunnel, they are equally secure but **MagicDNS is recommended simply for ease of use.**
106
107 ### Extreme OPSEC: The Secret Tor-over-WARP Proxy
108 `nullexit` includes a completely isolated, headless Tor infrastructure proxy designed for terminals and hacking tools (e.g., `curl`, `sqlmap`, `nmap`).
109 To protect against local malware fingerprinting:
110 1. **Procedural Ports:** The proxy ports are randomized into the ephemeral range (10000-65000) on every single boot using a strict kernel socket collision-avoidance check (`netstat`).
111 2. **Honey-Port Tripwire:** The gateway spawns a fake trap port. If any malware attempts a sequential port-scan on `localhost` to find the proxy, it trips the Honey-Port, instantly triggering a macOS desktop notification and a critical log alert.
112
113 > [!NOTE]
114 > **Threat Model limitation:** Port randomization and the Honey-Port only defeat blind, generic port scanners. They do not protect against targeted local malware that runs `lsof -i` or reads the `/tmp/nullexit-ports.env` file directly. To actually prevent malicious local processes from reaching the proxy, you must run a host-level outbound firewall (like LuLu 4.3.0+ or Little Snitch) with **localhost/loopback filtering explicitly enabled** to gate access by process identity.
115
116 To use the Tor proxy, run `cat /tmp/nullexit-ports.env` to find your `$TOR SOCKS_PORT` for the current session, and point your hardened browser (LibreWolf/Tor Browser) or CLI tool to `127.0.0.1:$TOR SOCKS_PORT`.
117
118 > [!WARNING]
119 > **Preventing DNS Leaks (SOCKS5 vs. SOCKS5-Hostname):**

```

```

120 > When routing command-line tools through Tor, you must force the tool to delegate DNS resolution to the
    Tor proxy. Using plain SOCKS5 (e.g., `socks5://` or curl's `--socks5` flag) will resolve hostnames locally
    * using the host's DNS settings (AdGuard/WARP) before establishing the connection. While this DNS traffic
    is encrypted inside the WARP tunnel, the destination domain name is still leaked to AdGuard and Cloudflare,
    undermining your anonymity.
121 >
122 > Use these correct protocols and configurations:
123 > - curl:** Use `--socks5-hostname` or the `socks5h://` scheme (e.g., `curl -x socks5h://127.0.0.1:
    $TOR SOCKS_PORT https://check.torproject.org`). Do not use `--socks5` or `socks5://`.
124 > - sqlmap:** Pass the proxy URL using the `socks5h://` scheme to ensure remote resolution: `--proxy="
    socks5h://127.0.0.1:$TOR SOCKS_PORT"`.
125 > - nmmap:** Nmap's built-in `--proxies` option resolves hostnames locally. To safely scan targets without
    DNS leaks, tunnel it using `proxychains-ng` configured with `proxy_dns` enabled (add `socks5 127.0.0.1 <
    PORT>` to `/etc/proxychains.conf` or a local config, then run `proxychains4 nmap <args>`).
126
127
128 ##### Transparent .onion Browsing & Dark Web Access
129 In addition to the randomized SOCKS5 proxy port, `nullexit` provides seamless, transparent `.onion` domain
    browsing for all devices** on your Tailscale mesh:
130 - Zero-Config DNS:** AdGuard Home automatically intercepts queries for `.onion` domains and resolves them to
    Tor's virtual mapping subnet.
131 - Tailscale Auto-Routing:** The virtual subnet is mapped to the `198.18.0.0/15` benchmark range (RFC 2544).
    Since this range is not a private IP subnet, Tailscale's Exit Node mode automatically routes it to the
    gateway (bypassing local LAN exclusions).
132 - Transparent NAT Proxying:** Kernel firewall rules in the gateway redirect all TCP traffic destined for
    `198.18.0.0/15` directly into Tor's transparent port (`9040`).
133 - Exit Node Exclusion:** Supports the ability to exclude specific countries from being used as Tor exit nodes
    via the `TOR_EXCLUDE_EXIT_NODES` environment variable in `.env`.
134
135 ##### Enable in Web Browsers
136 To prevent accidental DNS leaks, web browsers block `.onion` lookups by default. To browse dark web sites
    natively:
137 - Firefox:** Go to `about:config`, search for `network.dns.blockDotOnion`, and set it to `false`. Type any
    onion address (e.g. `duckduckgogg42xjoc72x3sjasowoarfbgcmvfimafitt6twagswzczad.onion`) directly into the URL
    bar and it will load.
138 - Mobile Browsers (iOS/Android):** Connect to your Tailscale exit node, open Firefox (with blockDotOnion
    disabled), and browse onion sites natively on the go.
139
140 ##### Hardening: Using obfs4 Tor Bridges
141 If you want to disguise your Tor traffic from the WARP exit node provider, you can optionally enable Tor
    bridges:
142 1. Obtain private `obfs4` bridge lines from [bridges.torproject.org](https://bridges.torproject.org) or the
    official Telegram bot.
143 2. Create a file named `tor-bridges.txt` in the root of the `nullexit` folder and paste your bridge lines
    inside it (one per line).
144 3. Set `TOR_USE_BRIDGES=true` in your `.env` file and restart the gateway. The proxy will refuse to start if
    the bridges file is empty or missing.
145
146 ##### Example: Browse your Android phone's files from your Mac
147
148
149 1. On your Android phone, install [Termux](https://termux.dev/) and [Termux:Boot](https://f-droid.org/
    packages/com.termux.boot/) from F-Droid.
150 2. In Termux, install and start the SSH server:
151 ```bash
152 pkg install openssh
153 sshd
154 ```
155 Termux defaults to port 8022 (ports below 1024 require root on Android).
156 3. Find your phone's Tailscale IP:
157 ```bash
158 tailscale ip -4 # e.g. 100.87.42.87
159 ```
160 4. Stop Android from killing Termux do ALL of these (Samsung One UI is especially aggressive):
161 - Settings Apps Termux Battery Unrestricted**
162 - Device Care Battery Background usage limits Sleeping apps / Deep sleeping apps** remove Termux if
    listed
163 - In Termux, run `termux-wake-lock` to hold a persistent wakelock
164 5. On your Mac/PC, open [Cyberduck](https://cyberduck.io/) (or any SFTP client: WinSCP, FileZilla, FE File
    Explorer on iOS) and connect to:
165 - Server:** `100.87.42.87` (your phone's Tailscale IP)
166 - Port:** `8022`
167 - Protocol:** SFTP (SSH File Transfer Protocol)
168 - Username:** (your Termux username, usually `u0_a123` run `whoami` in Termux to check)
169 - Password:** (your Termux password set with `passwd` in Termux if you haven't already)
170
171 That's it you can now browse, download, and upload files to your phone from any device on the mesh, no matter
    where either device is physically located. This works identically for any mesh device: Mac Mac, Windows
    Linux, phone NAS, etc.

```

```

173 > **Troubleshooting:** If the connection hangs for ~30 seconds before failing, your phone's SSH server isn't
      running. Open Termux and run `sshd` again. If Android keeps killing it despite Unrestricted battery, the
      termux-wake-lock` command is your fix.
174
175 ---
176
177 ## 5. Multi-Layer Firewall & Kill-Switch
178
179 ### Layer 1 DNS Sinkhole (AdGuard Home)
180 Blocks ads and tracking domains before they are downloaded. In automated Lighthouse tests on ad-heavy sites:
181 - **Time to Interactive:** ~22% faster (51.0s 41.8s)
182 - **Speed Index:** ~20% faster (15.3s 12.8s)
183
184 Customize blocking via `black_list.txt` and `white_list.txt`. Rule profiles (`light` / `medium` / `heavy`) are
      set in `.env`.
185
186 ### Layer 2 Kernel IP Firewall (ipset`/iptables`)
187 On every startup, the `rule-compiler` fetches threat-intelligence feeds (Spamhaus DROP, Feodo Tracker, Emerging
      Threats, CINS) and compiles **~16,700 unique malicious IPs/CIDRs** into the kernel `FORWARD` chain
      blocking both outbound C2 connections and inbound attack traffic. Zero configuration required.
188
189 ### Layer 3 Geo-IP Blocking
190 Dynamically blocks all traffic to and from specific countries using live IP ranges from `ipdeny.com`.
191 To add countries to your blocklist, open your `.env` file and set the `BLOCKED_COUNTRIES` variable with 2-
      letter ISO country codes (e.g. `BLOCKED_COUNTRIES="kp il cn ru"`), then restart the gateway.
192
193 ### Layer 4 macOS PF Kill-Switch
194 A native macOS Packet Filter (`pf`) kill-switch that enforces a strict default-deny policy on your physical Wi-
      Fi interface (`en0`). When `KILL_SWITCH=true` is set in your `.env`, it drops all outgoing traffic except
      the Cloudflare WARP endpoints and Tailscale DERP relays.
195
196 - **Failsafe Design:** If the VPN tunnel crashes or Docker dies, your Mac completely loses internet access
      rather than leaking your real IP.
197 - **Remote-Access Safe:** Carefully designed to whitelist Tailscale's control plane (`192.200.0.0/24`), `utun*`
      tunnel traffic, and local LAN subnets. This guarantees that your remote SSH sessions and local AirDrop
      will survive even when the kill switch engages.
198 - **Setup Requirement:** The toggle scripts need permission to manipulate the network and firewall in the
      background without prompting you for a password. You must configure your passwordless sudo config by
      running exactly:
199 ```bash
200 echo "$USER ALL=(root) NOPASSWD: /sbin/pfctl, /usr/sbin/networksetup, /usr/bin/dscacheutil, /usr/bin/killall,
      /usr/bin/pkill, /bin/kill, /sbin/route, /sbin/ifconfig, /usr/bin/true, /opt/homebrew/bin/brew, /usr/local/
      bin/brew, /usr/bin/python3 -I $PWD/scripts/dns-proxy.py, /opt/homebrew/bin/python3 -I $PWD/scripts/dns-
      proxy.py, /usr/local/bin/python3 -I $PWD/scripts/dns-proxy.py" | sudo tee /etc/sudoers.d/nullexit
201 ```
202
203 ---
204
205 ## 6. Toggle Scripts
206
207 ### macOS `toggle.sh`
208 Fully automated lifecycle script. Also supports a native `.app` launcher:
209 ```bash
210 osacompile -o "Toggle Gateway.app" Toggle-Gateway.applescript
211 ```
212
213 Optional `.env` settings (common subset; the full canonical table lives in `devref.md`):
214 - `GATEWAY_BYPASS_PING=true` proceed even if pre-flight connectivity checks fail.
215 - `GATEWAY_USE_EXIT_NODE=false` DNS-only mode, skip exit-node routing.
216 - `GATEWAY_HIJACK_HOST=false` skip DNS hijacking on the host (provides VPN/adblocking only to external
      Tailscale peers).
217 - `GATEWAY_MSS=1180` override the TCP MSS clamp for the double-tunnel (default `1120`; raise to `1180` for
      more speed on a healthy path).
218 - `HOST_LEAK_PROBE=false` disable background host-egress leak prober (default: true).
219 - `HOST_MTU=1200` override the host Tailscale interface MTU. Defaults to 1200 to fit inside the 1280-byte WARP
      tunnel.
220 - `KILL_SWITCH=true` enforce strict network lock that breaks SSH if the VPN fails.
221 - `STOP_COLIMA_ON_EXIT=true` fully shut down the Colima VM on toggle-off (saves battery on dedicated hosts).
222 - `TAILSCALE_ALLOW_LAN_P2P=true` allow direct Tailscale LAN P2P connections between devices on the same
      network.
223 - **Default: `false`** On campus or enterprise Wi-Fi (WPA2-Enterprise / 802.1x), AP Client Isolation blocks
      local device-to-device traffic. When Tailscale still attempts a local P2P upgrade, macOS's `gvproxy` layer
      SNAT-mangles the reply's source port. WireGuard's endpoint roaming treats this as a legitimate address
      change and updates the phone's outbound path to the newly-mangled port which has no listener. This
      blackholes 100% of the phone's traffic while the DERP relay path is abandoned. Setting this to `false`
      preemptively drops all RFC1918-destined Tailscale UDP from the gateway so the phone receives a clean
      failure and safely falls back to DERP.
224 - **Auto-managed:** `watcher.sh` automatically detects WPA2-Enterprise via `system_profiler SPAirPortDataType`
      (the `airport` binary was removed in macOS 14.4) and probes for AP isolation by pinging the default
      gateway (`route -n get 1.1.1.1`) on every network change. It writes the detected value to
      `lan_p2p_detected` in the repo root, which `routing-fix.sh` re-reads every 30 seconds **no restart required

```



```

    .** Only set this manually if auto-detection fails. See `devref.md 15.7.1` for the full SNAT endpoint
    poisoning analysis and `devref.md 15.11.6` for the `airport` removal background. *(Note: Direct P2P between
    the gateway container and the host Mac itself is always permitted via the Docker bridge to bypass DERP
    relays, regardless of this setting).*
225 - Set to `true` on trusted home networks or Windows/mobile hotspots (no AP isolation) for a faster, lower-
    latency direct path.
226 - `WARP_FAIL_THRESHOLD=3` number of consecutive failed checks before forcing a gateway teardown (default: 6
    checks = 30s).
227 - `WARP_ENDPOINT_1` / `WARP_ENDPOINT_2` override the Cloudflare WARP WireGuard endpoints.
228
229 ### Linux `toggle.sh` / `recover.sh`
230 ```bash
231 ./toggle.sh # toggle ON/OFF (cross-platform, dispatches on $OSTYPE)
232 ./recover.sh # recovery tool (cross-platform)
233 ```
234
235 ### Sleep Prevention (macOS)
236 When the gateway is ON, `caffeinate -i` runs in the background to prevent idle system sleep, keeping Docker and
    all services alive even on battery. The display can still sleep normally.
237
238 ### Auto-Recovery (macOS post-wake / post-roam)
239 Install the launchd LaunchAgent so the gateway self-heals after sleep/wake and Wi-Fi roams:
240 ```bash
241 cp ./launchd/com.nullexit.wake-recovery.plist ~/Library/LaunchAgents/
242 sed -i '' "s|__NULLEXIT_HOME__|$(pwd)|" ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
243 launchctl load -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
244 ```
245 See `devref.md 15.5.1` for the full deep dive.
246
247 ---
248
249 ## 7. Networking Notes
250
251 - **Port conflicts:** None. Tailscale uses dynamic UDP ports; WARP uses outbound UDP:2408. The gateway binds
    `0.0.0.0:41642/udp` on the host for Tailscale's WireGuard listener this is intentional and required to
    receive inbound hole-punch packets from peers on the internet. All other internal ports (AdGuard :5335,
    SOCKS5 :1080) are bound to `127.0.0.1` only and are not reachable from the LAN.
252 - **Tailscale P2P vs DERP:** On the same Wi-Fi, Tailscale establishes a fast P2P connection. On cellular (CGNAT
    ), it always falls back to a DERP relay (~80-200ms). See `devref.md 5.1`.
253 - **AirDrop / AirPlay:** Unaffected the gateway only touches standard Wi-Fi/Ethernet interfaces, not `awdl0`.
254 - **VPN coexistence:** Do not run a local VPN client (WARP, Mullvad, NordVPN) simultaneously with the exit
    node. Both fight for the default route and will blackhole your connection.
255 - **Banking & financial sites:** May intermittently block logins through WARP. Banks use anti-fraud databases
    that flag datacenter IP ranges (like Cloudflare's `104.28.x.x`) as VPN/proxy traffic. Success fluctuates
    depending on which WARP IP you land on. If a site blocks you, either disable the exit node temporarily for
    that session or use the bank's mobile app (which often uses different fraud detection).
256 - **MSS clamping (Bufferbloat):** Double-tunnelling adds ~120-160 bytes of overhead per packet. This is now
    automatically handled natively via the macOS `pf` firewall, which universally applies TCP MSS Clamping to
    all outbound packets to strictly prevent MTU fragmentation and bufferbloat. No manual `.env` configuration
    is required. Additionally, the host's Tailscale interface MTU is explicitly lowered to 1200 (configurable
    via `HOST_MTU` in `.env`) to guarantee packets fit inside the container's WARP tunnel without fragmenting.
257
258 ---
259
260 ## 8. Privacy & Security Hardening
261
262 The stack shifts trust across four layers: WARP hides traffic from your ISP, AdGuard blocks tracking at DNS,
    Tailscale encrypts device-to-device, and `ipset` blocks known malicious IPs at the kernel.
263
264 For higher security, the gateway supports these drop-in upgrades:
265
266 | Layer | Default | Hardened |
267 |---|---|---|
268 | VPN Exit | Cloudflare WARP (US) | Mullvad (Swedish, no-logs, anonymous payment) |
269 | DNS Resolver | AdGuard via WARP | AdGuard via Mullvad DoH |
270 | Coordination | Tailscale (US) | Headscale (self-hosted) |
271 | Auth | Persistent OAuth keys | Ephemeral auth keys |
272 | Content | WireGuard asymmetric | WireGuard + Pre-Shared Keys (PSK) |
273
274 #### Python Runtime Airgap (Isolated Mode)
275 Never install third-party `pip` packages (like `requests` or `pyyaml`) into the Python environment that `
    nullexit` uses. All python scripts in this project (e.g., the `dns-proxy.py` daemon) are strictly
    engineered to run on the zero-dependency Python Standard Library. To strictly isolate the environment from
    supply-chain attacks, `nullexit` executes these scripts using Python's Isolated Mode (`python3 -I`).
    This deliberately blinds the execution environment to any malicious user-installed `pip` packages on the
    host, permanently air-gapping the gateway from PyPI.
276
277 See `devref.md 9` for the full threat model, Snowden programme analysis, and trust assumptions.
278
279 ---
280

```

```

281 ## 9. Debugging
282
283 - **`output.log`** All stderr from `toggle.sh`, `recover.sh`, `setup.sh`, and the rule-compiler is written
    here. The WARP Watcher (`WARP_FAIL_THRESHOLD`) and the Host Leak Probe (`HOST_LEAK_PROBE`) also write all
    their events here `LEAK`, `ROTATE`, `HOST-PROBE failed`, and WARP shutdown notices. Check this first.
284 - Every run now records lifecycle breadcrumbs regardless of `DEBUG_TRACE`: an `EXEC:` / `EXIT <code>:` line
    brackets each heavyweight command (`colima start`, `docker compose up -d --build`, `docker compose down`),
    and an `EXIT: code=<N> phase=<init|stop|start>` line is written when the script exits **for any reason**
    including a killed terminal, `kill`, or OOM. This closes a prior blind spot where an interrupted **START**
    (e.g. a `--restart` race while Wi-Fi was re-associating) left the log ending abruptly after the teardown
    with no indication of what failed. If a toggle "does nothing" or the gateway ends up half-up, `grep -E '
    EXEC:|EXIT ' output.log | tail` shows the last command run and its result.
285 - **`DEBUG_TRACE=true`** (in `.env`) Full command-by-command execution trace of `toggle.sh`, mirrored to `
    output.log` for deep post-mortems of intermittent failures. Each traced line is timestamped and tagged with
    `file:line:function` (via a custom `PS4`), so you can follow the *exact* code path a run took including
    subshells, Docker/Colima calls, and where it aborted. Off by default because it is verbose (the log grows
    quickly); enable it only while reproducing a specific problem, then set it back to `false`. Implementation
    note: it prefers bash's `BASH_XTRACEFD` (bash 4.1, e.g. most Linux) to send the trace to the log while
    keeping your terminal clean; on macOS's stock `/bin/bash` 3.2 it transparently falls back to tee'ing stderr
    to the log (trace also appears in the terminal). It never uses the dynamic-fd (`exec {var}>>`) form, so it
    is safe on 3.2. To read a trace: `grep -nE '^+ ' output.log`.
286 - **`bash scripts/diagnose-host-leak.sh`** One-shot host-routing diagnostic. Runs 8 checks and classifies your
    state into one of four known scenarios (System Extension conflict, SOCKS5 fallback, IPv6 leak, route-table
    freeze) or OK, and prints the exact fix command. Check 5 uses `route -n get 1.1.1.1` to ask the kernel
    which interface real traffic actually resolves through (more accurate than `netstat -rn | grep default` on
    macOS). Pass `--fix` to apply the remediation automatically. Pass `--watch` (or `--watch 30`) to run a full
    baseline diagnostic then continuously monitor warp/IPv6/default-route for leaks, alerting on any state
    change.
287 - **`bash scripts/host-leak-probe.sh`** Sub-second host-egress prober (enabled automatically via `
    HOST_LEAK_PROBE=true` in `.env`). Polls `cdn-cgi/trace` every 300ms directly from the host NIC not via `
    docker exec` so it catches flash-leaks invisible to the in-container WARP Watcher. All state changes, curl
    errors, and probe timeouts land in `output.log`. Grep with `grep LEAK output.log`.
288 - **`bash scripts/fix-docker-bridge-collision.sh`** One-shot fix for Docker bridge IP conflicts. If your local
    Wi-Fi router assigns you an IP in the `172.17.0.0/16` range, it will violently collide with Docker's
    default internal bridge, permanently killing your internet. This script detects the collision and auto-
    patches your Colima VM to use a safe subnet (e.g. `10.200.0.1/24`).
289 - **`devref.md`** Complete architecture reference, routing deep-dives, and full resolved-issues log. Read this
    before modifying any routing or firewall logic.
290
291 > [!CAUTION]
292 > **Two infinite routing loops are possible.** (1) If a hotspot device providing internet to the gateway host
    is *also* using the gateway as its exit node, packets bounce forever between the two. (2) When the host's
    default route is redirected to `utun`, WARP endpoint packets can loop back into the tunnel. Both are
    documented in `devref.md 15.2.1` and `15.2.2` and are automatically handled by `toggle.sh`.
293
294 ---
295
296 ## 10. Unified Project Document (Quine)
297
298 The project includes a `generate_tex.py` script that compiles the entire source code, configurations, and
    documentation into a single, unified LaTeX document (`nullexit_unified.tex` / `.pdf`).
299
300 Funnily enough, this document acts as a "project-level quine." It encapsulates the entirety of its own source
    code, its documentation, and the exact Python code required to generate itself. It serves as a self-
    contained, long-term archiving strategy providing everything needed to reconstruct and understand the `
    nullexit` system from a single file.
301
302 ---
303
304 ## 11. Acknowledgements
305 - **[SyameimaruKoa](https://github.com/SyameimaruKoa):** Dual-stack TCP MSS clamping, `SIGHUP` state-tracking
    in the routing sidecar, and `TS_AUTH_ONCE` integration.
306
307 ## 12. License
308 GNU Affero General Public License v3. See [LICENSE](./LICENSE).
309
310 ## 13. Changelog
311
312 - **July 12, 2026** Tor ControlPort dynamic password authentication (unified with `ADGUARD_PASSWORD`),
    enabling live circuit inspection via `/sweep`. See `devref.md 15.10.2`.
313 - **July 11, 2026** Tor container hardening: `obfs4proxy` restored (base image `debian:bookworm-slim`),
    robust bridge-file validation, and image pinning. See `devref.md 15.10.1`.
314 - **July 10, 2026** Roaming/startup stabilization suite: fixed the Wi-Fi-roam host-IP leak (raw ISP `132.x`)
    caused by the WARP Watcher racing post-wake recovery, plus a batch of pre-flight/concurrency/kill-switch/
    Tailscale-flag bugs and the macOS SNAT endpoint-poisoning P2P blackhole. Full post-mortems in the Incident
    Log see `devref.md 15.2.7`, `15.5.3` `15.5.5`, `15.2.8` `15.2.9`, `15.7.1`, `15.7.3` `15.7.5`, `15.12.18`.
315 - **July 6, 2026** Edge-case security and stability hardening: DNS Watcher interface independence, robust lock
    -file PID verification, fail-closed crypto integrity check, strict `iptables` pinning for tailscale0tun0,
    Docker port binding to `127.0.0.1` to prevent LAN exposure, routing verification via `netstat -nr`. See `
    devref.md 17`.

```

```

316 - **July 4, 2026** Colima bridged networking, SOCKS5 proxy migrated natively to Tailscale, headless prompt
      crashes fixed, proxy bypass domains added to fix LAN P2P. Python dependencies completely eliminated: `
      logger` and `rule-compiler` ported to blazing-fast Go multi-stage Docker builds. Added `crypto.sh` to
      enforce cryptographic signature validation on all core bash scripts. Massively refactored `toggle.sh` and `
      recover.sh` to eliminate ~200 lines of duplicated code by migrating network and process logic to `scripts/
      common.sh`. Added a concurrency mutex to `toggle.sh`, resolved unbound `TS_AUTHKEY` check crashes on setup,
      and integrated Go validation to the CI pipeline.
317 - **July 3, 2026** **Critical Security Updates:** Addressed the overnight silent IP leak and improved geo-
      blocking. Introduced `scripts/host-leak-probe.sh` (a sub-second host-egress leak detector) and a
      statistical threshold auto-shutdown watcher. Added `unlock-files.sh` to safely reset file permissions.
      Resolved numerous startup regressions and system bugs.
318 - **July 2, 2026** Two infinite routing loops resolved (`devref.md 15.2.2`): host-VM tunnel loop (WARP bypass
      routes now via gateway IP + manual `utun` redirection) and Docker Compose subnet takeover (`10.200.1.0/24`
      lock). `--accept-routes=true` now explicit on all `tailscale up --reset` calls.
319 - **July 2, 2026** Auto-recovery daemon documented in 6 above (`scripts/watcher.sh` + launchd plist). See `
      devref.md 15.5.1`.
320 - **July 2, 2026** Linux scripts (`scripts/toggle-linux.sh`, `recover-linux.sh`, `setup-linux.sh`) documented
      in 6.
321 - **July 1, 2026** `recover.sh --post-wake` ships; wired to watcher daemon. Triggered on every sleep/wake or
      network-state change.
322 - **June 28, 2026** 8 internal scripts moved to `scripts/`. User-facing files (`toggle.sh`, `recover.sh`, `
      setup.sh`, `docker-compose.yml`) stay at repo root.
323 - **June 28, 2026** All hardcoded install paths removed. `SCRIPT_DIR`-relative resolution and `
      _NULLEXIT_HOME` placeholder used throughout.
324
325 ### Cryptographic Integrity Verification
326
327 nullexit uses a built-in cryptographic integrity checker designed to provide tamper-*evidence* against
      accidental edits, logic bugs, or non-privileged malware. During setup, a 256-bit `NULLEXIT_SEED` is
      injected into your `.env` file. The core entrypoint scripts (`toggle.sh`, `recover.sh`, etc.) are hashed
      using HMAC-SHA256, and their signatures are stored in `signatures`.
328
329 *(Note: This is not a strict boundary against an attacker who already has write access to the repo, as they
      could simply read the seed and re-sign the scripts. It is designed as a strict footgun-catcher).*
330
331 If any script is modified without authorization, the gateway will loudly fail to boot. If you make intentional
      edits to the bash scripts, you must re-sign them by running:
332 ```bash
333 ./scripts/crypto.sh --sign
334 ```

```

3 agent.md

```

1 # nullexit Detailed Agent Analysis
2
3 > Complete per-file breakdown of every function, constant, duplication, and bug found.
4 > **Note (July 2026):** `sync-rules.py` and `logger.py` have been fully ported to Go (`scripts/rule-compiler/
      main.go` and `scripts/logger/main.go`) to dramatically improve performance and reduce container footprint
      via multi-stage Alpine builds. The analysis below reflects their previous Python states.
5
6 ## Important Agent Instruction
7 - **NEVER use `cat << EOF` or terminal redirections to write or modify files.** You must always use your native
      file-editing tools (`replace_file_content` or `multi_replace_file_content`). Using `cat EOF` leads to
      duplicated text, broken markdown headers, and structural corruption (which previously destroyed the section
      numbering in `devref.md`).
8 - **Always run `bash scripts/crypto.sh --sign` after making any modifications to the core bash scripts.** This
      is required to update the cryptographic HMAC-SHA256 signatures; otherwise, the startup integrity checks
      will fail and block execution.
9 - **Always run `git diff` and print the changes to the user *before* requesting or executing any git commit
      command.** The user must be shown the diff so they can review and understand the changes. Based on this
      diff, formulate a highly precise, detailed, and descriptive commit message (explaining exactly what was
      changed and why) rather than a generic summary, and present it to the user alongside the diff.
10 - **Always run `git status` before `git diff` to identify all modified and untracked files, ensuring no files (
      such as new scripts or configuration assets) are missed before formulating commit diffs and staging.**
11 - **Always use the `--restart` flag when asked to restart the toggle (e.g., run `./toggle.sh --restart`).**
12 - **NEVER execute any `sudo` commands (especially `sudo route`, `sudo dscacheutil`, or any network-modifying
      commands) without explicitly explaining to the user what the command does and why it is needed FIRST. Do
      not assume implicit permission. Wait for the user's approval before running it.**
13 - **When fixing an error and using logs to debug, always show the user the reasoning and the specific logs that
      support this theory.**
14 - **When the user tells you to "push", this means read git diffs and prepare commit messages that correspond to
      these changes (do not ignore any changes). If the changes can be atomized into multiple commits for easier
      understanding, do so.**
15 - **When the user says "latex this", it means you should run `python3 scripts/generate_tex.py` to regenerate
      the unified LaTeX document.**
16 - **NEVER apply changes to running gateway containers directly via `docker compose up` or `docker compose
      restart`.** Recreating or restarting containers (especially the `warp` container which hosts the network

```

```

namespace) breaks the shared network stack for all other services (`adguardhome`, `tailscale`, `routing-fix`), severing host connectivity and freezing macOS DNS. If you need to apply changes or rebuild containers, cleanly stop the gateway first using `./toggle.sh`, or trigger `recover.sh --post-wake` to rebuild and re-verify the network stack safely in the correct dependency order. Do NOT run `./toggle.sh --restart` or any network-rebuilding commands yourself if the execution could sever host network access; instead, explain the changes and ask the USER to run `./toggle.sh --restart` (or `./toggle.sh` stop and start) themselves to safely apply the modifications.
17
18 ## How to Verify Gateway is Working
19 Whenever modifications are made to the gateway scripts, routing, or containers, execute these verification checks in order:
20 1. **Container Health:** Run `docker compose ps` to ensure all containers (`warp`, `adguardhome`, `routing-fix`, `tailscale`) are `running` and `healthy`.
21 2. **DNS Hijacking Status:** Run `networksetup -getdnsservers "Wi-Fi"` (or appropriate network service) and ensure it is set exclusively to the gateway's Tailscale static IP (typically `100.100.21.8`).
22 3. **DNS Query Interception:** Run `dig @100.100.21.8 google.com` (using the gateway static IP from `gateway_ip`) to ensure AdGuard Home is active, intercepting, and resolving queries.
23 4. **Double-Tunnel Egress:** Run `curl -s https://www.cloudflare.com/cdn-cgi/trace | grep warp` to verify the egress is routed through Cloudflare WARP (`warp=on`).
24 5. **Host Leak Monitoring:** Run `tail -n 30 output.log` and verify the background watchers and host leak probe are reporting healthy status without `LEAK` warnings.
25
26 ---
27
28 ## Part 1: macOS Main Scripts
29
30 ### toggle.sh (1281 lines, 56KB)
31
32 **Functions (27 total):**
33
34 | # | Function | Lines | Purpose |
35 |---|---|---|---|
36 | 1 | `run_with_timeout()` | L3064 | Run a command with a timeout watchdog; kills after N seconds |
37 | 2 | `run_gui_cmd()` | L6880 | Run a GUI command as the logged-in console user |
38 | 3 | `write_gateway_active_marker()` | L8890 | Write timestamp to `/tmp/nullexit-gateway-active.marker` |
39 | 4 | `clear_gateway_active_marker()` | L9295 | Remove gateway active marker + TUNNEL_FAILED_CLOSED.marker |
40 | 5 | `start_sleep_prevention()` | L108154 | Start `caffeinate` in background with shutdown trap to revert DNS |
41 | 6 | `stop_sleep_prevention()` | L157167 | Stop caffeinate process via PID file |
42 | 7 | `start_dns_watcher()` | L173191 | Background loop to re-hijack DNS every 30s for Wi-Fi roaming |
43 | 8 | `stop_dns_watcher()` | L193203 | Stop DNS watcher via PID file |
44 | 9 | `start_warp_watcher()` | L220279 | Background WARP liveness monitor; triggers recover.sh after N consecutive failures |
45 | 10 | `stop_warp_watcher()` | L281291 | Stop WARP watcher via PID file |
46 | 11 | `start_host_leak_probe()` | L297310 | Start host-egress leak probe (300ms polling) |
47 | 12 | `stop_host_leak_probe()` | L312322 | Stop host leak probe via PID file |
48 | 13 | `cleanup_handler()` | L325382 | Error/signal handler: resets DNS, stops all watchers, tears down containers |
49 | 14 | `restart_tailscaled_daemon()` | L449463 | Restart tailscaled via brew services (user then system fallback) |
50 | 15 | `disconnect_tailscale_host()` | L465477 | Disconnect host Tailscale with retry on failure |
51 | 16 | `get_active_service()` | L480508 | Detect active macOS network service name (e.g., "Wi-Fi") |
52 | 17 | `add_warp_bypass_routes()` | L524546 | Add static routes for WARP endpoints (162.159.192.1, 162.159.193.1) |
53 | 18 | `remove_warp_bypass_routes()` | L548552 | Remove WARP bypass routes |
54 | 19 | `setup_exit_node_routing()` | L558577 | Override default route to point through Tailscale utun interface |
55 | 20 | `reset_dns()` | L588595 | Reset DNS to 1.1.1.1 on ACTIVE_SERVICE + ENO_SERVICE |
56 | 21 | `cleanup_network_state()` | L600645 | Nuclear cleanup: disable proxies, flush DNS cache, flush routes, power-cycle Wi-Fi, kill sharing services |
57 | 22 | `enable_socks_proxy()` | L663682 | Enable system-wide SOCKS5 proxy on macOS |
58 | 23 | `disable_socks_proxy()` | L684689 | Disable SOCKS5 proxy |
59 | 24 | `stop_local_dns_proxy()` | L702710 | Kill local Python DNS proxy |
60 | 25 | `start_local_dns_proxy()` | L713768 | Start Python DNS proxy (UDP:53 TCP:5354) |
61 | 26 | `force_dns_to_gateway()` | L780820 | Force DNS to a specific IP with 3-retry verification loop |
62 | 27 | `is_gateway_active()` | L833854 | Check if gateway is running (containers, DNS, SOCKS proxy) |
63
64 **Constants:**
65
66 | Constant | Value | Line |
67 |---|---|---|
68 | `LOG_FILE` | `$PWD/output.log` | L8 |
69 | `PID_FILE` | `/tmp/nullexit-caffeinate.pid` | L105 |
70 | `DNS_WATCHER_PID_FILE` | `/tmp/nullexit-dns-watcher.pid` | L169 |
71 | `WARP_WATCHER_PID_FILE` | `/tmp/nullexit-warp-watcher.pid` | L170 |
72 | `HOST_LEAK_PROBE_PID_FILE` | `/tmp/nullexit-host-leak-probe.pid` | L171 |
73 | `SOCKS_PROXY_PORT` | `1080` | L661 |
74 | `PATH` export | `/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH` | L396 |
75 | Gateway active marker | `/tmp/nullexit-gateway-active.marker` | L89 |
76 | TUNNEL_FAILED_CLOSED marker | `${SCRIPT_DIR}/TUNNEL_FAILED_CLOSED.marker` | L94 |
77 | WARP endpoints | `162.159.192.1`, `162.159.193.1` | L535-544 |

```

```

78 | DNS fallback | `1.1.1.1` | L592 |
79 | DNS search domain | `ts.net` | L794 |
80 | `LOCK_FILE` | `/tmp/nullexit-toggle.lock` | L62 |
81
82 ---
83
84 ### recover.sh (566 lines, 23KB)
85
86 **Functions (7 total):**
87
88 | # | Function | Lines | Purpose |
89 | ---|-----|-----|-----|
90 | 1 | `step()` | L40 | Print bold step header |
91 | 2 | `ok()` | L41 | Print green success message |
92 | 3 | `warn()` | L42 | Print yellow warning message |
93 | 4 | `fail()` | L43 | Print red failure message |
94 | 5 | `die()` | L44 | Print red error and exit |
95 | 6 | `run_with_timeout()` | L5674 | Run command with timeout (bash-native, no GNU coreutils) |
96 | 7 | `restart_tailscaled_daemon()` | L118129 | Restart tailscaled daemon via brew services |
97
98 **Constants:**
99
100 | Constant | Value | Line |
101 | ---|-----|-----|
102 | Color codes | `RED`, `GREEN`, `YELLOW`, `BOLD`, `NC` | L37-38 |
103 | `SCRIPT_DIR` | `${dirname "${BASH_SOURCE[0]}"}` | L50 |
104 | `PATH` export | `/opt/homebrew/bin:/usr/local/bin:$PATH` | L77 |
105 | `PID_FILE` | `/tmp/nullexit-cafeinate.pid` | L387 |
106 | `DNS_WATCHER_PID_FILE` | `/tmp/nullexit-dns-watcher.pid` | L407 |
107 | `HOST_LEAK_PROBE_PID_FILE` | `/tmp/nullexit-host-leak-probe.pid` | L423 |
108 | WARP endpoints | `162.159.192.1`, `162.159.193.1` | L329-336 |
109 | TUNNEL_FAILED_CLOSED marker | `${SCRIPT_DIR}/TUNNEL_FAILED_CLOSED.marker` | L51 |
110
111 **Duplicated logic from toggle.sh:**
112 *(Note: As of July 2026, **ALL** of the below duplicated logic has been successfully extracted to `scripts/`
113   common.sh`!)*
114 - `run_with_timeout()` simpler subshell version vs toggle's PID-tracking version
115 - `restart_tailscaled_daemon()` near-identical (uses warn()/ok() vs echo)
116 - Active network service detection inline at L217-233 (toggle has `get_active_service()` function)
117 - ENO service resolution inline at L230-233 (same as toggle L515-516)
118 - WARP bypass routes inline at L329-337 (toggle has `add_warp_bypass_routes()` )
119 - Exit node routing setup inline at L340-345 (toggle has `setup_exit_node_routing()` )
120 - DNS cache flushing L296-302 (same as toggle L611-616)
121 - Wi-Fi power-cycling L442-461 (similar to toggle L626-637)
122 - Proxy disabling L282-288 (same as toggle L604-608)
123 - Sharing services reset L586 (same as toggle L642)
124 - PID file stop pattern L387-399, L407-419, L423-435 (same as toggle L157-167, L193-203, L312-322)
125 - KILL_SWITCH check L194 (same as toggle L141, L469)
126 - .gateway_ip reading L138-142, L209-213 (same pattern as toggle L1080)
127
128 ---
129
130 ### setup.sh (410 lines, 18KB)
131
132 **Functions (4 total):**
133
134 | # | Function | Lines | Purpose |
135 | ---|-----|-----|-----|
136 | 1 | `step()` | L10 | Print bold blue step header |
137 | 2 | `ok()` | L11 | Print green success message |
138 | 3 | `warn()` | L12 | Print yellow warning message |
139 | 4 | `die()` | L13 | Print red error and exit |
140
141 **Constants:**
142
143 | Constant | Value | Line |
144 | ---|-----|-----|
145 | Color codes | `RED`, `GREEN`, `YELLOW`, `BLUE`, `BOLD`, `NC` | L7-8 |
146 | `RECOMMENDED_TS_VERSION` | `1.98.5` | L71 |
147 | `ADGUARD_PASSWORD` | `nullexit` | L215 |
148 | Default .env values | `GATEWAY_RULE_PROFILE=medium`, `GATEWAY_MSS=1120`, etc. | L240-248 |
149
150 ---
151
152 ## Part 2: Linux Scripts
153
154 ### toggle.sh Linux path (unified into the root cross-platform script)
155
156 The Linux toggle is no longer a separate file. `scripts/toggle-linux.sh` was
157 merged into the repo-root `toggle.sh`, which dispatches on `OSTYPE`: an early
158 `if [[ "$OSTYPE" != darwin* ]]` block runs the self-contained Linux toggle

```

```

158 (shuf / ss / nmcli / ip / systemd) and exits before the macOS body. The shared
159 DNS/proxy/daemon primitives it used were already moved to `common.sh` in the
160 `313dc32` unification, so both branches call them by the same names.
161
162 ---
163
164 ### recover.sh Linux path (unified into the root cross-platform script)
165
166 Linux recovery is no longer a separate file. `scripts/recover-linux.sh` was
167 merged into the repo-root `recover.sh`, which dispatches on `$OSTYPE`: the Linux
168 branch runs a self-contained teardown (resolvectl / nmcli / ip) and exits before
169 the macOS dual-mode body. All formatting/timeout helpers come from `common.sh`.
170
171 ---
172
173 ### scripts/setup-linux.sh (416 lines, 18KB)
174
175 **Functions (4 total):**
176
177 | # | Function | Line | Purpose |
178 |---|-----|-----|-----|
179 | 1 | `step()` | L10 | Print formatted step header |
180 | 2 | `ok()` | L11 | Print green success message |
181 | 3 | `warn()` | L12 | Print yellow warning message |
182 | 4 | `die()` | L13 | Print red error + exit 1 |
183
184 **~95% identical to macOS setup.sh.** Only differences:
185 1. Desktop shortcuts (L366-391 Linux `.desktop` files vs macOS `.app` via `osacompile`)
186 2. Final instructions text
187 3. .env template: macOS adds `GATEWAY_USE_EXIT_NODE`, `WARP_FAIL_THRESHOLD`, `HOST_LEAK_PROBE`, `KILL_SWITCH`
   Linux omits these
188
189 ---
190
191 ## Part 3: Utility Scripts
192
193 ### scripts/diagnose-host-leak.sh (722 lines, 38KB)
194
195 **Purpose:** Comprehensive host-routing diagnostic. 8 checks classify host IP leaks. Supports `--fix` and `--
   watch` modes.
196
197 **Duplicated logic:**
198 - `resolve_active_service()` identical pattern as toggle.sh, recover.sh, fix-docker-bridge-collision.sh
199 - `run_with_timeout()` reimplemented timeout
200 - Color codes RED/GREEN/YELLOW/BOLD/NC with tty detection
201 - WARP check via `cloudflare.com/cdn-cgi/trace`
202 - `export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"`
203 - .gateway_ip reading pattern
204
205 ### scripts/fix-docker-bridge-collision.sh (402 lines, 18KB)
206
207 **Purpose:** Fixes Docker bridge subnet collisions with `10.200.1.0/24`.
208
209 **Duplicated logic:**
210 - `resolve_active_iface()` same interface detection as diagnose-host-leak.sh
211 - Color codes, step()/ok()/warn()/fail()/die() formatting helpers
212
213 ### scripts/fix-ssh-delay.sh (63 lines, 2.7KB)
214
215 **Purpose:** One-shot fix for ~20s SSH delay. Standalone, minimal duplication (uses hardcoded / instead of
   color functions).
216
217 ### scripts/host-leak-probe.sh (110 lines, 5.5KB)
218
219 **Purpose:** Sub-second host-egress leak prober at 300ms intervals.
220
221 **Duplicated logic:**
222 - WARP check via cdn-cgi/trace (same curl + awk as toggle.sh WARP Watcher)
223 - Color codes (hardcoded; status updates and warnings are conditioned on TTY detection to prevent log pollution
   )
224 - LOG_FILE = `$PROJECT_ROOT/output.log`
225
226 ### scripts/reinstall-host-tailscale.sh (740 lines, 31KB)
227
228 **Purpose:** Complete uninstall + reinstall of host Tailscale.
229
230 **Duplicated logic:**
231 - `with_timeout()` yet another bash timeout implementation (polling-based, different from diagnose and toggle
   versions)
232 - Color codes + logging functions
233 - System Extension detection logic (similar to diagnose-host-leak.sh)

```



```

234
235 ### scripts/routing-fix.sh (266 lines, 10.5KB)
236
237 **Purpose:** Runs INSIDE warp container. Maintains routing table 200, FORWARD rules, country blocking, IP
    blocklists, tunnel health checks.
238
239 **Duplicated logic:**
240 - (Fixed) WARP_ENDPOINT defaults are now centralized in common.sh
241 - WARP health check via cdn-cgi/trace (another instance)
242 - iptables dual-backend pattern (for-loop over `iptables iptables-legacy`)
243
244 ### scripts/watcher.sh (269 lines, 13KB)
245
246 **Purpose:** Long-running launchd daemon for post-wake/post-roam recovery.
247
248 **Key function `detect_lan_p2p_mode()`:**
249 Added July 10, 2026. Called on startup and on every network state change (Listener 2 NET: events). Determines
    whether the current Wi-Fi network safely allows direct Tailscale LAN P2P connections:
250 1. **WPA2-Enterprise check:** `system_profiler SPAirPortDataType` reads the `Security:` field for the current
    association. Enterprise = 802.1x = always AP-isolated sets `allow=false`.
251 2. **AP isolation probe:** `route -n get 1.1.1.1` finds default gateway IP, then `ping -c 3 -t 1 -q "$gw"` if
    0 responses on a non-empty network, AP isolation is active sets `allow=false`.
252 3. **Output:** Writes `true` or `false` to `.lan_p2p_detected` in the repo root.
253 4. **Consumer:** `routing-fix.sh` reads this file every 30 seconds and enforces/relaxes the RFC1918 DROP rule
    accordingly no restart required.
254
255 **Duplicated logic:**
256 - PATH export (similar to toggle.sh/recover.sh)
257 - Marker file pattern (`/tmp/nullexit-gateway-active.marker`)
258
259 ### scripts/dns-proxy.py (92 lines, 2.9KB) Clean, standalone
260
261 ### scripts/logger.py (144 lines, 5.9KB) Clean, standalone
262
263 ---
264
265 ## Part 4: Cross-Cutting Issues
266
267 ### Functions Identical Across macOS Linux
268
269 | Function | macOS toggle | macOS recover | macOS setup | Linux toggle | Linux recover | Linux setup |
270 |-----|-----|-----|-----|-----|-----|-----|
271 | `run_with_timeout()` | | | identical | identical | | |
272 | `stop_local_dns_proxy()` | | | identical | | | |
273 | `start_local_dns_proxy()` | | | ~95% similar | | | |
274 | `is_gateway_active()` | | | ~80% similar | | | |
275 | `step/ok/warn/die` | | | identical | identical | | |
276
277 ### Platform-Adapted Functions (same logic, different OS commands)
278
279 | Function | Linux Command | macOS Command |
280 |-----|-----|-----|
281 | `get_active_service()` | `ip route get 1.1.1.1 \| awk '{print $5}'` | `route get default` + `networksetup -
    listnetworkserviceorder` |
282 | `reset_dns()` | `resolvectl revert` | `networksetup -setdnsservers` |
283 | `cleanup_network_state()` | `ip link set dev down/up` | `networksetup -setairportpower off/on` + proxy
    disable |
284 | `force_dns_to_gateway()` | `resolvectl dns` + `resolvectl domain -ts.net` | `networksetup -setdnsservers` +
    `--setsearchdomains ts.net` |
285 | `start_sleep_prevention()` | `systemd-inhibit --what=sleep` | `caffeinate -i` |
286 | `enable/disable_socks_proxy()` | Stub (no-op on Linux) | `networksetup -setsocksfirewallproxy` |
287 | DNS cache flush | `resolvectl flush-caches` | `dscacheutil -flushcache` + `killall -HUP mDNSResponder` |
288
289 ### Features Missing From Linux Scripts
290
291 These exist in `toggle.sh`'s macOS branch but NOT its Linux branch:
292
293 1. `write_gateway_active_marker()` / `clear_gateway_active_marker()`
294 2. `restart_tailscaled_daemon()`
295 3. `start_warp_watcher()` / `stop_warp_watcher()` WARP liveness monitor
296 5. `add_warp_bypass_routes()` / `remove_warp_bypass_routes()` / `setup_exit_node_routing()`
297 6. `LOG_FILE` structured logging (timestamps to output.log)
298 7. MSS clamping injection (step 4c)
299 8. Rule count reporting
300 9. `--post-wake` mode in recover
301 10. Container teardown on failure in cleanup_handler
302
303 ---
304
305
306

```

```

307 ## Part 5: Constant Duplication Map
308
309 | Value | Files |
310 |-----|-----|
311 | `162.159.192.1` (WARP endpoint) | docker-compose.yml, routing-fix.sh (now dynamically loaded via .env/common.sh!) |
312 | `5335` (AdGuard DNS port) | docker-compose.yml, AdGuardHome.yaml, post-rules.txt |
313 | `5354` (mapped DNS port) | docker-compose.yml, dns-proxy.py |
314 | `41642` (Tailscale WireGuard) | docker-compose.yml, routing-fix.sh, post-rules.txt |
315 | `1080` (SOCKS5 port) | docker-compose.yml, toggle.sh |
316 | `1120` (MSS clamp) | post-rules.txt, .env |
317 | `admin:nullexit` (AdGuard creds) | AdGuardHome.yaml (bcrypt) |
318 | `10.200.1.0/24` (Docker subnet) | (Fixed) no longer duplicated |
319 | `America/New_York` (timezone) | docker-compose.yml (4 services) |
320 | `/tmp/nullexit-caffeinate.pid` | (Fixed) centralized in common.sh |
321 | `/tmp/nullexit-dns-watcher.pid` | (Fixed) centralized in common.sh |
322 | `/tmp/nullexit-host-leak-probe.pid` | toggle.sh, recover.sh |
323 | `/opt/homebrew/bin:...` (PATH) | (Fixed) centralized in common.sh |
324
325 ---
326
327 ## Part 6: Refactoring Outcome (Implemented July 2026)
328
329 **Decision: "Scripts-only" architecture**
330 Instead of introducing a new `lib/` directory, all shared code was moved into the existing `scripts/` directory to keep the project hierarchy flat and simple.
331
332 ### 1. `scripts/common.sh` (288 lines, 10KB)
333 Extracts the fundamental primitives and networking logic used across orchestrator scripts:
334 - **Formatters:** `step()`, `ok()`, `warn()`, `fail()`, `die()`
335 - **Timeout Wrapper:** `run_with_timeout()` (Canonical version with PID tracking, graceful SIGTERM, and SIGKILL fallback)
336 - **Daemon Lifecycle:** `restart_tailscaled_daemon()`, `stop_pidfile_daemon()` (collapsed caffeinate, DNS watcher, WARP watcher, and host leak probe teardown logic)
337 - **Network Interface/Routing:** `get_active_service()`, `get_en0_service()`, `bounce_wifi_interfaces()`, `add_warp_bypass_routes()`, `remove_warp_bypass_routes()`
338 - **Proxy/DNS Cleanup:** `disable_all_proxies()`, `flush_dns_cache()`, `reset_sharing_services()`
339 - **Configuration Helpers:** `read_adguard_ip()`, `is_kill_switch_enabled()`, `get_warp_endpoint_1()`, `get_warp_endpoint_2()` (which centralizes the hardcoded 162.159.192.1 / .193.1 into .env logic)
340
341 ### 2. `scripts/setup-common.sh` (~350 lines)
342 Extracts the heavy, duplicated installation logic shared between `setup.sh` (macOS) and `scripts/setup-linux.sh`:
343 - Docker and Colima prerequisite checks
344 - `wgcf` binary download and WARP key generation
345 - `.env` configuration file generation
346 - Interactive Tailscale Auth Key and AdGuard Rule Profile prompts
347
348 ### Sourcing Pattern
349 - macOS root scripts (`toggle.sh`, `recover.sh`, `setup.sh`) source via:
350   `source "$(dirname "${BASH_SOURCE[0]}")" && pwd)/scripts/common.sh`
351 - Scripts under `scripts/` (`watcher.sh`, `diagnose-host-leak.sh`, etc.) source via:
352   `source "$(dirname "${BASH_SOURCE[0]}")/common.sh`
353
354 ## Agent Verbs / Protocols
355
356 ### `/sweep` (or `sweep the gateway`)
357 When the user invokes this verb, the agent MUST perform a full end-to-end connectivity, health, security, and performance audit of the gateway:
358 1. **WARP Validation:** Run `curl -s https://www.cloudflare.com/cdn-cgi/trace | grep warp=` (must equal `on`).
359 2. **Tor Proxy & Control Validation:**
360    - Source the dynamically generated ports from `/tmp/nullexit-ports.env` (or identify them using `docker compose ps`).
361    - **SOCKS5 Check:** Run `curl -s --socks5-hostname 127.0.0.1:$TOR SOCKS_PORT https://check.torproject.org/api/ip` to confirm the SOCKS5 proxy successfully connects and routes through the Tor network.
362    - **Control Port Check:** Query the Tor Control Port using netcat: `echo "PROTOCOLINFO" | nc 127.0.0.1:$TOR_CONTROL_PORT`. Verify it returns a standard `250-PROTOCOLINFO` response, confirming the Control Port is active and communicating.
363    - **Circuit & Node Lookup:** Query the Tor Control Port for the active path (`GETINFO circuit-status`). For each active built circuit, parse the relay fingerprints, lookup their IP addresses (`GETINFO ns/id/<fingerprint>`), and resolve their country codes using Tor's local GeoIP database (`GETINFO ip-to-country/<IP>`). Include the list of active circuits, showing the role (Guard/Middle/Exit), nickname, IP, and country of each hop in the final sweep report.
364    - **Transparent Onion Validation:** Run `nslookup duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion` to verify it resolves to an IP within `198.18.0.0/15` benchmark range. Verify transparent dark web routing by running `curl -sI -H "Host: duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion" http://<resolved-ip>/` (should return HTTP 301/200).
365 3. **DNS Leak Validation:**
366    - Audit the upstream resolver currently utilized by the host. Run `curl -s https://edns.ip-api.com/json` to fetch details of the DNS resolver processing the client's requests.

```



```

367 - Verify that the resolver IP and organization returned belong to Cloudflare (e.g., AS13335) or match the
368 WARP tunnel's exit range, and that no DNS traffic leaks to the user's raw physical ISP DNS.
369 4. Performance Benchmark Audit:
370 - Run the python benchmark script `python3 benchmarks/test_load.py`.
371 - Report the overall load speed and the ratio of successfully fetched vs. blocked/failed subresources for
372 the targeted domains to ensure ad-blocking and MTU encapsulation are performing correctly.
373 5. Tailscale Mesh Validation:
374 - Run `ping -c 1 100.100.21.8` to ensure the gateway is reachable.
375 - Run `tailscale status` to identify all active/online nodes in the mesh.
376 - For every online peer returned in the status output, execute a ping check (`ping -c 1 <peer-ip>`) to
377 verify bidirectional connectivity and confirm healthy routing to all active mesh devices.
378 6. Log Audit: Run `tail -n 100 output.log | grep -iE "(error|fail|warn)"` to catch any underlying component
failures or warnings.
### `/latex` (or `generate latex`)
When the user invokes this verb, the agent MUST run `python3 -I scripts/generate_tex.py` to regenerate the
documentation source code (`nullexit_unified.tex`). The agent MUST NOT attempt to compile the `.tex` file
into a PDF (the user handles PDF compilation locally).

```

4 devref.md

```

1 # nullexit Development Reference & Resolved Issues
2
3 > Last updated: July 12, 2026
4 > Purpose: Provide any LLM or developer with complete project understanding, debugging history, and
5 resolved issues so they can make informed changes without re-reading every file.
6 > Diagrams: See [diagrams.md](./diagrams.md) for system architecture, toggle.sh flowcharts, monitoring
7 layer, traffic sequence, recover.sh decision tree, and the full failureself-healing map.
8
9 This reference is organized into four parts:
10 > - Part I Architecture & Reference (18)
11 > - Part II Design Decisions & Threat Model (914)
12 > - Part III Incident Log & Resolved Issues (15)
13 > - Part IV Observations, Changelog & TODO (1618)
14
15 ---
16 # Part I Architecture & Reference
17
18 ## 1. What Is nullexit?
19
20 nullexit is a Tunnel-in-Tunnel VPN gateway that chains Tailscale (mesh VPN) through Cloudflare WARP
21 (exit VPN). It runs on a macOS host inside Docker containers managed by Colima (lightweight Linux VM).
22 The goal: any device on the user's Tailscale mesh can use this gateway as an exit node, and all traffic
23 exits through Cloudflare WARP achieving double encryption, ISP invisibility, and network-wide ad-blocking
24 via AdGuard Home.
25
26 ### Traffic Flow
27
28 Phone/Laptop Tailscale mesh (WireGuard) Mac host Docker container
29 Tailscale decrypts Gluetun re-encrypts WARP tun0 Cloudflare Internet
30
31 ### SOCKS5 Failover Path (if exit node is unavailable)
32
33 DNS: Browser host 127.0.0.1:53 Python proxy TCP:5354 AdGuard WARP DNS
34 TCP: Apps macOS SOCKS5 proxy (127.0.0.1:1080) container tun0 WARP Internet
35
36 ---
37
38 ## 2. Architecture
39
40 ### Containers (all share `warp`'s network namespace via `network_mode: service:warp`)
41 | Container | Image | Role |
42 |-----|-----|-----|
43 | `warp` | `qmcgaw/gluetun:v3.41.1` | Gluetun WireGuard client Cloudflare WARP. Owns the network namespace.
44 | Strict firewall. |
45 | `tailscale` | `tailscale/tailscale:v1.98.4` | Advertises as exit node on the mesh. Also provides the built-in
46 | SOCKS5 proxy fallback via `TS_SOCKS5_SERVER=1080`. |
47 | `routing-fix` | `alpine:3.20` | Sidecar that maintains routing tables + iptables rules every 30 seconds. |
48 | `rule-compiler` | `golang:1.22-alpine` / `alpine:3.20` | One-shot startup container that compiles 500k+ DNS
49 | block rules in <2s and immediately exits. |
50 | `adguardhome` | `adguard/adguardhome:v0.107.77` | DNS sinkhole for ads/trackers. Listens on port 5335.
51 | Upstream DNS: `127.0.0.1:53` (through WARP). |

```

```

46 ### Port Mappings (on host via Colima)
47 | Host Port | Container Port | Protocol | Purpose |
48 |-----|-----|-----|-----|
49 | 5354 | 5335 | TCP+UDP | AdGuard DNS |
50 | (Tailscale IP):3000 | 3000 | TCP | AdGuard web UI (Internal to Mesh) |
51 | 41641 | 41641 | UDP | Tailscale WireGuard direct |
52 | 1080 | 1080 | TCP | SOCKS5 proxy |
53
54 ### Host Components
55 - **Colima VM** Linux VM running Docker (600MB RAM + 400MB swap)
56 - **tailscaled** Standalone Tailscale daemon via `brew install tailscale` + `brew services start tailscale` (
  NOT the GUI `.app`)
57 - **macOS network settings** DNS, SOCKS proxy, routing all manipulated by `toggle.sh`
58
59 ---
60
61 ## 3. Key Files
62
63 | File | Lines | Purpose |
64 |-----|-----|-----|
65 | `toggle.sh` | ~2109 | **Main script (macOS + Linux).** Dispatches on `$OSTYPE` an early Linux block (shuf/ss
  /nmcli/ip, systemd) runs and exits before the macOS body (jot/netstat/networksetup, launchd/pf). Detects
  state, toggles gateway ON/OFF. Handles DNS hijacking, Tailscale exit node, SOCKS proxy, sleep prevention,
  timeouts, cleanup. |
66 | `setup.sh` | 406 | **One-time setup.** Installs deps (Docker, Tailscale, wgcf), generates WARP keys, writes
  `.env`, configures AdGuard via API, starts containers. |
67 | `docker-compose.yml` | 194 | Service definitions for all 5 containers. |
68 | `scripts/routing-fix.sh` | 201 | Maintains routing tables (table 200 for SOCKS5, table 52 for CGNAT) and
  FORWARD iptables rules in both nftables and legacy backends. Loads and enforces the IP blocklist via `ipset`
  . Runs in a 30-second loop. |
69 | `post-rules.txt` | 18 | Gluetun iptables rules loaded at container start. FORWARD accept for tailscale0, NAT
  MASQUERADE on tun0, DNS redirect to port 5335, IPv6 drop, TCP MSS clamping. |
70
71 | `scripts/dns-proxy.py` | 91 | Local DNS proxy. UDP:53 TCP:5354 with proper 2-byte length prefix. Fallback
  when exit node is unavailable. |
72 | `scripts/rule-compiler/` | ~800 | Unified threat compiler (ported to Go from Python). **DNS pipeline:**
  fetches remote blocklists, deduplicates against AdGuard native filters, optimizes subdomains (~50%
  reduction), compiles to AdGuard syntax. **IP pipeline:** fetches threat-intel feeds (Spamhaus, Feodo, ET,
  CINS), strips private/Tailscale ranges, outputs atomic `ipset` restore file. Built dynamically in Docker
  multi-stage build. |
73 | **`adguard/work/`** | | **Bind-mounted container data folder. Off-limits from outside Docker READ-ONLY-
  EXCEPT-VIA-`sync-rules`. See README 6.** |
74 | `recover.sh` | ~742 | Nuclear recovery script (**macOS + Linux** dispatches on `$OSTYPE`; the Linux branch
  runs a self-contained teardown via `resolvectl`/`nmcli`/`ip` and exits, macOS falls through to the dual-
  mode body). Gain: use `--post-wake` (macOS) for non-destructive refresh after sleep/wake or Wi-Fi roam
  keeps the gateway live while still re-hijacking DNS + refreshing Tailscale exit-node + force-recreating
  warp if unhealthy. Resets DNS on ALL services, disables proxies, flushes routes, stops containers, kills
  sleep prevention, power-cycles Wi-Fi. Cryptographically verified. |
75 | `scripts/diagnose-host-leak.sh` | 580+ | **One-shot host-routing diagnostic.** Runs 8 checks (Tailscale,
  SOCKS5, CDN-cgi/trace, IPv6 leak, default route, output.log hints, gateway IP, Tailscale System Extension
  conflict), classifies into ONE of three known leak scenarios (A=SOCKS5 fallback, B=IPv6 leak, C=route
  freeze) or OK, writes a timestamped report to `host-leak-diagnostic-<UTC>.txt`, and prints a ready-to-run
  fix on stdout. Pass `--fix` to apply the matched remediation and re-verify egress. Pass `--watch` (or `--
  watch 30`) to run a full baseline then continuously monitor warp/IPv6/default-route every N seconds,
  alerting on any state change. See 15.6.1. |
76 | `scripts/host-leak-probe.sh` | ~110 | **Continuous sub-second host-egress prober.** Launched automatically by
  `toggle.sh` when `HOST_LEAK_PROBE=true` in `.env` (default). Polls `cdn-cgi/trace` every 300ms directly
  from the host NIC via `curl` not via `docker compose exec` so it detects flash-leaks invisible to the in-
  container WARP Watcher. Logs only on state change (`LEAK`, `ROTATE`, `HOST-PROBE failed/timeout`) to
  `output.log`. curl errors (`2>>output.log`) also land there. Stopped gracefully (SIGTERM) by `toggle.sh`
  STOP path, `cleanup_handler`, and `recover.sh`. See 15.6.1 (Host Leak Probe deep dive). |
77 | `scripts/fix-docker-bridge-collision.sh` | ~290 | **One-shot fix for 15.2.3 (Docker bridge subnet collision)
  .** Detects Docker's `172.17.0.0/16` bridge colliding with the host Wi-Fi subnet (the symptom that produces
  `default 172.17.0.1 UGScg en0` in `netstat -rn`), picks a non-colliding `docker.bip` from a small
  candidate set, idempotently edits `~/colima/default/colima.yaml` with a timestamped backup, restarts
  Colima, rebinds Wi-Fi DHCP, verifies the new default route is no longer Docker's bridge, and re-runs
  `toggle.sh` + `scripts/diagnose-host-leak.sh`. Supports `--dry` and `--skip-toggle`. |
78 | `scripts/watcher.sh` | 194 | **Long-running post-wake + post-roam daemon.** Launched by launchd LaunchAgent;
  subscribes to `com.apple.powermanagement` events via `log stream` and to network-state changes via `scutil
  n.watch`. Both fire `recover.sh --post-wake`. Single-instance lock + SCRIPT_DIR-relative resolve of
  RECOVER. See 15.5.1. |
79 | `scripts/common.sh` | ~40 | **Shared script primitives.** Centralizes formatted echo statements (`step`, `ok`
  , `warn`, `die`) and the safe background `run_with_timeout` execution wrapper. Sourced by all orchestrator
  scripts across macOS and Linux. |
80 | `scripts/setup-common.sh` | ~350 | **Shared installation logic.** Centralizes logic for verifying Docker,
  installing Tailscale/wgcf, generating `.env`, and prompting for auth keys. Sourced by `setup.sh` and `setup
  -linux.sh`. |
81 | `scripts/setup-linux.sh` | ~80 | **Linux sibling of `setup.sh`.** Sources `setup-common.sh`. Distro detection
  (apt/dnf/pacman), installs Linux-specific dependencies, creates `.desktop` shortcuts. |
82 | `scripts/Toggle-Gateway.bat` | ~300 | **Windows Toggle helper.** Moved from root to `scripts/`. Helps invoke
  the framework on Windows if applicable. |

```

```

83 | `scripts/reinstall-host-tailscale.sh` | ~740 | **Nuke-and-reinstall tool for host Tailscale.** Completely
    | purges Tailscale, its settings, macOS Background Items (`.btm` database), and stale System Extensions.
    | Wipes ALL Login Items via `sftool reset-login-items` (requires sudo). Useful for crisis recovery but
    | destructive to other login items. |
84 | `scripts/fix-ssh-delay.sh` | ~100 | **Fixes SSH connection delays.** One-shot script to address SSH delays
    | caused by DNS or routing issues, useful in crisis situations. |
85 | `scripts/logger/` | ~100 | **Internal logger piped from `scripts/routing-fix.sh`** inside the `warp`
    | container. Ported to Go from Python. Built via Docker multi-stage build. |
86 | `black_list.txt` | 71 | Custom domains to block (ads, trackers, telemetry). Supports `$important` modifier. |
87 | `white_list.txt` | 222 | Domains to force-allow (YouTube, Apple services, etc.). Always wins over blocks. |
88 | `.env` | ~14 | WARP WireGuard keys, Tailscale auth key, rule profile. **Contains secrets & NULLEXIT_SEED.** |
89 | `.gateway_ip` | 1 | Static gateway Tailscale IP (fallback for dynamic resolution). |
90 | `scripts/unlock-files.sh` | ~20 | **One-shot stale permission fix.** Uses atomic rename (`cp` + `mv`) to
    | replace locked inodes (mode `000`/`0444` from old `chmod` decisions) with fresh writable ones no `chmod`
    | called. |
91 | `scripts/crypto.sh` | ~50 | **Cryptographic integrity enforcement.** Uses `NULLEXIT_SEED` from `.env` to sign
    | core bash scripts (HMAC-SHA256) and strictly verify them on start to prevent manipulation. Pass `--sign`
    | or `--verify`. |
92 | `scripts/pf.conf` | 35 | **macOS native firewall ruleset.** Enforces the default-deny kill-switch, allows
    | local LAN / Tailscale traffic, and applies TCP MSS Clamping (`max-mss 1160`) to prevent WireGuard MTU
    | fragmentation. |
93
94 ---
95
96 ## 4. toggle.sh Flow
97
98 ### State Detection
99 `is_gateway_active()` returns true if EITHER containers are running OR host DNS is not `1.1.1.1`. This dual
    | check prevents the script from starting when it should stop (e.g., containers crashed but DNS is still
    | hijacked).
100
101 ### START Path (gateway OFF ON)
102 1. **Reset DNS to 1.1.1.1** Prevents deadlocks during startup
103 2. **Disconnect host Tailscale** `tailscale down` (prevents exit-node routing during container boot)
104 3. **Compile DNS rules** `docker compose run --rm rule-compiler` (Go binary inside multi-stage Docker build)
105 4. **Boot Colima VM** (if not running) via `colima_start_until_ready()`, which returns as soon as Docker is
    | reachable (~12s) instead of blocking on Colima's ~2-min post-provision hang (15.8.7). The networking mode
    | is **gated on the LAN-P2P signal** (`.lan_p2p_detected`, falling back to `TAILSCALE_ALLOW_LAN_P2P` in `.env`
    | ): a trusted home network requests `--network-address` or `--network-mode bridged` (or `shared` on WPA2-
    | Enterprise), while the common AP-isolated / untrusted case boots plain fast user-mode networking (`.colima`
    | start --memory 0.6 --vm-type vz).
106 - **Why Bridged Networking (when enabled)?** The `--network-mode bridged` and `--network-address` flags
    | force the VM to receive its own IP directly from your local router's DHCP server, placing it on the same
    | physical subnet as the host. This bypasses macOS network isolation and is what lets **external LAN devices
    | ** (phones, laptops on the same LAN), mDNS, and local service discovery reach across the container boundary
    | . It requires the `socket_vmnet` helper and is only worthwhile on a trusted LAN, which is why it is gated
    | rather than always requested. **It is *not* required for direct hostcontainer P2P** that path (the gateway
    | its own Mac) works even in user-mode networking via the routing-fix `host_ips` RETURN-rule carve-out;
    | see 15.3.5 and 15.8.7.
107 - **Enterprise Wi-Fi Fallback & Dynamic Roaming:** `toggle.sh` dynamically queries `system_profiler`
    | SPAirPortDataType` on boot. If it detects a **WPA2-Enterprise** connection (802.1X), it forces Colima to
    | fall back to `--network-mode shared`. This prevents aggressive network intrusion systems on corporate/
    | university Wi-Fi switches from terminating the host's Wi-Fi connection due to "MAC spoofing" (detecting
    | multiple MAC addresses originating from a single authenticated port). P2P connections are impossible on
    | these networks anyway due to AP Client Isolation.
108 - **Dynamic Roam Downgrading:** `toggle.sh` writes its selected network mode to `/tmp/nullexit_colima_mode`
    | .txt. When roaming between networks (e.g., Home to University), `recover.sh` actively checks this state
    | against the new network's security requirement (`.lan_p2p_detected`). If a mismatch is detected (e.g.,
    | bridged Colima on an Enterprise network), `recover.sh` automatically forces a full `toggle.sh --restart` in
    | the background to safely transition the Virtual Machine's network mode and protect the host from de-
    | authentication.
109 5. **Configure VM swap** 400MB swap file inside the VM to prevent OOM
110 6. **Clean corrupted AdGuard config** Remove empty `AdGuardHome.yaml`
111 7. **Start containers** `docker compose up -d`
112 8. **Wait for gateway Tailscale** Poll `tailscale status` for "offers exit node" (up to 60s, abort at 40
    | consecutive NoState)
113 9. **Resolve gateway IP** From `.gateway_ip` or `docker compose exec tailscale tailscale ip -4`
114 10. **Connect host to mesh** Verify `tailscaled` is running (auto-start if needed), then `tailscale up --reset`
    | --ssh=true --accept-dns=false --accept-routes=true --exit-node= ( `--accept-routes=true` must be explicit
    | --reset` reverts it to `false` otherwise, silently preventing the default route from switching to `utun`
    | *)
115 11. **Pre-flight checks** [1/3] `tailscale ping` gateway, [2/3] `dig +tcp` AdGuard DNS, [3/3] WARP container
    | internet
116 12. **If all pass** Set exit node + hijack DNS to gateway IP (single server, no fallback)
117 13. **If any fail** Enable SOCKS5 proxy + local DNS proxy as fallback
118 14. **Final DNS re-force** `force_dns_to_gateway` called again after exit-node transition (tailscaled clobbers
    | DNS briefly)
119 15. **Enable sleep prevention** `caffeinate -i` in background to prevent idle sleep
120
121 ### STOP Path (gateway ON OFF)
122 1. Disconnect host Tailscale (`tailscale down`)

```

```

123 2. `docker compose down -t 5`
124 3. Reset DNS to 1.1.1.1
125 4. Stop local DNS proxy
126 5. **Stop sleep prevention** Kill caffeinate process
127 6. Full network state cleanup (proxies off, DNS cache flush, route flush, Wi-Fi power cycle)
128
129 ### Safety Mechanisms
130 - `run_with_timeout()` Pure-bash watchdog for every system command (15s120s)
131 - `cleanup_handler()` Trap on ERR/INT/TERM/HUP restores DNS to 1.1.1.1, kills caffeinate, and tears down
    Tailscale
132 - `force_dns_to_gateway()` Sets DNS and VERIFIES via `networksetup -getdnsservers` (3 attempts with backoff)
133 - `SUCCESS_RUN` flag Cleanup only runs if script didn't complete successfully
134 - `start_sleep_prevention()` / `stop_sleep_prevention()` Manages `caffeinate -i` via PID file at `/tmp/
    nullexit-caffeinate.pid`
135
136 ---
137
138 ## 5. Routing & Firewall Architecture (Inside Container)
139
140 ### IP Rules (`ip rule show`)
141 | Priority | Match | Table | Purpose |
142 |-----|-----|-----|-----|
143 | 99 | `to 100.64.0.0/10` | 52 | CGNAT range Tailscale routes (injected by routing-fix) |
144 | 100 | `from 172.18.0.2` | 200 | Container's own traffic (SOCKS5 proxy) tun0 |
145 | 101 | all | 51820 | Gluetun's WireGuard fwmark tun0 |
146
147 ### Table 200 (SOCKS5 proxy traffic)
148 - `162.159.192.1 via $DOCKER_GW dev eth0` WARP endpoint exception (prevents tunnel loop)
149 - `default dev tun0` Everything else through WARP
150
151 ### iptables (TWO separate stacks!)
152 - **nftables backend** (`iptables`) Used by Gluetun's `post-rules.txt`. FORWARD policy DROP.
153 - **legacy backend** (`iptables-legacy`) Used by Tailscale's `ts-forward`. FORWARD policy ACCEPT.
154 - Both stacks apply to every packet. FORWARD RELATED,ESTABLISHED must exist in BOTH.
155
156 ### post-rules.txt (loaded by Gluetun)
157 ```
158 FORWARD: ACCEPT for tailscale0 in/out
159 INPUT: ACCEPT for tailscale0
160 NAT POSTROUTING: MASQUERADE on tun0
161 MANGLE: TCP MSS clamp to 1180
162 NAT PREROUTING: Redirect DNS (53) 5335 on tailscale0 and eth0
163 ip6tables: DROP FORWARD on tailscale0
164 ```
165
166 ---
167
168 ### 5.1 Tailscale Local Network Discovery (DERP Bypass)
169 By default, Gluetun's strict NAT swallows all of Tailscale's UDP hole-punching packets, forcing Tailscale to
    fall back to incredibly slow DERP relay servers (often adding 500ms+ latency).
170
171 To fix this, we use `FIREWALL_OUTBOUND_SUBNETS=192.168.0.0/16,172.16.0.0/12,10.0.0.0/8` in `docker-compose.yml`
    (on the `warp` container). This creates `ip rule` bypasses (Priority 99, Table 199) that route all packets
    destined for private IP ranges *outside* the WARP tunnel directly to the host's `eth0`.
172 - **172.16.0.0/12** is especially critical because many modern Wi-Fi routers (and Docker itself) assign IPs in
    this block (e.g., `172.17.x.x`).
173 - This bypass allows Tailscale's UDP packets to reach devices on the same Wi-Fi directly, establishing a fast
    peer-to-peer connection while the actual web traffic inside the tunnel remains fully routed through WARP.
174
175 ---
176
177 ### 5.2 Firewalling & Per-Device Access Control
178 Because every mesh device's traffic passes through the `warp` container's network namespace, this namespace
    serves as the ultimate choke point for network-wide firewall rules.
179
180 **Where to Put Rules**
181 The right place to add firewall rules is `scripts/routing-fix.sh`. It runs a 30-second loop inside the
    namespace and already handles re-injecting rules that Gluetun resets on reconnect. **Do not use `post-rules
    .txt` for dynamic rules**, as Gluetun flushes and resets it upon VPN reconnects.
182
183 **The Dual iptables Backend Constraint (CRITICAL)**
184 The container runs two simultaneous iptables backends: `iptables` (for the nftables backend used by Gluetun)
    and `iptables-legacy` (for Tailscale). Every rule **must** be added to both stacks, or it will silently
    fail for certain traffic.
185
186 **Avoiding Duplication**
187 Because `scripts/routing-fix.sh` runs in a loop, you must always check for a rule's existence with `-C` before
    appending with `-A`. Otherwise, your rules will duplicate every 30 seconds.
188
189 **Per-Device Identification & Filtering**

```

```

190 Each mesh device has a stable `100.x.x.x` Tailscale IP, which is visible as the `src` IP in the `FORWARD` chain
    . This is how you identify devices for filtering.
191 * Domain-level: AdGuard Home (port 3000, credentials `admin/nullexit`) provides a REST API that supports
    per-client blocking rules based on their Tailscale IP.
192 * IP-level: Use `iptables` in `scripts/routing-fix.sh` (e.g., `iptables -I FORWARD -s 100.x.x.x -d <
    blocked_ip> -j DROP`).
193 * Country-level: Add `ipset` to the `routing-fix` apk install step in `docker-compose.yml`, and load CIDR
    ranges from `ipdeny.com` into an ipset, then block that set in the `FORWARD` chain. See 5.3 (Geo-IP
    Blocking) for the full configuration flow.
194
195 ### 5.3 Geo-IP Blocking
196 To block traffic to and from specific countries, the system dynamically downloads country-specific IP ranges
    from [ipdeny.com](http://www.ipdeny.com/ipblocks/data/countries/) and drops them via iptables `FORWARD`
    rules.
197
198 To add a new country to the blocklist:
199 1. Find the 2-letter ISO country code (e.g., `cn` for China, `ru` for Russia, `il` for Israel, `kp` for North
    Korea).
200 2. Open your `.env` file.
201 3. Update the `BLOCKED_COUNTRIES` variable: e.g., `BLOCKED_COUNTRIES="kp il cn ru"`.
202 4. Run `toggle.sh` to restart the gateway and apply the new environment variables to the routing-fix container.
203
204 > [!NOTE]
205 * Limitations of Geo-IP Blocking: IP-based blocking is highly effective at blocking servers, direct apps,
    and raw infrastructure physically located and registered in a specific country (e.g., `www.gov.il`).
    However, it will not block high-profile websites (e.g., `jpost.com`) that route their traffic through
    global Content Delivery Networks (CDNs) like Google Cloud, Cloudflare, or Fastly. Because the CDN's IP
    addresses are registered in the US or globally, the network traffic physically never routes to the blocked
    country, allowing it to bypass the Geo-IP firewall.
206
207 ## 6. macOS Quirks to Remember
208
209 1. Colima SSH tunnel is TCP-only UDP ports (like 5354/udp) are NOT accessible from host. Use `dig +tcp` or
    the Python DNS proxy (UDPTCP converter).
210 2. docker compose exec -T injects `\r` Always `tr -d '\r'` when capturing output for comparisons or `
    networksetup` commands.
211 3. brew services` can get stuck Fix with `sudo brew services restart tailscale`. Common state: `brew
    services list` shows `started` but `tailscale status` shows "Tailscale is stopped."
212 4. No `timeout` command macOS lacks GNU `timeout`. Use the `run_with_timeout()` bash function.
213 5. sudo -v` prompts even with NOPASSWD Use `sudo -n` (non-interactive) everywhere.
214 6. DNS is scoped to en0 macOS scutil DNS resolver is scoped to en0 (Wi-Fi). DNS changes on other
    interfaces (like USB Ethernet) are ignored unless en0's service is also updated. The script always sets DNS
    on BOTH `$_ACTIVE_SERVICE` and `$_ENO_SERVICE`.
215 7. tailscaled clobbers DNS during exit-node transition `force_dns_to_gateway` is called a second time at
    the end of the ENABLE branch this.
216 8. tailscale up --reset resets all unspecified flags to defaults Every `--reset` call MUST also pass `--
    accept-dns=false` explicitly or tailscaled re-enables DNS management.
217 9. docker compose ps` CWD pitfall `docker compose` commands require a `docker-compose.yml` in the current
    working directory. Use `docker ps` for raw container listing from any CWD; `docker compose ps` requires
    the project directory.
218
219 ---
220
221 ## 7. Environment Variables
222
223 ### `.env` File
224 | Variable | Purpose |
225 |-----|-----|
226 | `WIREGUARD_PRIVATE_KEY` | WARP WireGuard private key (from `wgcf generate`) |
227 | `WIREGUARD_PUBLIC_KEY` | WARP WireGuard public key |
228 | `WIREGUARD_ADDRESSES` | WARP WireGuard address (e.g., `172.16.0.2/32`) |
229 | `TS_AUTHKEY` | Tailscale auth key for the container |
230 | `NULLEXIT_SEED` | 256-bit seed for HMAC-SHA256 script integrity signing (generated by `setup.sh`) |
231 | `GATEWAY_RULE_PROFILE` | Rule compilation tier: `light`, `medium`, `heavy` |
232 | `GATEWAY_BYPASS_PING` | (Optional) `true` to proceed even if pre-flight checks fail |
233 | `GATEWAY_USE_EXIT_NODE` | (Optional) `false` to skip exit node, DNS-only mode |
234 | `GATEWAY_HIJACK_HOST` | (Optional) `false` to skip DNS hijacking on the host (VPN/adblocking for Tailscale
    peers only) |
235 | `GATEWAY_MSS` | (Optional) TCP MSS clamp value (default 1120); 1180 for speed on healthy paths |
236 | `WARP_FAIL_THRESHOLD` | (Optional) Consecutive `warp=off` polls before auto-shutdown (default 6 = 30s; 3 = 15
    s) |
237 | `WARP_ENDPOINT_1` | (Optional) Override Cloudflare WARP WireGuard endpoint IP 1 (default `162.159.192.1`) |
238 | `WARP_ENDPOINT_2` | (Optional) Override Cloudflare WARP WireGuard endpoint IP 2 (default `162.159.193.1`) |
239 | `KILL_SWITCH` | (Optional) `true` to enforce strict PF lock drops all host traffic if VPN fails. Breaks SSH
    if VPN dies. |
240 | `HOST_LEAK_PROBE` | (Optional) `false` to disable the 300ms host-egress leak prober (default: `true`) |
241 | `STOP_COLIMA_ON_EXIT` | (Optional) `true` to fully shut down the Colima VM on toggle-off (saves battery on
    dedicated hosts) |
242 | `ADGUARD_USER` | (Optional) AdGuard Home web UI username (default: `admin`) |
243 | `ADGUARD_PASSWORD` | (Optional) Shared password for AdGuard Home and Tor ControlPort (default: `nullexit`) |

```

```

244 | `BLOCKED_COUNTRIES` | Space-separated 2-letter ISO codes to block via ipdeny.com CIDR ranges (e.g. `kp il cn
    ru`) |
245
246 ---
247
248 ## 8. Diagnostic Commands
249
250 ### Container Health
251 ```bash
252 docker ps --format '{{.Names}} {{.Status}}'
253 docker compose exec -T tailscale tailscale status
254 docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace
255 ```
256
257 ### SOCKS5 Proxy
258 ```bash
259 curl --socks5-hostname 127.0.0.1:1080 --max-time 5 -s https://www.cloudflare.com/cdn-cgi/trace
260 ```
261
262 ### AdGuard DNS
263 ```bash
264 dig +tcp @127.0.0.1 -p 5354 google.com +short +timeout=5
265 ```
266
267 ### Firewall & Routing
268 ```bash
269 # nftables FORWARD chain
270 docker compose exec -T warp iptables -L FORWARD -v --line-numbers
271 # Legacy FORWARD chain
272 docker compose exec -T warp iptables-legacy -L FORWARD -v --line-numbers
273 # IP rules + routing tables
274 docker compose exec -T warp ip rule show
275 docker compose exec -T warp ip route show table 199
276 docker compose exec -T warp ip route show table 200
277 ```
278
279 ### Host macOS
280 ```bash
281 tailscale status
282 brew services list | grep tailscale
283 networksetup -getdnsservers Wi-Fi
284 networksetup -getsocksfirewallproxy Wi-Fi
285 sysctl net.inet.ip.forwarding
286 ```
287
288 ---
289
290 # Part II Design Decisions & Threat Model
291
292 ## 9. Privacy Architecture & Threat Model
293
294 ### Layered Defense
295 The gateway stacks four layers targeting different attack surfaces:
296
297 - **Layer 1 ISP-Level Surveillance (WARP):** Your ISP sees only an encrypted WireGuard tunnel to a Cloudflare
    IP. In the US, ISPs can legally sell your browsing metadata and are subject to secret NSLs (National
    Security Letters). WARP removes their visibility entirely.
298 - **Layer 2 DNS Tracking (AdGuard Home):** DNS is the phone book of the internet. By default, queries go to
    your ISP's resolver, giving them a complete log of your traffic. AdGuard intercepts all DNS from mesh
    devices, blocks trackers, and resolves queries through the WARP tunnel.
299 - **Layer 3 Device Identity & IP Exposure (Tailscale):** On untrusted networks (coffee shops, public Wi-Fi),
    your device's IP is exposed. Tailscale creates an encrypted WireGuard mesh between your devices, routing
    all traffic to your gateway exit node.
300 - **Layer 4 Kernel-Level IP Blocking (ipset/iptables):** Sophisticated malware bypasses DNS entirely with
    hardcoded IPs. The `rule-compiler` fetches ~16,700 threat-intelligence IPs/CIDRs (Spamhaus, Feodo Tracker,
    Emerging Threats, CINS) and enforces them as `DROP` rules in the kernel `FORWARD` chain blocking both
    outbound C2 connections and inbound attack traffic.
301
302 ### The Documented Threat: Why This Matters
303 This architecture is calibrated against documented mass-surveillance programs revealed in the Snowden
    disclosures:
304 - **PRISM:** Bulk collection of communication content from major tech providers.
305 - **UPSTREAM:** Tapping physical backbone fiber, collecting metadata and content in bulk.
306 - **XKeyscore:** Retroactive search of unencrypted traffic.
307 - **MUSCULAR:** Infiltration of unencrypted internal datacenter interconnect links.
308
309 Passive bulk collection is not paranoia it is a documented reality. By double-encrypting and routing traffic,
    nullexit prevents your ISP from harvesting your metadata.
310
311 ### Trust Assumptions & Shifting
312 Every component shifts trust rather than eliminating it:

```



```

313 1. **Physical Device Integrity:** You trust your physical devices are not compromised.
314 2. **Cloudflare Integrity:** You trust Cloudflare not to correlate your WARP tunnel with exit traffic.
315 3. **Tailscale Integrity:** You trust Tailscale's coordination server not to MITM WireGuard keys.
316
317 The goal: no single provider has a complete picture. Your ISP sees an encrypted tunnel. WARP sees traffic with
    no account identity. Tailscale sees node topology but not content.
318
319 ### 9.1 Exit Node IP Rotation (Design Decision)
320
321 A common question for privacy architectures is whether the system should aggressively rotate its public exit IP
    (e.g., every 5 minutes, like a Tor circuit or a proxy rotator).
322
323 `nullexit` uses Cloudflare WARP via Gluetun. WARP provides **anonymity in a crowd** rather than aggressive
    rotation. Thousands of users share the same Cloudflare Edge IP simultaneously, making your traffic
    indistinguishable from the herd. The IP may occasionally rotate on its own (which `host-leak-probe.sh` logs
    as a `ROTATE` event), but it remains generally stable.
324
325 **Why aggressive IP rotation was intentionally excluded:**
326 1. **Broken TCP Connections:** Every rotation instantly drops all long-lived TCP sessions (SSH, large downloads
    , WebSockets, gaming, Zoom).
327 2. **CAPTCHA & 2FA Hell:** Modern web security (e.g., Cloudflare, Akamai) aggressively flags IP-hopping as bot
    behavior. Aggressive rotation guarantees the user will be bombarded with "Verify you are human" CAPTCHAs
    and "New Login Detected" 2FA emails constantly.
328 3. **WireGuard Architecture:** WireGuard is stateless and does not natively support IP hopping on the fly. To
    rotate an IP, the VPN client must actively drop the tunnel, request a new endpoint, and re-handshake. This
    introduces latency gaps where the kill-switch would repeatedly trigger and blackhole the user's internet.
329
330 **Verdict:** For a daily-driver secure gateway, shared-IP crowding (WARP) is superior to aggressive rotation.
    As an added benefit, Cloudflare WARP IPs are actually highly static (tied to the persistent WireGuard
    public key generated for your device identity). Because your IP stays static and is part of Cloudflare's
    own trusted network, it builds a high "trust score," virtually eliminating CAPTCHA loops while still hiding
    you within the massive crowd of other users in that data center. All mesh devices routing through your
    gateway will reliably share this exact same trusted IP.
331
332 *(Note: The only exception where aggressive IP rotation is genuinely required is for specialized offensive
    security tasklike high-volume web scraping or dark web research where avoiding IP bans or active tracking
    overrides the need for TCP stability.)*
333
334 ### 9.2 Cloudflare WARP Privacy & Logging
335
336 Because `nullexit` uses Cloudflare WARP (via Gluetun) as its upstream exit node, the system inherits Cloudflare
    's consumer privacy policy.
337
338 **What Cloudflare DOES NOT log (Strict No-Logs Policy):**
339 * **Browsing History:** They do not log the domains or URLs you visit.
340 * **Traffic Destinations:** They do not log the IP addresses of the servers you connect to.
341 * **Content:** They do not inspect or log the payload/data of your traffic.
342 * **Data Brokering:** They explicitly guarantee they will never sell or rent your data to advertisers.
343
344 **What Cloudflare DOES log (Minimal Telemetry):**
345 * **Data Transfer Volume:** Total bandwidth used (required to enforce 10GB/mo quotas if you are not using a
    paid WARP+ key).
346 * **Performance Metrics:** Anonymized connection speeds and latency to optimize their network routing.
347 * **Aggregate Volume:** Total traffic volumes to popular destinations in aggregate (e.g., "datacenter X sent 50
    TB to Netflix"), stripped of all user IDs.
348 * **Device ID (Public Key):** They store the persistent WireGuard public key generated by your container to
    authorize the connection.
349
350 **Privacy Verdict:**
351 Cloudflare WARP provides excellent **historical privacy**. Because they do not log your destinations, they
    cannot comply with historical subpoenas asking what websites you visited yesterday. However, it is **not
    absolute anonymity**. They still know your real ISP IP address while you are actively connected. For daily-
    driver privacy against ISPs, hackers, and corporate trackers, it is top-tier. For nation-state evasion, use
    Tor (see 11).
352
353 ### 9.3 OPSEC: Python Runtime Airgap (Isolated Mode)
354 `nullexit` explicitly relies on the host's native system Python 3 runtime for critical components like `scripts
    /dns-proxy.py`. Because this script runs with elevated privileges (`sudo -n`) to bind to the privileged UDP
    /53 socket, modifying or polluting this Python environment is strictly forbidden.
355
356 **The Zero-Dependency Rule & Isolated Mode (-I)**
357 You must **NEVER** install third-party `pip` packages (e.g., `requests`, `pyyaml`) into the system Python
    environment that `nullexit` uses.
358 - All Python scripts in this repository must be written using *only* the Python Standard Library (`socket`, `
    urllib`, `json`, `ssl`).
359 - Third-party packages introduce a massive supply-chain attack vector. A compromised `pip` package installed
    globally could allow local malware to hook into the Python runtime and exploit the `sudo` privilege of the
    DNS proxy to gain full root access.
360 - To enforce strict immunity against user-installed malicious packages, `toggle.sh` explicitly invokes the
    proxy using **Python's Isolated Mode** (`python3 -I`). This flag forces the interpreter to completely
    ignore `PYTHONPATH`, `PYTHONHOME`, and the user's `site-packages` directory, executing exclusively from the

```

```

        read-only, Apple-signed system framework.
361
362 ### 9.4 Recommended Upgrades
363
364 | Layer | Default | Upgraded |
365 |---|---|---|
366 | VPN Exit | Cloudflare WARP (US) | Mullvad (Swedish, proven no-logs, anonymous payment) |
367 | DNS Resolver | AdGuard via WARP | AdGuard via Mullvad DoH (Cloudflare-blind) |
368 | Coordination | Tailscale (US, OAuth) | Headscale (self-hosted, no third-party) |
369 | Auth | OAuth / persistent keys | Ephemeral auth keys (no identity persistence) |
370 | Content | WireGuard asymmetric | WireGuard + PSKs (coordination server blind) |
371
372 ##### Mullvad (Replacing WARP)
373 Swedish-based. Police raided their offices in 2023 and left empty-handed no logs exist to seize. Accounts are
    random numbers. Payment by cash or Monero. Replace `VPN_SERVICE_PROVIDER=custom` with `VPN_SERVICE_PROVIDER
    =mullvad` in `docker-compose.yml`.
374
375 ##### Mullvad DoH Upstreams
376 Point AdGuard's upstream resolvers to `https://adblock.dns.mullvad.net/dns-query`. Cloudflare WARP only sees
    encrypted HTTPS to Mullvad's IPs completely blind to your DNS queries.
377
378 ##### Headscale
379 Self-hosted Tailscale coordination server. Eliminates Tailscale the company, keeping all node topology under
    your control.
380
381 ##### Ephemeral Auth Keys
382 Generate in the Tailscale admin panel. Used once to authenticate; node disappears from the admin panel after
    disconnect. Breaks persistent identity linkage.
383
384 ##### Pre-Shared Keys (PSKs)
385 Add a WireGuard PSK between peer nodes. Adds a symmetric encryption layer that Tailscale's coordination server
    has no knowledge of, blinding it from reading transit content.
386
387 ### 9.5 Structural Limitations
388 - **Traffic Analysis / Metadata:** Passive fiber tapping (UPSTREAM) can observe timestamps, data volumes, and
    IP ranges without reading content. Defeating traffic analysis requires mixing networks (Tor) or continuous
    dummy packet padding both heavily degrade performance.
389 - **Targeted State-Level Adversaries:** Consumer-grade VPNs are not a complete shield against a nation-state
    actively targeting you. This stack is designed to defeat bulk passive surveillance and corporate dragnet
    collection.
390
391 ---
392
393 ## 10. Per-Device Access Control
394
395 Because every mesh device routes internet-bound traffic through the `warp` container's `FORWARD` chain, they
    all appear with their stable `100.x.x.x` Tailscale IP as the `src` address. This gives two powerful
    enforcement surfaces:
396
397 ### IP & Port Level (`scripts/routing-fix.sh`)
398 Write raw `iptables` rules targeting specific devices. The 30-second idempotent loop auto-survives Gluetun
    reconnects:
399
400 ```bash
401 # Block a specific device from a target subnet
402 iptables -I FORWARD -s 100.x.x.x -d 203.0.113.0/24 -j DROP
403
404 # Restrict a device to only HTTP/HTTPS
405 iptables -I FORWARD -s 100.x.x.x -p tcp ! --dport 3000 -j DROP
406 iptables -I FORWARD -s 100.x.x.x -p tcp ! --dport 443 -j DROP
407
408 # Block a device from internet egress entirely (mesh only)
409 iptables -I FORWARD -s 100.x.x.x -j DROP
410
411 # Time-based (e.g., cut off 10 PM 7 AM)
412 iptables -I FORWARD -s 100.x.x.x -m time --timestart 22:00 --timestop 07:00 -j DROP
413 ```
414
415 ### DNS Level (AdGuard REST API)
416 AdGuard Home supports per-client rules keyed by Tailscale IP:
417
418 ```bash
419 curl -X POST http://localhost:80/control/clients/add \
420   -H "Content-Type: application/json" \
421   -d '{
422     "name": "kids-ipad",
423     "ids": ["100.x.x.x"],
424     "blocked_services": ["youtube", "tiktok", "instagram"],
425     "safesearch_enabled": true
426   }'
427 ```

```



```

428 ---
429 ---
430
431 ## 11. Tor & OPSEC Architecture
432
433 ### 11.1 Tor-over-WARP Topology
434
435 When extreme anonymity is required, integrating the Tor network into `nullexit` provides a powerful Tor-over-
VPN topology:
436 * **The University/ISP** sees only an encrypted Cloudflare WireGuard tunnel, rendering them completely blind to
the fact that you are using Tor (effectively bypassing Enterprise firewall Tor blocks).
437 * **Cloudflare** sees an encrypted TCP stream heading to a Tor Guard Node. They cannot see your DNS lookups,
the destination website, or the content of your traffic.
438 * **The Tor Exit Node** sees your traffic, but not your real IP (only the Cloudflare IP).
439
440 ##### The "Transparent Proxy" Trap (What NOT to do)
441 It is tempting to build a system-wide "Transparent Tor Proxy" that forces all host traffic (e.g., standard
Chrome/Safari browsers, background iCloud syncs) through Tor automatically. **This is a massive OPSEC trap
and destroys anonymity.**
442 Modern operating systems and browsers will instantly leak your identity through background syncing (e.g., Apple
Mail, Google Drive) and browser fingerprinting (canvas fingerprints, WebRTC, screen resolution). The Tor
network protects your *IP*, but if your payload contains identifying cookies or unique fingerprints, the
Tor Exit Node (which could be run by malicious actors) will instantly tie your session to your real
identity.
443
444 ##### The nullexit Implementation: Isolated Procedural Proxy
445 To safely integrate Tor without triggering the transparent proxy trap, `nullexit` implements an **Isolated Tor
SOCKS5 Proxy** with **Procedurally Generated Ports**:
446 1. **Isolated Container:** A lightweight `tor` Docker container runs securely inside the `warp` network
namespace. It does not hijack system traffic.
447 2. **Opt-In Usage:** You must consciously configure a highly-hardened browser (like the official Tor Browser or
a dedicated Firefox profile) to use the proxy. This prevents accidental background sync leaks.
448 3. **Procedurally Generated Ports:** Rather than hardcoding the standard proxy ports (like `127.0.0.1:9050` or
`1080`), `toggle.sh` randomizes all internal proxy ports on every boot into the ephemeral range
(10000-65000) and silently writes them to `/tmp/nullexit-ports.env`. This completely defeats static
fingerprinting by malware.
449 4. **Kernel-Level Collision Avoidance:** To prevent the randomizer from picking a port already in use by a root
daemon or Apple service (which would crash the gateway), the script directly queries the macOS kernel
socket table (`netstat -an -p tcp`) to verify the port is absolutely free before assigning it.
450 5. **The Honey-Port Tripwire:** While procedural ports defeat static fingerprinting, advanced malware might
attempt a full sequential port scan of `localhost` to find the hidden proxy. To counter this, `toggle.sh`
spawns a "Honey-Port" (a fake random port with a netcat listener attached). If any local application
attempts to connect to the Honey-Port, it instantly logs a critical security alert to `output.log` and
fires a macOS desktop notification, serving as an early-warning Intrusion Detection System (IDS) for local
malware.
451
452 > **Threat Model note:** The Tor SOCKS proxy and Honey-Port Tripwire only bind to `127.0.0.1` (loopback). Port
randomization and the Honey-Port only defeat blind, generic port-scanners they do NOT protect against a
targeted local process that reads `/tmp/nullexit-ports.env`, greps process memory, or runs `lsof -i`. The
proper mitigation is a host-level loopback-filtering firewall (LuLu/Little Snitch). See 15.9 (Threat Model:
Local Proxy Discovery) for the full analysis and the rogue-binary verification test.
453
454 ### 11.2 Hardening: Tor Bridges and obfs4 Pluggable Transports
455 For users operating under severe Deep Packet Inspection (DPI), `nullexit` supports configuring Tor to connect
exclusively through private bridges using the `obfs4` pluggable transport.
456
457 When `TOR_USE_BRIDGES=true` is set:
458 - **What it does:** It hides the entry IP from being matched against the public Tor consensus list, and
disguises the traffic's protocol fingerprint from the WARP exit provider (Cloudflare).
459 - **What it does NOT do:** It does *not* add any extra layers of anonymity beyond what Tor already provides. It
is purely a censorship-circumvention and obfuscation mechanism.
460 - **Limitations:** `obfs4` is highly effective against passive DPI, but it is not guaranteed to defeat active-
probing-resistant detection by a highly sophisticated state-level adversary.
461
462 > **Architecture Rule:** The `nullexit` gateway deliberately does NOT bundle, hardcode, or automatically fetch
bridge lines. Bridge lines are highly sensitive and expire. Users must obtain their own bridge lines from
`https://bridges.torproject.org` and manually populate the `tor-bridges.txt` file. If bridges are enabled
but the file is missing or empty, the Tor container will intentionally crash and fail to start rather than
silently falling back to public guard relays.
463
464 **The Bootstrapping Paradox (Out-of-Band Key Exchange)**
465 Automating the bridge acquisition process (e.g., scripting the gateway to log into the Telegram `@GetBridgesBot`
via MTPROTO API) introduces catastrophic OPSEC flaws:
466 1. **Identity Leaks:** Baking a Telegram session into the container explicitly links the "anonymous" gateway to
a real-world physical phone number.
467 2. **The Chicken & Egg Deadlock:** In heavily censored regimes, Telegram itself is usually blocked. If the
container requires Telegram to fetch bridges to bypass the firewall, it will fail because it cannot bypass
the firewall to reach Telegram.
468 3. **Bridge Exhaustion & CAPTCHAs:** Automated scraping behaves identically to state-sponsored scrapers,
triggering Tor's anti-bot CAPTCHAs which leads to silent proxy failures and exhausts the limited public
bridge pool.

```

```

469 Instead, users should rely on a less secure layer to manually reach Telegram, obtain the bridges, and populate
470 `tor-bridges.txt`. This can be done via the standard `nullexit` WARP tunnel, a mobile phone on cellular
data, Mullvad VPN, or a secure remote VPS. *(Note: We plan on fully integrating Mullvad and VPS
architectures directly into `nullexit` in the future, but have not had the time to build it yet).* This
intentionally "air-gaps" the cryptographic key exchange from the proxy infrastructure, leaving zero API
tokens and zero network traces for an adversary to exploit.
471
472 ### 11.3 Tor Container Kill-Switch Inheritance & Fail-Closed Egress Verification
473
474 ##### Context
475 Unlike the host's SOCKS5 fallback path (whose kill-switch interaction is documented in 15.7), the `tor`
container's connection status and fail-closed behavior under VPN tunnel drops is not governed by macOS `pf`
rules. Instead, it relies on network namespace inheritance inside Docker Compose.
476
477 ##### Mechanics
478 1. **Shared Network Namespace:** The `tor` service is configured with `network_mode: service:warp` in `docker-
compose.yml`. This shares the network namespace of the `warp` (GlueTun) container.
479 2. **Inherited Egress Rules:** All socket communication within the namespace is bound to the same networking
stack. GlueTun manages this stack by redirecting traffic through `tun0` (the WireGuard/WARP tunnel) and
applying `iptables` rules that explicitly drop outbound traffic originating on `eth0` (the physical/bridge
interface), except for permitted local subnets or the VPN server's endpoint.
480 3. **Tunnel Fail-Closed:** If the WARP connection drops or the `tun0` interface is brought down, the routing
table entries for `tun0` become invalid or disappear. Because the `iptables` rules in the shared namespace
continue to drop any outgoing internet traffic attempting to exit via `eth0`, the `tor` container cannot
leak raw traffic to the host's underlying networks. Egress fails closed.
481
482 ##### Verification / Manual Test Case
483 To manually verify that the Tor container fails closed and does not leak traffic when the tunnel dies:
484
485 1. **Verify active path:** Ensure the gateway is active (`toggle.sh start`) and fetch the Tor port (`
$TOR SOCKS_PORT` in `/tmp/nullexit-ports.env`). Verify Tor traffic routes correctly:
486 ```bash
487 curl --socks5-hostname 127.0.0.1:$TOR SOCKS_PORT https://check.torproject.org/api/ip
488 ```
489 *(This should output a Tor exit node IP).*
490
491 2. **Simulate tunnel drop:** Bring down the virtual interface inside the shared network namespace:
492 ```bash
493 docker compose exec warp ip link set dev tun0 down
494 ```
495
496 3. **Re-run the egress check:**
497 ```bash
498 curl --socks5-hostname 127.0.0.1:$TOR SOCKS_PORT https://check.torproject.org/api/ip
499 ```
500 * **Expected Result:** The request must hang and eventually time out, or immediately fail with a proxy error
(e.g., `curl: (97) SOCKS5: connection failed` or `curl: (7) Failed to connect`).
501 * **Leaked Egress Check:** Check the host's Wi-Fi router or firewall logs. No outbound packets from the Tor
container should route directly over the bridge network to the internet.
502
503 ### 11.4 Transparent .onion Routing via RFC 2544 Subnet (198.18.0.0/15)
504
505 ##### Context
506 To enable seamless dark web browsing across the entire mesh, the gateway requires a mechanism to dynamically
map `.onion` domains to IP addresses that the host operating system and network layers can route.
507
508 ##### IP Address Conflict & Solution
509 1. **Initial Conflict:** Initially, the Tor virtual address network was configured to map `.onion` sites within
the `10.192.0.0/10` range. However, the Colima Docker daemon dynamically allocated `10.200.1.0/24` to the
containers. Because this Docker subnet mathematically falls within the `10.192.0.0/10` block, a routing
loop occurred: all host packets destined for the Docker containers on ports like `9050` or `9051` were
intercepted by the transparent proxy NAT rules and dropped, causing proxy failures.
510 2. **Tailscale Private IP Exclusion:** Shifting the subnet to another private range like `10.123.0.0/16`
resolved the port conflict, but led to connection timeouts. On macOS, Tailscale's Exit Node routing
actively excludes RFC 1918 private subnets (e.g. `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`) to
maintain accessibility to local LAN resources (like printers/routers). Thus, packets targeting `10.123.x.x`
were dropped locally by Tailscale before reaching the gateway tunnel.
511 3. **The Benchmarking Subnet Solution:** To bypass these private IP exclusions without manual static routing
tables, Tor's virtual network was changed to `198.18.0.0/15` (reserved by RFC 2544 for benchmarking).
Since this is not a private RFC 1918 range, Tailscale exit-node routing automatically encapsulates and
routes all traffic destined for `198.18.0.0/15` through the tunnel to the gateway.
512 4. **Transparent NAT Redirection:** The `routing-fix` sidecar applies NAT rules in the shared network namespace
:
513 ```bash
514 iptables -t nat -I PREROUTING -d 198.18.0.0/15 -p tcp -j REDIRECT --to-ports 9040
515 ```
516 This intercepts all incoming TCP traffic targeting the mapped `.onion` range and transparently proxies it
through Tor's transparent port (`9040`).
517
518 ##### Exit Node Exclusions

```

```

519 The gateway supports the ability to exclude specific countries from being used as exit nodes (e.g. to avoid
    certain jurisdictions or nodes with poor network latency). This is configured globally via the `
    TOR_EXCLUDE_EXIT_NODES` environment variable in `.env`, which gets dynamically appended to Tor's `
    ExcludeExitNodes` and strictly enforced with `StrictNodes 1`.
520
521 ---
522
523 ## 12. Censorship-Resistant Transport & Egress Compartmentalization
524
525 ### 12.1 Why this is needed
526 WARP can be blocked from deployment countries with strict internet controls. The most likely cause isn't that "
    encrypted traffic is blocked" in general it's that WireGuard has a recognizable handshake signature, and `
    WIREGUARD_ENDPOINT_IP=162.159.192.1` on `WIREGUARD_ENDPOINT_PORT=2408` is a fixed, easily-enumerable target
    (Cloudflare's known WARP range). That combination is exactly what IP/ASN-based and DPI-based blocking is
    good at catching.
527
528 ### 12.2 Important correction: Gluetun's `SHADOWSOCKS=on` is the wrong direction
529 Gluetun's built-in Shadowsocks option runs a Shadowsocks `**server**` inside the already-connected VPN tunnel, so
    other LAN devices can reach out through Gluetun's `*existing*` connection via the SS protocol. It is not a
    way to make Gluetun itself connect `**outbound**` through a Shadowsocks server Gluetun only speaks OpenVPN
    or WireGuard as its actual transport. There is no `VPN_TYPE=shadowsocks`. Confirmed against Gluetun's own
    maintainer/community discussion: people have asked for a "Shadowsocks-client" mode specifically to chain
    through Gluetun, and the standing answer is "run a separate Shadowsocks client container."
530
531 ### 12.3 Diagnose before building anything
532 The right fix depends entirely on what's actually being blocked:
533 - Point Gluetun at a `**different provider/IP/ASN**` (any other Gluetun-supported WireGuard/OpenVPN provider, or
    a self-hosted WireGuard server) and test. If that gets through, the block is Cloudflare/WARP-specific, not
    a general WireGuard block `**no new component needed**`, just swap `VPN_SERVICE_PROVIDER` / the endpoint.
534 - If WireGuard fails regardless of endpoint, the block is protocol-level (DPI fingerprinting the handshake)
    proceed to obfuscation.
535
536 ### 12.4 Path A: Swap WARP for a different Gluetun-native transport
537 Simplest fix, only applies if 12.3 shows the block is Cloudflare-specific. Change `VPN_SERVICE_PROVIDER` / `
    VPN_TYPE` / endpoint in `docker-compose.yml` and in the now-centralized `WARP_ENDPOINT_*` values in `.env`.
    Nothing else in the stack needs to change.
538
539 ### 12.5 Path B: Add an obfuscation hop
540 Needed if WireGuard itself is being fingerprinted/blocked. Two ways to slot it in:
541
542 **B1 Wrap (recommended):** Run a Shadowsocks (or obfs4/v2ray) client as a new sidecar container. Point Gluetun
    's `WIREGUARD_ENDPOINT_IP` at `127.0.0.1:<local-port>` instead of Cloudflare directly; the sidecar relays
    that traffic to a Shadowsocks server you run yourself abroad. Keeps Gluetun's kill-switch, healthchecks,
    and the entire rest of the stack (Tailscale, AdGuard, ipset, `routing-fix.sh`) untouched, since none of
    them know the transport changed underneath `warp`.
543
544 **B2 Replace:** Swap the `warp` service entirely for a Shadowsocks-client + `tun2socks` container that owns
    the network namespace instead of Gluetun. More invasive loses Gluetun's built-in kill-switch/healthcheck,
    which would need to be rebuilt by hand (see Section 12.9 for the pluggable architecture to support this
    natively).
545
546 **Recommendation: B1.** Smaller surface area, preserves the self-healing architecture already built and
    documented.
547
548 ### 12.6 Fingerprinting caveat
549 Plain Shadowsocks can still be caught by active-probing DPI in more aggressive censorship environments. If
    plain SS gets blocked too, the next step up is an obfuscation plugin (`v2ray-plugin`, `Cloak`) or a newer
    protocol designed against active probing (VLESS+Reality, Hysteria2). Test plain SS first don't over-build
    before confirming it's needed.
550
551 ### 12.7 AmneziaWG (The High-Speed Alternative)
552 If you want to maintain the bare-metal speeds of WireGuard while bypassing DPI (e.g. in Egypt or Russia), the
    best alternative to Shadowsocks/V2Ray is AmneziaWG. AmneziaWG is a fork of WireGuard that pads the
    handshake packets with randomized "junk" data, entirely destroying the 148-byte DPI signature that
    firewalls look for.
553 - **Pros:** Because it runs entirely as UDP without TCP-over-TCP overhead, it completely avoids "TCP meltdown"
    latency spikes associated with Shadowsocks/V2Ray wrappers.
554 - **Cons:** You cannot use Cloudflare WARP. Furthermore, it requires completely ripping Gluetun out of the `
    nullexit` stack and building a custom Docker container, meaning you lose Gluetun's built-in kill-switches
    and health-check logic.
555
556 ### 12.8 Infrastructure Costs & Privacy
557 To use either Shadowsocks (Path B) or AmneziaWG, you cannot use the free Cloudflare WARP infrastructure,
    because Cloudflare servers only speak standard WireGuard. The software for both custom protocols is 100%
    free and open-source, but you must host the remote server infrastructure yourself.
558 - **Free Tier (Oracle Cloud):** You can host your remote server on Oracle Cloud's "Always Free" tier for $0.
    However, this routes your highly sensitive, censorship-evading traffic directly through Oracle's massive
    corporate data broker. If privacy is the core objective, handing all your metadata to Oracle defeats the
    purpose.
559 - **Paid Tier (Hetzner / Mullvad):** The recommended path is to rent a standard ~$4/month Virtual Private
    Server (VPS) in a free country (e.g., Hetzner in Germany, subject to strict EU GDPR privacy laws) and self-

```



```

610 # 4. Package into DMG
611 hdiutil create -volname "Nullexit Installer" -srcfolder "./Nullexit.app" -ov -format UDZO Nullexit.dmg
612
613 # 5. Bypass Gatekeeper (unsigned app)
614 xattr -cr /Applications/Nullexit.app
615 ```
616
617 Without a paid Apple Developer certificate ($99/yr), users see an "App is damaged" Gatekeeper warning and need
    to run step 5.
618
619 ### 14.2 Moonlight/Sunshine + Tailscale Race Condition
620
621 When streaming from a remote Windows host running Sunshine over a Tailscale mesh (e.g., while the client is on
    a cellular hotspot), a race condition can occur on boot:
622
623 1. Both Tailscale and Sunshine are set to start automatically on boot.
624 2. Tailscale takes a few seconds to fully initialize its `100.x.x.x` virtual adapter and establish a connection
625 3. **Sunshine starts too fast.** It launches before Tailscale is fully ready. Since Sunshine only binds to
    network interfaces that are active at the exact moment of startup, it misses the Tailscale adapter entirely
626 4. Moonlight clients (even when configured with the correct Tailscale IP) will fail to connect because Sunshine
    isn't listening on that interface.
627
628 **The "Magic Toggle" Symptom:**
629 If the user manually enables or disables Wi-Fi on the host PC, it triggers a global Windows network change
    event. Sunshine listens for these events, re-enumerates all network adapters, and says, "Oh, there's a
    Tailscale adapter here!" and finally binds to it. Suddenly, Moonlight can connect.
630
631 **The Permanent Fix:**
632 On the Windows host, open `services.msc`, locate the **Sunshine Service**, and change its Startup type from **
    Automatic** to **Automatic (Delayed Start)**. This forces Windows to wait a minute or two after booting
    before launching Sunshine, ensuring Tailscale's virtual adapter is fully initialized first.
633
634 ### 14.3 Chrome Remote Desktop Performance (UDP NAT)
635
636 ##### Observation (July 11, 2026)
637 When the user connects to their host Mac using Chrome Remote Desktop (CRD) while `nullexit` is active, the
    connection suffers from massive latency, lag, and degraded video quality. AdGuard Home's `blocked.log`
    shows zero blocked DNS requests for Google, WebRTC, or `chromoting` endpoints, indicating this is not a DNS
    sinkhole issue. *(See also 15.11 for the separate CRD-on-remote-device connection-failure quirk.)*
638
639 ##### Root Cause
640 This is a fundamental limitation of tunneling all host traffic through a strict commercial NAT (Cloudflare WARP
    ).
641 1. Chrome Remote Desktop attempts to use **STUN (UDP hole-punching)** to establish a lightning-fast, direct
    peer-to-peer WebRTC connection between the client device and the host Mac.
642 2. Because all non-Tailscale traffic is forcefully routed through the `warp` container, the STUN packets exit
    the network from a Cloudflare IP.
643 3. Cloudflare's enterprise NAT infrastructure strictly drops unsolicited inbound UDP packets. The UDP hole-
    punch fails.
644 4. Because a direct P2P connection cannot be established, CRD falls back to **Relay Mode** (TURN), bouncing all
    video data back and forth through Google's cloud servers instead of sending it directly over the local or
    mesh network. This relaying introduces massive latency.
645
646 ##### Workaround / Fix
647 This lag is the accepted "tax" for maintaining absolute cryptographic anonymity; the only way to restore CRD
    speed would be to leak its UDP traffic outside the tunnel (violating zero-trust).
648 To achieve low-latency remote access, users should bypass CRD entirely and use macOS's built-in **Screen
    Sharing (VNC)** directly over the Tailscale IP. Tailscale's sophisticated custom DERP/WireGuard
    architecture successfully UDP hole-punches through strict NATs, providing a direct, encrypted, high-speed
    connection without relying on external cloud relays.
649
650 ### 14.4 Dynamic IP Fetch vs. MagicDNS
651
652 ##### Observation
653 The `nullexit` gateway uses a dynamically fetched IP address (cached in `.gateway_ip`) to configure host
    routing and the `pf` kill-switch, rather than utilizing Tailscale's user-friendly MagicDNS hostname (e.g.,
    `nullexit-gateway`).
654
655 ##### Why MagicDNS is Not Used
656 1. **The "Chicken and Egg" Bootstrapping Problem:**
657 macOS's Packet Filter (`pf`) and low-level routing commands (`route add`) operate strictly at Layer 3 and
    require raw IP addresses. If `toggle.sh` or `recover.sh` attempted to use `nullexit-gateway`, the host Mac
    would need to perform a DNS lookup to resolve it. However, the very first step of the gateway's
    initialization is to completely hijack the host's DNS and point it *at the gateway itself*. The Mac cannot
    resolve the gateway's MagicDNS name if it needs the gateway's IP to know where to send the DNS query!
658 2. **Dynamic Self-Healing (Avoiding Stale Cache):**
659 To solve the bootstrapping problem without causing bugs if the node is deleted and re-authenticated on the
    Tailscale Admin Console, `toggle.sh` explicitly runs `docker compose exec ... tailscale ip -4` during boot
    to fetch the absolute freshest IP dynamically. It saves this to `.gateway_ip` so that fast-roaming scripts

```

```

like `recover.sh` can instantly retrieve the routing target without incurring the latency of querying
660 Docker.
661 ### 14.5 Battery & Power Measurement Quirks
662
663 When trying to optimize this framework (or any daemon) for battery life, developers often look for a way to
programmatically measure the exact Wattage consumed by a specific process (e.g., "How many Watts is `toggle
664 .sh` using?").
665
666 **This is a hardware impossibility on macOS and Linux.**
667
668 ##### Why Activity Monitor is Lying to You
Your machine's logic board only has physical power sensors for the *entire* CPU package, the GPU, and the RAM.
It knows the CPU is pulling 5W total, but it has no physical hardware capability to know whether Docker is
using 3W and Chrome is using 2W.
669
670 The "Energy Impact" score you see in macOS Activity Monitor is a **synthetic, heuristic score** calculated by `
powerd`. It penalizes apps for waking up the CPU (idle wakes), doing disk I/O, or keeping the screen awake.
It is a relative score, not actual Wattage.
671
672 ##### The Proper A/B Testing Methodology
673 If you want to truly know how many Watt-hours or mAh your background daemons (`routing-fix.sh`, `host-leak-
probe.sh`, etc.) are costing you, you must perform a hardware-level A/B drain test:
674
675 1. Unplug the laptop, run the gateway, and note your exact battery mAh using:
676     ```bash
677     ioreg -l | grep "AppleRawCurrentCapacity"
678     ```
679 2. Leave the laptop idle for exactly 1 hour.
680 3. Run the command again to see exactly how many mAh were drained.
681 4. Turn the gateway off and repeat the test for another hour.
682
683 The delta in mAh drained between the two tests is the true, exact cost of running the software. When optimizing
polling loops (e.g., changing `sleep 5` to `sleep 30`), this is the only reliable way to measure the
actual battery life returned to the user.
684
685 ### 14.6 Host Lockdown Mode (Why It's Impossible on macOS)
686
687 A common privacy feature request is a "Host Lockdown Mode" (Mode 3): A mode where the Docker exit node remains
perfectly functional for remote devices, but the host machine itself is completely blocked from accessing
the internet (to prevent even encrypted host traffic from leaking metadata).
688
689 **Why this is impossible on macOS:**
690 On Linux (e.g., a Raspberry Pi), Docker runs natively. The host's traffic traverses the `OUTPUT` iptables chain
, while Docker traffic traverses the `FORWARD` chain. We could easily drop all `OUTPUT` traffic to kill the
host's internet, and Docker would continue routing traffic perfectly.
691
692 However, Docker on macOS runs inside a Linux Virtual Machine (via `qemu` or Apple `Virtualization.framework`).
This VM runs as a standard macOS user-space application. If you configure the macOS firewall (`pf`) to drop
all host outbound traffic, it will indiscriminately drop the VM application's traffic as well, instantly
killing the Cloudflare WARP tunnel and the exit node.
693
694 Writing complex macOS `pf` rules to "allow the VM app, allow Cloudflare WARP IPs, allow Tailscale DERP IPs, but
block everything else" is highly fragile and heavily discouraged. Since the current `toggle.sh` default
mode already fully encrypts the host's traffic via the WARP tunnel, the host is already protected from ISP
leaks.
695
696 ---
697
698 # Part III Incident Log & Resolved Issues
699
700 This is the single, deduplicated log of every incident, bug, and resolved issue. Entries are organized into
subsystem groups (15.115.12). Each entry follows a **Symptom / Root Cause / Fix** shape where applicable;
packet-level analyses are preserved as clearly-labeled **Deep dive** blocks. The terse dated changelog
lives in 17; this section holds the full post-mortems.
701
702 ## 15. Incident Log
703
704 ### 15.1 Exit-Node Return Path & SOCKS5 Failover
705
706 ##### 15.1.1 Exit Node Return Path (`rx 0`) & The SOCKS5 Failover RESOLVED
707
708 **Symptom:** Gateway showed `tx 936 rx 0` transmitted but never received return traffic. For days, Tailscale's
exit node refused to return traffic back to the client, leading to the assumption that Tailscale's
userspace forwarding was bypassing `iptables` and ignoring the WARP `tun0` interface. In a desperate
attempt to fix this, a **SOCKS5 proxy** (`scripts/socks5-proxy.py`) was introduced as a replacement.
709
710 **Root Cause:** Tailscale's userspace forwarding was *not* the problem. `docker-compose.yml` included `
FIREWALL_OUTBOUND_SUBNETS=100.64.0.0/10`. Gluetun created a strict routing rule (`ip rule add to
100.64.0.0/10 lookup 199`, priority 99) that forced all Tailscale CGNAT traffic to bypass the VPN via the
unencrypted Docker bridge (`eth0`). Return packets were sent to the macOS host instead of back through `
tailscale0`, blackholing all return packets back to the client.

```



```

710
711 **Fix:** Removed `FIREWALL_OUTBOUND_SUBNETS` entirely, immediately restoring the Tailscale exit node. `scripts/
    routing-fix.sh` now injects `lookup 52`, and return packets correctly flow back into `tailscale0`. Instead
    of removing the SOCKS5 proxy, we repurposed it into a **bulletproof failover** for when restrictive Wi-Fi
    networks block Tailscale UDP ports.
712
713 Architecture after the fix:
714 ```
715 PRIMARY (Exit Node):
716 DNS/TCP/UDP: Apps Tailscale Exit Node gateway container tun0 WARP internet
717
718 FAILOVER (SOCKS5 - if Tailscale is blocked by local Wi-Fi):
719 DNS: Browser 127.0.0.1:53 Python proxy TCP:5354 AdGuard WARP
720 TCP: Apps SOCKS5:1080 gateway container tun0 WARP internet
721 Mesh: SSH/ping 100.x.x.x utun5 WireGuard peers (independent of proxy)
722 ```
723
724 **Deep dive: Exit Node Return Path Analysis (June 26, 2026).**
725 This preserves the detailed packet-level analysis that led to identifying and fixing the `rx 0` bug.
726
727 *The exit node was probably never going to work in this container topology.* Here's the packet path traced by
    reading `ip rule show` and the routing tables live:
728
729 **Outbound (Phone-1 internet):**
730 1. Phone-1 sends packet to gateway via Tailscale mesh
731 2. Gateway's tailscale0 (kernel TUN, `TS_USERSPACE=false`) decrypts packet enters FORWARD chain
732 3. Packet has src=`100.87.42.87` (Phone-1), dst=some internet IP
733 4. FORWARD chain (nftables): Rule 1 matches (`-i tailscale0 -j ACCEPT`)
734 5. Routing decision for the forwarded packet. Which ip rule matches?
735 - Rule 98: `to 172.18.0.0/16` no (dst is internet)
736 - Rule 99: `to 100.64.0.0/10` no (dst is internet)
737 - Rule 100: `from 172.18.0.2` **NO** (src is `100.87.42.87`, not the container IP!)
738 - Rule 101: `not fwmark 0xca6c lookup 51820` **YES** (no fwmark on forwarded packets)
739 6. Table 51820: `default dev tun0` packet goes out through WARP
740 7. NAT POSTROUTING: `MASQUERADE on tun0` src becomes WARP IP
741 8. Packet exits through WARP to internet (in theory)
742
743 **Return (internet Phone-1):**
744 1. Return packet arrives on tun0, conntrack de-MASQUERADES dst back to `100.87.42.87`
745 2. Routing decision for dst=`100.87.42.87`:
746 - Rule 99: `to 100.64.0.0/10 lookup 199` **YES**
747 3. Table 199: `100.64.0.0/10 via 172.18.0.1 dev eth0` **sends it to the Docker gateway!**
748 4. **This is wrong.** The packet should go to tailscale0 (to be re-encrypted and sent to Phone-1), but table
    199 sends it out eth0 to the Docker bridge host.
749
750 **Table 199 is the smoking gun.** It has one route: `100.64.0.0/10 via 172.18.0.1 dev eth0`. This sends ALL
    Tailscale CGNAT return traffic out the Docker bridge to the host not back through tailscale0. The return
    packet leaves the container's network namespace entirely, hits the host's routing stack, and gets dropped
    because the host has no idea what to do with a raw packet for `100.87.42.87`.
751
752 *Why table 199 exists and why it's wrong for exit node traffic.* Table 199 + the CGNAT rule (pref 99) was
    designed so that the **container itself** can reach other Tailscale peers (e.g., for mesh management, DERP
    relay). For that use case, routing CGNAT traffic via the Docker bridge to the host (which has its own
    tailscaled) makes sense. But for **forwarded exit node traffic**, the return packet needs to go back
    through tailscale0 inside the container not out to the host. The CGNAT rule intercepts the return packet
    before it can be routed back to tailscale0 through the main table or Tailscale's own routing (table 52,
    which is at pref 5270).
753
754 *The SOCKS5 path works because it dodges all of this.* SOCKS5 proxy creates connections **from the container's
    own IP** (`172.18.0.2`). So:
755 - ip rule 100 (`from 172.18.0.2 lookup 200`) matches
756 - Table 200 has `default dev tun0`
757 - Return traffic comes back to `172.18.0.2` on tun0, conntrack de-NATs, proxy sends response to client
758
759 No FORWARD chain involved. No CGNAT rule involved. No table 199. It just works.
760
761 *Summary of findings:*
762
763 | Finding | Status | Evidence |
764 |-----|-----|-----|
765 | Table 199 CGNAT route hijacks exit node return traffic | **Fixed** | Changed `lookup 199` to `lookup 52` in `
    scripts/routing-fix.sh`. Return packets now route via tailscale0. |
766 | FORWARD RELATED,ESTABLISHED missing | **Fixed** | Installed `iptables` via apk in `docker-compose.yml`. Rule
    now injects successfully. |
767 | DOCKER_GW variable parsing error | **Fixed** | Added `head -1` in `scripts/routing-fix.sh`. |
768 | `FIREWALL_OUTBOUND_SUBNETS` was the root cause | **Fixed** | Removed from `docker-compose.yml`. Gluetun no
    longer hijacks CGNAT routing. |
769 | Host tailscaled error state from aggressive `tailscale down` | **Mitigated** | Toggle script now detects and
    auto-restarts. |
770
771 #### 15.1.2 Container Default Route Bypasses WARP

```



```

772 **Problem:** Inside the WARP container's network namespace, the main routing table's default route was via `
    eth0` (Docker bridge), not `tun0` (WARP tunnel). While gluetun uses policy routing (rule 101 table 51820
    `tun0`) for most traffic, traffic originating from the container's own IP (`172.18.0.2`) hits rule 100 (`
    from 172.18.0.2 lookup 200`), whose default was also via `eth0`. This caused the SOCKS5 proxy's outbound
    connections to bypass WARP.
773
774 **Fix:** Updated the `routing-fix` container to:
775 1. Change the main table's default route to `default dev tun0`
776 2. Change table 200's default route to `default dev tun0`
777 3. Add a specific route for the WARP WireGuard endpoint (`162.159.192.1`) through `eth0` via the Docker gateway
    in table 200, preventing a tunnel loop
778 4. Re-assert all routes every 30 seconds (gluetun may reset them on health checks)
779 5. Dynamically detect the Docker subnet from `ip route show dev eth0` instead of hardcoding `172.18.0.0/16`
780
781 #### 15.1.3 Hardcoded Docker Subnet in Routing Fix
782 **Problem:** The `routing-fix` container hardcoded `172.18.0.0/16` and `172.18.0.1` for the Docker bridge
    subnet and gateway. If Docker's bridge network changed (after `docker network prune`, Colima restart, or on
    a different machine), these routes would be wrong.
783
784 **Fix:** Detection is now dynamic using `ip route show`:
785 - `DOCKER_NET=$(ip route show dev eth0 | grep -v default | head -1 | awk '{print $1}')
786 - `DOCKER_GW=$(ip route show default | awk '{print $3}')
787
788 This extracts the actual subnet and gateway directly from the routing table, working with any subnet size
    (`/16`, `/24`, `/20`, etc.).
789
790 ### 15.2 Routing Loops & Subnet Collisions
791
792 #### 15.2.1 The Infinite Recursive Routing Loop (Hotspot Paradox)
793 **Problem:** A critical network topology crash occurs if the host Mac connects to a Mobile Hotspot (e.g., a
    Windows PC or Phone) that is *also* connected to the Tailscale mesh and has "Use Exit Node" enabled.
    Because the Mac is hosting the Exit Node, it routes its internet traffic to the Hotspot. The Hotspot
    intercepts the Mac's traffic on its way to the cellular tower and routes it *back* to the Exit Node (the
    Mac) via Tailscale. The Mac receives it, tries to send it out again, and the Hotspot intercepts it again.
    This creates an infinite, recursive encryption loop that instantly destroys network bandwidth and drops all
    connections.
794
795 **Fix:** This is a physical routing paradox that cannot be resolved in software. The upstream physical router
    providing internet to the Mac **must** be allowed to route its traffic directly to the physical WAN
    interface. The user must manually disable "Use Exit Node" on the upstream device providing the hotspot.
796
797 **Advanced Workaround:** If the upstream hotspot is a Windows machine and the user explicitly wants it to
    remain connected to the Exit Node, a permanent static route can be injected into the Windows routing kernel
    to forcefully bypass the Tailscale WFP interception for WARP packets. By binding the route to the physical
    adapter's Interface Index (IF), it becomes a permanent, roaming-aware bypass.
798
799 On Windows (Admin Command Prompt):
800 ```cmd
801 route print (find Interface List, note the IF number for the active Wi-Fi adapter, e.g. 15)
802 route -p add 162.159.192.1 mask 255.255.255.255 0.0.0.0 IF 15
803 route -p add 162.159.193.1 mask 255.255.255.255 0.0.0.0 IF 15
804 ```
805
806 On Linux (Root Terminal):
807 ```bash
808 ip route add 162.159.192.1 dev wlan0
809 ip route add 162.159.193.1 dev wlan0
810 ```
811
812 On macOS (Root Terminal):
813 ```bash
814 route add -host 162.159.192.1 -interface en0
815 route add -host 162.159.193.1 -interface en0
816 ```
817
818 *(Note: If the upstream router is an iPhone or Android device, you cannot inject static routes without
    jailbreak/root. You must simply disable "Use Exit Node" in the mobile Tailscale app).*
819
820 This punches a tiny hole straight through the paradox, letting the Mac's WARP packets escape to the physical
    internet while keeping the rest of the upstream router's traffic securely trapped in the Tailscale Exit
    Node.
821
822 #### 15.2.2 Infinite Egress Routing Loops & Subnet Takeover Paradoxes
823 **Problem:** Two distinct network loops can break host egress connectivity entirely when the exit node is
    active:
824
825 1. **The Recursive Host-VM Tunnel Loop:**
826 - **Mechanism:** The host Mac's default route points to the Tailscale exit node (`utun`). The exit node
    container sends its encrypted tunnel packets (destined for the WARP endpoint `162.159.192.1`) back to the
    host via the VM NAT. Without a static route exception on the host Mac, the host Mac routes those packets
    right back into the exit node (`utun`), creating an infinite loop that blackouts all host network egress.

```

```

827 - **ARP-Scoping Pitfall:** Simply binding the bypass route to the interface (`route add -host 162.159.192.1
828 -interface en0`) fails when the default route is deleted or redirected to `utun*`. Since `162.159.192.1` is
not local to the physical link, macOS fails to resolve it via ARP, dropping all tunnel packets.
829
830 - **Fix:** Dynamically resolve the active physical gateway IP (e.g. `172.17.0.1`) on startup, and add the
static host routes for the WARP endpoints via the gateway IP directly (`route add -host 162.159.192.1 <
gateway_ip>`). Additionally, because standalone `tailscaled` on macOS does not automatically modify the
system default route via the CLI, we manually redirect the default gateway to `utun*` using `
setup_exit_node_routing`.
831
832 2. **The Docker Compose Subnet Takeover Collision:**
833 - **Mechanism:** To avoid collisions with the host's campus network, Colima's default bridge (`docker.bip`)
is moved (e.g. to `10.200.0.1/24`). However, because `172.17.0.0/16` is now free inside the VM, Docker
Compose's IPAM dynamically takes it over for the project's custom network (`nullexit_default`). Because the
containers receive IPs on `172.17.0.0/16`, the host Mac cannot send local peer discovery packets (e.g.,
Tailscale `magicsock` packets on `172.17.0.2:41641`) directly. The host routing table sends them out `en0`
to the physical Wi-Fi, returning `host is down` or `no route to host`.
834 - **Fix:** Explicitly configure a custom default network subnet `10.200.1.0/24` in `docker-compose.yml` to
prevent Docker Compose from ever reclaiming the conflicting `172.17/172.18` ranges.
835
836 > See also 15.2.1 (The Hotspot Paradox) for the related infinite loop that occurs when the physical upstream
router is also a Tailscale exit-node client.
837
838 ##### 15.2.3 Docker Default Bridge vs Host Wi-Fi Subnet Collision (The 172.17.0.0/16 Subnet Overlap)
839 **Context:** When attempting to ping a local Windows client (`172.17.22.187`) or the default gateway
(`172.17.0.1`) directly over Wi-Fi, the Mac host returns `No route to host` (displaying a `REJECT` / `!`
flag in the routing table). Tailscale on the client also drops offline shortly after starting when using
the exit node, failing to establish direct P2P connections and falling back to slow DERP relays.
840
841 **Root Cause & Subnet Overlap Mechanism:**
842 1. **Docker Default Subnet:** By default, the Docker daemon initializes its default bridge network (`docker0`)
on the **172.17.0.0/16** IP range.
843 2. **Subnet Conflict:** The host's physical Wi-Fi network happens to be assigned the exact same
**172.17.0.0/16** range by the local network router.
844 3. **Routing Clash:** Colima launches a Linux VM on the Mac host to run the Docker engine. Because Docker
inside that VM creates `docker0` and claims the `172.17.0.0/16` subnet, any packet sent to `172.17.x.x`
from within the containers (such as Tailscale local discovery) is captured by the VM's internal virtual
bridge interface (which is down/empty) and is blackholed. On the Mac host, macOS dynamically marks routes
as `REJECT` (!) when local ARP resolution fails, causing local pings to immediately abort with `No route
to host`.
845 4. **Tailscale Relay Saturation:** Because local peer-to-peer discovery is blackholed, Tailscale falls back to
public DERP relay servers (e.g. `relay "tor"` in Toronto). When exit-node routing is enabled, all client
web traffic is routed through the relay. Public relays impose strict rate limits and throttle high-volume
traffic, resulting in packet drops. This saturation drops Tailscale's control packets, causing the
connection to time out and showing the client as offline.
846
847 **Fix:**
848 The canonical script for this is `scripts/fix-docker-bridge-collision.sh`. **One command applies the entire fix
:**
849
850 ```bash
851 bash scripts/fix-docker-bridge-collision.sh # apply
852 bash scripts/fix-docker-bridge-collision.sh --dry # preview changes without applying
853
854 It auto-detects the host's actual Wi-Fi subnet (no more guessing whether the campus DHCP handed out `172.x`,
`10.x`, or `192.168.x`), picks a non-conflicting `docker.bip` from a small candidate set (defaulting to
`172.26.0.1/24` if no overlap is matched), backs up `~/colima/default/colima.yaml` with an ISO-8601
timestamp, edits the file **in place** (idempotent replaces an existing `docker.bip` if present, appends
under an existing `docker:` block, or creates a fresh block), restarts Colima, rebinds Wi-Fi DHCP, verifies
the new default route is no longer Docker's bridge, and re-runs both `./toggle.sh` and `scripts/diagnose-
host-leak.sh` to print the post-fix verdict.
855
856 For reference, the underlying manual fix is: edit the Colima configuration file (`~/colima/default/colima.yaml
`) on your Mac to define:
857
858 ```yaml
859 docker:
860   bip: 172.26.0.1/24 # any /24 that does NOT overlap your Wi-Fi subnet
861
862 Then, restart Colima to apply the new subnet inside the VM:
863
864 ```bash
865 colima restart
866
867 This forces Docker to use `172.26.x.x` for its default bridge, freeing the physical `172.17.x.x` range and
letting pings and direct Tailscale peer-to-peer connections flow normally.
868
869 ##### 15.2.4 July 4, 2026: The "Network Unreachable" Host Leak & Post-Wake Container Destructions
870 **Symptom:**
871 1. Running `docker compose config` with dummy keys corrupted the user's `.env` file by appending invalid
WireGuard keys. This caused the `warp` container to become unhealthy on boot, which triggered a full
teardown of the gateway by `toggle.sh`.

```

869 2. After fixing `.env`, `toggle.sh` successfully booted the gateway, but the host's internet started bypassing the tunnel entirely (a massive host leak). The Cloudflare trace returned `warp=off` and exposed the host's physical ISP IP.

870 3. Running `recover.sh --post-wake` would violently force-recreate all gateway containers and drop the host's internet connection.

871

872 ****Root Cause:****

873 - ****Bug 1 (The Host Leak):**** In `toggle.sh`, the logic to find the Tailscale `utun` interface used `ifconfig | grep -B4 "inet 100."`. Due to the 4 lines of before-context, this regex was accidentally capturing the interface `*above*` the actual Tailscale interface (e.g. grabbing `utun4` instead of `utun5`). Because `utun4` had no IP address, the subsequent `sudo route add -net 0.0.0.0/1 -interface utun4` silently failed with "Network is unreachable", leaving the host's default route exposed.

874 - ****Bug 2 (The Post-Wake Destructions):**** The health check in `recover.sh --post-wake` relied on `docker compose exec -T warp curl -sf https://www.cloudflare.com/cdn-cgi/trace`. However, the newer `qmcgaw/glutun` Alpine-based Docker image no longer ships with `curl` pre-installed. The health check failed with `executable file not found in $PATH`, tricking `recover.sh` into thinking the tunnel was completely dead and needed to be forcefully recreated via `docker compose up -d --force-recreate`.

875 - ****Bug 3 (Restart Flags):**** Previous refactors incorrectly permitted `toggle.sh restart` instead of strictly enforcing `toggle.sh --restart`.

876

877 ****Resolution:****

878 - Cleaned the `.env` file of duplicate dummy entries.

879 - Replaced the brittle `grep -B4` interface parsing in `toggle.sh` with a strict AWK parser: `awk '/^[a-z0-9]+:{iface=$1}/inet 100\.{print iface; exit}' | tr -d ':'`. This mathematically isolates the exact interface possessing the Tailscale `100.x.x.x` IP address, preventing the `0.0.0.0/1` route from silently failing.

880 - Changed the health check in `recover.sh` to use `wget -q0- --timeout=3` which is natively installed in the Gluetun image. This stopped the unnecessary destructive recreations of the containers during post-wake events.

881 - Reverted the `restart` alias in `toggle.sh` to strictly require `--restart`.

882

883 ##### 15.2.5 July 4-5, 2026: Tailscale Data-Plane Loop (The DERP Relay Deadlock)

884 ****Symptom:****

885 After bypassing the Tailscale control plane (`192.200.0.0/16`) to fix the "offline" mesh status, the Mac appeared online. However, external peer devices (like the user's phone on cellular) could not successfully ping or SSH into the Mac. `tailscale netcheck` reported `Nearest DERP: unknown (no response to latency probes)`.

886

887 ****Root Cause:****

888 While bypassing the control plane kept the daemon connected to the coordination servers, Tailscale's actual peer-to-peer data plane relies on STUN (for NAT traversal) and DERP relays. These relays span ~80 dynamically allocated IP addresses across multiple cloud providers. Because we were forcing `0.0.0.0/1` through the `utun*` exit node, the Mac's own outbound connection attempts to DERP relays were being routed back into the tunnel. To prevent an infinite loop, Tailscale's daemon dropped its own packets when it saw them returning on the TUN interface. This effectively left the daemon deaf and blind to peer connections.

889

890 ****Resolution:****

891 - Modified `add_warp_bypass_routes` in `scripts/common.sh` to execute a live API fetch (`curl -s https://login.tailscale.com/derpmap/default`) during startup.

892 - The script parses out all active DERP relay IPv4 addresses (~80 IPs) and adds a physical host bypass route for every single one of them.

893 - These IPs are written to a temporary file (`/tmp/nullexit-derp-ips.txt`) so they can be cleanly un-routed by `remove_warp_bypass_routes` when the gateway shuts down. Peer connectivity (ping, SSH, SFTP) was fully restored.

894

895 ##### 15.2.6 July 5, 2026: Hardcoded WARP Endpoints in docker-compose

896 ****Symptom:****

897 The bash scripts allowed overriding the default Cloudflare WARP IP endpoints via `.env` variables (`WARP_ENDPOINT_1` and `WARP_ENDPOINT_2`) to establish bypass routes. However, `docker-compose.yml` statically hardcoded `162.159.192.1` for the `warp` container's `WIREGUARD_ENDPOINT_IP`.

898

899 ****Root Cause & Risk:****

900 If a user set a custom endpoint in `.env`, the script would successfully bypass the custom IP on the host level. However, Gluetun would still attempt to connect to the hardcoded `162.159.192.1`. Since `162.159.192.1` was no longer explicitly bypassed, its packets would be sucked into the `utun*` exit node, causing an infinite WireGuard routing loop that instantly breaks the gateway.

901

902 ****Resolution:****

903 Modified `docker-compose.yml` to dynamically read the environment variable via `WARP_ENDPOINT_1:-162.159.192.1`, ensuring both the host routing scripts and the container use the exact same endpoint.

904

905 ##### 15.2.7 Incident Post-Mortem: WARP Watcher Race vs Post-Wake (July 10, 2026)

906 ****Symptom:**** After switching Wi-Fi networks, the host IP appeared as the raw ISP IP (`132.x.x.x`) instead of a Cloudflare/WARP IP. The gateway appeared to complete post-wake recovery successfully in the logs, but nuclear shutdown fired immediately after and killed everything.

907

908 ****Root Cause:**** A fatal race condition between two concurrent processes:

909

910 1. ****Wi-Fi roam**** `watcher.sh` fires `recover.sh --post-wake`

911 2. ****Post-wake**** finds the `warp` container missing/dead calls `docker compose up -d --force-recreate` (~35 seconds)

```

912 3. **In parallel**, the WARP Watcher (running as a child of `toggle.sh`) polls the warp container every 5s
    during failures
913 4. While the container is being recreated it returns `warp=off` for those 35 seconds 7 consecutive failures
    **WARP SHUTDOWN fires**, calling nuclear `recover.sh`
914 5. Nuclear `recover.sh` tears down Tailscale, resets DNS to 1.1.1.1, stops all containers **gateway dead, IP
    exposed**
915 6. The post-wake `docker compose up` finishes at ~35s mark, container healthy but the gateway is already gone
916
917 The evidence in `output.log`:
918 ```
919 [2026-07-10T06:07:27Z] NET: ... bash recover.sh --post-wake
920 [2026-07-10T06:07:47Z] WARP DOWN cdn-cgi/trace reports warp=off watcher starts counting
921 ...
922 add net 0.0.0.0: gateway utun5 post-wake successfully re-routes at 02:08:22
923 ...
924 [2026-07-10T06:09:04Z] WARP SHUTDOWN 6 consecutive failures watcher fires nuclear
925 ```
926
927 **Fix (July 10, 2026):** Added a **WARP Watcher inhibit marker** (`/tmp/nullexit-warp-inhibit.marker`):
928 - `recover.sh --post-wake` writes the marker at startup **before** any container operations
929 - The WARP Watcher loop checks for the marker at the top of every iteration; if present it resets `consec_off`
    =0` and sleeps 5s (skips the poll entirely)
930 - `recover.sh` removes the marker at the very end (on both success and failure via `trap cleanup_inhibit EXIT`)
931 - This guarantees the watcher never counts failures during the intentional container-down window of a post-wake
    force-recreate
932
933 The marker file path is hardcoded to `/tmp/nullexit-warp-inhibit.marker` in both `toggle.sh` and `recover.sh`.
934
935 #### 15.2.8 Incident Post-Mortem: Concurrency Collision between Nuclear Teardown and Post-Wake (July 10, 2026)
936 **Symptom:** When the background WARP Watcher triggers a nuclear shutdown (due to a real or simulated tunnel
    failure), the gateway is completely torn down. However, the network configuration changes made during the
    teardown trigger a `State:/Network/Global/IPv4` network change event. The LaunchAgent-based network watcher
    (`watcher.sh`) intercepts this event and concurrently spawns `recover.sh --post-wake` to heal/restore the
    gateway. This causes `docker compose down` (from nuclear teardown) and `docker compose up` (from post-wake)
    to run in parallel, resulting in container errors like `dependency failed to start: container warp exited
    (0)` and the gateway becoming wedged.
937
938 **Root Cause:** A lack of coordination/locking between the lifecycle control scripts. While `toggle.sh` used `/
    tmp/nullexit-toggle.lock` to prevent concurrent `toggle.sh` runs, `recover.sh` (both nuclear and `--post-
    wake` modes) did not utilize or check any lock files.
939
940 **Fix (July 10, 2026):** Implemented a shared lifecycle lock file (`/tmp/nullexit-toggle.lock`) across both
    scripts:
941 - **`recover.sh` (all modes):** Checks `/tmp/nullexit-toggle.lock` at startup. If the lock is held by another
    running lifecycle script (`toggle.sh` or `recover.sh`):
942 - In `--post-wake` mode, it logs a skip message to `output.log` and exits cleanly (`exit 0`). This safely
    prevents the auto-recovery agent from recreating containers during a teardown.
943 - In nuclear recovery mode, it exits with an error.
944 - **`recover.sh` Lock Cleanup:** Cleans up the lock file upon completion using `trap cleanup_recover EXIT` (
    which checks if the lock file belongs to the current PID).
945 - **`toggle.sh` update:** The lock check in `toggle.sh` was updated to look for both `toggle.sh` and `recover.
    sh` in the running processes to enforce proper exclusion.
946
947 #### 15.2.9 Incident Post-Mortem: Infinite Auto-Recovery Loop after WARP Watcher Nuclear Shutdown (July 10,
    2026)
948 **Symptom:** When the background WARP Watcher triggered a nuclear shutdown (due to a tunnel outage), it
    successfully executed `recover.sh` (nuclear). However, immediately after the teardown completed, the system
    started spawning `recover.sh --post-wake` and rebuilding the containers again. This created an infinite
    loop of tearing down and auto-restarting the gateway, keeping the host's internet route permanently wedged.
949
950 **Root Cause:** An active-state marker file leak. The LaunchAgent-based network watcher (`watcher.sh`) only
    fires `recover.sh --post-wake` when `/tmp/nullexit-gateway-active.marker` is present on disk. While `toggle
    .sh` (STOP path) correctly deleted this marker, the nuclear `recover.sh` script did not. When the WARP
    Watcher ran `recover.sh` (nuclear), the containers were stopped but the active-state marker was left on
    disk. The network changes from the teardown then triggered `watcher.sh`, which saw the marker, believed the
    gateway was still supposed to be active, and triggered `--post-wake` to start it back up.
951
952 **Fix (July 10, 2026):** Modified the start of `recover.sh` to explicitly delete `/tmp/nullexit-gateway-active.
    marker` if `POST_WAKE` is `false` (nuclear mode). This ensures that any subsequent network configuration
    changes made during the teardown are correctly ignored by `watcher.sh` because the marker file is gone.
953
954 ### 15.3 MTU, Fragmentation & Throughput
955
956 #### 15.3.1 Network Bufferbloat & MTU Optimizations (Double-VPN UDP Crash)
957 **Symptom:** When running aggressive UDP bandwidth tests (such as macOS `networkQuality`, which uses QUIC/HTTP3
    ), the `nullexit` tunnel completely crashes or exhibits severe lag (6,000ms+ bufferbloat).
958
959 **Root Cause:** The bug is a cascading MTU mismatch. Tailscale generates a UDP packet. Cloudflare WARP (also
    WireGuard) receives the packet, wraps it in another layer of encryption, pushing the packet size beyond the
    standard 1500 byte limit. Unlike TCP, UDP packets cannot be resized dynamically via MSS negotiation,
    resulting in massive IP packet fragmentation. The Apple Virtualization framework (`vz`), which Colima uses

```

for bridged networking, silently drops heavily fragmented UDP packets under high load. This blackholes the entire UDP upload stream, instantly dropping the connection.

960

961 ****Fix (Partial):**** To prevent MTU fragmentation for standard web traffic, ****TCP MSS Clamping**** is strictly enforced via the macOS ``pf`` firewall. By adding ``scrub out all max-mss 1160`` to ``scripts/pf.conf``, macOS unilaterally shrinks all outgoing TCP headers to fit comfortably inside the double-encrypted tunnel, bypassing Path MTU Discovery blackholes. ``TS_DEBUG_MTU=1200`` was also added to ``docker-compose.yml`` to minimize internal fragmentation.

962 ****Unresolved:**** The only way to solve the UDP crash bug natively is to artificially cap the maximum tunnel bandwidth using SQM (``fq_codel``). Because a static bandwidth cap would artificially throttle high-speed networks, ``nulloxit`` leaves the bandwidth uncapped, optimizing perfectly for TCP while accepting UDP fragmentation under extreme edge-case load tests.

963 ****Why not Dynamic SQM (Aurate)?**** Implementing an EMA/PID-controlled aurate daemon (like ``sqm-aurate``) requires a constant 100ms polling loop to measure the kernel packet queue, which severely degrades laptop battery life by preventing deep C-state CPU idling. Furthermore, macOS's native firewall (``dummynet``) only supports ``fq_codel`` and does not support the newer Linux ``CAKE`` algorithm, which is the only algorithm that offers kernel-native, event-driven ``aurate-ingress`` (zero polling penalty). Therefore, dynamic SQM is computationally hostile to battery-powered macOS hosts.

964

965 **#### 15.3.2 Double-Tunneling MSS Clamping (Stalling Bug)**

966 ****Problem:**** Traffic double-wrapped through Tailscale (WireGuard) and WARP (WireGuard) suffered 120 bytes of MTU overhead (60 + 60). The default MSS clamp of 1180 was still slightly too large, causing large packets to fragment or drop, which resulted in mysterious web page stalls on strict-MSS endpoints.

967

968 ****Fix:**** Lowered the TCP MSS clamp in ``post-rules.txt`` to ``1120`` to guarantee double-tunneled payloads fit safely within standard internet MTUs.

969

970 **#### 15.3.3 Wi-Fi Edge Packet Loss via QUIC (UDP) Fragmentation**

971 ****Problem:**** Applications that heavily utilize HTTP/3 (QUIC) over UDP such as Facebook, Instagram, and Google services experience massive stuttering and stalled image loading when the client device is far away from the router (weak Wi-Fi signal). The issue did not occur when close to the router.

972

973 ****Root Cause:**** The ``nulloxit`` gateway uses a double-encrypted tunnel (Tailscale + WARP). To prevent packets from fragmenting when entering these tunnels, a firewall rule in ``post-rules.txt`` uses ``--set-mss 1120`` to safely clamp TCP packets. However, because QUIC runs entirely over UDP, it bypasses the TCP MSS handshake completely. QUIC blasts maximum-MTU (1280 byte) UDP packets. The inner ``tailscale0`` interface (MTU 1200) forces these large UDP packets to fragment into two pieces. When the client is on the edge of Wi-Fi range (where 5-10% packet loss is common), losing *either* UDP fragment destroys the *entire* image frame. This exponential amplification of packet loss stalled QUIC streams.

974

975 ****Fix & Trade-offs:**** Appended an ``iptables`` rule to ``post-rules.txt`` that explicitly blocks ``UDP --dport 443`` (QUIC). When modern web apps detect QUIC is blocked, they instantly fall back to HTTP/2 (TCP 443). Because TCP is routed through the MSS clamp, the packets are resized *before* they leave, entirely eliminating IP fragmentation and restoring high-speed loading over weak Wi-Fi.

976 ***Consequences:*** This adds a minor latency penalty (TCP 3-way handshake) compared to QUIC's zero-RTT connection. It may also force WebRTC video calls (Google Meet/Discord) to fall back to TCP TURN servers, slightly inflating real-time video latency. However, in a constrained double-tunnel mesh, the absolute stability gained from eliminating fragmentation vastly outweighs these trade-offs.

977

978 **#### 15.3.4 Cellular Networks & PMTUD Blackholes (The 1280 Byte Limit)**

979 ****Experiment:**** Conducted a live packet sweep from the PC-1 host to Phone-1 connected via a cellular network + Tailscale DERP relay using ``ping -D -s <size>``.

980

981 ****Result:****

982 - Packets with a payload of ``1252`` bytes (+ 28 bytes IP/ICMP headers = ****1280 bytes total****) succeeded perfectly with ~60ms latency.

983 - Packets with a payload of ``1253`` bytes (****1281 bytes total****) and above resulted in a silent timeout.

984

985 ****Analysis:**** The cellular network (or the Tailscale DERP relay) enforces a strict MTU of 1280 bytes. Critically, it acts as a "PMTUD Blackhole" it silently drops packets larger than 1280 bytes instead of returning an ICMP "Fragmentation Needed" warning. If the exit node tries to send a standard 1500-byte internet packet to the phone over this link, it vanishes, causing web pages to stall permanently.

986

987 ****Conclusion:**** This empirically validates the absolute necessity of the TCP MSS clamping rule in ``post-rules.txt``. By artificially clamping the MSS to ``1120`` (or ``1180``), we ensure the TCP payload + headers + double-WireGuard overhead never exceeds the strict 1280-byte ceiling of the cellular mesh link.

988

989 **#### 15.3.5 Low Gateway Throughput (DERP Relay & MTU Fragmentation) Fixed**

990 ****Problem:**** The host Mac's internet throughput through the gateway dropped significantly (e.g., from 34Mbps raw to 2.1Mbps via gateway). This was caused by a combination of two bottlenecks:

991

992 ****1. Tailscale DERP Relay Bottleneck:****

993 Because the host Mac and the Docker container operate behind the same public IP, standard hole-punching for Tailscale P2P failed, forcing traffic through Tailscale's NYC DERP relay. Normally, ``routing-fix.sh`` explicitly drops container Tailscale UDP packets destined for local subnets (when ``TAILSCALE_ALLOW_LAN_P2P=false`` in ``env`` or auto-detected). We nuke these P2P connections on purpose to prevent the severe macOS ``gvproxy`` SNAT endpoint poisoning issue we faced earlier (see 15.7 SNAT Endpoint Poisoning). However, this strict policy unintentionally blocked direct communication between the container and the Mac host itself via the Docker bridge network.

994


```

995 **Fix:** Tailscale's control plane registers the Mac host using its physical IP (e.g., `172.17.52.223`), not
    the Docker bridge IP. If this physical IP falls within the dropped local subnets (like `172.16.0.0/12`),
    the P2P handshake is dropped. To solve this natively, `scripts/common.sh` now features a `write_host_ips()`
    function that actively grabs all physical IPv4 addresses on the Mac and writes them to `.host_ips`.
    `routing-fix.sh` dynamically reads this file every 30 seconds and injects priority `RETURN` rules for those
    exact physical IPs. This guarantees direct container-to-host P2P over the local bridge (bypassing DERP)
    while continuing to block external LAN devices (like phones) to prevent the 15.7 SNAT poisoning bug.
996
997 **2. MTU Double-Encapsulation Fragmentation:**
998 The host's Tailscale interface (`utun5`) defaulted to an MTU of 1280. The WARP tunnel inside the container also
    uses an MTU of 1280. When a full 1280-byte Tailscale packet from the host entered the container and was
    encapsulated by WARP (adding ~60 bytes of overhead), the resulting packet exceeded 1280 bytes, leading to
    severe fragmentation and performance degradation.
999
1000 **Fix:** In `toggle.sh`'s `setup_exit_node_routing` function, the host's Tailscale `utun` interface MTU is
    explicitly lowered to `1200` (configurable via `HOST_MTU` in `.env`). This ensures that host-originated
    packets can comfortably fit inside the container's WARP tunnel encapsulation without fragmenting.
1001
1002 > **Design Note Is direct hostcontainer P2P a Docker bug, or are we "exploiting a VM/host relationship"?
1003 > Neither. It is expected, well-defined behavior, and the RETURN-rule carve-out is a *defensive de-restriction
    *, not an exploit. The reasoning:
1004 > - **On native Linux**, the container and host share one kernel; the Docker bridge is a first-class local
    interface and hostcontainer is *trivially* direct. Nobody calls that an exploit. Our RETURN rules simply
    reproduce that same local reachability on macOS.
1005 > - **The RETURN rules add nothing new; they *stop blocking* something legitimate.** `routing-fix.sh` broadly
    DROPS containerLAN Tailscale UDP to prevent the real 15.7 `gvproxy` SNAT endpoint-poisoning bug caused by *
    external* LAN peers (e.g. a phone on another AP). Tailscale's control plane registers the Mac host by its *
    physical* LAN IP (e.g. `172.17.52.223`), which falls inside those DROPPed ranges so the host, the one peer
    that is literally the same physical machine, became collateral damage. The `.host_ips` RETURN rules are
    the minimal, precise exception that re-permits exactly that same-machine path and nothing else.
1006 > - **The genuinely VM-specific wart is `gvproxy` SNAT poisoning (15.7), and we route *around* it, not through
    a hole in it.** Keeping hostcontainer traffic on the Docker bridge (direct) avoids letting it get NATed
    through the userspace `gvproxy` endpoint rewriter that confuses Tailscale's peer-endpoint learning. That is
    the opposite of exploiting the VM boundary it is respecting it.
1007 > - **What genuinely *doesn't* work on a VM host is UDP hole-punching to arbitrary *external* peers** (see
    15.13). That is a real limitation of userspace VM networking, and it is why remote peers fall back to DERP
    unless bridged mode is enabled on a trusted LAN. Direct hostcontainer P2P is the one case that works
    regardless because both endpoints live on the same physical machine, no hole-punching is needed at all. It
    was verified `direct 172.17.52.223` even in plain user-mode networking (no `--network-address`).
1008 >
1009 > In short: the DROP is the deliberate policy, the RETURN is a surgical exception for same-machine traffic, and
    the whole thing is standard bridge networking not a Docker bug and not an exploit.
1010
1011 ### 15.4 DNS Interception & Proxy
1012
1013 #### 15.4.1 DNS Proxy: TCP Wire Format Mismatch (socat / dnsmasq)
1014 **Problem:** When the Tailscale data plane is unavailable (DERP relay only), the script falls back to a local
    DNS proxy that forwards queries from host UDP:53 to Docker's TCP:5354 (AdGuard). The `socat` and `dnsmasq`
    approaches both failed.
1015
1016 - **socat** forwards raw bytes it does not prepend the 2-byte length prefix that DNS-over-TCP requires.
    AdGuard parses the stream incorrectly and returns garbage.
1017 - **dnsmasq** tries UDP first to reach the upstream (`127.0.0.1#5354`), but Colima's SSH tunnel only forwards
    TCP. With a single upstream server, dnsmasq doesn't fall back to TCP even with `--timeout=1`.
1018
1019 **Solution:** Replaced both with a **Python DNS proxy** (`scripts/dns-proxy.py`, ~25 lines) that properly
    handles the DNS-over-TCP wire format: reads a UDP query from the host, prepends the 2-byte length prefix,
    sends it over TCP to AdGuard, strips the prefix from the response, and sends it back over UDP.
1020
1021 *Architectural Note: Why is the DNS proxy in Python while the SOCKS5 proxy was moved to a compiled Go Docker
    container?
1022 The SOCKS5 proxy handles heavy data streaming (e.g., video, downloads). Python introduces massive CPU/battery
    overhead for raw socket streaming, which is why we rely on the compiled Go implementation built directly
    into the `tailscaled` daemon via `TS_SOCKS5_SERVER=:1080`.
1023 Conversely, the DNS proxy only handles intermittent UDP queries (a few bytes per minute) generated solely by
    the local host machine. The Python overhead for this is effectively 0.001 Watts. We deliberately kept `dns-
    proxy.py` in Python because it runs natively on the macOS/Linux host rewriting it in Go would force users to
    install a Go compiler (`brew install go`) on their host machine for zero noticeable battery gain. (Note:
    If this architecture is ever adapted to serve as a public DNS resolver for thousands of users or for
    massive automated web-scraping clusters, `dns-proxy.py` must be rewritten in Go to survive the concurrent
    packet flood).
1024
1025 #### 15.4.2 Tailscale MagicDNS Bypassing the AdGuard PREROUTING Intercept
1026 **Context:** The `routing-fix.sh` script applies an iptables NAT rule (`PREROUTING -i tailscale0 -p udp --dport
    53 -j REDIRECT`) to intercept all incoming DNS queries from connected Tailscale peers and force them into
    the AdGuard container on port 5335.
1027 **Problem:** When remote clients (like Android or iOS) have "Use Tailscale DNS settings" turned on, they query
    Tailscale's MagicDNS IP (`100.100.100.100`). This packet enters the exit node, but the `tailscaled` daemon
    running *inside* the gateway container intercepts the `100.100.100.100` packet internally. `tailscaled`
    then generates a brand new, local DNS request to the upstream DNS server to resolve the query. Because this
    new request is generated *locally* by the daemon (not arriving externally via the `tailscale0` interface),

```

it bypasses the `PREROUTING` chain completely. The request glides straight out the WARP tunnel to Cloudflare/Google, entirely bypassing the AdGuard sinkhole.

1028 ****Fix:**** The only way to fix this blindspot is to force `tailscaled` to use AdGuard as its upstream. In the Tailscale Admin Console, the gateway's Tailscale IPv4 address (e.g., `100.x.x.x`) must be added as the sole Custom Global Nameserver, and "Override local DNS" must be enabled. Additionally, to block Android devices from bypassing this via Private DNS (DNS-over-TLS), outbound Port 853 traffic is explicitly REJECTED in both `routing-fix.sh` and `post-rules.txt`.

1029

1030 **#### 15.4.3 Seamless Wi-Fi Roaming & The macOS DNS Wipeout Loophole**

1031 ****Problem:**** The Tailscale, WireGuard, and Colima NAT stacks are natively designed for connectionless roaming and will seamlessly survive when the macOS host switches Wi-Fi networks (e.g., roaming from home Wi-Fi to a cellular hotspot). However, the internet completely breaks upon network transition. This occurs because macOS intentionally wipes out all custom DNS settings (`networksetup -setdnsservers`) and reverts to the new network's default DHCP nameserver whenever the active Wi-Fi BSSID changes. Since the Mac is locked to the exit node, its DNS requests are swallowed by WARP and dumped onto the public internet, which cannot route private DHCP IPs. This results in a total DNS failure.

1032

1033 ****Solution:**** Implementing a silent background "DNS Watcher" daemon (`nullexit-dns-watcher`) in `toggle.sh`. When the gateway starts, it spawns a background polling loop that checks the active Wi-Fi DNS every 30 seconds. If macOS resets the DNS during a network change, the watcher instantly detects the drift and forcefully re-injects `100.100.21.8` via `networksetup`. When the gateway stops, the watcher process is cleanly killed. This allows true, seamless roaming across physical networks without ever dropping the gateway state.

1034

1035 **#### 15.4.4 docker compose exec `r` Injection**

1036 ****Fix:**** All `docker compose exec` output piped through `tr -d '\r'`.

1037

1038 **#### 15.4.5 Stderr Invisibly Swallowed**

1039 ****Symptom:**** Failures were silent due to stderr redirected to `output.log`.

1040

1041 ****Fix:**** Tailscale wait loop now shows output. Critical commands show errors inline.

1042

1043 **### 15.5 Roaming, Sleep/Wake & Auto-Recovery**

1044

1045 **#### 15.5.1 Gateway Breakage on Lid Close / Wi-Fi Roam Post-Wake + Post-Roam Auto-Recovery**

1046 ****Context / Symptom:**** Closing the lid (sleep/wake) OR losing + reconnecting Wi-Fi (e.g. in an elevator, caf, or roaming between APs) leaves the gateway in a partly-dead state. Symptoms range from a ~30s window of dead DNS to a permanent outage that only full `recover.sh` or `toggle.sh` fixes. The root cause is that nullexit has ****no daemon watching macOS sleep/wake or network-state-change events**** the existing DNS Watcher inside `toggle.sh` only runs in-process and is suspended along with the rest of the user shell on lid close, so the gateway cannot self-heal after either event.

1047

1048 ****Diagnosis:**** Two distinct failure modes share the same root gap.

1049

1050 *** ** (A) Lid close sleep wake.** `caffeinate -i` (used by `toggle.sh`) blocks ***idle*** sleep but ****does not** block forced sleep from lid close******. Closing the lid hard-suspends Colima VM + every container + every Tailscale-quit-shells-except-tailscaled. On wake the VM clock needs ~5-30s to resync via NTP, during which TLS cert validation fails: Cloudflare WARP `162.159.192.1:2408` rejects the handshake gluetun's healthcheck (interval: 2s, retries: 15) eventually flips unhealthy Docker `unless-stopped` restarts the `warp` container (~30s gap). `mDNSResponder`'s in-memory cache still holds pre-sleep entries (stale A records for tailnet hosts) so the first dozen DNS queries after wake time out. `tailscaled` on the host often keeps its stale DERP relay preference and routes packets through a dead relay for another 30-60s.

1051 *** ** (B) Wi-Fi roam / loss in elevators, captive portals.** WARP is a UDP tunnel to Cloudflare's anycast edge. Carrier NATs and captive-portal networks silently invalidate the UDP binding on roam and on hotspot-bound re-association. macOS `networksetup` ***should*** preserve DNS settings on the same Wi-Fi service across a roam, but most captive-portal networks DHCP-replace DNS to the captive-portal resolver ****during the captive-portal dance itself****, before the user has authenticated so even DNS hijack survives a clean roam and quietly breaks on captive-portal handoff.

1052

1053 ****Resolution.**** A single ****launchd LaunchAgent**** (`com.nullexit.wake-recovery`) runs a long-running shell daemon (`scripts/watcher.sh`) that listens for both event sources and, on each fire, executes `bash recover.sh --post-wake` a ***lighter-touch*** recovery mode that's the opposite of the existing **``--nuclear``** semantics: it keeps the gateway live while still refreshing every stale subsystem.

1054

1055 ****Deep dive: Apple Power Management Primer (for the unfamiliar).****

1056 macOS exposes several relevant surfaces; the right primitive depends on what you actually need to detect. None of these are obvious and none of them have a standard splash screen in the docs.

1057

1058 *** **`caffeinate <flags>`**** creates I/O Kit power assertions via `IOPMAssertionCreateWithName`. It does NOT block forced sleep (lid close, low-battery shutdown, sleep timer firing on a per-set Energy Settings policy). Flags:

1059 *** `i`** PreventIdleSleep (the only one `toggle.sh` uses; protects against the OS going idle while the user is active but a clamshell close still hard-puts it to sleep)

1060 *** `s`** PreventSystemSleep (only honored on AC power; the user may be on battery)

1061 *** `u`** DeclareUserActive (resets the idle timer every 30s by default; equivalent to keeping the cursor moving)

1062 *** `d`** PreventDisplaySleep

1063 *** `m`** PreventDiskIdleSleep

1064 *** There is NO `l` (lid-block) flag in any modern macOS. Lid close is a kernel-firmware-level event that ignores user-space assertions.**


```

1065 * To prove any of this on live hardware: `pmset -g assertions` lists every active assertion with type /
    process / reason / timeout.
1066 * `pmset` is the read-side of the same system:
1067 * `pmset -g log | grep -E 'Sleep|Wake|DarkWake'` shows the recent timeline.
1068 * `pmset -g history` is the per-day summary.
1069 * `pmset -g assertions` is the live assertion list (matches `-i -s -d -u` to processes).
1070 * `launchd` does NOT have a native "on-wake" hook. Not in any version of macOS as of Sonoma. The canonical
    recipe is `StartCalendarInterval` with a sub-minute cadence and let launchd queue missed intervals for
    replay-on-wake, or use a third-party daemon (Sleepwatcher). For our use case a long-running shell daemon
    listening to the unified log is more responsive than a polling agent.
1071 * `com.apple.system.powermanagement.*` Darwin notifications (`willSleep`, `didWake`, `didDim`, `didUndim`)
    are the C-level signal. From a shell, they are best reached through the unified log with `log stream --
    predicate` see the watcher implementation.
1072 * `scutil n.watch` is the canonical CLI for live-following network-state changes. Pairs with `n.add State:/
    Network/Global/IPv4` to get a single tap that fires on any IPv4 link/route/DNS/address change across all
    interfaces. This is what `scripts/watcher.sh`'s network-change listener uses.
1073 * `com.apple.networkChange` Darwin notification is the C-level equivalent but has no shell-friendly
    listener; use `scutil` instead.
1074 * `SCNetworkReachability` is the Objective-C API used by SOCKS5/HTTP clients to detect route changes per
    target. Never a fit for shell scripts.
1075
1076 **The Fix (component-by-component).**
1077
1078 1. `recover.sh --post-wake` (new flag, non-destructive by design). Adding `--post-wake` to the existing
    nuclear recovery script inverts the semantics. The default mode still tears down Tailscale, resets DNS to
    empty, stops caffeinate, runs `docker compose down`, power-cycles Wi-Fi. The new mode does:
1079 * Skip Tailscale disconnect (we WANT it to keep the mesh connection)
1080 * Re-hijack DNS to the gateway Tailscale IP read from `gateway_ip` (instead of resetting to empty)
    applies to both `ACTIVE_SERVICE` and `ENO_SERVICE` because the system resolver is scoped to en0
1081 * Refresh the exit-node preference with `tailscale up --reset --ssh=true --accept-dns=false --exit-node
    =\`$TS_IP\` --exit-node-allow-lan-access=true` (the exact flags `toggle.sh` START uses). This re-asserts
    DERP mapping without dropping the mesh
1082 * Inspect `docker inspect --format '{{.State.Health.Status}}` nullexit-warp-1 and only `docker compose
    up -d --force-recreate warp` if gluetun is unhealthy this single targeted recreate nudges the UDP NAT
    binding back to Cloudflare without disturbing Tailscale or AdGuard
1083 * Run the sharingd-reset step (existing fix from `toggle.sh` START path; see 15.11) so AirDrop doesn't freeze
    on the new IP lease
1084 * Verify the gateway with `dig +tcp @127.0.0.1 -p 5354 google.com` (AdGuard via TCP through Colima's SSH
    tunnel) and a 4-second `tailscale ping` to the gateway Tailscale IP, instead of the default mode's direct-
    to-1.1.1.1 check
1085 * Crucially: skip `sudo -v` because the LaunchAgent has no TTY to prompt on; all privileged calls use `
    sudo -n` and lean on `/etc/sudoers.d/nullexit` NOPASSWD entries (`networksetup`, `dnsmasq`, `dscacheutil`,
    `killall`, `pkill`).
1086 2. `toggle.sh` writes and clears `/tmp/nullexit-gateway-active.marker` (an ISO-8601 UTC timestamp) at the
    bottom of the START path and the top of the STOP path / `cleanup_handler`. Two new helpers: `
    write_gateway_active_marker` and `clear_gateway_active_marker`. The watcher uses this file as its only
    signal that the gateway is live: if `toggle.sh` was stopped (or never started), the watcher no-ops. This
    prevents waking-up-only-on-counterfactual events from churning DNS.
1087 3. `scripts/watcher.sh` (long-running daemon, sibling-style source file). Two backgrounded pipelines:
1088 * Wake listener: `log stream --predicate 'composedMessage CONTAINS[c] "didWake" OR composedMessage
    CONTAINS[c] "Wake from Sleep" OR composedMessage CONTAINS[c] "Waking from" OR composedMessage CONTAINS[c] "
    DarkWake" --style compact --info` piped through `while read; do run_recover "WAKE: ..."; done`. `[c]`
    makes the predicate case-insensitive (the unified log writes phrases in mixed case across versions).
1089 * Network listener: `(echo "n.add State:/Network/Global/IPv4"; echo "n.watch"; while true; do sleep
    86400; done) | scutil 2>/dev/null` piped through a `case` match on `n.state*` / `SCEventUpdate*`.
1090 * `run_recover` checks the marker, debounces via `/tmp/nullexit-watcher.last-recovery` (default 10s,
    configurable via `NULLEXIT_DEBOUNCE_SECONDS`), and shells out to `bash recover.sh --post-wake`. The 10s
    debounce prevents a single wake event (which typically emits 2-3 log lines) from triggering multiple
    recoveries.
1091 * `trap ... TERM INT` `kill 0` cleanly tears down the process group on launchctl bootout` so the `sleep
    86400` inside the scutil pipe also dies.
1092 * `wait` blocks indefinitely while both pipelines feed it.
1093 4. `launchd/com.nullexit.wake-recovery.plist` (LaunchAgent in the user domain runs as the console user at
    every login without needing sudo).
1094 * `Label: com.nullexit.wake-recovery`
1095 * `ProgramArguments: ["/bin/bash", "<absolute-path-to-your-nullexit-install>/scripts/watcher.sh"]`
1096 * `RunAtLoad: true` starts immediately on launchctl load` (no need to log out and back in)
1097 * `KeepAlive: { SuccessfulExit: false, Crashed: false }` don't restart on clean SIGTERM exit (so `
    launchctl bootout` doesn't get stuck in a relaunch loop). Also don't restart on hard crash: we observed
    that macOS Sonoma's sleep/wake suspend-resume path accumulates orphan watcher.sh PIDs because SIGCONT
    does not reliably clean up the prior instance's listener grandchildren, and a `KeepAlive.Crashed=true`-
    driven relaunch actively races with the still-live prior process. The script enforces single-instance on
    its own via a `flock`-style PID-file lock, so a relaunch-on-crash would be skipped by the lock and produce
    0 listener coverage until the live instance happened to die. Trade-off: a real crash leaves the user
    without auto-recovery until they run launchctl load -w` manually acceptable because (a) gateway still
    works without auto-recovery, (b) a relaunch wouldn't fix whatever caused the crash, and (c) the gateway-
    recovery itself is observably broken (no DNS, no exit-node) if it does go down.
1098 * `ProcessType: Background` hint to App Nap not to suspend us.
1099 * `StandardOutPath/StandardErrorPath: /tmp/nullexit-watcher.{out,err}.log` separates launchd-level
    diagnostics from the script's own `/tmp/nullexit-watcher.log` (which `exec >>` redirects).
1100

```

```

1101 **Install (one-time).** The plist lives in the repo at **`launchd/com.nullexit.wake-recovery.plist`** for
      version control. The installed (live) copy must land in ~/Library/LaunchAgents/` so launchd picks it up on
      the user session.
1102
1103 ```bash
1104 # Copy the plist to the user-launchd location (run from repo root)
1105 cp ./launchd/com.nullexit.wake-recovery.plist \
1106 ~/Library/LaunchAgents/
1107
1108 # Substitute __NULLEXIT_HOME__ with your local install absolute path.
1109 # The plist uses WorkingDirectory + a relative program arg, so this
1110 # single substitution is the only place it references your machine.
1111 sed -i 's|__NULLEXIT_HOME__|$(pwd)|' \
1112 ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1113
1114 # Validate the XML
1115 plutil -lint ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1116
1117 # Load + persist this user session + every future login
1118 launchctl load -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1119
1120 # Confirm it actually started
1121 launchctl list | grep nullexit
1122 ```
1123
1124 To **stop** the watcher (without uninstalling):
1125
1126 ```bash
1127 launchctl bootout gui/$UID/com.nullexit.wake-recovery
1128 # or:
1129 launchctl unload -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1130 ```
1131
1132 To **uninstall** completely:
1133
1134 ```bash
1135 launchctl bootout gui/$UID/com.nullexit.wake-recovery 2>/dev/null || true
1136 rm ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1137 ```
1138
1139 **Why this fix was preferred over alternatives.**
1140
1141 * **Polling with `StartCalendarInterval`** would have given ~30-60s detection latency per wake. The unified-log
      stream is sub-second.
1142 * **Sleepwatcher** would have covered wake events but not network changes. We'd still need a separate `scutil`
      listener, and introducing a third-party dependency for what amounts to ~150 lines of shell is unjustified.
1143 * **Forcing `caffeinate -l`** doesn't exist and the closest equivalent (`caffeinate -s`) only works on AC power
      . The whole point of the gateway is to stay usable on battery while traveling lid close must NOT silently
      kill the gateway.
1144 * **A kernel extension** is impossible (no entitlement) and out of scope.
1145
1146 **Reproduction recipe.** Test both events in isolation to confirm the fix actually closes the gap.
1147
1148 * **(A) Lid-only repro**: with the gateway up, run in a terminal: `pmset displaysleepnow && sleep 30 && pmset
      wake` (without physically closing the lid). Watch the wake wake-fires the watcher post-wake routine runs
      login should still work in <5s after wake. Compare to baseline pre-fix: pre-fix the gateway would be dead
      for 30-90s.
1149 * **(B) Roam-only repro**: with the gateway up: `networksetup -setairportpower off && sleep 30 && networksetup
      -setairportpower on` (or simply walk out of Wi-Fi range and back). The captive-portal WILL repave DHCP DNS
      for a few seconds, the watcher fires on the second `State:/Network/Global/IPv4` change (after the Wi-Fi re-
      associates) and the post-wake routine re-hijacks DNS within 2-3s.
1150
1151 **Operative files.**
1152
1153 * `recover.sh` added arg parsing; wrapped destructive sections behind `POST_WAKE`; added re-hijack DNS /
      refresh exit-node / force-recreate-if-unhealthy path.
1154 * `toggle.sh` added `write_gateway_active_marker` / `clear_gateway_active_marker` called at the END of START
      and TOP of STOP / cleanup_handler.
1155 * `scripts/watcher.sh` new long-running daemon.
1156 * `launchd/com.nullexit.wake-recovery.plist` launchd LaunchAgent descriptor.
1157 * `~/Library/LaunchAgents/com.nullexit.wake-recovery.plist` the installed copy (one-time copy).
1158
1159 **Honest assessment (read before testing).** This fix has two asymmetric halves:
1160
1161 * **Network / roam / captive-portal path RELIABLE.** `scutil n.watch` on `State:/Network/Global/IPv4{4,6}`
      catches every Wi-Fi roam, captive-portal DHCP-rebind, hotspot handoff, and VPN change with zero missed
      events. To validate, drop Wi-Fi for 30 s and re-add it; `/tmp/nullexit-watcher.log` will record a `NET:`
      trigger and `recover.sh --post-wake` will run within ~10 s of the network coming back.
1162 * **Lid close / wake path BEST-EFFORT on macOS Sonoma+. Apple `suspends` LaunchAgents process-state during
      forced sleep and `resumes` them on wake; it does NOT re-launch them every wake. As a result: (a) `log
      stream` is a live subscription events emitted during the agent's suppressed window are DROPPED, not

```

buffered, so the wake event that prompted the resume is invisible to `log stream` on resume; (b) the "fire on startup" guard runs once on `launchctl load -w` and after a `KeepAlive` crash-recovery, NOT on every successive wake; (c) `subsystem == "com.apple.powermanagement"` will catch FUTURE emissions (display dim/restore, manual sleep/wake) but not the lid-close/open wake directly.

In practice the lid-wake gap is short, because `toggle.sh`'s existing DNS Watcher already polls every 5 s and re-hijacks DNS (so DNS recovers within 5 s of any roam or wake), `tailscaled` self-heals via DERP fallback within 3060 s, and `gluetun` reconnects via its healthcheck retry within 30 s. The user-visible pain on lid open is ~30 s of wonky DNS, not a permanent outage.

****Test the ROAM path FIRST.**** If ROAM works in your testing, the lid-wake gap is acceptable. If lid-wake handling is critical, the two clean follow-ups are:

- **Sleepwatcher**** (one canonical install): `brew install sleepwatcher`; drop `recover.sh --post-wake` into `~/sleep` and `~/wake` so Sleepwatcher invokes it directly on every wake without the launchd propagation problem.
- **Move the watcher to a LaunchDaemon**** (root) and use `IOPMSchedulePowerEvent` via a tiny C wrapper not portable to shell.

A truly reliable shell-only solution to "wake events from inside a LaunchAgent" does not exist on macOS Sonoma +.

****Empirical lessons from deployment.**** Two findings came up while deploying this fix end-to-end; recording so the next operator doesn't lose an evening to each.

- **Bash function-call-before-definition gotcha.**** While introducing the gateway-active marker helpers, the defensive `clear\_gateway\_active\_marker` call was inserted at the very top of `toggle.sh` (right after `set -e`), but the function definition lived near `PID\_FILE="/tmp/nullexit-cafeinate.pid"` ~90 lines down. Bash parses top-to-bottom and resolves names at first invocation, so the call site exited with code 127 (`clear\_gateway\_active\_marker: command not found`). The gateway stayed unreachable from the START path even though `bash -n` passed (the parser doesn't catch order-of-resolution errors). Rule: define functions BEFORE any block that calls them. Verify with `grep -n 'name()\s\*{'` followed by `grep -n 'name\s\*\${` the latter must appear on a line number AFTER the former.
- **networksetup -setairportpower off+on` is not a faithful roam reproduction.\*\*** Synthetic Wi-Fi radio-cycling (`sudo networksetup -setairportpower Wi-Fi off` for 30 s, then back on) is expected to produce a `NET:` trigger in `scutil n.watch` but produced ZERO during testing because `setairportpower` powers the radio at the driver level while leaving the network SERVICE in the "up" state from scutil's perspective, so no `n.state State:/Network/Global/IPv4` event is generated. A real roam (macOS briefly loses the AP and re-associates into a different one) DOES fire the event because the link actually changes.

Better reproduction recipes in priority order:

- * Walk between two real SSIDs via `networksetup -switchtolocation` (or actual physical station movement) guaranteed to fire the scutil event.
- * Force a DHCP rebind on the same SSID with `sudo ipconfig set en0 DHCP` triggers `State:/Network/Global/IPv4` to update.
- * Trust the existing 30-second DNS Watcher inside `toggle.sh` for the DNS path: it re-hijacks DNS automatically. The post-wake watcher adds clear value mostly for tailscale exit-node re-assertion and warp gluetun force-recreate, both of which self-heal within ~30 s anyway.

15.5.2 July 8, 2026: Network Watcher Event Mismatch, Sudo-in-Background Crash, and Loop-Prevention

****Symptom:****

- Switching Wi-Fi or waking the Mac from sleep did not trigger a post-wake recovery (`recover.sh --post-wake`) automatically.
- The network connection would eventually get completely sinkholed/blocked. If the user tried to restart or wait, it did not self-heal.
- During the recovery attempt, checking the public IP on the host would return the unencrypted campus/ISP IP (starting with `132.`) instead of the encrypted tunnel IP.

****Root Cause:****

- **Bug 1 (Network Watcher Mismatch):**** In `scripts/watcher.sh`, the network change listener watched `State:/Network/Global/IPv4` changes via `scutil n.watch` but matched them against `\*n.state[\*SCEventUpdate\*]`. On macOS, the actual output is `changedKey [0] = State:/Network/Global/IPv4`. Because the pattern did not match, all network switches were silently ignored.
- **Bug 2 (Sudo Caching Background Crash):**** When the background WARP Watcher eventually detected the broken tunnel (after 6 failures/30s), it triggered the default nuclear `recover.sh` (with `POST\_WAKE=false`). Because there was no active TTY in the background context, the `sudo -v` caching check aborted instantly with a terminal required error. Due to `set -e`, `recover.sh` died immediately before resetting DNS, disabling the packet-filter killswitch, or stopping containers, leaving the host network completely sinkholed.
- **Bug 3 (Post-Wake Infinite Loop):**** Once the watcher patterns were fixed, `recover.sh --post-wake` triggered successfully on network changes. However, because the script deleted the default routing entries at the start, spent ~15 seconds rebuilding 88 DERP relay bypass routes, and re-added the default routes at the end, this execution time exceeded the watcher's 10-second debounce threshold. Furthermore, the watcher recorded the `\*start\*` timestamp in the debounce file. Consequently, the route changes made by the script itself triggered the watcher again immediately after completion, trapping the system in an infinite loop of recovery runs. During this loop, the VPN routes were constantly deleted, resulting in the host leaking traffic through the real ISP interface (an IP starting with `132.`) for 15s out of every cycle.

****Resolution:****

- **Fixed Watcher Matching:**** Updated `scripts/watcher.sh` to match `\*changedKey\*` and `\*State:/Network/Global/IPv4` to correctly catch network changes.
- **Bypassed Background Sudo:**** Added `--auto`/`--non-interactive` flags to `recover.sh` and added a TTY check `[ -t 0 ]` to skip interactive `sudo -v` authentication when running headlessly in the background. Updated

```

the WARP Watcher call in `toggle.sh` to pass `--auto`.
1195 - **Prevented Infinite Loops:** Modified `scripts/watcher.sh` to write the **completion** timestamp of `recover
.sh` to `DEBOUNCE_FILE` (instead of only the start timestamp). This ensures all buffered network config
events generated during route reconstruction are evaluated against the end-of-run timestamp and
successfully debounced, breaking the infinite loop.
1196 - **Centralized & Timestamped Logs:** Redirected `watcher.sh` stdout/stderr from `/tmp/nullexit-watcher.log` to
the repository's main `output.log` and updated `scripts/common.sh`'s `step`, `ok`, `warn`, `fail`, and `
die` helpers to prefix logs with a local `[YYYY-MM-DD HH:MM:SS]` timestamp.
1197
1198 ##### 15.5.3 Incident Post-Mortem: Startup Pre-Flight Check Tailscale Race (July 10, 2026)
1199 **Symptom:** Immediately after startup via `toggle.sh`, the pre-flight connectivity check `[1/3] Gateway
reachable via Tailscale` returned `FAIL`, forcing `toggle.sh` to skip the full exit-node activation and
fall back to SOCKS5/local DNS proxy mode.
1200
1201 **Root Cause:** A timing race condition during startup. In `toggle.sh`, the host joins the mesh using `
tailscale up --reset` (Phase A) right before performing the pre-flight check. Because `tailscale up` resets
and reconfigures the host's `tailscaled` daemon, the local daemon must fetch the latest netmap
asynchronously from the coordination servers. This network map propagation and route establishment takes a
few seconds. Running `tailscale ping` immediately after `tailscale up` returns (with no delay) causes the
check to fail because the local daemon has not yet discovered the gateway node `100.100.21.8`.
1202
1203 **Fix (July 10, 2026):** Upgraded pre-flight Check 1 in both `toggle.sh` and `scripts/toggle-linux.sh` to use a
5-attempt retry loop with a 1-second delay between checks. This allows up to 5 seconds for local tailnet
routing paths to settle, ensuring the gateway is correctly reached and a pong is received before deciding
whether to enable exit-node routing.
1204
1205 ##### 15.5.4 Incident Post-Mortem: Pre-Flight Check False Negatives due to VPN Firewall STUN Blocks (July 10,
2026)
1206 **Symptom:** Immediately after a cold boot or full restart of the gateway, the pre-flight connectivity check
`[1/3] Gateway reachable via Tailscale` failed, causing `toggle.sh` to skip the full exit-node
configuration and fall back to SOCKS5/local DNS proxy mode. However, manually running `tailscale ping
100.100.21.8` immediately after startup succeeded in 1ms.
1207
1208 **Root Cause:** An asynchronous Tailscale handshake delay caused by firewall restrictions. Inside the container
network namespace, the `warp` container (gluetun) implements a strict firewall that blocks all outbound
UDP traffic to the public internet except to the designated WireGuard endpoint. Because of this, the `
tailscale` container's STUN requests to public STUN servers are blocked, causing the local daemon to report
`UDP is blocked, trying HTTPS`. Without UDP STUN capability, Tailscale is forced to fall back to a slower
DERP-assisted handshake (relayed via HTTPS/TCP). This negotiation and direct-path punch-through takes
roughly 30 to 35 seconds to establish on cold boots (after a full `tailscale up --reset`). Since the pre-
flight check only waited 15 seconds (15 attempts with 1-second sleeps), the check timed out and aborted
before the path completed.
1209
1210 **Fix (July 10, 2026):** Increased the pre-flight ping retry limit from 15 to 45 attempts (45 seconds) in both
`toggle.sh` and `scripts/toggle-linux.sh`. This provides sufficient time for the DERP-assisted handshake to
complete on cold boots, while still passing immediately (in 1-2 seconds) on warm starts or once the path
is established.
1211
1212 ##### 15.5.5 Incident Post-Mortem: Docker Image Build DNS Failures on Immediate Restart (July 10, 2026)
1213 **Symptom:** When executing `toggle.sh --restart`, the script successfully stops the gateway but then
immediately fails during startup during `docker compose build` with DNS resolution errors:
1214 `failed to do request: Head "https://registry-1.docker.io/...": dial tcp: lookup registry-1.docker.io on
192.168.5.1:53: no such host`.
1215
1216 **Root Cause:** A race condition between physical link negotiation and virtual machine startup. The STOP path
of the restart command invokes `cleanup_network_state` which power-cycles the host's physical Wi-Fi
interface (`en0`) to flush stale routing states. Immediately after, `toggle.sh` starts the gateway boot
sequence. Because `toggle.sh` did not verify the status of the physical interface, it proceeded to boot
Colima and build Docker containers while `en0` was still negotiating a DHCP lease. Consequently, the host (
and by extension, the VM bridge) had no active internet gateway/DNS path, causing Docker's registry check
to fail.
1217
1218 **Fix (July 10, 2026):**
1219 1. Created a unified `wait_for_dhcp_settle` helper function in `scripts/common.sh` that polls `ipconfig
getpacket` and `ifconfig` until a valid IP and router are assigned. To handle typical macOS Wi-Fi
reassociation and DHCP negotiation latency (which commonly takes 15-20 seconds after a power-cycle), this
loop was configured to poll up to 60 times (30 seconds total).
1220 2. Injected a call to `wait_for_dhcp_settle` at the beginning of the `START` action in `toggle.sh` (right
before checking Colima status) to block container builds until the host's physical network is online.
1221 3. Refactored `recover.sh` to reuse the unified `wait_for_dhcp_settle` function.
1222
1223 ##### 15.5.6 Watcher.sh Suppressed Exit Code Bug (Fixed)
1224 **Observation:** The `scripts/watcher.sh` script is a long-running daemon that invokes `recover.sh --post-wake`
when it detects system wake or network roam events. Previously, it contained the following bug:
1225 ```bash
1226 bash "$RECOVER" --post-wake
1227 local exit_code=$?
1228 ...
1229 return $exit_code || true
1230 ```

```

```

1231 The `|| true` mistakenly swallowed the actual `$exit_code` returned by `recover.sh`, causing `run_recover()` to
    always return `0` (success), masking any underlying failures in the wake recovery process.
1232
1233 **Fix:** The `|| true` statement was removed from the return line:
1234 ```bash
1235     return $exit_code
1236 ```
1237 Since `watcher.sh` explicitly omits `set -e` (to prevent pipe breakages from killing the daemon), removing `||
    true` safely allows the underlying exit code to propagate without risking daemon termination. The watcher
    daemon was then reloaded (`launchctl kickstart -k`) to pick up the script changes in memory.
1238
1239 #### 15.5.7 Network Readiness Race Condition on `--restart` (Fixed)
1240 **Problem:** When running `./toggle.sh --restart`, the script tears down the gateway and immediately begins the
    startup sequence. On systems with Wi-Fi (or any wireless interface), the OS takes a moment to re-associate
    with the network after the teardown's DNS/routing changes. If the startup sequence ran before the OS had a
    default route, `get_active_service` would return nothing, routing bypass targets would be wrong, and DNS/
    WARP setup would silently fail resulting in a partially configured gateway that self-heals only once the
    `watcher.sh` re-runs.
1241
1242 **Root cause:** The original code had no gate before the first network-dependent call (`ACTIVE_SERVICE=$(
    get_active_service)` at the top of the start branch). This was a pure timing race.
1243
1244 **First fix (Wi-Fi specific):** A polling loop on `ipconfig getifaddr en0` was added to wait for an IPv4
    address on `en0` before proceeding. This worked but was brittle it only handled Wi-Fi and would fail for
    Ethernet, USB-C adapters, or iPhone USB tethering.
1245
1246 **Generalized fix:** Replaced the `en0` check with `route get default | grep 'gateway:'`. This detects a valid
    default gateway on any interface, covering all connection types. The loop polls every 2 seconds with a
    60-second hard timeout (then proceeds with a warning rather than hanging forever). The output prints the
    detected gateway IP so failures are easy to diagnose in logs.
1247
1248 ```bash
1249 # In toggle.sh, before ACTIVE_SERVICE=$(get_active_service)
1250 while true; do
1251     if route get default 2>/dev/null | grep -q 'gateway: '; then
1252         _gw=$(route get default 2>/dev/null | awk '/gateway:/{print $2}')
1253         echo " Network ready (default gateway: $_gw)."
1254         break
1255     fi
1256     ...
1257 done
1258 ```
1259
1260 **Why not ping?** Pinging an IP (e.g. `1.1.1.1`) during restart is unreliable because DNS may still be hijacked
    from the previous session, and the exit node routing may not yet be cleared, causing the ping to loop back
    through the tunnel. `route get default` is a pure local kernel query no packets sent.
1261
1262 ### 15.6 WARP Tunnel Liveness & Host Leaks
1263
1264 #### 15.6.1 Host-Side Traffic Leaking Past the Exit Node (with Remote Clients Routing Correctly)
1265 **Symptom:** `toggle.sh` reports success. Remote tailnet clients (e.g. an S24 phone) correctly egress through
    Cloudflare WARP and see a Cloudflare IP on `whatismyip.com`. But on the **HOST Mac running the gateway,
    `whatismyip.com` reports the underlying physical ISP's ASN (e.g. `campus_isp` for a university campus Wi-Fi).
    The gateway container itself is healthy; the leak is specifically on the host.
1266
1267 **Root causes:** Three distinct mechanisms can each produce this exact symptom on the host alone, while leaving
    remote clients unaffected. They are mutually rankable so the diagnostic picks the active one
    deterministically.
1268
1269 * **(A) SOCKS5 fallback lane active.** `toggle.sh`'s three Phase-B pre-flight checks (`tailscale ping` AdGuard
    via `dig +tcp` WARP internet, `toggle.sh` -lines 750-820) sometimes flake during the first minute. When ANY
    of the three fails, the script sets `SKIP_EXIT_NODE=true` (toggle.sh:849) and instead activates the SOCKS5
    fallback path (toggle.sh:894). The SOCKS5 fallback hijacks DNS to `127.0.0.1` (so AdGuard filtering still
    works) but **modern browsers on macOS do NOT honor the system SOCKS5 proxy** they ignore `networksetup -
    setsocksfirewallproxy`. Result: DNS gets ad-filtered but actual HTTP/S traffic exits direct through the
    campus ISP. This branch is invisible from a remote tailnet client because the client's tunnel is unaffected
    .
1270
1271 * **(B) IPv6 leak over campus Wi-Fi.** The Tailscale exit node and WARP tunnel are both IPv4-only (gluetun +
    `post-rules.txt` drop all IPv6 forwarded traffic; see 15.12 IPv6 Exit-Node Leak). But many university and
    enterprise APs are dual-stack the host happily accepts AAAA DNS responses and sends IPv6 traffic straight
    out `en0`. macOS happily encrypts that traffic through Tailscale ONLY if Tailscale has IPv6 advertised;
    with an IPv4-only exit node, IPv6 traffic skips Tailscale entirely. This branch is invisible from a phone
    because iOS/Android Tailscale apps actively sinkhole IPv6 in their VPN profile, but the macOS host's
    `tailscaled` does not.
1272
1273 * **(C) Route-table freeze and `--accept-routes` reset after `tailscaled` wake/toggle.** Running `tailscale up
    --reset` clears the exit-node preference and reverts `--accept-routes` back to its default (`false`).
    Without explicitly passing `--accept-routes=true`, `tailscaled` will not attempt to install the default
    route through the `utun` tunnel interface even if the exit node preference is reconciled. Additionally,
    macOS occasionally freezes and fails to apply route-table changes (see 15.11 Standalone Tailscale Daemon

```



```

Freeze). In both cases, `netstat -rn` shows the default route still pointing to the physical Wi-Fi gateway
(e.g. `172.17.0.1`) instead of the Tailscale `utun*` interface, causing host traffic to exit direct while
remote clients route correctly.
1274
1275 **Why remote clients don't see this.** Each remote phone/laptop uses its OWN Tailscale tunnel to reach the
container's `100.x.x.x:443` over the mesh they're end-to-end encrypted regardless of how the host Mac
itself is configured. Only the HOST's own egress sockets (HTTP requests from the host's browser, curl, etc
.) are affected.
1276
1277 **Resolution.** A single script: `scripts/diagnose-host-leak.sh`. It runs the 8 checks, classifies into one of
A / B / C / OK, prints a verdict + a ready-to-run fix command on stdout, AND writes the full report to `
host-leak-diagnostic-<UTC>.txt` so the user can paste the file back instead of scrolling-and-copying from
the terminal. With `--fix`, the matched remediation is applied and the two checks that could change (warp +
ipv6) are re-verified. With `--watch` (or `--watch 30`), it runs the full baseline diagnostic then enters
a continuous loop checking warp/IPv6/default-route every N seconds, alerting only on state changes logs to
`host-leak-watch.log`.
1278
1279 ```bash
1280 # Diagnose only:
1281 bash scripts/diagnose-host-leak.sh
1282
1283 # Diagnose + apply matched fix + re-verify:
1284 bash scripts/diagnose-host-leak.sh --fix
1285 ```
1286
1287 **What it prints.** A 7-section report (`tailscale status`, SOCKS5 state, `cdn-cgi/trace` warp=, IPv6 leak
probe, host default route, last toggle.sh output.log hints, resolved gateway Tailscale IP), followed by a
classification block (`Scenario: A | B | C | OK`) with the exact command(s) the user needs to paste into
their terminal they don't have to figure out which `tailscale up` flags match their environment.
1288
1289 **What it does NOT do.** It does not touch `toggle.sh` or `toggle.sh`'s pre-flight logic, does not modify `.env`
(except `--fix` mode appends `GATEWAY_BYPASS_PING=true` when fixing Scenario A), and does not restart
Docker. It is a read-only diagnostic with an opt-in remediation flag. Safe to run on a healthy gateway the
worst case is a 30-line report that says "OK".
1290
1291 **Why cdn-cgi/trace over whatismyip.com for the verdict.** `whatismyip.com`'s ISP database routinely mislabels
campus IPv6 ranges as residential ISPs (that's why the local ISP's ASN appears even when you're routing
through Cloudflare). `cdn-cgi/trace` is hosted by Cloudflare ITSELF and reports `warp=on|off` plus the
exact edge colo serving the request. It is the only public endpoint that can truthfully confirm whether the
WARP tunnel is in use.
1292
1293 **Common false alarms.**
1294
1295 * **`warp=on` but whatismyip.com still shows the local ISP.** `whatismyip.com`'s ISP-detection database is
unreliable on academic networks. Trust `cdn-cgi/trace`'s `warp=on` over `whatismyip.com`'s ISP string. Use
`https://api.ipify.org` (returns just the IP, no ISP DB lookup) as a third confirmatory check.
1296
1297 * **`tailscale status` shows the host online but exit node preference is missing.** `tailscale up --reset --
exit-node=` (empty value) clears any stale preference; `tailscale up --reset --exit-node=<100.x.x.x>` re-
establishes it. The diagnostic prints the exact command with the resolved TS_IP filled in.
1298
1299 * **Both IPv6 leak AND route-freeze flags set simultaneously.** Scenario B outranks Scenario C fixing IPv6
makes IPv4 the only viable egress, after which the route-freeze usually self-resolves within ~30s (or one
more `toggle.sh` cycle).
1300
1301 **Deep dive: Host Leak Probe Continuous Sub-Second Egress Monitor (`scripts/host-leak-probe.sh`).**
1302
1303 **What it is.** `scripts/host-leak-probe.sh` is a lightweight background daemon (~1.4 MB RSS, ~01% CPU) that
polls `https://www.cloudflare.com/cdn-cgi/trace` every 300ms directly from the host via `curl` not via `
docker compose exec`. This is a fundamentally different vantage point from the WARP Watcher (15.6.2): the
WARP Watcher asks the WARP *container* whether it sees `warp=on`; the Host Leak Probe asks what a real
browser request leaving the host's physical NIC looks like. The two blind spots it covers that the WARP
Watcher cannot:
1304
1305 1. **Host-side flash-leaks** a Cloudflare anycast edge re-route, a browser reusing a stale pooled connection
across a tunnel state change, or a split-second routing gap during a Gluetun healthcheck restart (see
15.12.13 Routing Fix 30-second polling acknowledged 14s window).
1306
1307 2. **Host egress while container is healthy** the container can report `warp=on` while the host's own traffic
exits direct (the three-scenario leak class documented above in 15.6.1).
1308
1309 **Lifecycle.* Started by `toggle.sh`'s START path (`start_host_leak_probe`) immediately after `
start_warp_watcher`. Controlled by:
1310
1311 | Signal path | What happens |
1312 |---|---|
1313 | `toggle.sh` STOP | `stop_host_leak_probe()` SIGTERM + PID file cleanup |
1314 | `toggle.sh` `cleanup_handler` (error/INT/TERM/HUP) | Same |
1315 | `recover.sh` (default, non `--post-wake`) | 6d block kill by PID file |
1316 | `recover.sh --post-wake` | Left running the gateway is still live |
1317
1318 PID file: `/tmp/nullexit-host-leak-probe.pid`. Toggle with `HOST_LEAK_PROBE=false` in `.env` to disable at next
gateway start.
1319

```



```

1317 *Output format.* All events are written to `output.log` (repo root) with UTC timestamps. Only state *changes*
      are logged the probe is silent during normal healthy operation.
1318
1319 ...
1320 [09:10:26] LEAK warp=off ip=45.23.1.1 prev_warp=on prev_ip=104.28.246.50
1321 [09:10:26] ROTATE warp=on ip=104.28.248.11 prev_ip=104.28.246.50
1322 [09:10:26] HOST-PROBE failed/timeout (count=1)
1323 ...
1324
1325 curl transport errors (`TLS handshake failed`, `Connection refused`, etc.) are also appended to `output.log`
      via `2>>output.log` nothing is silenced to `/dev/null`.
1326
1327 *Grep cheatsheet.*
1328
1329 ```bash
1330 grep LEAK output.log           # real warp=off events from the host
1331 grep ROTATE output.log         # Cloudflare anycast edge rotations (harmless)
1332 grep 'HOST-PROBE' output.log   # curl failures / timeouts
1333 ```
1334
1335 *Resource budget.* ~1.4 MB RSS, ~0% CPU at rest, ~3 HTTPS requests/second to Cloudflare. Safe to leave running
      for hours. Back off the polling interval (`bash scripts/host-leak-probe.sh 1.0`) if you observe probe-
      failure pile-ups indicating Cloudflare rate-limiting.
1336
1337 *Detection vs prevention.* This script detects leaks; it does not prevent them. A sub-300ms flash-leak can
      still slip between two polls undetected. If `grep LEAK output.log` shows confirmed leak events, the next
      step is a kill-switch (prevention) an `iptables` rule on the host's `en0` that drops non-Tailscale, non-
      local egress whenever the gateway is active. That is a separate engineering task not yet implemented.
1338
1339 ##### 15.6.2 In-Flight WARP Tunnel Liveness Monitor & Statistical Auto-Shutdown
1340 **Why.** nullexit's core promise is that your IP always appears as Cloudflare. If the WARP tunnel silently dies
      due to a UDP NAT timeout, a Cloudflare edge rotation, a Colima VM clock skew causing TLS cert rejection,
      or any of a dozen other failure modes the host silently falls back to egressing through the physical ISP.
      The user sees the gateway as "up" (containers running, Tailscale connected, DNS hijacked) but their real IP
      is exposed. This is the exact class of failure that Ingo Blechschmidt documented in his chilling 2024
      Linux kernel post-mortem: for over two years, LUKS disk encryption on suspend was silently broken because a
      refactored kernel syscall returned success while doing nothing ([source](https://mathstodon.xyz/@iblech
      /116769502749142438)). The lesson: **a security mechanism that fails silently is worse than no mechanism at
      all** it creates a false sense of safety that prevents the user from taking corrective action.
1341
1342 **Design principle.** The WARP Watcher (`start_warp_watcher()` in `toggle.sh`) is a background daemon that
      polls `cdn-cgi/trace` every 30 seconds and applies a **statistically calibrated threshold** before
      triggering any safety response. This threshold distinguishes transient network blips (which self-heal
      within seconds) from genuine outages (which require intervention). A single `warp=off` reading is
      meaningless Docker might be slow, the network might be congested, a container healthcheck might be
      restarting. But 6 consecutive off readings (30 continuous seconds of downtime) has vanishingly low
      probability of being a false positive: even at an unrealistically high 10% false-positive rate per poll, P
      (6 consecutive) = 0.1 0.0001%. In practice, the per-poll false-positive rate is far lower than 10%, making
      the real probability astronomically small.
1343
1344 **What it does.**
1345
1346 1. **Always logs.** Every state transition (onoff, offon) is timestamped to `output.log`. The user can `grep
      WARP output.log` to audit the tunnel's entire lifetime. This is the "never fail silently" mandate.
1347
1348 2. **Tracks consecutive failures.** A counter increments on each `off` reading and resets to 0 on any `on`
      reading. This ensures the watcher never fires on intermittent flapping.
1349
1350 3. **Auto-shuts down at threshold.** When the counter reaches `WARP_FAIL_THRESHOLD` (default 6, configurable in
      `.env`), the watcher declares a statistically significant outage and runs `recover.sh` the nuclear
      recovery script that disconnects Tailscale, stops Docker containers, resets DNS to `1.1.1.1`, flushes
      routes, and power-cycles Wi-Fi. This instantly reverts the host to normal (un-encrypted) internet,
      guaranteeing no traffic leaks through a dead tunnel while the user thinks they're protected. A trigger file
      at `/tmp/nullexit-warp-shutdown-triggered` prevents double-firing.
1351
1352 4. **Exits cleanly.** After shutdown, the watcher exits (the gateway is dead, there's nothing left to monitor).
      `stop_warp_watcher()` is also called by `toggle.sh`'s STOP path and `cleanup_handler` to terminate the
      daemon cleanly during normal teardown.
1353
1354 **Configuration.** Set `WARP_FAIL_THRESHOLD=3` in `.env` for a more aggressive 15s timeout, or `
      WARP_FAIL_THRESHOLD=12` for a conservative 60s window. The variable is parsed from `.env` using the same `
      grep` pattern as `GATEWAY_MSS` and `GATEWAY_BYPASS_PING`.
1355
1356 **Log sample.** After a real outage (or a forced test via `docker compose pause warp`):
1357
1358 ...
1359 [2026-07-03T21:15:17Z] WARP DOWN cdn-cgi/trace reports warp=off
1360 [2026-07-03T21:15:47Z] WARP SHUTDOWN 6 consecutive failures (threshold=6). Running recover.sh to kill the
      gateway and restore normal internet. Your IP is NO LONGER Cloudflare.
1361 [2026-07-03T21:15:52Z] WARP SHUTDOWN recover.sh completed, watcher exiting. Your IP is now your real ISP IP.
1362 ...

```

```

1363
1364 Or, if WARP self-recovers before the threshold:
1365
1366
1367 [2026-07-03T21:15:17Z] WARP DOWN   cdn-cgi/trace reports warp=off
1368 [2026-07-03T21:15:32Z] WARP RECOVERED   warp=on after 3 consecutive off readings (threshold was 6)
1369
1370
1371 **Relationship to other monitors.** The WARP Watcher is the **innermost safety layer** it runs as a child of `
toggle.sh` and only while the gateway is active. It complements (does not replace) the `scripts/watcher.sh`
daemon (15.5.1, which handles sleep/wake and Wi-Fi roam) and `scripts/diagnose-host-leak.sh` (15.6.1,
which is a manual diagnostic tool). The WARP Watcher is the only component that actively verifies end-to-
end tunnel liveness and takes autonomous action when it fails.
1372
1373 #### 15.6.3 Incident: The Overnight Silent IP Leak (July 2026)
1374 **The Problem:** The host leaked traffic over its raw, unencrypted `eth0` IP continuously for roughly 8 hours,
completely bypassing the VPN. The `toggle.sh` state and the container liveness checks all reported the
gateway as healthy and active.
1375
1376 **The Investigation:**
1377 1. **Sleep/Wake Checks:** We initially suspected macOS sleep cycles broke the routing table. A check of `pmset
-g log` confirmed the laptop literally never slept (0 sleep events recorded since boot), ruling out all
sleep/wake code paths.
1378 2. **Keepalive Expiry:** We discovered `docker-compose.yml` was missing `
WIREGUARD_PERSISTENT_KEEPA_LIVE_INTERVAL`. Without a keepalive, standard router NAT tables (which typically
garbage collect UDP after 30 to 120 seconds of silence) quietly expired the WireGuard port mapping
overnight.
1379 3. **Container State Masking:** Because WireGuard is stateless, the `warp` container didn't immediately crash.
It stayed "Up", which meant Docker's healthchecks and our `toggle.sh` liveness checks never noticed the
tunnel inside it was totally dead.
1380
1381 **The Solution:**
1382 1. Added `WIREGUARD_PERSISTENT_KEEPA_LIVE_INTERVAL=25s` (the WireGuard standard to beat standard 30s NAT
timeouts) to keep the UDP tunnel permanently pinned open through the router.
1383 2. Injected a **Fail-Closed Kill-Switch** into `routing-fix.sh`: A 30-second loop now actively verifies tunnel
health by running `curl` over `tun0` to Cloudflare's trace endpoint. If it fails 3 consecutive times, it
immediately rewrites the default route to `blackhole`, severing all traffic instead of letting it leak
through `eth0`.
1384 3. Added a listener in `watcher.sh` that detects this fail-closed event and instantly pushes a native macOS UI
notification to the desktop, alerting the user to manually intervene.
1385
1386 > [!NOTE]
1387 > **Why dual failure systems?**
1388 > The system now relies on two independent failure handlers that cover each other's blind spots:
1389 > 1. **Automated Self-Healing (WARP Watcher + `recover.sh`):** Triggered when the `warp` container logs `warp=
off`. This happens when the server actively kicks the client. Because the container *knows* it is
disconnected, it triggers `recover.sh` to safely reboot Docker and reconnect.
1390 > 2. **Fail-Closed Kill-Switch (`routing-fix.sh` + `watcher.sh`):** Triggered when a silent network failure
occurs (e.g., NAT timeouts). The container incorrectly believes it is connected (`warp=on`), so auto-
recovery never fires. Instead of auto-recovering (which risks falling back to raw Wi-Fi mid-process while
the user isn't looking), this kill-switch actively blackholes the internet and forces a manual intervention
via a desktop notification.
1391
1392 ### 15.7 Tailscale P2P, SNAT & Control Plane
1393
1394 #### 15.7.1 macOS SNAT Endpoint Poisoning & The P2P Blackhole (July 10, 2026)
1395 **Symptom:** When a Tailscale client (like an S24) connected to the exact same Wi-Fi network as the macOS
gateway, it completely lost internet connectivity. However, if the S24 connected to a Windows PC's hotspot
on the same network, the tunnel worked flawlessly.
1396
1397 **Root Cause:** Claude's critique correctly dismantled the port collision theory: the gateway daemon wasn't
failing to bind, and macOS wasn't load-balancing ports. The real culprit was **macOS SNAT Source Port
Mangling poisoning Tailscale's path discovery**.
1398
1399 Here is the exact step-by-step of the "infinite loop" blackhole:
1400 1. **The P2P Handshake:** The S24 and the Mac are on the same Wi-Fi (`192.168.1.x`). The S24 discovers the Mac's
local IP and sends a UDP hole-punch packet to `192.168.1.5:41641`. Colima successfully forwards this to
the gateway container.
1401 2. **The Bypass Rule:** The gateway replies. Because `routing-fix.sh` forces all Tailscale UDP traffic out `
eth0` to bypass WARP, the reply goes to the macOS host.
1402 3. **macOS SNAT Mangling:** macOS routes the reply out `en0` to the S24. Because the packet crosses from the
Colima VM bridge to the Wi-Fi interface, macOS applies Source NAT (SNAT). Critically, macOS **randomizes
the source port** (e.g., changes it from `41641` to `58291`).
1403 4. **Endpoint Poisoning:** The S24 receives the authenticated Tailscale packet from `192.168.1.5:58291`.
Tailscale's `magicsock` is designed to handle NAT dynamically, so it assumes the Mac has roamed to port
`58291`! The S24 updates its endpoint and starts sending all its encrypted internet traffic to
`192.168.1.5:58291`.
1404 5. **The Blackhole:** macOS has no port forward for `58291`. It drops 100% of the S24's outbound data packets.
The connection is completely dead.
1405

```

1406 ****Why did the Hotspot work?*** When the S24 connected to the PC hotspot, it was behind the PC's NAT. The S24 sent the packet, PC NATted it, and the Mac replied (mangled to `58291`). When the reply hit the PC, the PC's strict NAT dropped it because the source port (`58291`) didn't match the destination port it originally sent to (`41641`). Because the mangled packet was dropped, the S24 never poisoned its endpoint! It safely gave up on P2P, fell back to the 100% reliable TCP DERP relay, and the internet worked perfectly.

1407

1408 ****Fix & Resolution (July 10, 2026):***

1409

1410 ****Phase 1 Port disambiguation:*** Changed the gateway container's Tailscale listen port from the default `41641` to `41642` across `docker-compose.yml`, `post-rules.txt`, and `routing-fix.sh`. This prevents any bind conflict with the macOS host's native Tailscale daemon.

1411

1412 ****Phase 2 RFC1918 DROP rules:*** Updated `routing-fix.sh`'s `add\_tailscale\_p2p\_bypass()` to explicitly drop all outgoing Tailscale UDP packets destined for private subnets (`192.168.0.0/16`, `10.0.0.0/8`, `172.16.0.0/12`) when `TAILSCALE\_ALLOW\_LAN\_P2P=false`. This surgically kills the local P2P hole-punch attempt *before* it can trigger the macOS goproxy SNAT mangling. The S24 receives a clean failure and immediately locks onto the public DERP relay with 0% packet loss.

1413

1414 ****Phase 3 Automatic network detection:*** Because `TAILSCALE\_ALLOW\_LAN\_P2P=false` breaks P2P on trusted hotspots that genuinely don't have AP isolation, a `.env` toggle and a full auto-detection system were implemented:

1415

1416 - ****`.env` toggle:*** `TAILSCALE\_ALLOW\_LAN\_P2P=true/false` static fallback, used only if auto-detection cannot run.

1417 - ****`watcher.sh` auto-detection (`detect\_lan\_p2p\_mode`):*** On every network change, `watcher.sh` runs two checks:

1418 1. ****WPA2-Enterprise detection:*** `airport -I` is used to read the current Wi-Fi `link auth` field. If the auth type does not contain `-psk` (e.g., it is plain `wpa2` = 802.1x), the network is classified as enterprise and P2P is forced off. *(Note: the `airport` binary was removed in macOS Sonoma 14.4 the detection now uses `system\_profiler SPAirPortDataType`; see 15.11 for the full migration.)*

1419 2. ****AP Isolation probe:*** For WPA2-Personal networks, a short `ping` is sent to the default gateway. If the gateway responds (it always should on a real home/hotspot network), P2P is allowed. If not, AP isolation is inferred and P2P is forced off.

1420 - The result (`true` or `false`) is written to `.lan\_p2p\_detected` in the repo root.

1421 - ****`routing-fix.sh` dynamic reading:*** `add\_tailscale\_p2p\_bypass()` re-reads `.lan\_p2p\_detected` on every 30-second loop iteration and atomically adds or removes the RFC1918 DROP rules. ****No container restart is needed when switching networks.****

1422

1423 ****Known Unknowns (Pending Verification):***

1424 - The goproxy SNAT source port mangling theory is mechanistically sound and backed by observed behavior (goproxy's UDP proxy changelog has documented edge cases in this exact code path), but has not yet been confirmed via `tcpdump -i en0 udp portrange 40000-60000` while triggering a P2P handshake from the phone. A packet capture cross-referenced with `tailscale ping` endpoint reports on the phone side would confirm the theory definitively.

1425 - Claude's recommendation: run `tcpdump -i en0 udp port 41642 or portrange 40000-60000` on the Mac while triggering the handshake from the phone to see whether the reply leaves with a different source port than `41642`.

1426 - ****Client-Side VPN Cycling (Roaming Recovery):*** An observation (July 10, 2026) suggests that when a hung DNS probe state is triggered by roaming between Access Points (APs) on a WPA2-Enterprise network, simply toggling the VPN connection (Tailscale) off and on locally on the client phone immediately resolves the issue. You do not need to cycle the phone's physical Wi-Fi. This reinforces the theory that roaming on Enterprise networks leaves a stale/poisoned P2P endpoint in the phone's Tailscale magicsock state, which gets instantly flushed upon a local VPN restart. (Status: Possible but not formally verified).

1427

1428 **#### 15.7.2 July 6, 2026: Packet Filter (pfctl) Defaulting Local LAN Rules to TCP-Only**

1429 ****Symptom:*** Devices on the exact same local Wi-Fi network as the gateway were experiencing ~500ms latency to the gateway instead of the expected ~80ms.

1430

1431 ****Root Cause:*** When writing the `pf.conf` rules to whitelist local LAN traffic (`pass out quick on \$ext\_if to { 192.168.0.0/16, ... }`), the rule omitted an explicit protocol. The macOS `pfctl` compiler implicitly applies state tracking to all `pass` rules. However, because state tracking (`keep state`) natively evaluates TCP sessions, `pfctl` automatically appended the `flags S/SA` modifier to the compiled rule in the kernel. This silently restricted the whitelist to ****TCP only****, causing all local UDP traffic to be dropped by the default-deny kill-switch. Since Tailscale relies on UDP port 41641 for direct P2P connections, the local devices were unable to hole-punch and were forced to fall back to routing traffic through a remote Tailscale DERP relay, massively inflating latency.

1432

1433 ****Resolution:*** Split the single implicit rule into explicit `proto tcp`, `proto udp`, and `proto icmp` rules in `scripts/pf.conf` to guarantee the macOS compiler allows all three core transport protocols on the local subnet without mutating the rule into a TCP-only lock.

1434

1435 **#### 15.7.3 Incident Post-Mortem: PF Kill-Switch Bricks Host Internet in SOCKS5 Fallback Mode (July 10, 2026)**

1436 ****Symptom:*** When the pre-flight checks fail (e.g. because host Tailscale was unauthenticated/logged out), the script falls back to SOCKS5 proxy mode but the host immediately loses all internet connectivity and tailscaled reports `You are logged out... connect: no route to host`. Even running `sudo tailscale up` to log in fails because the connection to the control plane times out.

1437

1438 ****Root Cause:*** An architectural mismatch. The macOS Packet Filter (PF) Kill-Switch works at the IP level: it blocks all outbound traffic on the physical interface (`en0`) except to explicitly whitelisted VPN endpoints, expecting all other traffic to go through the virtual VPN interface (`utun\*). In SOCKS5 fallback mode, the exit-node (`utun*) is NOT enabled; instead, only specific applications (like browsers)

use the localhost SOCKS5 proxy, while all other host traffic (including the `tailscaled` daemon's UDP packets) continues to go through `en0`. Because the script still enabled the PF Kill-Switch during SOCKS5 fallback, it blocked all non-SOCKS5 traffic on `en0`, completely bricking the host's internet and locking the Tailscale daemon out of the control plane.

1439

1440 ****Fix (July 10, 2026):**** Modified `toggle.sh` to remove the `enable\_killswitch` call from the SOCKS5 fallback path. The firewall kill-switch is now strictly reserved for exit-node VPN mode, ensuring SOCKS5 fallback leaves the host's direct internet route open for system daemons to authenticate.

1441

1442 **#### 15.7.4 Incident Post-Mortem: Host Tailscale Logouts due to Aggressive reset Flags (July 10, 2026)**

1443 ****Symptom:**** Every time the user runs `toggle.sh` or `recover.sh` which disconnects/reconnects the host client, the host client randomly gets logged out, forcing them to re-run `sudo tailscale up` to authenticate.

1444

1445 ****Root Cause:**** An overly aggressive configuration reset. The host-side `tailscale up` calls (used in `disconnect\_tailscale\_host` and the startup/enable exit node paths) all passed the `--reset` flag. On macOS, `--reset` resets the entire configuration state and authentication token cache of the daemon. Since the script does not pass a `TS\_AUTHKEY` to the host's commands (which are meant for the user's private interactive client), the `--reset` flag effectively logged the user out of their tailnet, requiring interactive re-login.

1446

1447 ****Fix (July 10, 2026):**** Removed the `--reset` flag from all host-side `tailscale up` invocations in `toggle.sh` and `scripts/common.sh`. This ensures that exit-node state transitions (enabling/disabling) are handled safely while preserving the host client's authenticated session.

1448

1449 **#### 15.7.5 Incident Post-Mortem: macOS Tailscale Flag Requirement Abort (July 10, 2026)**

1450 ****Symptom:**** When the `toggle.sh` stop trap or the roaming watcher triggered, the gateway failed to shut down or disconnect properly. The `output.log` showed the error: `Error: changing settings via 'tailscale up' requires mentioning all non-default flags. To proceed, either re-run your command with --reset...`

1451

1452 ****Root Cause:**** In a previous attempt to stop host-side logouts (15.7.4), we removed the `--reset` flag from all `tailscale up` invocations. However, if the Tailscale daemon on macOS is already running with non-default flags (like `--ssh` or `--accept-routes`), invoking `tailscale up` with only a subset of flags (like `--exit-node=`) triggers a strict safety check in the Tailscale CLI that aborts the command completely. Because the command aborted, exit-node routing remained stuck.

1453

1454 ****Fix (July 10, 2026):**** Migrated all specific preference changes in `toggle.sh` and `scripts/common.sh` from `tailscale up` to a two-step sequence:

1455 1. `tailscale up` (with no flags) to safely ensure the interface is brought up using the current authenticated state.

1456 2. `tailscale set --exit-node=...` to modify the specific target preference cleanly without triggering the missing flags error or requiring `--reset`.

1457

1458 **### 15.8 Docker / Colima Lifecycle**

1459

1460 **#### 15.8.1 Colima VM Memory / Swap Thrashing Latency**

1461 ****Symptom:**** After running nullexit flawlessly for over 24 hours straight to test stability, the 0.5GB VM RAM configuration began experiencing severe network latency on the Tailscale mesh later in the day.

1462

1463 ****Root Cause:**** Services (like AdGuard, Tailscale, Docker logs) slowly accumulate cache and state over extended periods. Under the tight 512MB physical RAM limit, this slow creep forced the Linux kernel to aggressively swap memory to the 512MB SSD swap file. The constant disk I/O thrashing blocked process execution, causing DNS resolutions and packet forwarding to delay significantly (making the internet feel "very slow").

1464

1465 ****Proof (Empirical Observation):**** While in this OOM-thrashing state, direct DNS queries through the Tailscale mesh (`dig @100.100.21.8 google.com`) would intermittently hang or take several seconds to resolve. After restarting with the 600MB configuration, the exact same query immediately dropped to a lightning-fast **41 ms** response time. (Note: Querying the mapped host port via `127.0.0.1:5354` will always time out due to Gluetun's strict leak-prevention firewall blocking non-VPN ingress traffic).

1466

1467 ****Fix:**** Increased VM base physical RAM to 600MB (`--memory 0.6`) and reduced the swap file to 400MB. This provides enough native RAM headroom for long-running state caches without forcing heavy I/O swapping. Subdomain deduplication still reduces blocklists by ~60%. Memory profiles (`light`/`medium`/`heavy`) let users trade off coverage for memory.

1468

1469 **#### 15.8.2 Missing iptables in routing-fix Container**

1470 ****Root Cause:**** `alpine:3.20` does not have iptables installed. Commands failed silently with `2>/dev/null || true`.

1471

1472 ****Fix:**** Updated `docker-compose.yml` to `apk add --no-cache iptables iproute2` before `scripts/routing-fix.sh`.

1473

1474 **#### 15.8.3 DOCKER_GW Variable Parsing Bug**

1475 ****Root Cause:**** `ip route show default` printed two lines; `awk '{print \$3}'` picked up both, causing route commands to fail.

1476

1477 ****Fix:**** Added `head -1` to the parsing chain.

1478

1479 **#### 15.8.4 Hard Reboots Leave Stale Docker Socket in Colima VM**

1480 ****Problem:**** If the macOS host machine is subjected to a hard reboot, sudden power loss, or a kernel panic, the Colima VM running in the background is abruptly killed. Upon next boot, running `toggle.sh` resulted in the contradictory output:

1481 `Colima is already running.` followed by `Cannot connect to the Docker daemon at unix:///~/.colima/default/
1482 docker.sock.`

1483 ****Root Cause:**** The hard crash prevents the inner Docker daemon from executing its graceful shutdown routines.
It leaves behind a corrupted `docker.sock` file inside the VM. On boot, the outer VM spins up (hence "
1484 already running"), but the inner Docker daemon sees the stale socket, assumes another instance is running,
and crashes.

1485 ****Fix:**** Added an Auto-Healing routine directly into `toggle.sh` (Step 4b). The script now explicitly tests the
Docker daemon with `docker ps`. If the daemon is unresponsive despite Colima reporting as active, it
assumes a stale socket and automatically runs `colima restart` in the background to cleanly rebuild the VM
and socket before proceeding.

1486 **#### 15.8.5 Cloudflare WARP 24-Second Startup Delay (UDP Session Rate-Limiting)**

1487 ****Context:**** When running `toggle.sh` to turn the gateway ON, `docker compose up -d` often hangs for exactly
1488 24-25 seconds waiting for the `warp` container to become `healthy`. Users might mistakenly blame the Go
`rule-compiler` processing 400k+ rules for this delay, as Docker prints `rule-compiler Exited 24.3s` at the
end of the wait.

1489 ****Root Cause:**** The `rule-compiler` actually finishes its work in <2 seconds. The 24-second blockage is
entirely due to Cloudflare WARP's edge server rate-limiting. WireGuard is a stateless protocol with no "
disconnect" packet. When the toggle is turned OFF, the old UDP session state remains active on Cloudflare's
backend for 20-30 seconds. When the toggle is instantly turned back ON, Docker starts a new WireGuard
connection from a new ephemeral UDP source port but uses the **same static cryptographic key** (from `.env`
). Cloudflare detects this as a potential replay attack or key-sharing abuse, and intentionally drops all
packets (rate-limits) for the new connection until the old session's ghost state fully expires (~20-25
seconds).

1490 ****Result:**** Gluetun's healthcheck fails repeatedly every 2 seconds until Cloudflare lifts the penalty box, at
which point traffic flows and Docker unblocks the rest of the startup sequence. This is a known, expected
behavior of reusing static keys rapidly on Cloudflare's network and cannot be bypassed.

1491 **#### 15.8.6 July 4, 2026: Multi-stage Docker Builds for Go Ports & Teardown Hang Fix**

1492 ****Symptom:**** The Python implementations of `logger.py` and `sync-rules.py` were slow and required a heavy
1493 python runtime inside Alpine containers. Also, `routing-fix` container teardown was hanging for 30s during
`toggle.sh` shutdown.

1494 ****Root Cause:****

1495 - Python is slower than Go and installing it dynamically via `apk add` added overhead and complexity to the
containers.

1496 - Docker containers ignore `SIGTERM` if the init process doesn't handle it, causing a full 30s timeout on
shutdown.

1497 ****Resolution:****

1498 - Ported `logger` and `sync-rules` to Go using Goroutines for massive concurrent speedups.

1499 - Implemented ****Multi-stage Docker builds**** (using `golang:1.22-alpine` as builder) to compile the static
binaries inside Docker. The final containers are raw Alpine with zero dependencies.

1500 - Added `--build` to `docker compose up -d` in `toggle.sh` to ensure updates are consistently applied on boot.

1501 - Added `stop\_grace\_period: 1s` to `routing-fix` in `docker-compose.yml` which eliminates the 30-second
teardown hang instantly.

1502 - Implemented a cryptographic integrity checker (`scripts/crypto.sh`) using HMAC-SHA256 and `NULLEXIT\_SEED` in
.env` to prevent tampering of bash scripts. (See also 15.9.4.)

1503 **#### 15.8.7 Colima `start` Post-Provision Hang (~2 min) & the Docker-Readiness Poll (July 14, 2026)**

1504 ****Symptom:**** `toggle.sh` took ~197s of wall-clock time to bring the gateway up, and almost the entire delay
1505 lived in a single step: `colima start`. Timestamps in `output.log` showed the VM reaching `READY` and
creating the `colima` Docker context in ~8-12 seconds, after which `colima start` simply ***did not return***
for ~110 more seconds until the `run\_with\_timeout 120` watchdog `SIGKILL`ed it. The rest of the boot (
containers, routing) then proceeded normally.

1506 ****Root Cause:**** A colima-internal hang that occurs ***after*** `Successfully created context colima` is printed. It
is ****mode-independent****. It was initially misattributed to `--network-address` / `--network-mode bridged`
needing the `socket\_vmnet` helper, but a controlled test with plain user-mode networking (`colima start --
memory 0.6 --vm-type vz`, no `--network-address`) reproduced the identical ~120s hang (04:40:34 04:42:34,
watchdog-killed) while Docker was fully usable the entire time. Crucially, killing the hung `colima start`
process does ****not**** stop the VM and does ****not**** corrupt state `colima status` correctly reports `running`
afterwards and `docker` keeps working via the `colima` context. The ~110s was therefore pure dead time
waiting on a CLI that had already done its real work.

1508 ****Fix:**** `scripts/common.sh` now provides `colima\_start\_until\_ready()`. It launches `colima start "\$@"` in the
1509 background and polls `docker --context colima info` every 3s. The moment Docker answers (~12s in practice)
it stops waiting, reaps the (usually still-hung) starter with `kill`, runs `docker context use colima` to
guarantee the default context is set, and returns 0. If Docker never answers within a 90s cap it returns
non-zero and `toggle.sh` continues to the existing Step 4b `docker ps` auto-heal (15.8.4). Both Colima
start call sites in `toggle.sh` the LAN-P2P bridged/shared path and the user-mode fast path now go
through this helper instead of `run\_with\_timeout 120 colima start`.

1510 ****Result:**** The Colima phase dropped from ~120s to ~12s and total `toggle.sh --restart` wall time from ~197s to
1511 ~106s, with the double-tunnel and direct hostcontainer P2P confirmed intact after the change. The key
insight: treat "Docker is reachable" not "the `colima start` CLI returned" as the definition of ***started***.

1512 **### 15.9 Permissions, Crypto & Security Hardening**

1513 **#### 15.9.1 Sudo Credential Caching Removed**


```

1516 **Problem:** The script used `sudo -v` at startup to cache sudo credentials, plus a background `SUDO_KEEPER_PID`
    loop refreshing them every 60 seconds. This required interactive password entry even with NOPASSWD
    sudoers configured, because `sudo -v` itself prompts.
1517
1518 **Fix:** Removed the entire `sudo -v` + `SUDO_KEEPER_PID` section. All privileged commands now use `sudo -n` (
    non-interactive), which works silently because the user has NOPASSWD rules in `/etc/sudoers.d/nullexit` for
    the specific commands needed (`networksetup`, `python3`, `dscacheutil`, `killall`, `pkill`, `route`, `
    ifconfig`).
1519
1520 ##### 15.9.2 Background Sudo Failures
1521 **Context:** The `--post-wake` script is intentionally designed to skip the `sudo -v` interactive prompt
    because it is launched by `watcher.sh` running via `launchd` in the background (which has no TTY and cannot
    accept password input). Thus, it relies entirely on `sudo -n` (non-interactive sudo) for all its internal
    routing changes.
1522 **Problem:** If the user has not configured the `/etc/sudoers.d/nullexit` file properly, any automated
    background wake event will silently fail: `sudo -n` commands drop out, the WARP bypass routes fail to apply
    , and the `warp` container loses internet connectivity.
1523
1524 ##### 15.9.3 Tailscale Hijacking the Default Route (PHYSICAL_GW Bug)
1525 **Problem:** When `--post-wake` runs, it clears the routing table using `sudo route -n flush` to ensure a clean
    slate after the Wi-Fi card roams or wakes up. Because it skips bouncing the Wi-Fi interface (to make the
    reconnect faster), macOS's DHCP client does not automatically rebuild the physical default route. To
    counteract this, the script must manually restore the physical default route. Originally, the script tried
    to capture the physical gateway via `route get default`. However, because the gateway is already running
    and Tailscale is active, the default route is often already pointing to `utunX`. When this happens, `route
    get default` does not output a `gateway:` field (it only outputs the `interface:`), causing the captured `
    PHYSICAL_GW` variable to be empty.
1526 When `PHYSICAL_GW` is empty, the physical default route is never restored, causing the `en0` interface to be
    isolated from the physical network. The WARP bypass routes then fall back to targeting `interface en0` (
    instead of routing via the gateway), which breaks WireGuard's remote connections completely.
1527 **Fix:** The script now bypasses the OS routing table entirely and directly queries the hardware DHCP lease (
    `ipconfig getpacket en0`) to reliably extract the physical router IP, ensuring the physical route is
    correctly restored even when Tailscale is active.
1528
1529 ##### 15.9.4 July 4, 2026: Consolidation of Core Bash Scripts (Refactoring)
1530 **Symptom:** Over 200 lines of bash logic were directly duplicated across `toggle.sh` and `recover.sh` (e.g. `
    restart_tailscaled_daemon`, `stop_sleep_prevention`, WARP bypass routes). This caused drift when one script
    was updated without the other.
1531 **Root Cause:** Historical separation of responsibilities allowed `recover.sh` to grow complex alongside `
    toggle.sh`.
1532 **Resolution:**
1533 - Massively refactored both scripts by extracting all duplicated networking logic, PID lifecycle management,
    and environmental configuration down to `scripts/common.sh`.
1534 - Specifically, the `stop_sleep_prevention`, `stop_dns_watcher`, `stop_host_leak_probe`, and `stop_warp_watcher`
    functions were collapsed into a single `stop_pidfile_daemon()` abstraction.
1535 - Proxy disable loops were abstracted to `disable_all_proxies()`.
1536 - The `162.159.192.1/193.1` WARP edge IPs are now dynamically resolved from `.env` instead of hardcoded.
1537
1538 ##### 15.9.5 Threat Model: Local Proxy Discovery
1539 The Tor SOCKS proxy and Honey-Port Tripwire only bind to `127.0.0.1` (loopback). The real security boundary
    here is "can an unprivileged local process reach the loopback interface," not "does the malware know the
    port number."
1540
1541 The port-randomization and the Honey-Port trap only catch blind or generic port-scanners that do not already
    know where to look. They **do not protect** against a process that directly reads the `/tmp/nullexit-ports.
    env` file, greps the `toggle.sh` memory space, or runs `lsof -i` to inspect open socket bindings.
1542
1543 Because modifying file ownership on `/tmp/nullexit-ports.env` via `chown`/`chmod` to restrict read-access
    introduces an unavoidable Time-Of-Check to Time-Of-Use (TOCTOU) race condition (the file is created user-
    owned before privilege escalation, meaning file descriptors can be snatched), it is deliberately left as a
    standard user-owned file.
1544
1545 The actual, proper mitigation for preventing a malicious local process from reaching the proxy is not hiding
    the port it is running a host-level outbound firewall with strict localhost/loopback filtering enabled (e.g
    ., LuLu 4.3.0+ or Little Snitch). These firewalls gate access by cryptographic process identity (binary
    signature) at the kernel/network-extension level, rather than by port secrecy.
1546
1547 **Verifying Localhost Filtering (The Rogue Binary Test).** To empirically validate that the host firewall
    properly intercepts local proxy hijacking, you can compile and execute a completely unknown, un-whitelisted
    "rogue" binary to attempt a connection to the proxy port:
1548
1549 ```c
1550 // lulu-test.c
1551 #include <stdio.h>
1552 #include <stdlib.h>
1553 #include <arpa/inet.h>
1554
1555 int main(int argc, char *argv[]) {
1556     int port = atoi(argv[1]);
1557     int sock = socket(AF_INET, SOCK_STREAM, 0);
1558     struct sockaddr_in server = { .sin_family = AF_INET, .sin_port = htons(port) };

```



```

1559     server.sin_addr.s_addr = inet_addr("127.0.0.1");
1560
1561     printf("Rogue binary attempting connection to 127.0.0.1:%d...\n", port);
1562     if (connect(sock, (struct sockaddr *)&server, sizeof(server)) < 0) {
1563         printf("Connection BLOCKED by firewall.\n");
1564         return 1;
1565     }
1566     printf("Connection SUCCESSFUL (Firewall bypassed!).\n");
1567     return 0;
1568 }
1569 ...
1570 Compile with `gcc -o lulu-test lulu-test.c`. When executed against the `$TOR SOCKS_PORT`, a properly configured
    host firewall (with "Allow Loopback" disabled in LuLu Preferences) will immediately pause the socket
    execution and generate a desktop alert for the unrecognized binary signature. This physically stops
    targeted local malware from exfiltrating data through the Tor tunnel, proving the threat model mitigation.
1571
1572 ##### 15.9.6 Unlocking Mode-Locked Output Files Without `chmod` (`unlock-files.sh`)
1573 **The one-command fix:** `bash scripts/unlock-files.sh` replaces every known locked inode in the project with
    a fresh writable one. No `chmod`. Covers `.env` (mode `000`), `adguard/work/userfilters/compiled_rules.txt`
    `, `ip_blocklist.ipset`, `cache/*.txt`, `cache/ip/*.txt`, and `adguard/work/data/filters/*.txt` (mode
    `0444`). Run it once after setup or after pulling an old repo; `toggle.sh` and `sync-rules.py` will then
    work without permission errors.
1574
1575 **Context:** The nullexit project has a strict project-wide policy of `no chmod from scripts` (see README 6).
    This rule exists because every prior `chmod 0444` (post-write tamper-proof lock) and `chmod 000` (post-
    toggle `.env` lock) call from `scripts/sync-rules.py` / `toggle.sh` could leave a file permanently
    unreadable on disk if the script exited early, was killed mid-flight, or simply set a restrictive mode
    without ever being re-flipped. We've stripped every `chmod` from the codebase. But files **written by older
    versions of those scripts** are still on disk in those restrictive modes (`0444` for `compiled_rules.txt`,
    `ip_blocklist.ipset`, `data/filters/<id>.txt`; `000` for `.env` after end-of-toggle lock). Re-running `
    toggle.sh` or `scripts/sync-rules.py` against those leftovers hits `PermissionError: [Errno 13] Permission
    denied` immediately.
1576
1577 **Why the stale permissions also block `mv`/`rm` of the parent folder:** The restrictive modes (`000` on `.env
    `, `0444` on output files) don't just block writes they prevent the kernel from unlinking the file's
    dentry during a `mv` or `rm` of the **containing directory** (`adguard/work/`). macOS enforces this at the
    VFS layer: you own the directory, but you can't delete a directory that contains entries you can't write to
    . This made the entire `adguard/work/` tree unmovable/undeletable until the locked inodes inside it were
    replaced. `unlock-files.sh` resolves this permanently by swapping each locked inode for a fresh `0644` one
    via atomic rename (`cp` + `mv`), which operates on the directory entry rather than the file contents no `
    chmod` called, no permission bypass needed.
1578
1579 **Problem:** You (the owner) cannot `cat`, `cp`, `rm`, `>` redirect, or any normal POSIX write against a `mode
    =000` file even though you own it the basic mode bits block ALL access for owner, group, and other. The
    script does not call `chmod` and you want a permanent fix that never relies on a chmod from any future run
    either.
1580
1581 **Solution:** Replace the locked inode with a fresh readable one. `mv` is atomic; mode bits live in the inode,
    not the directory entry. Two flavors cover every case we hit on a real machine:
1582
1583 **Flavor A locked file is mode `000` (owner can't even READ it):**
1584 NOPASSWD sudoers (`/etc/sudoers.d/nullexit`) already includes `/usr/bin/python3`. So we read via `sudo python3`
    (which has the DAC override root has) and pipe the bytes around as base64 in user space, where the
    redirect happens with the user's normal umask = `0644`:
1585 ```bash
1586 sudo -n /usr/bin/python3 -c "import base64,sys; sys.stdout.write(base64.b64encode(open('<FILE>','rb').read()).
    decode())" > /tmp/snap.b64
1587 base64 -d < /tmp/snap.b64 > <FILE>.new
1588 mv -f <FILE>.new <FILE>
1589 ls -la <FILE>          # mode is now 0644
1590 ```
1591 The old `mode=000` inode is unlinked by `mv`; the new `mode=0644` inode (created by base64-decode via the
    calling user's umask) takes the path. No chmod ever called. The parent directory just needs to be writable
    by you (it always is, since you own it).
1592
1593 **Flavor B locked file is mode `0444` (you can READ but cannot WRITE; `scripts/sync-rules.py` needs to re-
    write):**
1594 You own the file and can read it, you just cannot truncate/overwrite it. The simplest path is to rename it
    aside and let the writer create a fresh inode from scratch (which gets the umask-default `0644`):
1595 ```bash
1596 mv <FILE> /tmp/old.<FILE>.$$
1597 python3 scripts/sync-rules.py          # now writes <FILE> cleanly at mode 0644
1598 ```
1599 Or apply Flavor A if you'd prefer to preserve the contents while resetting the mode.
1600
1601 **Why this is the canonical recovery, not a hack:**
1602 - Mode bits live in the inode, not the path. `mv -f` replaces the path's inode reference; the old locked inode
    drops out of the directory entirely (dentry eviction) and loses its last link, so the kernel reclaims it.
    The new inode (whatever it is: a freshly-written one, or a base64-decoded copy) gets the path.
1603 - The only prerequisite to apply Flavor B's `mv` is: you own the parent directory (so you can unlink entries
    from it) AND you have a NOPASSWD-capable binary that can read the byte stream if mode prohibits user read.

```

```

1604 - This pattern generalizes. Any time a script ends up producing a "locked file that the next run can't update"
      situation, the same trick applies without a single chmod anywhere.
1605
1606 **Empirical verification (user machine, July 1, 2026):**
1607 - `.env` was mode=000` (owner $USER, i.e. you): Flavor A unblocked it in 3 commands, no chmod.
1608 - Before: ----- 1 $USER staff 899 Jun 28 07:07 .env`
1609 - After: ~-rw-r--r--@ 1 $USER staff 899 Jul 1 20:39 .env`
1610 - `docker compose config` immediately picked up `TS_AUTHKEY` + `WIREGUARD_PRIVATE_KEY` substitution.
1611 - `compiled_rules.txt`, `ip_blocklist.ipset`, `data/filters/1782645604.txt` were mode=0444`: Flavor B (`mv`-
      aside) unblocked all three.
1612 - scripts/sync-rules.py then ran clean: 279587 block rules + 171 allow rules + 16710 IP entries compiled; new
      files came out at `0644`.
1613 - `toggle.sh` then went end-to-end in 48s (warp / routing-fix / adguardhome / tailscale all healthy, gateway
      mesh-joined, DNS hijack verified single-server).
1614
1615 **When this trick is appropriate vs. inappropriate:**
1616 - You own the parent directory and the file (typical desktop scenario).
1617 - The lock is a leftover from a removed `chmod` call that you want off disk forever.
1618 - NOPASSWD sudo includes at least one interpreter (`python3` on this system) that can be used to read mode-0
      files. If it does not, add it first: `<USER> ALL=(ALL) NOPASSWD: /usr/bin/python3`.
1619 - The file is owned by another user (root, another account) and the lock is intentional respect it; do not
      bypass another user's security boundary.
1620 - The parent directory is owned/writable only by another user escalate properly via sudo, or stop and ask the
      user.
1621 - You do not actually own the file and there is a legitimate reason for its lock state restore the file via a
      trusted source rather than circumventing the perms.
1622
1623 **Reusable cookbook:**
1624
1625 ```bash
1626 # Quick diagnostic: figure out which flavor you need.
1627 WHOAMI=$(whoami)
1628 ls -la <FILE>
1629 stat -f 'mode=%Sp owner=%Su group=%Sg' <FILE>
1630 # mode=----- or mode=----r----- : you cannot even read it Flavor A.
1631 # mode=r--r--r-- but writer fails : you are blocked from write but can read Flavor B.
1632 # mode=rw-r--r-- : not actually locked at all; unrelated write failure.
1633
1634 # Flavor A (mode=000):
1635 sudo -n /usr/bin/python3 -c "import base64,sys; sys.stdout.write(base64.b64encode(open('<FILE>','rb').read()).
      decode())" > /tmp/snap.b64
1636 base64 -d < /tmp/snap.b64 > <FILE>.new && mv -f <FILE>.new <FILE> && rm -f /tmp/snap.b64
1637
1638 # Flavor B (mode=0444, file is readable):
1639 mv <FILE> /tmp/old.<FILE>.$(date +%s)
1640 # (let the original writer recreate <FILE>; new file lands at mode 0644)
1641
1642 # Verify after either flavor:
1643 ls -la <FILE> # mode should be 0644
1644 <your normal validation command>
1645 ```
1646
1647 ### 15.10 Tor Container
1648
1649 #### 15.10.1 July 11, 2026: Tor Container Hardening & obfs4proxy Restoration
1650 **Symptom:**
1651 1. The Tor container entered a fatal crash loop upon boot, throwing `exec: "obfs4proxy": executable file not
      found in $PATH` when `TOR_USE_BRIDGES=true`.
1652 2. When the user pasted Tor bridge lines containing comments (`#`) or blank lines into `tor-bridges.txt`, the
      container failed to validate the file properly and crashed.
1653
1654 **Root Cause:**
1655 - **Bug 1 (Missing Pluggable Transports):** The Tor Dockerfile was built on `debian:bullseye-slim`, which had
      silently dropped or failed to resolve the `obfs4proxy` package in its latest apt repositories.
1656 - **Bug 2 (Fragile Validation):** The `entrypoint.sh` bridge validation check merely checked `[ -s tor-bridges.
      txt ]` (file size > 0). It did not verify if the file actually contained valid, uncommented bridge lines.
      Passing empty lines or comments tricked the validation script into appending invalid `Bridge` directives to
      the `torrc`, causing the Tor daemon to instantly exit.
1657
1658 **Resolution:**
1659 - **Upgraded Base Image:** Shifted the Tor Dockerfile base image to `debian:bookworm-slim`, restoring access to
      the `obfs4proxy` pluggable transport package.
1660 - **Robust Bridge Validation:** Replaced the fragile `[ -s ]` check with a strict `grep -vE '\s*(#|$)\'`
      command that strips comments and empty lines. If no valid lines remain, the container safely falls back to
      standard public guard relays instead of crashing, ensuring Tor remains available even with a misconfigured
      bridge file.
1661 - **Image Pinning:** Explicitly pinned `image: nullexit-tor:v1.0.0` in `docker-compose.yml` to prevent
      arbitrary local builds from drifting to `:latest`.
1662
1663 #### 15.10.2 July 12, 2026: Tor ControlPort Dynamic Password Authentication & User Config

```

```

1664 **Symptom:** Querying the Tor Control Port (e.g., via the /sweep script or check_circuits.py) returned
        empty responses or connection errors. Logs showed that Tor automatically closed its ControlPort:
1665 `You have a ControlPort set to accept unauthenticated connections from a non-local address. ... That's so bad
        that I'm closing your ControlPort for you.`
1666
1667 **Root Cause:** Docker port mapping (especially under Colima on macOS) requires container sockets to bind to
        `0.0.0.0` to be reachable from the host. However, when Tor's ControlPort is bound to `0.0.0.0` (non-local)
        with no authentication, Tor closes it by default as a safety precaution. Sourcing SOCKS5/ControlPort to
        `127.0.0.1` inside the container was not an option as it broke the Docker bridge routing path.
1668
1669 **Resolution:**
1670 - **Dynamic Password Authentication**: Updated the Tor container's entrypoint.sh to read TOR_PASSWORD and
        dynamically generate its HMAC hash via tor --hash-password, adding HashedControlPassword <hash> to torrc.
        This secures the ControlPort on `0.0.0.0` so Tor keeps it open.
1671 - **Unified Credentials in .env**: Exposed ADGUARD_USER=admin and ADGUARD_PASSWORD=nullexit in .env to
        serve as the unified gateway credentials. Sourced TOR_PASSWORD directly from ADGUARD_PASSWORD in docker-compose.yml.
1672 - **Client Authentication**: Updated scripts/check_circuits.py to fetch TOR_PASSWORD (defaulting to ADGUARD_PASSWORD
        or nullexit) and authenticate with the ControlPort using AUTHENTICATE "<password>" to
        regain access.
1673
1674 > See also 11.3 (Tor Container Kill-Switch Inheritance & Fail-Closed Egress Verification) for the design-level
        fail-closed guarantee.
1675
1676 ### 15.11 macOS Platform Quirks
1677
1678 #### 15.11.1 macOS Application Firewall Silently Blocking AirDrop & Continuity
1679 **Problem:** Users of complex network extensions (Tailscale, Lulu, Docker, etc.) often run into an issue where
        AirDrop, AirPlay, and Universal Clipboard silently fail, even with the gateway inactive and Wi-Fi/Bluetooth
        correctly configured. The internal Apple Wireless Direct Link (awdl0) interface appears healthy, but
        connections never establish.
1680
1681 **Root Cause:** The built-in macOS Application Firewall has a specific list of explicitly blocked applications.
        Apple's own core networking services (e.g., /usr/libexec/sharingd, /usr/libexec/rapportd, /usr/libexec/avconferenced)
        can be silently added to the "Block incoming connections" list either through
        accidental "Deny" clicks on permission prompts, execution of aggressive privacy-hardening scripts, or a
        known macOS bug that retains strict blocks even after toggling the "Block all incoming connections" setting
        off. Since these are background daemons, macOS provides no visible error.
1682
1683 **Fix:** The firewall rules must be cleared via the terminal (or System Settings > Network > Firewall > Options
        ):
1684 ```bash
1685 sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblockapp /usr/libexec/sharingd
1686 sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblockapp /usr/libexec/rapportd
1687 ```
1688 Once unblocked, sharingd immediately resumes broadcasting mDNS/Bonjour discovery packets, and AirDrop
        restores instantly without requiring a reboot.
1689
1690 #### 15.11.2 Standalone Tailscale Daemon Freeze after macOS Sleep/Wake
1691 **Problem:** When the macOS host goes to sleep and wakes up, the network interfaces and routing tables are
        rebuilt. The standalone tailscaled daemon (installed via Homebrew) occasionally fails to handle this
        transition and becomes completely frozen/unresponsive. In this state, any command like tailscale down or
tailscale status hangs indefinitely, and all DNS requests sent to the local Tailscale resolver
        (100.100.21.8) time out, causing a total internet blackout on the host.
1692
1693 **Fix:** Enhanced the Tailscale verification and teardown logic in toggle.sh and recover.sh:
1694 1. **Unresponsive Daemon Detection:** Instead of assuming the daemon is healthy just because the Unix socket /var/run/tailscaled.socket
        exists, the scripts now actively run a status check with a 5-second timeout (run_with_timeout 5 tailscale status).
        If the check times out (exit code 143), the daemon is classified as unresponsive.
1695 2. **Auto-Recovery Loop:** If the daemon is unresponsive, the script now automatically attempts to restart the
        service using brew services restart tailscale (falling back to sudo -n brew services restart tailscale).
1696 3. **Graceful Retry:** Once restarted, the script pauses for 3 seconds for the daemon to initialize before
        proceeding with connection or teardown commands.
1697
1698 #### 15.11.3 macOS Sharing Services (AirDrop/AirPlay) Freeze on Network Transitions
1699 **Problem:** When the gateway is turned on or off, the scripts perform network configuration cleanups which
        include flushing the routing tables (route -n flush), restarting the mDNSResponder daemon (killall -HUP mDNSResponder),
        and power-cycling the Wi-Fi interface (setairportpower or ifconfig en0 down/up). While AirDrop BLE discovery continues
        to function, the actual file transfers stall at "Waiting..." indefinitely.
1700
1701 **Root Cause:** The rapid tear-down and rebuild of the local routing table and Wi-Fi interface causes Apple's
        core sharing and connection daemons (sharingd and rapportd) to lose their socket bindings to the mDNS/Bonjour
        discovery interface. Instead of reconnecting gracefully, they enter a wedged state and try to route peer-to-peer
        (AWDL) IPv6/TCP transfer traffic into the default gateway (the Tailscale interface utun) instead of the direct
        wireless link, causing the TLS verification handshake to time out.
1702
1703 **Fix:** Integrated an automatic sharing services reset into both toggle.sh and recover.sh:

```

```

1704 1. The script now automatically runs `sudo -n killall sharingd rapportd` at the end of both the START path
    and the STOP path (as well as inside `recover.sh`).
1705 2. Killing these daemons forces macOS to immediately relaunch them, binding them fresh to the newly initialized
    network interfaces and routing tables, allowing AirDrop to work seamlessly while the gateway is active.
1706
1707 ##### 15.11.4 Chrome Remote Desktop Connection Failures (Tailscale on Remote Device)
1708 Context: When attempting to access a remote machine via Chrome Remote Desktop (CRD), the connection fails
    or hangs if Tailscale is active on the remote device. The connection works fine even if Tailscale (and
    the exit node) is active on the local client/viewer device, but only if Tailscale is disabled on the
    remote machine. (This is distinct from the CRD-through-WARP latency limitation in 14.3.)
1709
1710 Why: Having Tailscale active on the remote machine creates virtual network interfaces and DNS/routing
    modifications that conflict with Chrome Remote Desktop's WebRTC bindings and signaling traffic.
1711
1712 Workaround:
1713 - Use Tailscale Native RDP/RustDesk (Recommended): Connect directly to the remote device's Tailscale IP
    (`100.X.Y.Z`) via an RDP or RustDesk client. This is faster and avoids third-party relay conflicts.
1714 - Disable Tailscale on the Remote Device: If you must use CRD, disable Tailscale on the remote machine
    before connecting.
1715
1716 ##### 15.11.5 tailscaled Daemon Broken State (brew services)
1717 Symptom: `brew services list` shows `started` but `tailscale status` shows "stopped".
1718
1719 Fix: `sudo brew services restart tailscale`. Toggle script now detects and auto-restarts.
1720
1721 ##### 15.11.6 Apple Silently Removed the `airport` Binary in macOS Sonoma 14.4 (March 2024)
1722 What happened: `watcher.sh`'s `detect_lan_p2p_mode()` was written to use the `airport` CLI tool at:
1723 ```
1724 /System/Library/PrivateFrameworks/Apple80211.framework/Versions/Current/Resources/airport
1725 ```
1726 This path returned `exit 127: no such file or directory`. The function silently fell back to `reason="no-wifi"`
    even while the Mac was actively connected to WPA2 Enterprise Wi-Fi.
1727
1728 Why Apple removed it: `airport` was never a real public binary it lived inside a private framework and
    was always technically unsupported. Apple deprecated it quietly years ago and finally removed it in macOS Sonoma 14.4 (March 2024). The removal is part of a broader privacy clampdown on Wi-Fi metadata:
1729 - Starting in macOS 14.5, BSSIDs and other Wi-Fi association details are redacted (<redacted>) in
    most tools, including the official replacement wdutil
1730 - Apple's official recommendation is to use the CoreWLAN framework (Swift/Objective-C) with proper
    entitlements useless for a bash script
1731 - The official CLI replacement wdutil requires sudo for most useful output also useless for a background
    LaunchAgent
1732
1733 Fix: Replaced airport -I | awk '/link auth/' with:
1734 ```bash
1735 system_profiler SPAirPortDataType | awk '/Current Network Information:/{found=1} found && /Security:/{print $NF
    ; exit}'
1736 ```
1737 This returns human-readable strings like `WPA2 Enterprise`, `WPA2 Personal`, or `None` for the currently
    associated network without sudo, and without any removed private binary.
1738
1739 Confirmed working output on this machine:
1740 ```
1741 P2P detect: security='Enterprise' allow=false (reason: 802.1x-enterprise)
1742 ```
1743
1744 > Risk: system_profiler may not survive forever either. system_profiler SPAirPortDataType still works
    as of macOS Sequoia (15.x) without sudo and without redaction. However, given Apple's trajectory on Wi-Fi
    privacy, this could be locked down in a future release. If security comes back empty on a future macOS
    version while the Mac is actively on Wi-Fi, the function will silently fall back to allow=false (safe
    default), but the detection will be broken again. The long-term fix would be a small Swift helper using
    CoreWLAN that we compile once and bundle in the repo. (This forward-looking caveat is also tracked as an
    observation in 16.)
1745
1746 ##### 15.11.7 Changing macOS Local Hostname
1747 Context: Users may wish to change their Mac's computer name/hostname in macOS System Settings.
1748 Effect: Changing the Mac's hostname will not break Nullexit (which relies on internal routing/localhost
    ). However, Tailscale will automatically detect this change and update the machine name on the Tailnet.
1749 - The underlying Tailscale 100.x.x.x IPv4 address will remain exactly the same.
1750 - The MagicDNS name will change to match the new hostname. Users must remember to update their SFTP/SSH
    clients to use the new MagicDNS address.
1751
1752 ##### 15.11.8 Shell-Language Landmines: macOS /bin/bash 3.2 + set -e (Field Notes)
1753
1754 This project's lifecycle scripts (toggle.sh, recover.sh, common.sh, ) run under #!/bin/bash, which on
    macOS is GNU bash 3.2.57 (2007). Apple has never shipped a newer bash because bash 4+ is GPLv3. (
    Homebrew's bash 5 lives at /opt/homebrew/bin/bash but is not the shebang target, and cannot be assumed
    on PATH a bare bash --version on a stock Mac reports 3.2.) Every script here also runs set -e (exit-on-
    error) with an ERR/INT/TERM/HUP trap. That combination ancient bash + set -e + traps + set -
    x xtrace redirection is a minefield. The traps below were all hit and fixed during development;
    document them so they are not re-discovered.

```

```

1755
1756 **A. `set -e` + `if then else fi` can abort at the `else` (bash 3.2 heisenbug).**
1757 The nastiest one. A construct as innocent as:
1758 ```bash
1759 if [ -f "$MARKER_FILE" ]; then
1760     X="yes"
1761 else
1762     X="no"
1763 fi
1764 ```
1765 aborted `toggle.sh` at init under `set -e` whenever the file was **absent** the false status of the `[ -f ]`
    test leaked through the compound and `set -e` killed the script (`EXIT: code=1 phase=init`), before any
    real work ran. Two things made this vicious: (1) it is intermittent/context-dependent it did **not**
    reproduce in isolated `bash -c` snippets of the identical block; it only manifested inside the full
    script with the `set -x` xtrace fd-redirect (`exec 2> >(tee)` active. We only proved causation by **
    bisecting the live script** (replace the block with a trivial assignment the script sailed past init). (2)
    Static tools (`bash -n`, `shellcheck`) cannot see it it is a runtime interaction. **Safe pattern:**
    default-assign, then an `if` with **no `else`** (an `if` whose false condition has no else-branch yields
    status 0):
1766 ```bash
1767 X="no"
1768 if [ -f "$MARKER_FILE" ]; then X="yes"; fi
1769 ```
1770 **Confirmed root cause (2026-07-13, reproduced head-to-head).** The trigger is specifically **set -x plus a `
    PS4` that contains a command substitution** here the `DEBUG_TRACE` prompt `PS4='+ [$(date)] '`. Forcing
    the false-condition/else path (`gateway_state` `stopped`, so `if [ "$(gateway_state)" = "running" ]` takes
    the else/START branch) on stock `/bin/bash` 3.2.57: with the **default `PS4`** it survives (3/3, exit 0);
    with toggle's **$(date)` `PS4`** the `ERR` trap fires on the else branch (`rc=1` from the false condition,
    3/3, exit 42). The **$(date)` runs a subshell on **every** traced command; while tracing the **if-condition
    it perturbs **$? so the condition's non-zero status leaks past the normal **if-exemption and trips the `ERR`
    trap. This is a true **heisenbug** enabling tracing (`DEBUG_TRACE=true`) is what **causes** the abort,
    which is exactly why it's invisible with `DEBUG_TRACE=false` (the default) and why every isolated repro on
    the **default `PS4` passes (this misled an entire debugging session into briefly "disproving" the landmine).
    It aborted the gateway START decision (`toggle.sh:689`) under `DEBUG_TRACE=true`; with `DEBUG_TRACE=false`
    the gateway starts cleanly (verified: 192 s, all five containers healthy). **Fix (TODO):** give `
    DEBUG_TRACE` a subshell-free `PS4` (drop **$(date)`; bash 3.2 has no non-**$()` clock token for `PS4`, so
    either omit the per-line timestamp or accept that trace mode can abort at **if/else decisions). Until fixed
    , treat `DEBUG_TRACE` as **debug-only** safe for tracing non-decision code, not the STOP/START decision
    path.
1771
1772 More generally, any command whose "failure" is normal control flow (`grep -q`, `[ ]`, `diff`, `kill -0`) that
    ends up as the **last** command of a function/branch/script under `set -e` is a landmine; guard it (`|| true`
    ) or restructure so a non-zero status can't be the tail value.
1773
1774 **B. `BASH_XTRACEFD` and the `exec {var}>>file` dynamic-fd form are bash 4.1+ (hard-fail on 3.2).**
1775 Our `DEBUG_TRACE` feature wants xtrace (`set -x`) sent to `output.log` while keeping the terminal clean the
    clean way is `exec {BASH_XTRACEFD}>>"$LOG_FILE"`. On 3.2 this is a **syntax/runtime error**: `exec: {
    BASH_XTRACEFD}: not found` (dynamic-fd allocation **{var}>> didn't exist until 4.1, and `BASH_XTRACEFD`
    itself is 4.1+; on 3.2 it's just an ordinary variable that `set -x` ignores). Left unguarded it would abort
    the script at startup i.e. the debug feature would break the very thing it's meant to observe. **Safe
    pattern:** branch on **${BASH_VERSINFO[0]}.${BASH_VERSINFO[1]} on 4.1 use **exec 9>>"$LOG_FILE"; export
    BASH_XTRACEFD=9; on 3.2 fall back to **exec 2> >(tee -a "$LOG_FILE" >&2) (xtrace goes to stderr, which we
    duplicate to the log; the terminal also shows it, which is acceptable). Never use the **exec {var}>> form
    unless you **know** you're on 4.1.
1776
1777 **C. `trap ERR` + **$LINENO misattributes the failing line on 3.2.**
1778 When a command fails under `set -e`, our trap runs **cleanup_handler ERR $LINENO. On bash 3.2, **$LINENO inside
    an `ERR` trap frequently reports the line of the **enclosing `fi`/closing brace of a compound statement**,
    not the true failing line. During the STOP/START-decision bug this produced **Script failed at line 1202
    where 1202 is a bare **fi actively misleading. **Mitigation:** don't trust the trap's line number as
    gospel on macOS; corroborate with the **EXEC:/EXIT: lifecycle breadcrumbs and the **DEBUG_TRACE xtrace (
    which prints **file:line:function via a custom **PS4), which pin the **actual** last-executed command.
1779
1780 **D. `set -e` is not inherited by command substitutions / subshells the way people expect, and is suppressed
    inside **if/`&&/`||` conditions.**
1781 This cuts both ways and is a frequent source of "why didn't it fail?" and "why **did** it fail?": a non-zero
    command used as an **if condition (or left/right of **&&/`||`) is **exempt from **set -e (by design), but
    the moment that same call is a bare statement it aborts. **is_gateway_active() relies on this (it's always
    called as **if is_gateway_active; then`), which is precisely why moving the decision to a marker file
    instead of a probe that must be **if-guarded to be safe is more robust (see 15.12.19). Also note **local X=
    $(cmd) masks **cmd's exit status (the **local builtin's status wins) a **shellcheck SC2155 we see
    repeatedly; harmless when we don't check **$?, but a real footgun if you do.
1782
1783 **E. Process substitution (**>(), **<()) is a bashism, and its lifecycle is loose.**
1784 Our 3.2 xtrace fallback (**exec 2> >(tee -a "$LOG_FILE" >&2)) uses process substitution fine under bash, but
    (1) it is **not** POSIX **sh, so these scripts must never be invoked as **sh script.sh; and (2) the
    background **tee it spawns is reaped asynchronously, so the very last few lines written just before exit
    can occasionally be truncated in the log. Acceptable for a debug trace; do not rely on it for critical last
    -gasp logging (use a direct **>> "$LOG_FILE" append for must-capture lines, as the **EXIT breadcrumb does).
1785
1786 **F. GNU-vs-BSD userland divergence (not bash itself, but same class of pain).**

```



```

1787 macOS ships BSD variants of the core tools, so idioms differ from Linux: `sed -i ''` (BSD requires the empty
      backup-suffix arg) vs `sed -i` (GNU); `stat -fz` (BSD) vs `stat -c%s` (GNU); `date` format flags; `jot` (
      BSD) vs `shuf` (GNU) for random port generation; `route`/`networksetup`/`dscacheutil` (macOS-only) vs `ip
      `/`resolvectl` (Linux). This is *why* the codebase branches on `[[ "$OSTYPE" == "darwin"* ]]` everywhere
      `toggle.sh` and `recover.sh` are each single cross-platform scripts that dispatch on `$OSTYPE` (an early
      Linux block runs and exits before the macOS body). `timeout(1)` doesn't exist on stock macOS at all hence
      the pure-bash `run_with_timeout` in `common.sh`.
1788
1789 **Working rules distilled from the above:**
1790 - Assume **bash 3.2** semantics for anything under `#!/bin/bash` on macOS; gate any 4.0+ feature (
      BASH_XTRACEFD`, dynamic `{var}` fds, associative arrays, `${var^^}` case-conversion, `mapfile`/`readarray`,
      negative array indices) behind a `BASH_VERSINFO` check or avoid it.
1791 - Under `set -e`, never let a "benign non-zero" command (`[ ]`, `grep -q`, `kill -0`, `diff`) be the tail
      statement of a branch/function/script; use no-`else` `if`s, `|| true`, or explicit `${?}` handling.
1792 - Prefer **persisted state (marker files)** over **live probes** for control-flow decisions a probe forces an
      `if`-guard and can hang/abort; a file read cannot (see 15.12.19).
1793 - `bash -n` and `shellcheck` catch syntax and many smells but **cannot** catch `set -e`/trap runtime
      interactions a **live run is mandatory** to validate lifecycle-script changes; the `DEBUG_TRACE` +
      breadcrumb instrumentation exists precisely to make those live runs diagnosable.
1794 - Never run these scripts as `sh` they are bashisms end to end.
1795
1796 ### 15.12 Misc Resolved
1797
1798 #### 15.12.1 Gluetun Resets nftables FORWARD Rules
1799 **Symptom:** FORWARD RELATED,ESTABLISHED rule disappears after Gluetun health check.
1800
1801 **Fix:** `scripts/routing-fix.sh` re-adds to both iptables backends every 30 seconds. Legacy backend (policy
      ACCEPT) acts as safety net.
1802
1803 #### 15.12.2 Native SSH & SFTP Security (Zero Local Attack Surface)
1804 **Context:** macOS "Remote Login" (`sshd`) and "File Sharing" (`smbd`) open ports on the local network (e.g.
      `172.x.x.x`), exposing the host to brute-force attacks on public Wi-Fi.
1805
1806 **Implementation:** The script strictly enforces `tailscale up --ssh=true` whenever the mesh state is reset.
      This empowers the user to completely disable native macOS Remote Login and File Sharing. This provides an
      incredible zero-trust security advantage: even if an attacker steals your Mac username and password, they
      **cannot** access your files remotely. Disabling the native services completely closes the listening ports
      on the local Wi-Fi interface (`172.x.x.x`), rendering the Mac inaccessible to local network logins.
      Meanwhile, Tailscale intercepts port 22 traffic exclusively over the encrypted mesh and authenticates
      cryptographically based on your Tailscale SSO identity. Stolen Mac passwords are mathematically useless
      without first bypassing your Tailscale Two-Factor Authentication and registering a device onto the mesh.
1807
1808 **Cross-Platform File Exchange:** To easily browse and exchange files securely, simply use an SFTP client and
      connect to your Mac's MagicDNS name on port 22. Recommended clients: **WinSCP** or **FileZilla** (Windows),
      **Cyberduck** (macOS), and **FE File Explorer** or **Solid Explorer** (iOS/Android).
1809
1810 #### 15.12.3 Logging Architecture
1811 For debugging, logs are strictly segmented based on the component's lifecycle:
1812 - **`output.log` (Host-side):** Contains all standard error (`stderr`) and verbose output from the host scripts
      (`toggle.sh`, `recover.sh`, `setup.sh`). Since the `rule-compiler` container is ephemeral and deleted
      after running, its logs (and any Python errors from `scripts/sync-rules.py`) are extracted and appended to
      this file before deletion.
1813 - **Log Rotation Policy:** On startup, `toggle.sh` checks if `output.log` exceeds **1GB** (`1,073,741,824`
      bytes). If it does, it performs a metadata-only rename (`mv output.log output.log.old`), preserving history
      while resetting the active log to 0 bytes. This rename-based rotation avoids the heavy disk I/O and CPU
      overhead of rewriting a massive file line-by-line. The threshold is intentionally large because with
      `DEBUG_TRACE=true` the log accumulates a full command-by-command execution trace (effectively the framework's
      entire compilation/warning history), which is valuable to retain for post-mortems; a smaller cap would
      rotate that history away too quickly.
1814 - **Egress Probe Logging:** The background host-egress leak prober checks if stdout is a TTY (`[ -t 1 ]`) and
      will only print live status updates (using carriage return `\r`) or duplicate console warnings in
      interactive mode. It automatically detects macOS/BSD vs. Linux environments to dynamically construct
      timestamps with date and time (omitting milliseconds on macOS to prevent high CPU utilization from spawning
      sub-second processes).
1815 - **`docker logs <container>` (Guest-side):** The persistent containers (`warp`, `tailscale`, `routing-fix`,
      `adguardhome`) use the Docker `json-file` logging driver with a strict `max-size` (1m-10m) to prevent VM
      disk exhaustion. Use standard `docker logs` to view them.
1816
1817 #### 15.12.4 Pre-Flight Check 1: Wrong `tailscale ping` Flag
1818 **Problem:** Check 1 of the pre-flight connectivity checks used `--c 1` (double dash) but `tailscale ping`
      accepts `-c 1` (single dash). The invalid flag caused the command to exit with code 1, marking the check as
      FAIL even though the gateway was reachable.
1819
1820 **Fix:** Changed `--c 1` to `-c 1`.
1821
1822 #### 15.12.5 Pre-Flight Check 1: DERP Relay Exit Code
1823 **Problem:** Even with the correct `-c 1` flag, `tailscale ping` exits with code 1 when the connection goes
      through a DERP relay (direct connection not established) even though a pong was successfully received
      (28ms via DERP(tor)). The check treated relayed connections as failures.
1824

```



```

1825 **Fix:** Changed the check from relying on exit code to piping stdout through `grep -q "pong"`. This accepts
      relayed connections as valid (they work fine for the exit node use case), while still failing on true
      timeouts or unreachable peers.
1826
1827 ##### 15.12.6 IPv6 Exit-Node Leak
1828 **Problem:** Tailscale supports IPv6, but WARP/gluetun is IPv4-only. IPv6 packets forwarded through the exit
      node would leak through the Docker bridge unencrypted.
1829
1830 **Fix:** Added `ip6tables -A FORWARD -i tailscale0 -j DROP` to `post-rules.txt`, blocking all IPv6 forwarded
      traffic from the Tailscale interface. *(This is the fix referenced by the IPv6-leak scenario in 15.6.1.)*
1831
1832 ##### 15.12.7 SOCKS5 Proxy Threading Race Condition
1833 **Problem:** The `forward` function in `scripts/socks5-proxy.py` called `select.select([src, dst], [], [])`
      which raised `ValueError: file descriptor cannot be a negative integer (-1)` when one thread closed a
      socket while the other thread was selecting on it.
1834
1835 **Fix:** Added `ValueError` exception handling around `select.select()` and `recv()` calls. When a file
      descriptor becomes negative, the thread breaks out of the forwarding loop cleanly instead of crashing.
1836
1837 ##### 15.12.8 Docker Healthcheck: Multi-line Python in CMD Array
1838 **Problem:** The socks-proxy healthcheck used a `CMD` array with a multi-line Python string. YAML collapsed the
      newlines, causing the `#` comment to comment out the rest of the code and the `assert` statement to be
      mangled into `as`.
1839
1840 **Fix:** Changed from `[ "CMD", "python3", "-c", "..."]` to `[ "CMD-SHELL", "python3 -c '...'" ]` with a single-
      line Python expression. Uses a real SOCKS5 handshake (sends `[5, 1, 0]`, expects `[5, 0]`) instead of a
      simple TCP port check.
1841
1842 ##### 15.12.9 Graceful Shutdowns Leave DNS Hijacked
1843 **Problem:** The gateway overrides the macOS host's DNS settings (via `networksetup`) to route queries to the
      AdGuard container. Because macOS persists `networksetup` changes to disk, shutting down the Mac while the
      gateway is running causes the Mac to boot up later with hijacked DNS. Since the Colima VM and Docker
      containers don't auto-start on boot, the user would have no internet access until they manually run the
      toggle script.
1844
1845 **Root Cause:** The `toggle.sh` script exits after the gateway starts. There was no background daemon listening
      for macOS system shutdown signals to revert the network settings.
1846
1847 **Fix:** Wrapped the background `caffeinate` process (used to prevent system sleep) in a bash `trap` command.
      The trap listens for `SIGTERM`, `SIGINT`, and `SIGHUP`. When the user initiates a graceful shutdown, macOS
      sends `SIGTERM` to the background trap, which instantly executes `recover.sh` to flush the host DNS back to
      `1.1.1.1` right before the machine powers off.
1848 *Note: We explicitly use `recover.sh` instead of `toggle.sh` because during a shutdown, the OS limits process
      cleanup time and is already independently sending termination signals to Docker and Colima. Trying to run
      `docker compose down` inside a shutdown trap creates a race condition that can hang the script until macOS
      forcefully `SIGKILL`s it. `recover.sh` abandons container management and surgically flushes the macOS
      network settings in milliseconds, guaranteeing completion before the OS pulls the plug.
1849
1850 ##### 15.12.10 Linux Native Wi-Fi Bouncing
1851 **Problem:** The generated `scripts/recover-linux.sh` incorrectly retained the macOS `networksetup` commands,
      which would crash and fail to bounce Wi-Fi on Linux hosts.
1852
1853 **Fix:** Swapped the macOS logic for native Linux commands. The script now attempts `nmcli radio wifi off/on`
      first, and gracefully falls back to `ip link set wlan0 down/up` if NetworkManager is unavailable.
1854
1855 ##### 15.12.11 TS_AUTH_ONCE Cloud Footgun & Ephemeral Keys
1856 **Decision:** The compose file uses `TS_AUTH_ONCE=true` to prevent authentication loops. However, on headless
      cloud VPS deployments, if a standard auth key expires (usually 90 days), the container will silently drop
      off the mesh. We documented this trade-off explicitly in `docker-compose.yml` and recommended the use of **
      Tailscale Ephemeral Keys** for cloud deployments, which automatically clean themselves up and bypass this
      issue.
1857
1858 ##### 15.12.12 Hardcoded AdGuard Credentials
1859 **Decision:** AdGuard is deployed with the hardcoded credentials `admin / nullexit`. This is perfectly safe
      behind a NAT on a local macOS laptop. However, if deployed on a cloud host with exposed ports, this becomes
      a severe security risk. We added a stark warning comment in `docker-compose.yml` to ensure cloud users
      change this before deployment.
1860
1861 ##### 15.12.13 Routing Fix: 30-second polling vs Netlink Socket
1862 **Decision:** Instead of building a complex Netlink socket listener (`ip monitor`) to react instantly to
      iptables and routing flushes from Gluetun, we retained the 5-second `sleep` loop in `scripts/routing-fix.sh`
      .
1863 - **Reasoning:** Building a netlink listener in pure bash on Alpine is incredibly brittle and complex (
      especially for iptables events). The 30-second polling loop is bulletproof. The only trade-off is a
      potential 1-4 second stutter if Gluetun reconnects, which was deemed an acceptable edge case for absolute
      reliability.
1864
1865 ##### 15.12.14 Cache Poisoning and URL Rot in scripts/sync-rules.py
1866 **Problem:** If a remote blocklist URL went permanently offline (404 Not Found), the Python `urllib` request
      would return the 404 HTML string. The script would aggressively regex-parse this HTML, find no domains, and
      overwrite the healthy local cache with a 0-domain file. This effectively disabled ad-blocking until the

```

URL was fixed.

****Fix:**** Implemented a defensive programming sanity check in `scripts/sync-rules.py`. Before overwriting the cache, the script now verifies that the compiled domain count is greater than 10 (and 1 for IP feeds). If it falls below this threshold (indicating a catastrophic 404 failure across multiple URLs), it aborts the cache overwrite, raises a `ValueError`, and keeps the previous day's healthy cache intact.

15.12.15 Gluetun nf_tables Parser Crash (Bash Interpolation Failure)

****Problem:**** Attempting to make the TCP MSS dynamically configurable by using a standard bash variable inside `post-rules.txt` (`iptables ... --set-mss \${GATEWAY\_MSS:-1120}`) caused the Gluetun container to catastrophically crash during startup. Gluetun's internal parser executes `post-rules.txt` line by line without a shell, meaning the variable was never expanded. It literally attempted to inject the string `\${GATEWAY\_MSS:-1120}` into iptables, resulting in a fatal `bad value for option "--set-mss"` error.

****Fix:**** Removed the bash variable from `post-rules.txt` and reverted it to a pure hardcoded integer. To retain dynamic `.env` configuration, we moved the interpolation logic to `toggle.sh`. Right before Docker starts, the host script parses `GATEWAY\_MSS` from the `.env` file and uses a cross-platform `sed` command to dynamically overwrite the integer directly inside `post-rules.txt`. This perfectly mimics manual configuration and completely bypasses Gluetun's strict parser.

15.12.16 AdGuard Filter Redundancy & Memory Optimization

****Problem:**** AdGuard Home does not perform cross-list deduplication in memory. When it loads its own native subscription filters (e.g., `AdGuard DNS filter`) alongside our massive `compiled\_rules.txt`, overlapping rules are loaded twice, wasting significant RAM in the Colima VM. The UI would show ~500k rules, representing the sum of all lists rather than unique domains.

****Fix:**** Updated `scripts/sync-rules.py` to intelligently cross-reference AdGuard's configuration and deduplicate our list against it.

1. The script now reads `adguard/conf/AdGuardHome.yaml` to dynamically fetch the exact URLs of any enabled native AdGuard filters.
2. The parsing engine was upgraded to normalize basic AdGuard syntax (`||domain`) back into raw base domains during the build process.
3. The script subtracts the native AdGuard domains from our custom blocklist *before* compiling, immediately purging ~83,000 completely redundant rules.
4. The normalized base domains then pass through our subdomain optimizer, which squashes an additional ~50% of the remaining rules.

This reduces the final compiled output from ~325k to ~281k rules on the `heavy` profile, saving ~45k rules from being loaded into memory twice and reducing the overall RAM footprint of the `adguardhome` container.

15.12.17 Kernel-Level IP Blocklist (Threat Intelligence Firewall)

****Problem:**** DNS sinkholing cannot stop malware or botnets that bypass DNS entirely by hardcoding direct IP addresses (a common technique for C2 communication). These connections pass straight through AdGuard unseen.

****Fix:**** Added a second compilation pipeline to `scripts/sync-rules.py` that runs on every startup alongside the DNS pipeline.

1. The compiler concurrently fetches four curated threat-intelligence feeds: ****Feodo Tracker**** (abuse.ch botnet C2 IPs), ****Spamhaus DROP**** (IPs allocated to criminal organizations), ****Emerging Threats**** (Proofpoint active C2 and scanners), and ****CINS**** (active brute-force sources).
2. All entries are normalized via Python's `ipaddress` module. Any IP or CIDR that overlaps with RFC1918 private ranges, loopback, link-local, or the Tailscale CGNAT range (`100.64.0.0/10`) is stripped to prevent accidentally locking users out of their own LAN or mesh.
3. The cleaned list (16,721 unique IPs/CIDRs) is written as an `ipset restore` file using an ****atomic swap pattern****: `create\_new populate swap with live destroy\_new`. This guarantees the live `block\_malicious` ipset is never empty during a reload.
4. `scripts/routing-fix.sh` watches the file's mtime every 30 seconds. On change it runs `ipset restore` and idempotently re-injects `FORWARD DROP` rules for both `src` and `dst` into both iptables backends (nftables + legacy).
5. `docker-compose.yml` mounts `adguard/work/userfilters/` into `routing-fix` as `/userfilters:ro` and adds `rule-compiler: service\_completed\_successfully` to its `depends\_on`, eliminating the race condition where routing-fix could start before the file existed.

****Memory cost:**** The entire 16,721-entry ipset costs only ~1.6 MiB of kernel memory negligible in the 600MB VM budget.

15.12.18 Incident Post-Mortem: Discarded Docker Exec Errors in logs (July 10, 2026)

****Symptom:**** When the `warp` container is stopped, dead, or failing to connect to its endpoints, the `toggle.sh` and `recover.sh` connectivity health checks fail, but no details are printed to `output.log`. The logs only show generic `FAIL` outputs, making it hard to see if the failure was a network timeout, Docker daemon error, or a missing binary.

****Root Cause:**** Overly broad error silencing. The `docker compose exec` commands in the health checks (Check 3 in `toggle.sh` and the traffic check in `recover.sh`) both redirected stderr to `/dev/null` (`&>/dev/null` and `2>/dev/null` respectively), completely throwing away any diagnostic error messages produced by Docker or `wget`.

****Fix (July 10, 2026):**** Redirected stderr of the `docker compose exec` check commands to `output.log` (`2>> output.log`). This ensures that any command execution failures or connection timeout details are logged.

1905 ##### 15.12.19 Incident Post-Mortem: Toggle Decided STOP/START From a Live Probe, Aborting When Docker Was Down (July 13, 2026)

1906 ****Symptom:**** Running `./toggle.sh` (or `--restart`) while the Colima VM / Docker daemon was ****not**** running caused the script to hang for ~15 seconds and then abort during the START phase without ever booting Colima. With `DEBUG_TRACE` the lifecycle breadcrumb showed `toggle.sh EXIT: code=1 phase=start`, and the trace showed execution jumping straight from the START-branch entry to the `ERR` trap (`cleanup_handler ERR`) with the START body never executing. Historically this silent failure occurred 57 in `output.log`. Spam-clicking the toggle during the resulting window (each retry rejected by the lock guard, then finally succeeding) is what produced the earlier "pile of stacked toggle processes" and Wi-Fi-bounce confusion.

1907 ****Root Cause:**** `toggle.sh` decided whether to STOP or START by calling `is_gateway_active()` (`scripts/common.sh`), which ****live-probes**** the running system its first step is `run_with_timeout 15 docker compose ps --status running`. When `/var/run/docker.sock` is absent (Colima down), that call blocks for the full 15 s timeout and returns non-zero. Combined with the script's global `set -e` and `ERR` trap, a non-zero result evaluated at the `if is_gateway_active`; then else `fi` boundary aborted the whole script ****before**** the `else` (START) branch body ran precisely the branch whose job is to start Colima. Design-wise the deeper flaw is that a ****disable**** should not depend on whether the gateway is ****currently**** healthy; it should act on whether the gateway is ****supposed**** to be running (persisted intent). The correct signal already existed `MARKER_FILE=/tmp/nullexit-gateway-active.marker`, written at the end of a successful START and cleared on STOP but `toggle.sh` only wrote/cleared it and never read it for this decision. ****Note:**** this was a long-standing latent bug, not a regression from the July 2026 `common.sh` platform-function unification or the `DEBUG_TRACE` work; those changes only made the previously-silent abort ****visible**** (via the `EXEC:/EXIT:` breadcrumbs and `xtrace`).

1909 ****Fix (July 13, 2026):**** Introduced `gateway_should_be_running()` in `common.sh` and routed all four decision sites through it (`toggle.sh` main decision + `--restart` pre-check; `scripts/toggle-linux.sh` main decision + `--restart` pre-check). It decides from persisted intent:

1910 1. `toggle.sh` captures the marker's existence into `GATEWAY_MARKER_AT_STARTUP` ****before**** the top-of-script "defensive stale-marker clear" wipes it, and the helper honors that captured value (falling back to the live marker file for callers that run before the clear, e.g. the `--restart` pre-check).

1911 2. Marker present STOP path; marker absent START path. This is instant and cannot hang or abort.

1912 3. Only when the marker is absent ****and**** the Docker socket is actually reachable (`/var/run/docker.sock` or `~/colima/default/docker.sock` on macOS) does it consult `is_gateway_active` as a non-fatal reconciliation fallback (covers a marker lost to a `/tmp` wipe while containers are genuinely up). A dead socket short-circuits to "stopped" with no probe, so `set -e` can never be tripped.

1913 4. The STOP path's `docker compose down` is now `|| true`-guarded so a disable always completes even with Docker down; the START path's `docker compose up` is deliberately left unguarded (a failed start ****should**** trigger cleanup). On Linux the marker isn't maintained, so the helper degrades to the `is_gateway_active` fallback fast there because native Docker has no Colima VM to hang.

1914 ****How we got here (methodology note):**** This one is worth recording because the ****process**** mattered as much as the patch. The failure had been happening silently for a long time the log simply ended after a teardown with no error, so there was nothing to chase. We first attacked the ****observability gap**** rather than guessing at the cause: we wrapped the heavyweight lifecycle commands (`colima start`, `docker compose up/down`) so their output and exit codes always land in `output.log`, added an `EXIT: code= phase=` breadcrumb that fires even when the process is killed, and put a full opt-in `xtrace` behind `DEBUG_TRACE`. Only ****then**** did we reproduce the failure and this time the trace said, unambiguously, `EXIT: code=1 phase=start` with the START body never entered. The instrumentation didn't fix anything, but it converted an invisible abort into a precise, reproducible fact. (Lesson, consistent with 15.12.18 and the LUKS post-mortem referenced in 15.11: ****a mechanism that fails silently is worse than one that fails loudly**** so we make it fail loudly first.)

1915 The root-cause leap, though, came from a ****design-smell intuition**, not the stack trace: the observation that it is nonsensical for a ****disable**** to first check whether the gateway is ****currently online**** a teardown should act on whether the gateway is ****supposed**** to be running, not probe a subsystem to find out. That instinct turned out to name the bug exactly. The smell was a ****conflated responsibility**** with an inverted dependency: to decide whether to ****start**** the gateway, the code first had to ****talk to**** the gateway (via Docker) the very thing START is responsible for bringing up so the check was guaranteed to be unavailable in precisely the cold-start case that mattered. A second tell was ****false symmetry****: `enable` (builds state) and `disable` (tears it down) are not symmetric operations, yet both ran the identical expensive probe; whenever two operations that should differ share suspiciously identical machinery, the code is usually lying about the structure of the problem, and reality eventually calls the bluff. The fix invented nothing the honest signal (`MARKER_FILE`), literally "the gateway is supposed to be up" already existed and was maintained in all the right places; it was simply never consulted for the decision it was made for. We just pointed the decision at the answer the codebase already knew. General principle worth keeping: ****design smells here are load-bearing they tend to *be* latent bugs, not merely cosmetic****, and are worth chasing on intuition even before a trace confirms them.

1916 ****Acceptance verified (static + live):**** with marker present STOP (instant, even with Docker down); with marker absent + Docker down START (0 s, no `set -e` abort); a real run now proceeds past init to the gateway-status decision. `bash -n` clean on all three scripts; no new `shellcheck` findings; re-signed signatures (`toggle.sh`/`common.sh`/`toggle-linux.sh` are in the signed set) and `crypto.sh --verify` exits 0.

1917 ****Follow-up (same day)** a `set -e` regression the fix introduced, and why only a live run caught it. ****The first cut of the marker capture wrote the intent as a full `if [ -f "$MARKER_FILE" ]; then GATEWAY_MARKER_AT_STARTUP="yes"; else GATEWAY_MARKER_AT_STARTUP="no"; fi`. On macOS `/bin/bash` 3.2, with `set -e` active and the `DEBUG_TRACE` `xtrace` redirect (`exec 2> >(tee)`) in effect, that construct ****aborted the script at init**** the moment the marker was absent the false `[ -f ]` status leaked through the compound and `set -e` killed the run (`EXIT: code=1 phase=init`), before the gateway logic ran at all. Notably this did ****not**** reproduce in isolated `bash -c` snippets of the same block it only surfaced in**

```

the script's real runtime context so we had to bisect against the actual `toggle.sh` (swap the block for a
trivial assignment the script sailed past init) to prove causation. Fix: write it as a default assignment
plus an `if` **without an `else`** (`GATEWAY_MARKER_AT_STARTUP="no" then `if [ -f "$MARKER_FILE" ]; then
GATEWAY_MARKER_AT_STARTUP="yes"; fi`), which yields status 0 when the condition is false. Lesson reinforced
: `set -e` on legacy bash is genuinely treacherous around conditionals, and the observability work paid for
itself again the breadcrumb pinned the abort to init instantly, and a live run (not static checks) was
the only thing that exposed a heisenbug invisible to isolated reproduction.
1923
1924 ---
1925
1926 # Part IV Observations, Changelog & TODO
1927
1928 ## 16. Unverified Observations
1929
1930 ### 16.1 AdGuard Returns Two Different Sinkhole IPs (Unverified)
1931
1932 > **Status: Observed but not formally verified. Needs a deeper audit of the rule-compiler sources to confirm.**
1933
1934 **Observation (July 10, 2026):** When querying AdGuard Home for blocked ad domains, some domains resolve to
`0.0.0.0` and others resolve to `127.0.0.1`. Both are blocked, both cause connection failure, but the
different responses were unexpected.
1935
1936 ```
1937 doubleclick.net          0.0.0.0      (blocked)
1938 ads.google.com           127.0.0.1   (blocked)
1939 pagead2.googlesyndication.com 0.0.0.0     (blocked)
1940 scorecardresearch.com    127.0.0.1   (blocked)
1941 ```
1942
1943 **Likely Explanation (Unverified):** There are three historical schools of thought on the "correct" DNS
sinkhole response, and AdGuard faithfully preserves whichever the source blocklist used:
1944
1945 | Sinkhole IP | Convention | Origin |
1946 |---|---|---|
1947 | `0.0.0.0` | AdBlock-style lists (`\|\domain`) | Modern DNS-level blocking; AdGuard's native format. Fails
fast OS doesn't attempt a TCP connection. Works for both IPv4 and IPv6 without a separate rule. |
1948 | `127.0.0.1` | Hosts-file-style lists (`127.0.0.1 domain`) | 90s-era `/etc/hosts` tradition. Steven Black's
hosts list (500k+ domains) still uses this. |
1949 | `NXDOMAIN` | Purist / Pi-hole default | Returns "domain doesn't exist." Semantically most accurate but
requires browsers to handle NXDOMAIN gracefully. |
1950
1951 The dual-response behavior in this project is because `rule-compiler` pulls from both AdBlock-format sources (
Hagezi, uBlock origin lists) and hosts-format sources (Spamhaus DROP, Steven Black, Feodo Tracker), and
AdGuard returns whatever sinkhole IP the matching rule specifies.
1952
1953 **To Verify:** Run `docker compose exec tailscale cat /etc/hosts` to confirm no hosts-file rules are being
injected at the container level, and check which specific AdGuard rule matched each domain via the AdGuard
query log at `http://100.100.21.8:3000/#logs`.
1954
1955 ### 16.2 AGY CLI Appears to Generate Native macOS Notification Pop-ups (Unverified)
1956
1957 > **Status: Observed but source unverified. macOS security logs ruled out system-level origin.**
1958
1959 **Observation (July 10, 2026):** While running a DNS blocklist verification via the Antigravity CLI (`agy`), a
command was proposed that included `malware.wicar.org` (a known-safe DNS blocklist test domain) in a `dig`
loop. Shortly after, a macOS-style notification popup appeared describing a "malicious" or "blocked" event.
The AGY CLI terminal session then closed.
1960
1961 **Investigation:**
1962 - **macOS XProtect / Gatekeeper logs:** Empty. Zero events in the 30-minute window around the incident.
1963 - **macOS Endpoint Security / System Policy logs:** Empty.
1964 - **Broad `log show` search for "malicious", "malware", "blocked":** Empty.
1965 - **Conclusion:** macOS itself did not generate the notification. No system security subsystem fired.
1966
1967 **Likely Explanation (Unverified):** The notification appears to have originated from **AGY CLI's own internal
safety system**. AGY CLI is a native macOS application (not just a terminal process) and has access to the
macOS Notification Center API (`UNUserNotificationCenter`). When its guardrails detected a domain
containing the word "malware" (`malware.wicar.org`) in a proposed shell command, it likely:
1968 1. Blocked the command from executing
1969 2. Posted a native macOS-style notification via the Notification Center
1970
1971 This is surprising behavior for a CLI tool most terminal programs don't send GUI notifications but AGY CLI
appears to bridge both worlds: it runs in a terminal but is packaged as a native app with full system API
access. The notification would be indistinguishable from a system security alert at a glance.
1972
1973 **Notes:**
1974 - `malware.wicar.org` is the DNS/AV equivalent of the EICAR test file a deliberately benign domain that
triggers blocklist/AV systems to verify they work. It was included in the test set to confirm the AdGuard
blocklist was catching it. AGY's safety system is not aware of this distinction.
1975 - For future DNS blocklist testing, use only ad/tracking domains (e.g., `doubleclick.net`, `adnxs.com`) to
avoid triggering AGY's guardrails.

```

```

1976
1977 ### 16.3 `system_profiler SPAirPortDataType` Longevity (Wi-Fi Detection)
1978 The Wi-Fi security detection used by `watcher.sh`'s `detect_lan_p2p_mode()` currently relies on `
system_profiler SPAirPortDataType` (after the `airport` binary removal full history in 15.11.6). This
works as of macOS Sequoia (15.x) without sudo and without redaction, but given Apple's ongoing Wi-Fi
privacy clampdown, it may be locked down in a future release. If `security` comes back empty on a future
macOS version while on Wi-Fi, detection silently falls back to `allow=false` (a safe default). The long-
term fix would be a small Swift CoreWLAN helper compiled and bundled in the repo.
1979
1980 ## 17. Changelog / Recent Updates
1981
1982 Reflects the current branch's state. Each entry is a one-line summary + commit hash; read the commit message (
and the linked Incident Log entry) for full detail. Full post-mortems live in 15.
1983
1984 - **July 10, 2026 `fix(race): suppress WARP Watcher during post-wake force-recreate`** Fixed a race where
switching Wi-Fi caused the host IP to leak as the raw ISP IP (`132.x`) on every roam; `recover.sh --post-
wake`'s warp force-recreate tripped the WARP Watcher into a nuclear shutdown. Fixed via `/tmp/nullexit-warp
-inhibit.marker`. See 15.2.7.
1985 - **July 10, 2026 startup/roaming stabilization suite** Pre-flight Tailscale race (15.5.3), STUN-block cold-
boot false negative (15.5.4), restart DNS-build race via `wait_for_dhcp_settle` (15.5.5), nuclear/post-wake
concurrency lock (15.2.8), infinite auto-recovery loop marker leak (15.2.9), PF kill-switch bricking
SOCKS5 fallback (15.7.3), `--reset` host logouts (15.7.4), missing-flags abort (15.7.5), discarded docker-
exec errors (15.12.18), and the SNAT endpoint poisoning fix (15.7.1).
1986 - **July 6, 2026 `fix: resolve edge-case vulnerabilities and system state leaks`** Security-review hardening
suite:
1987 1. **DNS Watcher Interface Independence:** `start_dns_watcher` now detects the active interface via `
get_active_service()` instead of hardcoding a `grep` for "Wi-Fi".
1988 2. **Robust Lock Verification:** `toggle.sh` lock-file logic uses `ps -p` verification to prevent permanent
lockouts from a recycled PID.
1989 3. **Fail-Closed Crypto:** `crypto.sh` integrity check switched from `[ -x ]` to `[ -f ]` so it runs even if
git strips the `+x` bit.
1990 4. **Strict `iptables` Pinning:** `post-rules.txt` FORWARD-chain rules explicitly pin `tailscale0`/`tun0` both
directions, preventing escape via `eth0` during WARP drops.
1991 5. **Docker Local Network Isolation:** `docker-compose.yml` binds exposed WARP ports (`1080`, `5354`) to
`127.0.0.1` to prevent LAN access.
1992 6. **Routing Verification & Sudoers:** Added a `netstat -nr` default-route override check and ensured `/usr/
bin/python3` is permitted in the sudoers template.
1993 - **July 4, 2026 `cc789c5`** Concurrency mutex on `toggle.sh` (`/tmp/nullexit-toggle.lock`); defensive `
TS_AUTHKEY` init under `set -u`; Go `go vet`/`go build` steps in CI; AdGuard log capping; macOS BSD `grep`
fixes in `setup-common.sh`; `recover-linux.sh` quoting/PID fixes; removed redundant bypass-routes block in
`recover.sh`.
1994 - **July 4, 2026 `fix: post-wake routing loop & destructive recovery`** (1) `add_warp_bypass_routes` now
accepts explicit interface args so post-wake stops routing WARP traffic back into `utun*` (was an infinite
loop killing internet on every wake). (2) Replaced `sudo route -n flush` with targeted deletions of
`0.0.0.0/1`/`128.0.0.0/1` + WARP bypass routes, so recovery no longer destroys loopback/multicast routes
and wedges `configd`/`mDNSResponder` until reboot.
1995 - **July 4, 2026 `feat: secure execution`** Restricted the blanket `/usr/bin/python3` NOPASSWD sudo to
exactly `/usr/bin/python3 -I .../scripts/dns-proxy.py`; locked `dns-proxy.py` with `chown root:wheel` + `
chmod 755`; added native auth prompts to `Toggle Gateway.app`/`Recover Gateway.app`; `toggle.sh` now
captures `warp` logs on failure before teardown.
1996 - **July 4, 2026 Go ports + refactor** `logger` and `rule-compiler` ported to Go multi-stage Docker builds
(15.8.6); ~200 lines of duplicated bash extracted to `scripts/common.sh` (15.9.4); `crypto.sh` HMAC-SHA256
script integrity added.
1997 - **July 3, 2026 `fix: resolve numerous system regressions`** Fixed truncated `/etc/sudoers.d/nullexit` line;
temporary `.tmp`+`os.replace()` atomic writes in `sync-rules.py` (later reverted, see below); `recover.sh`
`ts_args` quoting; ANSI `%b`/`-e` output in `diagnose-host-leak.sh`/`fix-docker-bridge-collision.sh`; `
toggle-linux.sh` using `resolvectl dns` instead of macOS `networksetup`; AdGuard REST refresh POST
targeting port `3000`; restored `START_GATEWAY=true`; purged stale port `80` mapping; verified `output.log`
in `.gitignore`.
1998 - **July 3, 2026 `unlock-files.sh` + revert atomic writes** Reverted all 5 `.tmp`+`os.replace()` sites in `
sync-rules.py` to direct writes; created `scripts/unlock-files.sh` to permanently fix stale `0444`/`000`
file permissions via atomic rename (no `chmod`). Covers `.env`, `compiled_rules.txt`, `ip_blocklist.ipset`,
`cache/*.txt`, `cache/ip/*.txt`, `data/filters/*.txt`. See 15.9.6 + 3.
1999 - **July 3, 2026 Overnight leak + geo-blocking** Introduced `scripts/host-leak-probe.sh` (sub-second host-
egress detector, 15.6.1) and the statistical-threshold WARP auto-shutdown watcher (15.6.2); fixed the
overnight silent IP leak (15.6.3).
2000 - **July 2, 2026 `8c5fae1` + `181e1e1`** Two infinite routing loops fixed: host-VM tunnel loop (WARP bypass
routes via physical gateway IP + `setup_exit_node_routing` forcing default to `utun*`) and Docker Compose
subnet takeover (`10.200.1.0/24` lock). `--accept-routes=true` added explicitly to all `tailscale up --
reset` calls. See 15.2.2.
2001 - **July 2, 2026 auto-recovery daemon** `scripts/watcher.sh` + launchd LaunchAgent documented. See 15.5.1.
2002 - **July 1, 2026 `c5b92c0` (refactor: personal-leak cleanup)** Eliminated every residual personal-username
leak across source + config; launchd plist uses `WorkingDirectory` + relative program arg with the `
__NULLEXIT_HOME__` sentinel (install recipe gains a single `sed -i 's|__NULLEXIT_HOME__|$(pwd)|'` step);
8 devref prose sites genericized to `~/colima/...`, `~/Library/LaunchAgents/...`, `$USER`, `~USER`.
2003 - **July 1, 2026 `a5e2a5a` (refactor: script-relative paths)** Replaced 6 hardcoded `/Users/<user>/.../
nullexit/...` literals with script-relative resolution. `recover.sh`/`watcher.sh` inject `SCRIPT_DIR="$(cd
"${dirname "${BASH_SOURCE[0]}"}" && pwd)"`; `RECOVER="${SCRIPT_DIR}/../recover.sh"` resolves without launch-
path dependency.
2004 - **June 28, 2026 `f8f8e4e` (M1: scripts/ move)** 8 internal scripts consolidated into `scripts/`; the 4 user
-facing/orchestrator/config files (`toggle.sh`, `recover.sh`, `setup.sh`, `docker-compose.yml`) stayed at

```



```

repo root.
2005 - **June 28, 2026 `0875ef3` (ci: glob)** CI `py_compile` switched to `python3 -m py_compile scripts/*.py`
      after M1 left root with zero `.py` files.
2006 - **June 28, 2026 `25b37a1` (docs: scripts/ prefix)** `devref.md` + `README.md` updated to the `scripts/`
      prefix for all 8 moved files (~31 patches).
2007
2008 ### End-to-end verification (commit `c5b92c0`)
2009 Manual START STOP cycle ran clean on macOS after path relativization + personal-leak cleanup:
2010 - `bash toggle.sh` START: exit 0, ~135s (cold Colima boot); marker present (`2026-07-02T03:41:41Z`), all 5
      containers healthy, host DNS hijacked to `100.100.21.8` (no fallback), exit node selected, Cloudflare trace
      through WARP succeeds.
2011 - `bash toggle.sh` STOP: exit 0, ~12s; marker cleared, all nullexit containers gone, host DNS restored to
      `1.1.1.1`, Tailscale disconnected.
2012
2013 ## 18. TODO & Future Work
2014
2015 ### 18.1 TODO
2016 * **Decide toggle STOP/START from persisted *intent*, not a live probe:** ~~`toggle.sh` decided STOP-vs-START
      by calling `is_gateway_active()`~, which live-probes the running system (`docker compose ps` + DNS + SOCKS
      checks). When Colima/Docker was down the `docker compose ps` blocked for the full 15 s `run_with_timeout`
      window and, under `set -e` + the `ERR` trap, aborted the script *before the START body ran* so the path
      that would boot Colima never executed (`EXIT: code=1 phase=start`, seen 57 historically).~~ **Closed**
      added `gateway_should_be_running()` in `common.sh`, which reads persisted intent (`MARKER_FILE`) captured
      into `GATEWAY_MARKER_AT_STARTUP` before the defensive stale-marker clear, and only falls back to `
      is_gateway_active` when the marker is absent *and* the Docker socket is actually reachable (so a dead-
      socket probe can neither hang nor trip `set -e`). All four decision sites (`toggle.sh` main + `--restart`,
      `toggle-linux.sh` main + `--restart`) now use it, and the STOP `docker compose down` is `|| true`-guarded
      so a disable always completes even with Docker down. See 15.12 Misc Resolved.
2017 * **Verify Wi-Fi Roaming:** Investigate and test whether switching Wi-Fi networks now successfully and smoothly
      recovers the gateway end-to-end without any tailscale flag errors or dead tunnels. (Pending user
      verification).
2018 * **Fix Diagnostic Script Check 5/8:** ~~Investigate why `diagnose-host-leak.sh` check 5/8 (`Host default route
      `) sometimes reports "default route goes via physical Wi-Fi Tailscale route assertion failed" on macOS
      even though the `warp=on` egress checks confirm that traffic is successfully tunneling.~~ **Closed** fixed
      by switching check 5 from `netstat -rn | grep default` to `route -n get 1.1.1.1`. macOS Tailscale uses
      interface-scoped routes and does not replace the DHCP default route entry. See 15.7.1 Known Unknowns.
2019 * **Expose Tor Control Port (9051):** ~~Consider mapping the Tor Control port to localhost and adding `nyx` (
      the Tor terminal status monitor) to allow users to inspect their entry, middle, and exit node IPs in real-
      time.~~ **Closed** mapped the Tor Control Port to a procedurally generated, localhost-bound port `
      TOR_CONTROL_PORT`, configured `ControlPort` and disabled cookie authentication in `docker/tor/entrypoint.sh
      `, and integrated it into the `/sweep` verification protocol.
2020 * **Host-Only Tor Mode (Pluggable Egress):** Implement an `EGRESS_TYPE=tor_host_only` mode for users in heavily
      censored (DPI-restricted) environments. This mode would skip booting `warp` and `tailscale`, boot `
      adguardhome` and `tor` (using Telegram `obfs4` bridges), set AdGuard's upstream to Tor's `DNSPort`, and use
      the `pf` kill-switch to force all host Mac traffic strictly through the Tor network. This provides a 100%
      free, DPI-resistant evasion path without requiring an external VPS.
2021 * **The Technical Realities (What You Need to Watch Out For):**
2022 * **The UDP Problem:** Tor only transports TCP traffic. It does not support UDP (aside from DNS requests
      passed directly to its DNSPort). macOS is incredibly chatty over UDP (FaceTime, mDNS, QUIC protocols). Your
      pf rules defined in `scripts/pf.conf` must be configured to aggressively and silently DROP all host UDP
      traffic (except for local DNS queries to AdGuard). If you don't drop it cleanly, macOS services might hang
      indefinitely while waiting for timeouts.
2023 * **The "Daily Driver" Friction:** Routing a whole host OS through Tor is brutal on performance. Background
      daemons (iCloud sync, macOS software updates, Spotlight indexing) will blindly attempt to download
      gigabytes of data over Tor, which could saturate the circuits or time out completely.
2024 * **Bridge Rot:** State firewalls actively hunt and block obfs4 bridges. When a user's bridges burn out,
      they will lose all internet access due to the pf kill-switch. You will need to ensure the user has a
      frictionless way to update the `tor-bridges.txt` file and restart the Tor container without exposing their
      IP.
2025 * **Exit Node Discrimination:** Because all traffic will exit from known Tor nodes, the user will face an
      onslaught of Cloudflare CAPTCHAs, and many banking or streaming services will flat-out reject the
      connections.
2026
2027 ### 18.2 Future Work
2028 - **Direct P2P on VM Hosts:** Non-Linux hosts run Docker inside a VM, making true UDP hole-punching **to
      arbitrary external peers** impossible; those peers fall back to the DERP relay unless bridged mode is
      enabled on a trusted LAN. The only fully general solution is a native Linux host (Raspberry Pi, Intel NUC)
      where Docker runs without VM hypervisor translation. *(The one case that works regardless is direct **
      hostcontainer** P2P the gateway and its own Mac, the same physical machine re-permitted via routing-fix
      `.host_ips` RETURN rules and confirmed `direct` even in plain user-mode networking; see 15.3.5 and 15.8.7.)
      *
2029 - **Native Linux Deployment:** Benchmark on a Raspberry Pi native container footprint is ~75MB without macOS
      hypervisor overhead.
2030 - **Mesh-Wide Filesystem (SFTP over Tailscale):** Any mesh device passively exposes its filesystem over SFTP.
      Android via Termux + OpenSSH + wakelock; iOS is client-only due to background process limits.
2031 - **Post-Quantum Cryptography (PQC):** Tailscale uses Curve25519, theoretically vulnerable to "harvest now,
      decrypt later" quantum attacks. A future iteration could replace the Tailscale container with raw WireGuard
      + [Rosenpass](https://rosenpass.eu/) to negotiate post-quantum PSKs.
2032 - **Egress Compartmentalization (Pluggable Architecture):** See 12.9 for the full plan to make the entire
      egress layer swappable via a single `.env` variable.

```


5 diagrams.md

```
1 # nullexit Flow Diagrams & Architecture
2
3 ---
4
5 ## 1. System Architecture What's Running & Where
6
7 ```mermaid
8 graph TB
9     subgraph INTERNET[" Internet / Cloudflare Edge"]
10         CF["Cloudflare WARP\n(Exit Point)\nwarp=on"]
11     end
12
13     subgraph TAILNET[" Tailscale Mesh (100.x.x.x)"]
14         PHONE[" Phone / Laptop\n(Tailnet Client)"]
15     end
16
17     subgraph MACHOST[" macOS Host"]
18         direction TB
19         PFFIREWALL["macOS pf Firewall\n(Kill-Switch & TCP MSS Clamping)"]
20         TSDAEMON["tailscaled\n(brew service)\nhost Tailscale"]
21         HOSTDNS["Host DNS\n Gateway IP\n(hijacked by toggle.sh)"]
22         HOSTSOCKS["Host SOCKS5\n127.0.0.1:1080\n(fallback only)"]
23         HOSTTOR["Host Tor SOCKS5\n127.0.0.1:$TOR SOCKS_PORT\n(randomized)"]
24         HOSTTORCTRL["Host Tor Control\n127.0.0.1:$TOR_CONTROL_PORT\n(randomized)"]
25
26         subgraph MONITORS[" Background Daemons (toggle.sh children)"]
27             WARPWATCH["WARP Watcher\n(every 30s, via docker exec)\n output.log"]
28             LEAKPROBE["Host Leak Probe\n(every 300ms, via curl from HOST)\n output.log"]
29             DNSWATCHER["DNS Watcher\n(re-hijacks if drift detected)\n output.log"]
30             CAFFEINATE["caffeinate -i\n(sleep prevention)"]
31         end
32
33         subgraph COLIMAVM[" Colima VM (Linux, 600MB RAM)"]
34             subgraph NETNS["warp container network namespace (shared by ALL containers)"]
35                 GLUETUN["warp / Gluetun\nWireGuard tun0\n(owns the namespace)"]
36                 TS["tailscale container\nAdvertises exit node\ntailscale0 interface"]
37                 SOCKS["tailscale\nSOCKS5\nport 1080"]
38                 ADGUARD["adguardhome\nDNS sinkhole\nport 5335"]
39                 ROUTINGFIX["routing-fix sidecar\nGeo-IP Blocking & Go Logger\n(writes blocked.log)"]
40                 TOR["tor container\nSOCKS5: port 9050\nControl: port 9051\nTransparent Proxy: port 9040\n"]
41             end
42             nDNSPort["port 5353"]
43         end
44
45         subgraph LAUNCHD[" launchd (always-on)"]
46             WATCHER["scripts/watcher.sh\n(sleep/wake + Wi-Fi roam\n+ LAN P2P auto-detection)"]
47         end
48     end
49
50     subgraph RECOVERY[" Recovery"]
51         RECOVER["recover.sh\n(nuclear reset)"]
52         RECOVERPOST["recover.sh --post-wake\n(light refresh)"]
53     end
54
55     %% Traffic flow
56     PHONE -->|"WireGuard / Tailscale mesh"| TSDAEMON
57     TSDAEMON -->|"exit-node route utun*"| TS
58     TS -->|"FORWARD tun0 (MASQUERADE)"| GLUETUN
59     GLUETUN -->|"WireGuard UDP (macOS Host Egress)"| PFFIREWALL
60     PFFIREWALL -->|"TCP MSS Clamped / Kill-Switch check"| CF
61     CF -->|"198.18.0.0/15 PREROUTING redirect"| TOR
62     TOR -->|"internal path / exit nodes via tun0"| GLUETUN
63
64     %% DNS
65     HOSTDNS -->|"UDP:53 TCP:5354"| ADGUARD
66     ADGUARD -->|"DNS upstream via tun0"| CF
67     CF -->|"onion queries localhost:5353"| TOR
68
69     %% Monitoring
70     WARPWATCH -->|"docker exec warp wget cdn-cgi/trace"| GLUETUN
71     LEAKPROBE -->|"curl cdn-cgi/trace (HOST NIC)"| CF
72     DNSWATCHER -->|"networksetup -getdnsservers"| HOSTDNS
73
74     %% Logging
75     ROUTINGFIX -->|"writes"| BLOCKEDLOG["blocked.log\n(Host-side)"]
76     HOSTTOR -->|"port mapping"| TOR
77     HOSTTORCTRL -->|"port mapping"| TOR
```

```

78 %% Recovery triggers
79 WARPWATCH -->|"6 consecutive warp=off"| RECOVER
80 WATCHER -->|"sleep/wake / roam"| RECOVERPOST
81 RECOVERPOST -->|"if gateway unhealthy"| RECOVER
82 WATCHER -->|"writes .lan_p2p_detected"| ROUTINGFIX
83
84 style MONITORS fill:#1a1a2e,color:#e0e0ff
85 style COLIMAVM fill:#0d2137,color:#e0e0ff
86 style NETNS fill:#0a1628,color:#e0e0ff
87 style RECOVERY fill:#2d0a0a,color:#ffe0e0
88 style INTERNET fill:#0a2d1a,color:#e0ffe0
89 style TAILNET fill:#1a2d0a,color:#e8ffe0
90 style LAUNCHD fill:#2d1a2d,color:#ffe0ff
91 ...
92
93 ---
94
95 ## 2. toggle.sh Full START Flow
96
97 ``mermaid
98 flowchart TD
99     START([./toggle.sh])
100     CHECK{"is_gateway_active?\n(containers running OR\nDNS 1.1.1.1)"}
101
102     START --> CHECK
103     CHECK -->|"YES already ON"| STOPFLOW[" See STOP Flow"]
104     CHECK -->|"NO start it"| S1
105
106     S1["1. Reset DNS 1.1.1.1\n(prevent deadlocks)"]
107     S2["2. tailscale down\n(prevent exit-node routing during boot)"]
108     S3["3. Check Colima VM\n(colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged)"]
109     S4["4. Configure VM swap\n(400MB, prevent OOM)"]
110     S5["5. Clean corrupted AdGuard config\n(prevent container crash loop)"]
111     S6["6. docker compose up -d --build\n(compile Go threat rules & boot containers)"]
112     S7["7. Poll tailscale status\n(wait for 'offers exit node'\nup to 60s)"]
113     TSREADY{"container\nnTailscale ready?"}
114     ABORT(["ABORT\n cleanup_handler"])
115     S8["8. Resolve gateway Tailscale IP\n(.gateway_ip or docker exec)"]
116     S9["9. Verify tailscaled running\n(auto-start if needed)"]
117     S10["10. tailscale up --reset\n--exit-node=\n--accept-routes=true\n--ssh=true --accept-dns=false"]
118
119     PF1{"Pre-flight 1/3\ntailscale ping gateway"}
120     PF2{"Pre-flight 2/3\ndig +tcp AdGuard DNS"}
121     PF3{"Pre-flight 3/3\nWARP internet check"}
122     ALLPASS{"All 3 pass?"}
123
124     EXITPATH["EXIT NODE PATH\n tailscale up --exit-node=\n force_dns_to_gateway (3 retry)\n SOCKS5 disabled"]
125     SOCKSPATH["SOCKS5 FALLBACK PATH\n macOS SOCKS5 127.0.0.1:1080\n DNS proxy 127.0.0.1:53\n No exit node (\n SKIP_EXIT_NODE=true)"]
126
127     DAEMONS["Start background daemons\n caffeinate -i (sleep prevention)\n DNS Watcher\n WARP Watcher\n Host\n Leak Probe\n(all write output.log)"]
128
129     RULECOMPILE["Rule compiler status\n(log rule/IP counts)"]
130     WRITEMARKER["write_gateway_active_marker\n(/tmp/nullexit-gateway-active.marker)"]
131     DONE(["Gateway UP"])
132
133     S1 --> S2 --> S3 --> S4 --> S5 --> S6 --> S7
134     S7 --> TSREADY
135     TSREADY -->|"NO (40 NoState)"| ABORT
136     TSREADY -->|"YES"| S8
137     S8 --> S9 --> S10
138     S10 --> PF1 --> PF2 --> PF3
139     PF1 & PF2 & PF3 --> ALLPASS
140     ALLPASS -->|"YES"| EXITPATH
141     ALLPASS -->|"NO"| SOCKSPATH
142     EXITPATH --> DAEMONS
143     SOCKSPATH --> DAEMONS
144     DAEMONS --> RULECOMPILE --> WRITEMARKER --> DONE
145
146 style ABORT fill:#5c1010,color:#fff
147 style EXITPATH fill:#0a3d1a,color:#c0ffc0
148 style SOCKSPATH fill:#3d2d00,color:#ffe0a0
149 style DAEMONS fill:#0a1f3d,color:#c0d8ff
150 style DONE fill:#0a3d1a,color:#fff
151 ...
152
153 ---
154
155 ## 3. toggle.sh STOP Flow
156

```

```

157 ```mermaid
158 flowchart TD
159     STOP(["./toggle.sh\n(gateway already ON)"])
160
161     ST1["1. tailscale down\n(disconnect from mesh)"]
162     ST2["2. docker compose down -t 5\n(stop all containers)"]
163     ST3["3. Reset DNS 1.1.1.1\n(networksetup on all services)"]
164     ST4["4. stop_local_dns_proxy\n(kill Python UDP proxy)"]
165     ST5["5. stop_pidfile_daemon\n(kill caffeinate, rm PID)"]
166     ST6["6. stop_pidfile_daemon\n(kill DNS Watcher, rm PID)"]
167     ST7["7. stop_pidfile_daemon\n(kill WARP Watcher, rm PID)"]
168     ST8["8. stop_pidfile_daemon\n(kill Leak Probe, rm PID)"]
169     ST9["9. clear_gateway_active_marker\n(rm /tmp/nullexit-gateway-active.marker)"]
170     ST10["10. cleanup_network_state\n disable_all_proxies\n Flush DNS cache (dscacheutil)\n Flush stale routes\n n Power-cycle Wi-Fi (offon)"]
171
172     DONE([" Gateway DOWN\nInternet restored"])
173
174     STOP --> ST1 --> ST2 --> ST3 --> ST4
175     ST4 --> ST5 --> ST6 --> ST7 --> ST8 --> ST9 --> ST10 --> DONE
176
177     style DONE fill:#0a3d1a,color:#fff
178 ```
179
180 ---
181
182 ## 4. Monitoring Layer The 4 Background Daemons
183
184 ```mermaid
185 graph LR
186     subgraph DAEMONS["Background Daemons (all start with gateway, stop on teardown)"]
187         direction TB
188
189         subgraph D1[" Sleep Prevention"]
190             CAFF["caffeinate -i\nPID: /tmp/nullexit-caffeinate.pid\n\nKeeps Mac awake while\ngateway is active"]
191         end
192
193         subgraph D2[" DNS Watcher"]
194             DNSW["Polls networksetup every -30s\nPID: /tmp/nullexit-dns-watcher.pid\n\nIf DNS drifts from gateway IP\n re-hijacks automatically\n output.log"]
195         end
196
197         subgraph D3[" WARP Watcher"]
198             WARPW["Polls every 30s via:\ndocker compose exec warp wget cdn-cgi/trace\nPID: /tmp/nullexit-warp-watcher.pid\n\nCounts consecutive warp=off\n WARP_FAIL_THRESHOLD (default 6=30s)\n runs recover.sh (nuclear)\n output.log"]
199         end
200
201         subgraph D4[" Host Leak Probe"]
202             LEAKP["Polls every 300ms via:\ncurl cdn-cgi/trace (HOST NIC directly)\nPID: /tmp/nullexit-host-leak-probe.pid\n\nLogs ONLY on state change:\n LEAK / ROTATE / HOST-PROBE failed\nAll errors output.log\n(HOST_LEAK_PROBE=true in .env)"]
203         end
204     end
205
206     subgraph BLIND["What each monitor can/can't see"]
207         B1["WARP Watcher sees:\n Container WARP tunnel state\n Host NIC traffic\n Flash leaks < 5s"]
208         B2["Host Leak Probe sees:\n Host NIC egress (browser path)\n Sub-second transitions\n Container internals"]
209     end
210
211     D3 -->|"covers"| B1
212     D4 -->|"covers"| B2
213
214     style D1 fill:#1a2d0a
215     style D2 fill:#0a1a2d
216     style D3 fill:#2d0a2d
217     style D4 fill:#2d1a00
218     style BLIND fill:#1a1a1a,color:#aaa
219 ```
220
221 ---
222
223 ## 5. Traffic Flow Data Path (Normal Operation)
224
225 ```mermaid
226 sequenceDiagram
227     participant PHONE as Phone<br/>(Tailnet client)
228     participant TS_HOST as tailscaled<br/>(macOS host)
229     participant TS_CONTAINER as tailscale<br/>(container)

```

```

230 participant GLUETUN as Gluetun/warp<br/>(container)
231 participant CF as Cloudflare<br/>WARP Edge
232 participant INTERNET as Internet
233
234 Note over PHONE,INTERNET: Normal EXIT NODE path (all 3 pre-flights passed)
235
236 PHONE-->>TS_HOST: WireGuard packet<br/>(encrypted, UDP 41641)
237 TS_HOST-->>TS_CONTAINER: route via utun* tailscale0
238 TS_CONTAINER-->>GLUETUN: FORWARD chain<br/>(iptables MASQUERADE on tun0)
239 GLUETUN-->>TS_HOST: WireGuard tunnel (tun0) egresses macOS Host
240 Note over TS_HOST: macOS pf Firewall applies<br/>Kill-Switch & TCP MSS Clamping
241 TS_HOST-->>CF: re-encrypted UDP packet (MSS 1160)
242 CF-->>INTERNET: Plain HTTPS from<br/>Cloudflare IP
243
244 Note over PHONE,INTERNET: DNS Resolution path
245 PHONE-->>TS_HOST: DNS query
246 TS_HOST-->>TS_CONTAINER: UDP:53 TCP:5354 AdGuard :5335
247 TS_CONTAINER-->>GLUETUN: AdGuard upstream DNS via tun0
248 GLUETUN-->>CF: Encrypted DNS query
249 CF-->>GLUETUN: DNS response
250 GLUETUN-->>TS_CONTAINER: response
251 TS_CONTAINER-->>TS_HOST: response
252 TS_HOST-->>PHONE: DNS answer
253 ---
254
255 ---
256
257 ## 6. recover.sh Decision Tree
258
259 mermaid
260 flowchart TD
261     INVOKE(["recover.sh called"])
262     MODE["--post-wake\nflag?"]
263
264     subgraph POSTWAKE["--post-wake (light refresh, gateway stays UP)"]
265         PW1["Re-hijack DNS gateway IP"]
266         PW2["Flush DNS cache"]
267         PW3["Refresh tailscale exit-node"]
268         PW4["Force-recreate warp container\n(if Gluetun healthcheck failed)"]
269         PW5["Leave: containers, Wi-Fi,\ncaffeinate, DNS watcher,\nHost Leak Probe untouched"]
270     end
271
272     subgraph NUCLEAR["Default (nuclear gateway torn down)"]
273         N0["Sudo keep-alive loop (background)"]
274         N1["1. tailscale down\n(disconnect from mesh)"]
275         N2["2. Reset DNS 1.1.1.1\n(ALL network services)"]
276         N3["3. disable_all_proxies\n(all interfaces)"]
277         N4["4. Flush DNS cache\n(dscacheutil -flushcache)"]
278         N5["5. Flush stale routes\n(WARP endpoint routes)"]
279         N6a["6a. docker compose down -t 30\n(stop all containers)"]
280         N6b["6b. stop_pidfile_daemon\n(kill caffeinate)"]
281         N6c["6c. stop_pidfile_daemon\n(kill DNS Watcher)"]
282         N6d["6d. stop_pidfile_daemon\n(kill Leak Probe)"]
283         N7["7. Power-cycle Wi-Fi\n(networksetup offsleep 2on)"]
284         N8["8. Verify internet working\n(curl ifconfig.io)"]
285     end
286
287     DONE_PW["Gateway refreshed\n(still running)"]
288     DONE_NUC["Internet restored\n(gateway fully stopped)"]
289
290     INVOKE --> MODE
291     MODE -->|"yes"| PW1
292     PW1 --> PW2 --> PW3 --> PW4 --> PW5 --> DONE_PW
293     MODE -->|"no"| NO
294     NO --> N1 --> N2 --> N3 --> N4 --> N5
295     N5 --> N6a --> N6b --> N6c --> N6d --> N7 --> N8 --> DONE_NUC
296
297     style POSTWAKE fill:#0a2d1a,color:#c0ffc0
298     style NUCLEAR fill:#2d0505,color:#ffc0c0
299     style DONE_PW fill:#0a3d1a,color:#fff
300     style DONE_NUC fill:#3d1a00,color:#ffe0c0
301 ---
302 ---
303 ---
304
305 ## 7. Failure Paths & Self-Healing
306
307 mermaid
308 flowchart LR
309     subgraph EVENTS["Failure / Event"]
310         E1["WARP tunnel drops\n(warp=off)"]

```

```

311     E2["Mac wakes from sleep\nor Wi-Fi roams"]
312     E3["DNS drifts from\ngateway IP"]
313     E4["toggle.sh crashes /\nCtrl-C / SIGTERM"]
314     E5["Host-side flash leak\ndetected (HOST NIC)"]
315     E6["Silent NAT timeout\n(ping stops working)"]
316 end
317
318 subgraph DETECTORS["Detector"]
319     D1["WARP Watcher\n(30s poll, docker exec)"]
320     D2["scripts/watcher.sh\n(launchd, always on)"]
321     D3["DNS Watcher\n(30s poll, networksetup)"]
322     D4["cleanup_handler\n(ERR/INT/TERM/HUP trap)"]
323     D5["Host Leak Probe\n(300ms poll, HOST curl)"]
324     D6["routing-fix.sh\n(30s curl over tun0)"]
325 end
326
327 subgraph ACTIONS["Response"]
328     A1["threshold consecutive off\nrecover.sh (nuclear)"]
329     A2["recover.sh --post-wake\n(gentle refresh)"]
330     A3["Re-hijack DNS\n(networksetup setdnsservers)"]
331     A4["Stop all daemons\nReset DNS 1.1.1.1\nTear down Tailscale"]
332     A5["Log LEAK to output.log"]
333     A6["Blackhole internet traffic\n+ Drop TUNNEL_FAILED_CLOSED.marker"]
334     A7["Push macOS Desktop Notification\n(via watcher.sh listener)"]
335 end
336
337 E1 --> D1 --> A1
338 E2 --> D2 --> A2
339 E3 --> D3 --> A3
340 E4 --> D4 --> A4
341 E5 --> D5 --> A5
342 E6 --> D6 --> A6
343 A6 -. ->|"marker detected"| D2
344 D2 --> A7
345
346 style EVENTS fill:#2d1010,color:#ffcccc
347 style DETECTORS fill:#0a1a2d,color:#ccee0f
348 style ACTIONS fill:#0a2d0a,color:#ccffcc
349 ...
350
351 ---
352
353 ## 8. output.log Who Writes What
354
355 All events converge into a single `output.log` at the repo root.
356
357 | Writer | Event Types | Format |
358 |-----|-----|-----|
359 | `toggle.sh` | All startup/shutdown steps, errors, pre-flight results | Plain text with timestamps |
360 | `recover.sh` | Every recovery step (nuclear or post-wake) | Plain text |
361 | **WARP Watcher** | `WARP DOWN`, `WARP RECOVERED`, `WARP SHUTDOWN` | `[UTC timestamp]` prefix |
362 | **Host Leak Probe** | `LEAK`, `ROTATE`, `HOST-PROBE failed/timeout` | `[HH:MM:SS.mmm]` prefix |
363 | **DNS Watcher** | DNS re-hijack events | Appended inline |
364 | `docker compose` | Container logs on failure (last 100 lines of warp) | Dumped on ERR path |
365 | `routing-fix.sh` | (via `nullexit-logger` inside container) | Structured |
366
367 **Grep cheatsheet:**
368 ```bash
369 grep LEAK output.log # Host-side warp=off events (real leaks)
370 grep ROTATE output.log # Cloudflare edge IP rotations (harmless)
371 grep 'HOST-PROBE' output.log # curl failures from probe
372 grep 'WARP DOWN' output.log # Container-side tunnel drops
373 grep 'WARP SHUTDOWN' output.log # Auto-recovery triggered
374 grep 'WARP RECOVERED' output.log # Self-healed before threshold
375 ```

```

6 toggle.sh

```

1 #!/bin/bash
2 # toggle.sh nullexit gateway toggle script (macOS + Linux)
3 # Automatically handles Docker containers, Colima, DNS hijacking, and Tailscale exit node routing.
4 #
5 # One file, both OSes: the dispatch below runs the self-contained Linux
6 # implementation (shuf / ss / nmcli / ip / systemd) and exits; on macOS it
7 # falls through to the macOS implementation (jot / netstat / networksetup,
8 # launchd / pf).

```

```

9
10 set -e
11
12 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
13
14 #
15 # LINUX TOGGLE self-contained; runs the full toggle then exits before the
16 # macOS body below. Native Linux tooling (shuf / ss / nmcli / ip / systemd).
17 # (Body inlined verbatim at column 0 it contains multiline `bash -c ""`
18 # strings that must NOT be re-indented.)
19 #
20 if [[ "$OSTYPE" != "darwin"* ]]; then
21 # Define log file (used by the lifecycle breadcrumb + run_logged helper)
22 LOG_FILE="$PWD/output.log"
23
24 # --- LIFECYCLE PHASE TRACKING + EXIT BREADCRUMB ---
25 # TOGGLE_PHASE is updated as the script moves through STOP/START so that if the
26 # process is killed (terminal closed, pkill, OOM) or exits on error, the EXIT
27 # trap records the final exit code AND the phase it died in making an
28 # interrupted START traceable in output.log instead of ending abruptly.
29 TOGGLE_PHASE="init"
30 _log_exit_breadcrumb() {
31     local rc=$?
32     echo "[$(date '+%Y-%m-%d %H:%M:%S')] toggle.sh EXIT: code=$rc phase=$TOGGLE_PHASE (pid $$)" >> "$LOG_FILE"
33     2>/dev/null || true
34 }
35 trap '_log_exit_breadcrumb' EXIT
36
37 # --- DEBUG TRACE (opt-in via .env: DEBUG_TRACE=true) ---
38 # When enabled, mirror a full command-by-command xtrace to output.log. Off by
39 # default. Read directly from .env because common.sh (read_env_var) isn't
40 # sourced yet. Branch on bash version: BASH_XTRACEFD needs >= 4.1 (Linux
41 # usually has bash 5); fall back to teeing stderr on older bash. We never use
42 # the `exec {var}>>` dynamic-fd form (4.1+ only).
43 DEBUG_TRACE=$(grep -E '^DEBUG_TRACE=' .env 2>/dev/null | cut -d '=' -f2- | tr -d '\'" | tr '[:upper:]' '[:lower:]' | tr -d '[:space:]')
44 if [ "$DEBUG_TRACE" = "true" ]; then
45     echo "[$(date '+%Y-%m-%d %H:%M:%S')] DEBUG_TRACE enabled xtrace mirrored to output.log (bash ${BASH_VERSINFO[0]}.${BASH_VERSINFO[1]})" >> "$LOG_FILE"
46     export PS4='+ [$(date '+%H:%M:%S')] ${BASH_SOURCE##*/}:${LINENO}:${FUNCNAME[0]:-main}: '
47     if [ "${BASH_VERSINFO[0]}" -gt 4 ] || { [ "${BASH_VERSINFO[0]}" -eq 4 ] && [ "${BASH_VERSINFO[1]}" -ge 1 ]; }; then
48         exec 9>>"$LOG_FILE"
49         export BASH_XTRACEFD=9
50         set -x
51     else
52         # bash < 4.1: plain stderrfile redirect. NOT a process substitution under
53         # set -e, xtrace flowing through a backgrounded `tee` can abort the script
54         # (failed write/SIGPIPE trips set -e). See devref 15.11.8.
55         exec 2>>"$LOG_FILE"
56         set -x
57     fi
58 fi
59
60 # --- PROCEDURAL PORT GENERATION ---
61 # To prevent static port fingerprinting by malware, we randomize exposed ports
62 # on every boot and persist them to a hidden environment file.
63 PORTS_FILE="/tmp/nullexit-ports.env"
64 if [[ "$1" == "" || "$1" == "--restart" || "$1" == "restart" ]]; then
65
66     # Collision-avoidance function
67     GENERATED_PORTS=""
68     get_free_port() {
69         local port
70         while true; do
71             # Linux uses shuf or $RANDOM instead of jot
72             port=$(shuf -i 10000-65000 -n 1 2>/dev/null || echo $((10000 + RANDOM % 55000)))
73             # 1. Prevent self-collision within this loop
74             if [[ "$GENERATED_PORTS" == *"$port"* ]]; then
75                 continue
76             fi
77             # 2. Prevent collision with ANY process on the machine (root daemons, terminal apps, etc)
78             # We use ss to read the kernel socket table directly.
79             if ! ss -ltn | awk '{print $4}' | grep -q ":$port$"; then
80                 echo "$port"
81                 return
82             fi
83         done
84     }
85     export TOR SOCKS_PORT=$(get_free_port)

```



```

86 GENERATED_PORTS="$GENERATED_PORTS $TOR SOCKS_PORT"
87
88 export TOR_CONTROL_PORT=$(get_free_port)
89 GENERATED_PORTS="$GENERATED_PORTS $TOR_CONTROL_PORT"
90
91 export SOCKS_PROXY_PORT=$(get_free_port)
92 GENERATED_PORTS="$GENERATED_PORTS $SOCKS_PROXY_PORT"
93
94 export DNS_PROXY_PORT=$(get_free_port)
95
96 echo "export TOR SOCKS_PORT=$TOR SOCKS_PORT" > "$PORTS_FILE"
97 echo "export TOR_CONTROL_PORT=$TOR_CONTROL_PORT" >> "$PORTS_FILE"
98 echo "export SOCKS_PROXY_PORT=$SOCKS_PROXY_PORT" >> "$PORTS_FILE"
99 echo "export DNS_PROXY_PORT=$DNS_PROXY_PORT" >> "$PORTS_FILE"
100 elif [ -f "$PORTS_FILE" ]; then
101     # On STOP, source the existing ports so docker-compose down matches
102     source "$PORTS_FILE"
103 else
104     # Fallbacks if file is lost
105     export TOR SOCKS_PORT=9050
106     export TOR_CONTROL_PORT=9051
107     export SOCKS_PROXY_PORT=1080
108     export DNS_PROXY_PORT=5354
109 fi
110
111 # Start execution timer
112 TOGGLE_START_TIME=$SECONDS
113
114 if [[ "$1" == "restart" ]] || [[ "$1" == "--restart" ]]; then
115     echo "Executing Gateway Restart Sequence..."
116     # We must source common.sh here temporarily to check gateway state before we proceed
117     source "$SCRIPT_DIR/scripts/common.sh"
118     # Use persisted intent via gateway_state (echoes a string, never a return code;
119     # set -e-safe under set -x see devref 15.11.8). On Linux the marker is not
120     # maintained, so this degrades to the is_gateway_active fallback (fast native Docker).
121     if [ "$(gateway_state)" = "running" ]; then
122         echo "Gateway is currently running. Stopping it first..."
123         bash "$0"
124         echo "Gateway stopped. Now starting it..."
125     else
126         echo "Gateway is already stopped. Starting it..."
127     fi
128     bash "$0"
129     exit 0
130 fi
131
132 # Global flag to track if the script completed successfully
133 SUCCESS_RUN=false
134
135 # Global variable to track the currently running background command PID
136 CURRENT_BG_PID=""
137
138 # Source common bash functions
139 source "$SCRIPT_DIR/scripts/common.sh"
140
141
142 PID_FILE="/tmp/nullexit-cafeinate.pid"
143
144 # Start macOS caffeinate to prevent sleep while gateway is running
145 start_sleep_prevention() {
146     if ! command -v systemd-inhibit >/dev/null 2>&1; then
147         echo " [Warning] systemd-inhibit tool not found. Sleep prevention unavailable."
148         return 1
149     fi
150
151     # Stop any existing sleep prevention process first to avoid duplicates
152     stop_sleep_prevention
153
154     echo " Enabling system sleep prevention (systemd-inhibit)..."
155     # Run systemd-inhibit wrapped in a bash trap to prevent idle system sleep.
156     # Critically, this traps shutdown signals (SIGTERM) and automatically
157     # flushes hijacked DNS back to normal right before the PC powers off.
158     # We do not use recover.sh here because it is a heavy recovery script
159     # which takes too long and gets terminated before it can reset the DNS. Instead,
160     # we surgically and instantly restore normal DNS settings here.
161     nohup bash -c "
162         trap '
163             echo \"[Shutdown Trap] Reverting network settings...\" >> \"$PWD/output.log\" 2>&1
164             kill \$! 2>/dev/null || true
165             sudo -n resolvectl domain \"\$ACTIVE_SERVICE\" \"\" >> \"$PWD/output.log\" 2>&1 || true
166             sudo -n resolvectl revert \"\$ACTIVE_SERVICE\" >> \"$PWD/output.log\" 2>&1 || true

```

```

167     resolvectl domain \"\$ACTIVE_SERVICE\" \"\" >> \"\$PWD/output.log\" 2>&1 || true
168     resolvectl revert \"\$ACTIVE_SERVICE\" >> \"\$PWD/output.log\" 2>&1 || true
169     if command -v tailscale >/dev/null 2>&1; then
170         tailscale up --accept-dns=false --exit-node= >> \"\$PWD/output.log\" 2>&1 &
171         TS_DOWN_ARGS=\"\"
172         if [ \"\$KILL_SWITCH\" = \"true\" ]; then TS_DOWN_ARGS=\"--accept-risk=lose-ssh\"; fi
173         tailscale down \"\$TS_DOWN_ARGS >> \"\$PWD/output.log\" 2>&1 &
174     fi
175     sleep 1
176     echo \"[Shutdown Trap] Cleanup complete.\" >> \"\$PWD/output.log\" 2>&1
177     exit 0
178     ' SIGTERM SIGINT SIGHUP
179     systemd-inhibit --what=sleep --who=nullexit --why=\"gateway running\" --mode=block sleep infinity &
180     wait \$!
181     \" >> output.log 2>&1 &
182     local caffe_pid=$!
183     echo \"\$caffe_pid\" > \"\$PID_FILE\"
184     echo \" Sleep prevention active (PID \$caffe_pid). Your PC won't sleep while the gateway is running.\"
185 }
186
187 # Stop sleep prevention when gateway is stopped
188 stop_sleep_prevention() {
189     if [ -f \"\$PID_FILE\" ]; then
190         local caffe_pid
191         caffe_pid=$(cat \"\$PID_FILE\")
192         if [ -n \"\$caffe_pid\" ] && kill -0 \"\$caffe_pid\" 2>/dev/null && ps -p \"\$caffe_pid\" -o command= 2>/dev/null |
193             grep -q -E \"systemd-inhibit|recover.sh\"; then
194             echo \" Stopping system sleep prevention (systemd-inhibit wrapper PID \$caffe_pid)...\"
195             kill \"\$caffe_pid\" 2>/dev/null || true
196         fi
197         rm -f \"\$PID_FILE\"
198     fi
199 }
200 DNS_WATCHER_PID_FILE=\"/tmp/nullexit-dns-watcher.pid\"
201
202 stop_dns_watcher() {
203     if [ -f \"\$DNS_WATCHER_PID_FILE\" ]; then
204         local watcher_pid
205         watcher_pid=$(cat \"\$DNS_WATCHER_PID_FILE\")
206         if [ -n \"\$watcher_pid\" ] && kill -0 \"\$watcher_pid\" 2>/dev/null; then
207             echo \" Stopping background DNS Watcher (PID \$watcher_pid)...\"
208             kill -9 \"\$watcher_pid\" 2>/dev/null || true
209         fi
210         rm -f \"\$DNS_WATCHER_PID_FILE\"
211     fi
212 }
213
214 # Cleanup handler to restore DNS to 1.1.1.1 on error or user interrupt (Ctrl+C / SIGTERM / SIGHUP)
215 cleanup_handler() {
216     local exit_code=$?
217     local trigger_type=\"$1\"
218     local line_no=\"$2\"
219
220     if [ \"\$SUCCESS_RUN\" = \"false\" ]; then
221         echo -e \"\n[Self-Correction] Script interrupted or failed ($trigger_type). Restoring host DNS to 1.1.1.1...\"
222         if [ -n \"\$ACTIVE_SERVICE\" ]; then
223             reset_dns
224         fi
225
226         if [ -n \"\$CURRENT_BG_PID\" ]; then
227             echo \"Terminating active command (PID \$CURRENT_BG_PID)...\"
228             kill -15 \"\$CURRENT_BG_PID\" 2>> output.log || true
229         fi
230
231         # Disconnect host Tailscale and close the GUI application on failure
232         disconnect_tailscale_host
233
234         # Stop local DNS proxy if running
235         stop_local_dns_proxy
236
237         # Stop sleep prevention
238         stop_sleep_prevention
239         stop_dns_watcher
240
241         # Nuke leftover network state (proxies, routes, DNS cache, Wi-Fi)
242         if [ -n \"\$ACTIVE_SERVICE\" ]; then
243             cleanup_network_state
244         fi
245     fi

```

```

246
247     if [ "$trigger_type" = "ERR" ]; then
248         echo -e "\n=====
249         echo "ERROR: Script failed at line $line_no with exit code $exit_code."
250         echo "=====
251         echo "Troubleshooting steps:"
252         echo "1. Run 'colima status' to verify the VM is active."
253         echo "2. Run 'docker compose ps' to check container health."
254         echo "3. Run 'docker compose logs' to view application logs."
255         echo "4. Run 'tailscale status' to check host Tailscale status."
256         echo "=====
257     fi
258 fi
259 }
260
261 trap 'cleanup_handler ERR $LINENO' ERR
262 trap 'cleanup_handler INT' INT
263 trap 'cleanup_handler TERM' TERM
264 trap 'cleanup_handler HUP' HUP
265
266 # NOPASSWD sudo
267 # The user has NOPASSWD in /etc/sudoers.d/nullexit for the specific commands
268 # needed (resolvectl, ip, systemctl, systemd-inhibit, pkill, and
269 # python3 scripts/dns-proxy.py for the fallback DNS proxy).
270 # All privileged calls below use `sudo -n` which will run without prompting.
271 # No credential caching loop is needed.
272
273 # Add Homebrew and standard paths
274 export PATH="/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH"
275
276 # Check for local DNS proxy tools (used when tailnet data plane is unavailable)
277 # Uses a Python DNS proxy (handles TCP wire format correctly).
278 # Python3 is built into macOS no external dependencies.
279 DNS_PROXY_BIN=""
280 if command -v python3 >/dev/null 2>&1; then
281     DNS_PROXY_BIN="python3 -I"
282 elif command -v python >/dev/null 2>&1; then
283     DNS_PROXY_BIN="python -I"
284 fi
285
286 # Path to the DNS proxy script (sibling of this script)
287 DNS_PROXY_SCRIPT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/dns-proxy.py"
288
289 # Resolve Tailscale CLI on host. We rely on the standalone Homebrew formula
290 # (`brew install tailscale` + `systemctl start tailscaled`) NOT the
291 # Tailscale.app GUI so there is no .app bundle or Network Extension sandbox
292 # to launch, no menu-bar click to perform, and no scutil Network Service to
293 # start manually. tailscaled runs as a per-user LaunchAgent (or LaunchDaemon
294 # if you ran the install with sudo) and the control socket is always available.
295 TS_BIN=""
296 if command -v tailscale >> output.log 2>&1; then
297     TS_BIN="tailscale"
298 fi
299
300 # Baseline: disable Tailscale DNS management at the daemon level.
301 # `tailscale set` writes a persistent preference. However, `tailscale up --reset`
302 # (used in steps 7 and 9) RESETS all unspecified flags to defaults, which would
303 # re-enable `--accept-dns=true` and nullify this `set`.
304 # Therefore, EVERY `tailscale up --reset` call in this file MUST also pass
305 # `--accept-dns=false` explicitly. See steps 7 and 9 for the fix.
306 #
307 # This initial `set` is still useful as a baseline for non-reset operations
308 # (e.g. if the user runs `tailscale up` manually without --reset) and covers
309 # the brief window between this line and the first `--reset` call.
310 #
311 # Runs once at script start while tailscaled is still connected (if it was
312 # left running from a previous toggle), so the pref takes effect before any
313 # disconnection or reconnection logic.
314 if [ -n "$TS_BIN" ]; then
315     $TS_BIN set --accept-dns=false >> output.log 2>&1 || true
316 fi
317
318 # Disconnect host Tailscale from the mesh. With the standalone daemon, this is the
319 # *entire* teardown no need to mess with system network managers.
320 # tailscaled keeps running (as a systemd service), which is intentional:
321 # the next `tailscale up` then reconnects in seconds rather than waiting for a
322 # slow daemon launch. The `tailscale down` call also retains the user's
323 # auth state, so we don't force a browser re-login every cycle.
324 #
325 # Defined early (before the if/else branches and referenced by cleanup_handler's
326 # ERR trap) so that any failure before the main logic can still tear down Tailscale.

```

```

327 restart_tailscaled_daemon() {
328     echo " [Tailscale Recovery] tailscaled daemon appears to be wedged/unresponsive."
329     echo " [Tailscale Recovery] Attempting to restart tailscaled service via systemctl..."
330     if run_with_timeout 15 sudo -n systemctl restart tailscaled >> output.log 2>&1; then
331         echo " [Tailscale Recovery] Successfully restarted tailscaled service."
332     else
333         echo " [Tailscale Recovery] WARNING: Failed to restart tailscaled. Manual intervention may be needed."
334     fi
335     sleep 3
336 }
337 disconnect_tailscale_host() {
338     if [ -n "$TS_BIN" ]; then
339         echo "Disconnecting host Tailscale from mesh (tailscaled stays running as system service)..."
340         local ts_args=""
341         if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
342         if ! run_with_timeout 10 $TS_BIN down $ts_args >> output.log 2>&1; then
343             restart_tailscaled_daemon
344             run_with_timeout 10 $TS_BIN down $ts_args >> output.log 2>&1 || true
345         fi
346     fi
347 }
348
349 ACTIVE_SERVICE=$(get_active_service)
350
351 # Helper to nuke leftover network state after teardown (proxy settings, routing
352 # table, DNS cache, Wi-Fi). Prevents the "had to write a whole recovery script"
353 # problem where stale state kills internet after the gateway stops.
354 cleanup_network_state() {
355     echo -e "\nCleaning up network state (proxies, routes, DNS cache, Wi-Fi)..."
356
357     # 1. Disable any leftover proxy settings (needs root on many macOS versions)
358     # (Skipped for Linux since we don't set global SOCKS proxy)
359     echo " Proxies disabled."
360
361     # 2. Flush DNS cache
362     if command -v resolvectl >> output.log 2>&1; then
363         sudo -n resolvectl flush-caches 2>> output.log || true
364     elif command -v systemd-resolve >> output.log 2>&1; then
365         sudo -n systemd-resolve --flush-caches 2>> output.log || true
366     fi
367     echo " DNS cache flushed."
368
369     # 3. Flush stale routing table entries
370     if command -v ip >> output.log 2>&1; then
371         sudo -n ip route flush cache >> output.log 2>&1 || true
372     fi
373     echo " Routing table flushed."
374
375     # 4. Power-cycle Wi-Fi to clear any lingering interface state
376     echo " Restarting network interface..."
377     sudo -n ip link set dev "$ACTIVE_SERVICE" down 2>> output.log || true
378     sleep 1
379     sudo -n ip link set dev "$ACTIVE_SERVICE" up 2>> output.log || true
380     echo " Interface power-cycled ($ACTIVE_SERVICE)."
381     sleep 3
382
383     echo "Network state cleanup complete."
384 }
385
386 # Ensure DNS is set to 1.1.1.1 immediately at script start to prevent any DNS deadlocks/hangs
387 # during status checks, container startup, VM boot, or teardown. We *also* clear any leftover
388 # `ts.net` search domain from a previous `tailscale up --accept-dns=true` run, otherwise macOS
389 # would prepend it to every lookup and most public DNS queries would resolve to NXDOMAIN.
390 # Capture current DNS before resetting so we can check if it was hijacked
391 INITIAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>/dev/null | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | head
392 -1 || true)
393
394 echo "Initializing DNS to 1.1.1.1 to ensure reliable internet access..."
395 reset_dns
396
397 # Local Network Surveillance Check
398 echo -e "\n"
399 echo "Local Network Surveillance Check"
400 echo ""
401 # Count populated ARP cache entries to detect network scanning or high congestion
402 if command -v ip >> /dev/null 2>&1; then
403     ARP_COUNT=$(ip neigh | grep -iv 'incomplete' | wc -l | tr -d ' ')
404 else
405     # Using '-an' avoids hanging on DNS reverse resolution when the network state is in transition.
406     ARP_COUNT=$(arp -an 2>/dev/null | grep -iv 'incomplete' | wc -l | tr -d ' ')

```

```

407 fi
408 if [ "$ARP_COUNT" -gt 15 ]; then
409     echo -e " \033[1;33m[!] WARNING: High ARP activity detected ($ARP_COUNT devices in cache).\033[0m"
410     echo -e " \033[1;33m[!] This Wi-Fi network may be heavily congested or actively scanned (e.g., arp-scan)
        .\033[0m"
411     echo -e " [] Your traffic remains fully encrypted and invisible.\n"
412 else
413     echo -e " [] Local network looks quiet ($ARP_COUNT devices). No aggressive scanning detected.\n"
414 fi
415
416 echo "Checking Gateway Status..."
417 # Decide STOP vs START from persisted intent via gateway_state, which echoes a
418 # string (never a return code) to stay set -e-safe under set -x. See devref 15.11.8.
419 # On Linux this falls back to is_gateway_active (fast native Docker).
420 if [ "$(gateway_state)" = "running" ]; then
421     TOGGLE_PHASE="stop"
422     echo -e "\n=====
423     echo "Gateway is RUNNING. Stopping it now..."
424     echo -e "=====\\n"
425
426 # 1. Disconnect host Tailscale cleanly first before stopping the exit-node container
427 disconnect_tailscale_host
428 echo ""
429
430 echo "Stopping Docker containers..."
431 # `|| true`: a disable must always complete even if the Docker daemon is down.
432 run_logged docker compose down -t 5 || true
433
434 echo -e "\nDocker daemon handles container teardown automatically."
435
436 # The host's DNS was hijacked to the gateway IP during ENABLE; now that the
437 # gateway is down, restore DNS to 1.1.1.1 immediately so subsequent lookups
438 # don't stall for the macOS DNS timeout before the 1.1.1.1 fallback engages.
439 echo -e "\nRestoring host DNS to 1.1.1.1 (gateway is gone)... "
440 reset_dns
441
442 # 3. Stop local DNS proxy if running
443 stop_local_dns_proxy
444
445 # Stop sleep prevention
446 stop_sleep_prevention
447 stop_dns_watcher
448
449 # 4. Nuke leftover network state so internet actually works after teardown
450 cleanup_network_state
451
452 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
453 echo -e "\nGateway has been successfully STOPPED in ${ELAPSED} seconds."
454 else
455     TOGGLE_PHASE="start"
456     echo -e "\n=====
457     echo "Gateway is STOPPED. Starting it now..."
458     echo -e "=====\\n"
459
460 # 1. Prevent host exit-node deadlock during VM / Container startup
461 disconnect_tailscale_host
462
463 echo -e "\nUsing native Linux Docker..."
464
465 # 4. Clean up corrupted AdGuardHome configurations
466 if [ -f "adguard/conf/AdGuardHome.yaml" ] && [ ! -s "adguard/conf/AdGuardHome.yaml" ]; then
467     rm -f "adguard/conf/AdGuardHome.yaml"
468     echo "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop."
469 fi
470
471
472
473 # 5. Start compose services
474 write_host_ips
475 # Procedurally generated ports have already been exported at the top of the script.
476 echo " [Security] Procedurally randomized proxy ports generated to evade fingerprinting."
477
478 # --- HONEY-PORT TRIPWIRE ---
479 # Generate a random trap port. If any malware does a sequential port scan on localhost,
480 # it will inevitably hit this port. When it does, we instantly fire a desktop alert.
481 HONEY_PORT=$(get_free_port)
482 (
483     if nc -l -p "$HONEY_PORT" 127.0.0.1 >/dev/null 2>&1 || nc -l localhost "$HONEY_PORT" >/dev/null 2>&1; then
484         echo "[$(date +%Y-%m-%d %H:%M:%S)] [CRITICAL SECURITY ALERT] PORT SCAN DETECTED! An unknown application
            just pinged the local Honey-Port ($HONEY_PORT)!" >> "$PWD/output.log"
485         if command -v notify-send >/dev/null 2>&1; then

```

```

486     notify-send -u critical "nullexit OPSEC Alert" "An unknown application is aggressively scanning your
        local ports. Malware port-scan suspected."
487     fi
488 fi
489 ) &
490 disown %1 2>/dev/null || true
491 echo " [Security] Honey-Port Tripwire armed on random port to detect malware port-scans."
492
493 echo -e "\nStarting Docker containers..."
494 run_logged docker compose up -d
495
496 # Clean up the one-off rule compiler container so it doesn't clutter the Docker UI
497 docker compose rm -s -f rule-compiler >> output.log 2>&1
498
499 # 6. Wait for the gateway container's Tailscale connection to be ready
500 BYPASS_PING=$(read_env_var GATEWAY_BYPASS_PING | tr '[:upper:]' '[:lower:]')
501 USE_EXIT_NODE=$(read_env_var GATEWAY_USE_EXIT_NODE | tr '[:upper:]' '[:lower:]')
502
503 echo "Waiting for gateway container's Tailscale to connect to the tailnet..."
504 echo " (This can take 30-60s Tailscale goes through: NoState Starting Running Online)"
505 connected=false
506 consecutive_failures=0
507 for i in {1..60}; do
508     # Run tailscale status and capture its output so the user can see what's happening.
509     # Previously this was hidden behind 2>> output.log, making it impossible to debug hangs.
510     ts_output=$(run_with_timeout 5 docker compose exec -T tailscale tailscale status 2>&1 || true)
511
512     if echo "$ts_output" | grep -q "offers exit node"; then
513         connected=true
514         break
515     fi
516
517     # Track consecutive failures to detect a stuck state.
518     # If we see 40+ consecutive 'NoState' responses, give up early
519     # the daemon is alive but can't connect to the coordination server.
520     if echo "$ts_output" | grep -q "NoState"; then
521         consecutive_failures=$((consecutive_failures + 1))
522         if [ "$consecutive_failures" -ge 40 ]; then
523             echo ""
524             echo " [attempt $i/60] Stuck in 'NoState' for $consecutive_failures checks."
525             echo " Tailscale daemon is running but can't reach the coordination server."
526             break
527         fi
528     else
529         consecutive_failures=0
530     fi
531
532     # Print a condensed status every 10 seconds so the user can see progress
533     if [ $((i % 10)) -eq 0 ]; then
534         ts_short=$(echo "$ts_output" | head -1 | tr -d '\r\n')
535         echo " [attempt $i/60] $ts_short"
536     elif [ $((i % 5)) -eq 0 ]; then
537         echo -n "[i]"
538     else
539         echo -n "."
540     fi
541     sleep 1
542 done
543 echo ""
544
545 if [ "$connected" = "true" ]; then
546     echo "Gateway container is online on the Tailscale mesh."
547 elif [ "$BYPASS_PING" = "true" ]; then
548     echo "[Warning] Gateway container check timed out. Proceeding anyway (GATEWAY_BYPASS_PING is true)..."
549 else
550     echo "ERROR: Gateway container failed to initialize Tailscale (timed out after 60s)." >&2
551     echo " The container was seen in the following states during the wait:" >&2
552     echo "$ts_output" | sed 's/^/ /' >&2
553     echo "" >&2
554     echo " This could mean:" >&2
555     echo " 1. Tailscale auth key has expired check TS_AUTHKEY in .env" >&2
556     echo " 2. Network issue inside the container check: docker compose logs tailscale" >&2
557     echo " 3. gluetun VPN tunnel not connecting check: docker compose logs warp | tail -20" >&2
558     echo "" >&2
559     echo " Diagnose manually:" >&2
560     echo " docker compose exec -T tailscale tailscale status" >&2
561     echo " docker compose exec -T tailscale tailscale netcheck" >&2
562 cleanup_handler ERR $LINENO
563 exit 1
564 fi
565

```



```

566 # 6b. Resolve the gateway's Tailscale IP.
567 # Dynamically query the container for the IP (handles IP changes after re-auth)
568 # and cache it in .gateway_ip for fast retrieval by recover.sh during Wi-Fi roaming.
569 #
570 # CRITICAL: `docker compose exec -T` on macOS/Colima injects carriage
571 # returns (\r) into captured output. If $TS_IP ends with \r, then
572 # `networksetup -setdnsservers` silently rejects the malformed IP and
573 # DNS stays at 1.1.1.1 the primary bug this project was hitting.
574 # `tr -d '\r'` strips the CR; `awk 'NR==1{print $1; exit}'` takes only
575 # the first field of the first line.
576 TS_IP=""
577 if [ "$connected" = "true" ]; then
578   TS_IP=$(run_with_timeout 10 docker compose exec -T tailscale tailscale ip -4 2>> output.log | tr -d '\r' |
579     awk 'NR==1{print $1; exit}' || true)
580   if [ -n "$TS_IP" ]; then
581     echo "Resolved gateway Tailscale IP dynamically: $TS_IP"
582     echo "$TS_IP" > .gateway_ip
583   else
584     echo " [!] Failed to resolve gateway IP dynamically. Trying fallback cache..." >&2
585     TS_IP=$(read_adguard_ip || true)
586   fi
587 else
588   TS_IP=$(read_adguard_ip || true)
589 fi
590
591 # 7. Connect host Mac to Tailscale mesh.
592 # With the standalone tailscaled daemon (registered as a per-user LaunchAgent or
593 # system LaunchDaemon via 'systemctl start tailscaled'), the previous five-
594 # phase flow collapses into two trivial CLI calls. There is no .app GUI to launch,
595 # no menu bar item to click, and no Accessibility permission required.
596 #
597 # CRITICAL: If tailscaled is not running, `tailscale up` will silently fail.
598 # We auto-start it before proceeding, and SKIP the rest of the Tailscale setup
599 # if it can't be started (DNS hijack to a 100.x.x.x IP and exit-node routing
600 # are impossible without the host on the mesh).
601 #
602 # We connect in two phases:
603 #   Phase A Join mesh WITHOUT exit node (so pre-flight checks can run)
604 #   Phase B Set exit node only after pre-flight checks confirm the gateway works
605 HOST_ON_MESH=false
606 SKIP_EXIT_NODE=false
607 if [ -n "$TS_BIN" ]; then
608   echo -n "Verifying tailscaled is reachable"
609   daemon_ready=false
610   for i in {1..10}; do
611     # `tailscale status` returns exit code 1 when the daemon is alive but
612     # disconnected ("Tailscale is stopped."). We check the daemon socket
613     # directly as a fallback if the socket exists, tailscaled is running
614     # even if the host isn't connected to the mesh yet.
615     if run_with_timeout 5 $TS_BIN status >> output.log 2>&1 || [ -S /var/run/tailscaled.socket ]; then
616       daemon_ready=true
617       break
618     fi
619     echo -n "."
620     sleep 1
621   done
622   echo ""
623   if [ "$daemon_ready" != "true" ]; then
624     echo -e "\033[0;33m[!] tailscaled daemon is not running. Attempting to start it...\033[0m"
625
626     # Try system-wide daemon (with timeout sudo may prompt for password)
627     if [ "$daemon_ready" != "true" ]; then
628       if run_with_timeout 10 sudo systemctl start tailscaled 2>> output.log; then
629         echo " Started tailscaled as system LaunchDaemon."
630         for i in {1..15}; do
631           if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
632             daemon_ready=true
633             break
634           fi
635           sleep 1
636         done
637       fi
638     fi
639   fi
640
641   if [ "$daemon_ready" = "true" ]; then
642     echo "tailscaled is responsive (running as a system service)."
643
644     # Phase A: Join mesh without exit node
645     echo "Connecting host to Tailscale mesh (no exit node yet)..."

```

```

646 if $TS_BIN up --reset --ssh=true --accept-dns=false --exit-node=; then
647     HOST_ON_MESH=true
648     echo "Host is on Tailscale mesh."
649 else
650     echo -e "\033[0;33m[!] tailscale up failed even though the daemon is running.\033[0m"
651     echo " This usually means the host hasn't authenticated with your tailnet yet."
652     echo " Run this command in a terminal to authenticate:"
653     echo ""
654     echo "         sudo tailscale up"
655     echo ""
656     echo " A browser will open. Log in to your Tailscale account, then re-run toggle.sh."
657 fi
658 else
659     echo -e "\033[0;31m[!] tailscaled could NOT be started automatically.\033[0m"
660     echo " Run this in a terminal to start and authenticate Tailscale:"
661     echo ""
662     echo "         sudo systemctl start tailscaled"
663     echo "         sudo tailscale up"
664     echo ""
665     echo " Then re-run toggle.sh."
666 fi
667 else
668     echo -e "\n[Warning] tailscale CLI not found on PATH. Skipping host Tailscale configuration..."
669 fi
670
671 # Phase B: Pre-flight checks + enable exit node only if safe
672 if [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" != "false" ] && [ -n "$TS_IP" ]; then
673     echo -e "\n"
674     echo "Pre-flight connectivity check"
675     echo ""
676
677     # Check 1: Can we reach the gateway via Tailscale (uses tailscale ping, not ICMP)?
678     # NOTE: tailscale ping exits 1 when the connection goes through a DERP relay
679     # ("direct connection not established") even though the pong was received.
680     # We check for 'pong' in the output (stderr discarded) to accept relayed connections.
681     echo -n " [1/3] Gateway reachable via Tailscale... "
682     ping_ok=false
683     # The host tailscaled needs a few seconds to propagate the new network map and
684     # establish a connection route after 'tailscale up --reset'. We retry up to 45 times.
685     for attempt in {1..45}; do
686         if $TS_BIN ping --until-direct=false -c 2 --timeout 2s "$TS_IP" 2>/dev/null | grep -q "pong"; then
687             ping_ok=true
688             break
689         fi
690         sleep 1
691     done
692     if [ "$ping_ok" = "true" ]; then
693         echo "PASS"
694     else
695         echo "FAIL"
696         SKIP_EXIT_NODE=true
697     fi
698
699     # Check 2: Can we reach AdGuard via localhost (exposed port ${DNS_PROXY_PORT})?
700     echo -n " [2/3] AdGuard DNS via localhost... "
701     if command -v dig >/dev/null 2>&1; then
702         if dig +tcp @127.0.0.1 -p ${DNS_PROXY_PORT} google.com +short +timeout=5 >/dev/null 2>&1; then
703             echo "PASS"
704         else
705             echo "FAIL"
706             SKIP_EXIT_NODE=true
707         fi
708     elif command -v nslookup >/dev/null 2>&1; then
709         if nslookup -port=${DNS_PROXY_PORT} google.com 127.0.0.1 >/dev/null 2>&1; then
710             echo "PASS"
711         else
712             echo "FAIL"
713             SKIP_EXIT_NODE=true
714         fi
715     else
716         echo "SKIP (dig/nslookup not available)"
717     fi
718
719     # Check 3: Does the WARP container have external internet?
720     echo -n " [3/3] WARP container internet... "
721     if docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace &>/dev/null;
722     then
723         echo "PASS"
724     else
725         echo "FAIL"
726         SKIP_EXIT_NODE=true

```

```

726 fi
727
728 # Act based on check results
729 if [ "$SKIP_EXIT_NODE" != "true" ]; then
730     echo -e "\nAll checks passed. Enabling exit node $TS_IP..."
731     if $TS_BIN up --reset --ssh=true --accept-dns=false --exit-node="$TS_IP" --exit-node-allow-lan-access=
true; then
732         echo "Exit node enabled."
733     else
734         echo "[Warning] Failed to set exit node."
735         SKIP_EXIT_NODE=true
736     fi
737 fi
738
739 if [ "$SKIP_EXIT_NODE" = "true" ]; then
740     echo -e "\n033[0;33m[] Pre-flight checks failed EXIT NODE + DNS HIJACK SKIPPED.\033[0m"
741     echo " Host stays on Tailscale mesh but your internet routes through your normal connection."
742     echo " Troubleshoot with:"
743     echo "     docker compose logs warp | tail -20"
744     echo "     docker compose exec -T warp wget -q0- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace"
745     echo ""
746     echo " Falling back to SOCKS5 proxy for traffic routing..."
747 fi
748 elif [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" = "false" ]; then
749     echo "USE_EXIT_NODE is false. Skipping exit node and DNS hijack."
750     SKIP_EXIT_NODE=true
751 fi
752
753 # 8. Apply DNS Hijacking.
754 # If exit node is enabled: hijack DNS to gateway's Tailscale IP (routes through tailnet).
755 # Otherwise, leave DNS at 1.1.1.1 (already set by reset at the top).
756 # AdGuard is also available at localhost:${DNS_PROXY_PORT} for manual configuration in apps.
757 if [ -n "$TS_IP" ] && [ "$HOST_ON_MESH" = "true" ] && [ "$SKIP_EXIT_NODE" != "true" ]; then
758     echo -e "\nHijacking host DNS to point ONLY at AdGuard Home ($TS_IP)..."
759     echo "Setting MagicDNS search domain 'ts.net' so tailnet hostnames resolve..."
760     if force_dns_to_gateway "$TS_IP"; then
761         echo "DNS hijack verified: single server = $TS_IP."
762         start_dns_watcher "$TS_IP"
763     else
764         echo "[Warning] DNS hijack could not be verified."
765         echo " If DNS is broken, run manually:"
766         echo "     networksetup -setdnsservers \"\$ACTIVE_SERVICE\" $TS_IP"
767         echo "     networksetup -setsearchdomains \"\$ACTIVE_SERVICE\" ts.net"
768     fi
769 elif [ -n "$TS_IP" ]; then
770     echo -e "\n[Info] Exit node not enabled (pre-flight checks failed)."
771     echo " Trying local DNS proxy for ad-blocking..."
772     if start_local_dns_proxy; then
773         echo " Ad-blocking active via local DNS proxy through AdGuard."
774     else
775         echo " Local DNS proxy unavailable. DNS stays at 1.1.1.1."
776         echo " AdGuard available at http://localhost:80 (port ${DNS_PROXY_PORT} for DNS via TCP)."
777     fi
778
779 # Enable SOCKS5 proxy for traffic routing through WARP
780 # (enabled whenever exit node is skipped, regardless of HOST_ON_MESH)
781 echo -e "\n Enabling SOCKS5 proxy for TCP traffic routing through gateway WARP tunnel..."
782 echo " (SOCKS5 proxy runs inside gateway container, routes through tun0 -> WARP)"
783 enable_socks_proxy "$ACTIVE_SERVICE"
784
785 # Verify the SOCKS5 proxy works through WARP
786 echo -n " Verifying SOCKS5 proxy routes through WARP... "
787 if curl --socks5-hostname 127.0.0.1:${SOCKS_PROXY_PORT} --max-time 10 -s https://www.cloudflare.com/cdn-cgi/
trace 2>/dev/null | grep -q "warp=on"; then
788     echo "ok (warp=on)."
789     echo " All TCP traffic now encrypted through Cloudflare WARP tunnel."
790 else
791     echo "FAILED (proxy not routing through WARP)."
792     echo " Disabling SOCKS5 proxy internet stays on direct connection."
793     disable_socks_proxy
794     echo " SOCKS proxy available at localhost:${SOCKS_PROXY_PORT} for manual configuration."
795 fi
796 else
797     echo -e "\n[Warning] Could not resolve gateway Tailscale IP."
798 fi
799
800 # 9. Verify host connectivity
801 if [ "$HOST_ON_MESH" = "true" ] && [ -n "$TS_BIN" ]; then
802     if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
803         echo "Host is online on the Tailscale mesh."
804     else

```

```

805     echo "[Warning] Host is not responding on Tailscale."
806 fi
807 fi
808
809 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
810 echo -e "\nGateway has been successfully STARTED in ${ELAPSED} seconds."
811
812 # Prevent system from going to sleep while gateway is running
813 start_sleep_prevention
814 fi
815
816 SUCCESS_RUN=true
817
818 # Final DNS state summary
819 if [ -n "$ACTIVE_SERVICE" ]; then
820     FINAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>/dev/null | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | tr '
821 ' ' || true)
822     echo -e "\n"
823     echo -e "DNS STATE: $FINAL_DNS"
824     echo -e ""
825 fi
826
827 if [ "$START_GATEWAY" = "true" ] && command -v docker >/dev/null 2>&1; then
828     if docker compose logs rule-compiler 2>/dev/null | grep -q "Warning: Failed to fetch"; then
829         echo -e "\n[Warning] One or more blacklist URLs failed to download (404/Offline)."
830         echo " The gateway gracefully fell back to yesterday's cached blacklist."
831         echo " (Run 'docker logs rule-compiler' later to investigate which link died.)"
832     fi
833 fi
834
835 echo -e "\n"
836 read -rp "Press [r] and Enter to instantly reverse state, or just press Enter to exit: " USER_CHOICE
837 if [[ "$USER_CHOICE" == "r" || "$USER_CHOICE" == "R" ]]; then
838     echo "Reversing gateway state..."
839     exec bash "$0"
840 fi
841
842 echo -e "\nYou can close this terminal window now."
843
844 exit 0
845 fi
846
847 #
848 # macOS TOGGLE jot / netstat / networksetup, launchd / pf.
849 #
850
851 # Enforce Cryptographic Script Integrity
852 if [ -f "scripts/crypto.sh" ]; then
853     if ! bash scripts/crypto.sh --verify; then
854         exit 1
855     fi
856 fi
857
858 # Define log file
859 LOG_FILE="$PWD/output.log"
860
861 # Rotate log file if it exceeds 1GB (1073741824 bytes).
862 # Sized large on purpose: with DEBUG_TRACE the log holds a full command-by-command
863 # trace effectively the entire compilation/warning history of the framework
864 # which is valuable to keep for post-mortems. Raised from 50MB so always-on
865 # tracing doesn't churn that history away prematurely.
866 if [ -f "$LOG_FILE" ]; then
867     log_size=$(stat -f%z "$LOG_FILE" 2>/dev/null || stat -c%s "$LOG_FILE" 2>/dev/null || echo 0)
868     if [ "$log_size" -gt 1073741824 ]; then
869         mv "$LOG_FILE" "${LOG_FILE}.old" 2>/dev/null || rm -f "$LOG_FILE"
870         echo "[$(date '+%Y-%m-%d %H:%M:%S')] Log file exceeded 1GB; rotated to output.log.old" > "$LOG_FILE"
871     fi
872 fi
873
874 # --- LIFECYCLE PHASE TRACKING + EXIT BREADCRUMB ---
875 # TOGGLE_PHASE is updated as the script moves through STARTUP/STOP/START so that
876 # if the process is killed (window closed, pkill, OOM) or exits with an error,
877 # the EXIT trap records the final exit code AND the phase it died in. Previously
878 # an interrupted START left output.log ending abruptly after the teardown with no
879 # indication of what failed this makes every exit traceable.
880 TOGGLE_PHASE="init"
881 _log_exit_breadcrumb() {
882     local rc=$?
883     echo "[$(date '+%Y-%m-%d %H:%M:%S')] toggle.sh EXIT: code=$rc phase=$TOGGLE_PHASE (pid $$)" >> "$LOG_FILE"
884     2>/dev/null || true
885 }

```

```

885 trap '_log_exit_breadcrumb' EXIT
886
887 # --- DEBUG TRACE (opt-in via .env: DEBUG_TRACE=true) ---
888 # When enabled, mirror a full command-by-command xtrace to output.log for a
889 # complete post-mortem trail (e.g. a race during --restart while Wi-Fi is
890 # re-associating). Off by default. Read directly from .env here because
891 # common.sh (read_env_var) isn't sourced yet.
892 #
893 # BASH_XTRACEFD (send xtrace to its own fd, keeping the terminal clean) needs
894 # bash >= 4.1. macOS ships /bin/bash 3.2, so we branch:
895 #   - bash >= 4.1 (e.g. Linux): dedicate fd 9 to the log and point xtrace there;
896 #   the terminal stays clean and normal stderr is untouched.
897 #   - bash 3.2 (stock macOS): BASH_XTRACEFD is unsupported, so xtrace always goes
898 #   to stderr. We redirect stderr *straight to the log file* with a plain
899 #   `exec 2>>"$LOG_FILE"`. This deliberately does NOT use a process
900 #   substitution (`2> >(tee)`): under `set -e`, stderr flowing through the
901 #   backgrounded `tee` proved to abort the script mid-START on 3.2 (a failed
902 #   write / SIGPIPE in a pipeline returned non-zero and tripped `set -e`, with
903 #   $LINENO mis-blaming the enclosing `fi` see devref 15.11.8-A/E). The
904 #   tradeoff: on 3.2 the terminal no longer shows live progress while tracing
905 #   (everything is in output.log instead). Acceptable for an opt-in debug mode.
906 # We NEVER use the `exec {var}>>` dynamic-fd form (4.1+ only), which hard-fails on 3.2.
907 DEBUG_TRACE=$(grep -E '^DEBUG_TRACE=' .env 2>/dev/null | cut -d '=' -f2- | tr -d '"' | tr '[:upper:]' '[:lower:]' | tr -d '[:space:]')
908 if [ "$DEBUG_TRACE" = "true" ]; then
909     echo "[$(date '+%Y-%m-%d %H:%M:%S')] DEBUG_TRACE enabled xtrace mirrored to output.log (bash ${BASH_VERSINFO[0]}.${BASH_VERSINFO[1]})" >> "$LOG_FILE"
910     # Timestamped, location-aware xtrace prompt (works on 3.2 and 4.x).
911     export PS4='+ [$(date '+%H:%M:%S')] ${BASH_SOURCE##*/}:${LINENO}:${FUNCNAME[0]:-main}: '
912     if [ "${BASH_VERSINFO[0]}" -gt 4 ] || { [ "${BASH_VERSINFO[0]}" -eq 4 ] && [ "${BASH_VERSINFO[1]}" -ge 1 ]; }; then
913         exec 9>>"$LOG_FILE"
914         export BASH_XTRACEFD=9
915         set -x
916     else
917         # bash 3.2 (stock macOS): plain stderrfile redirect (no process substitution).
918         exec 2>>"$LOG_FILE"
919         set -x
920     fi
921 fi
922
923 # --- MAIN EXECUTION LOGGING ---
924 echo -e "\n===== " >> "$LOG_FILE"
925 echo "[$(date '+%Y-%m-%d %H:%M:%S')] toggle.sh invoked" >> "$LOG_FILE"
926 echo "===== " >> "$LOG_FILE"
927
928 # (Previously this script chmod 600'd .env at start to "unlock" it for docker compose,
929 # and chmod 000'd it on exit. That pattern broke docker compose on macOS/Colima when
930 # the umask produced 0600 the script removed its own ability to read .env before it could even proceed.)
931
932 # --- PROCEDURAL PORT GENERATION ---
933 # To prevent static port fingerprinting by malware, we randomize exposed ports
934 # on every boot and persist them to a hidden environment file.
935 PORTS_FILE="/tmp/nullexit-ports.env"
936 if [[ "$1" == "" || "$1" == "--restart" ]]; then
937
938     # Collision-avoidance function
939     GENERATED_PORTS=""
940     get_free_port() {
941         local port
942         while true; do
943             port=$(jot -r 1 10000 65000)
944             # 1. Prevent self-collision within this loop
945             if [[ "$GENERATED_PORTS" == *"$port"* ]]; then
946                 continue
947             fi
948             # 2. Prevent collision with ANY process on the Mac (root daemons, terminal apps, etc)
949             # We use netstat instead of lsof because netstat reads the kernel socket table
950             # directly and doesn't require sudo to see ports bound by other users/root.
951             if ! netstat -an -p tcp | awk '{print $4}' | grep -q "\.${port}"; then
952                 echo "$port"
953                 return
954             fi
955         done
956     }
957
958     export TOR SOCKS_PORT=$(get_free_port)
959     GENERATED_PORTS="$GENERATED_PORTS $TOR SOCKS_PORT"
960
961     export TOR_CONTROL_PORT=$(get_free_port)
962     GENERATED_PORTS="$GENERATED_PORTS $TOR_CONTROL_PORT"

```

```

963
964 export SOCKS_PROXY_PORT=$(get_free_port)
965 GENERATED_PORTS="$GENERATED_PORTS $SOCKS_PROXY_PORT"
966
967 export DNS_PROXY_PORT=$(get_free_port)
968
969 echo "export TOR SOCKS_PORT=$TOR SOCKS_PORT" > "$PORTS_FILE"
970 echo "export TOR_CONTROL_PORT=$TOR_CONTROL_PORT" >> "$PORTS_FILE"
971 echo "export SOCKS_PROXY_PORT=$SOCKS_PROXY_PORT" >> "$PORTS_FILE"
972 echo "export DNS_PROXY_PORT=$DNS_PROXY_PORT" >> "$PORTS_FILE"
973 ADGUARD_PASS=$(grep -E "^ADGUARD_PASSWORD=" .env 2>/dev/null | cut -d '=' -f2 - | tr -d '\n')
974 echo "export TOR_PASSWORD=${ADGUARD_PASS:-nullexit}" >> "$PORTS_FILE"
975 elif [ -f "$PORTS_FILE" ]; then
976 # On STOP, source the existing ports so docker-compose down matches
977 source "$PORTS_FILE"
978 else
979 # Fallbacks if file is lost
980 export TOR SOCKS_PORT=9050
981 export TOR_CONTROL_PORT=9051
982 export SOCKS_PROXY_PORT=1080
983 export DNS_PROXY_PORT=5354
984 fi
985
986 # Start execution timer
987 TOGGLE_START_TIME=$SECONDS
988
989 if [[ "$1" == "--restart" ]]; then
990 echo "Executing Gateway Restart Sequence..."
991 # We must source common.sh here temporarily to check gateway state before we proceed
992 source "$SCRIPT_DIR/scripts/common.sh"
993 # Use persisted intent (marker) via gateway_state, which echoes a string (never
994 # a return code) to stay set -e-safe under set -x. See devref 15.11.8.
995 if [ "$(gateway_state)" = "running" ]; then
996 echo "Gateway is currently running. Stopping it first..."
997 bash "$0"
998 echo "Gateway stopped. Now starting it..."
999 else
1000 echo "Gateway is already stopped. Starting it..."
1001 fi
1002 bash "$0"
1003 exit 0
1004 fi
1005
1006 # Global flag to track if the script completed successfully
1007 SUCCESS_RUN=false
1008
1009 # Global variable to track the currently running background command PID
1010 CURRENT_BG_PID=""
1011
1012 # Source common bash functions
1013 source "$SCRIPT_DIR/scripts/common.sh"
1014
1015 # Prevent concurrent execution of lifecycle scripts (toggle.sh / recover.sh)
1016 LOCK_FILE="/tmp/nullexit-toggle.lock"
1017 if [ -f "$LOCK_FILE" ]; then
1018 LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
1019 if [ -n "$LOCK_PID" ] && [ "$LOCK_PID" != "$$" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
1020 if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -q -E "toggle\.sh|recover\.sh"; then
1021 die "Another instance of toggle.sh or recover.sh (PID $LOCK_PID) is already running."
1022 fi
1023 fi
1024 fi
1025 echo "$$" > "$LOCK_FILE"
1026
1027 # Helper function to run a GUI command as the logged-in console user
1028 # (Prevents permission failures if the script is run with sudo)
1029 run_gui_cmd() {
1030 local console_user
1031 console_user=$(stat -f '%Su' /dev/console 2>> output.log || echo "$SUDO_USER")
1032 if [ -z "$console_user" ] || [ "$console_user" = "root" ]; then
1033 console_user=$(logname 2>> output.log || echo "$USER")
1034 fi
1035
1036 if [ -n "$console_user" ] && [ "$console_user" != "root" ] && [ "$EUID" -eq 0 ]; then
1037 sudo -u "$console_user" "$@"
1038 else
1039 "$@"
1040 fi
1041 }
1042
1043 # Active-state marker: written at end of START path so external watchers

```



```

1044 # (see launchd/com.nullexit.wake-recovery.plist + scripts/watcher.sh +
1045 # devref 10.29) know the gateway is currently up. They only fire
1046 # recover.sh --post-wake when this file is present. Cleared at the top
1047 # of the STOP path and inside cleanup_handler on any error/signal path
1048 # so a half-broken gateway never re-triggers post-wake refreshes.
1049 write_gateway_active_marker() {
1050     date -u +%FT%TZ > /tmp/nullexit-gateway-active.marker
1051     # Set the watcher debounce timestamp to now to prevent a redundant post-startup recovery run.
1052     date +%s > /tmp/nullexit-watcher.last-recovery 2>/dev/null || true
1053 }
1054
1055 clear_gateway_active_marker() {
1056     rm -f /tmp/nullexit-gateway-active.marker
1057     rm -f "$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/TUNNEL_FAILED_CLOSED.marker"
1058 }
1059
1060 # Capture whether the gateway-active marker exists BEFORE the defensive clear
1061 # below wipes it. The STOP-vs-START decision (gateway_should_be_running) reads
1062 # this captured value persisted *intent* instead of a live Docker probe, so
1063 # a disable is instant and works even when Colima/Docker is down. Must be read
1064 # before clear_gateway_active_marker or it would always look "stopped".
1065 #
1066 # Written as default-assign + `if` WITHOUT an `else`: under `set -e` (with the
1067 # DEBUG_TRACE xtrace redirect active) an `if [ -f ]; then ; else ; fi` whose
1068 # condition is false was aborting the script here at init on macOS /bin/bash 3.2
1069 # the false `[ -f ]` status leaked through the compound. An `if` with no `else`
1070 # yields status 0 when the condition is false, so nothing leaks.
1071 GATEWAY_MARKER_AT_STARTUP="no"
1072 if [ -f "$MARKER_FILE" ]; then
1073     GATEWAY_MARKER_AT_STARTUP="yes"
1074 fi
1075
1076 # Defensive: clear any stale marker from a prior crashed/aborted run. If a
1077 # previous toggle.sh wrote the marker but was OOM-killed in the middle of the
1078 # START block (before the END-of-START write) or if the user rebooted mid
1079 # session this prevents the watcher from firing post-wake against a
1080 # non-running gateway on every wake event. Placed AFTER the function
1081 # definitions so bash resolves the call at parse time.
1082 clear_gateway_active_marker
1083
1084 PID_FILE="$PID_CAFFEINATE"
1085
1086 # Start macOS caffeine to prevent sleep while gateway is running
1087 start_sleep_prevention() {
1088     if ! command -v caffeine >/dev/null 2>&1; then
1089         echo " [Warning] caffeine tool not found. Sleep prevention unavailable."
1090         return 1
1091     fi
1092
1093     # Stop any existing sleep prevention process first to avoid duplicates
1094     stop_pidfile_daemon "/tmp/nullexit-caffeinate.pid" "system sleep prevention"
1095
1096     echo " Enabling system sleep prevention (caffeinate)..."
1097     # Run caffeine wrapped in a bash trap to prevent idle system sleep.
1098     # Critically, this traps macOS shutdown signals (SIGTERM) and automatically
1099     # flushes hijacked DNS back to normal right before the Mac powers off.
1100     # We do not use recover.sh here because it is a heavy recovery script (managing
1101     # containers, restarting tailscaled, etc.) which takes too long and gets
1102     # terminated by macOS before it can reset the DNS. Instead, we surgically and
1103     # instantly restore normal DNS and proxy settings here.
1104     nohup bash -c "
1105     trap '
1106         echo \"[Shutdown Trap] Reverting network settings...\" >> \"\$PWD/output.log\" 2>&1
1107         kill \$! 2>/dev/null || true
1108         sudo -n networksetup -setdnsservers \"\$ACTIVE_SERVICE\" 1.1.1.1 >> \"\$PWD/output.log\" 2>&1 || true
1109         sudo -n networksetup -setdnsservers \"\$ENO_SERVICE\" 1.1.1.1 >> \"\$PWD/output.log\" 2>&1 || true
1110         sudo -n networksetup -setsearchdomains \"\$ACTIVE_SERVICE\" \"Empty\" >> \"\$PWD/output.log\" 2>&1 || true
1111         sudo -n networksetup -setsearchdomains \"\$ENO_SERVICE\" \"Empty\" >> \"\$PWD/output.log\" 2>&1 || true
1112         sudo -n networksetup -setsocksfirewallproxystate \"\$ACTIVE_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 ||
1113         true
1114         sudo -n networksetup -setsocksfirewallproxystate \"\$ENO_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
1115         sudo -n networksetup -setwebproxystate \"\$ACTIVE_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
1116         sudo -n networksetup -setwebproxystate \"\$ENO_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
1117         sudo -n networksetup -setsecurewebproxystate \"\$ACTIVE_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
1118         sudo -n networksetup -setsecurewebproxystate \"\$ENO_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
1119         if command -v tailscale >/dev/null 2>&1; then
1120             tailscale up >> \"\$PWD/output.log\" 2>&1 || true
1121             tailscale set --accept-dns=false --exit-node= >> \"\$PWD/output.log\" 2>&1 || true
1122             TS_DOWN_ARGS=""
1123             if [ \"\$KILL_SWITCH\" = \"true\" ]; then TS_DOWN_ARGS="--accept-risk=lose-ssh\"; fi
1124             tailscale down \$TS_DOWN_ARGS >> \"\$PWD/output.log\" 2>&1 &

```

```

1124     fi
1125     sleep 1
1126     echo \"[Shutdown Trap] Cleanup complete.\" >> \"\$PWD/output.log\" 2>&1
1127     exit 0
1128     ' SIGTERM SIGINT SIGHUP
1129     caffeinate -i &
1130     wait \$!
1131     \" >> output.log 2>&1 &
1132     local caffe_pid=\$!
1133     echo \"\$caffe_pid\" > \"\$PID_FILE\"
1134     echo \" Sleep prevention active (PID \$caffe_pid). Your Mac won't sleep while the gateway is running.\"
1135 }
1136
1137
1138
1139 DNS_WATCHER_PID_FILE=\"\$PID_DNS_WATCHER\"
1140 WARP_WATCHER_PID_FILE=\"/tmp/nullexit-warp-watcher.pid\"
1141
1142
1143
1144 # Background WARP tunnel liveness monitor. Polls cdn-cgi/trace every 30s
1145 # while healthy, but accelerates to polling every 5s if a failure is detected.
1146 # Always logs state transitions to output.log. When warp=off persists for
1147 # WARP_FAIL_THRESHOLD consecutive polls (default 6 = 30s of downtime), the watcher
1148 # triggers a nuclear recovery: runs recover.sh to tear down the entire
1149 # gateway disconnecting Tailscale, stopping containers, resetting DNS,
1150 # and power-cycling Wi-Fi. Note: This minimizes exposure window, but
1151 # only the pf kill switch guarantees absolutely zero leakage on drop.
1152 #
1153 # The threshold (default 6) is statistically chosen: even at an
1154 # unrealistically high 10% false-positive rate per poll, P(6 consecutive
1155 # false positives) = 0.1^6 0.0001%. Real-world false-positive rates
1156 # are far lower, making 30s of continuous downtime overwhelmingly likely
1157 # to be a genuine outage.
1158 #
1159 # Configurable via .env: WARP_FAIL_THRESHOLD=3 (15s, more aggressive)
1160 start_warp_watcher() {
1161     stop_warp_watcher
1162     # Parse threshold from .env (same pattern as GATEWAY_MSS, GATEWAY_BYPASS_PING, etc.)
1163     local threshold
1164     threshold=\"$(read_env_var WARP_FAIL_THRESHOLD)\"
1165     if [ -z \"\$threshold\" ]; then
1166         threshold=\"${WARP_FAIL_THRESHOLD:-6}\"
1167     fi
1168     local trigger_file=\"/tmp/nullexit-warp-shutdown-triggered\"
1169     rm -f \"\$trigger_file\"
1170     echo \" Starting background WARP Watcher (polling every 30s [accelerating to 5s on failure], shutdown after \$
1171         {threshold} consecutive failures)...\"
1172     nohup bash -c \"
1173         trap 'exit 0' SIGTERM SIGINT SIGHUP
1174         last_state='on'
1175         consec_off=0
1176         threshold='\$threshold'
1177         trigger_file='\$trigger_file'
1178         out_log='\$PWD/output.log'
1179         recover_bin='\$PWD/recover.sh'
1180         inhibit_file='/tmp/nullexit-warp-inhibit.marker'
1181         while true; do
1182             # Inhibit check
1183             # recover.sh --post-wake writes this marker while force-recreating the
1184             # warp container. During that window, the container is intentionally
1185             # down don't count it as a failure or we'll fire nuclear recover.sh
1186             # and kill a gateway that was in the process of healing itself.
1187             if [ -f \"\$inhibit_file\" ]; then
1188                 consec_off=0
1189                 sleep 5
1190                 continue
1191             fi
1192             state='off'
1193             if docker compose exec -T warp wget -q0- --timeout=5 \
1194                 https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null \
1195                 | grep -q 'warp=on'; then
1196                 state='on'
1197             fi
1198             # State-transition logging
1199             if [ \"\$state\" != \"\$last_state\" ]; then
1200                 if [ \"\$state\" = 'off' ]; then
1201                     echo \"[\$(date -u +%FT%TZ)] WARP DOWN cdn-cgi/trace reports warp=off\" >> \"\$out_log\"
1202                 fi

```

```

1204     last_state="\${state}"
1205 fi
1206
1207 # Consecutive-failure tracking + auto-shutdown
1208 if [ "\${state}" = 'off' ]; then
1209     consec_off=$((consec_off + 1))
1210     if [ "\${consec_off}" -ge "\${threshold}" ] && [ ! -f "\${trigger_file}" ]; then
1211         touch "\${trigger_file}"
1212         echo "\[${date -u +%FT%TZ}\] WARP SHUTDOWN \${consec_off} consecutive failures (threshold=\${threshold})
Running recover.sh to kill the gateway and restore normal internet. Your IP is NO LONGER Cloudflare.\" >>
\${out_log}"
1213         # Nuclear recovery: tear down the entire gateway.
1214         # recover.sh resets DNS 1.1.1.1, disconnects Tailscale,
1215         # stops Docker containers, flushes routes, power-cycles Wi-Fi.
1216         bash "\${recover_bin}" --auto >> \${out_log} 2>&1 || true
1217         echo "\[${date -u +%FT%TZ}\] WARP SHUTDOWN recover.sh completed, watcher exiting. Your IP is now
your real ISP IP.\" >> \${out_log}"
1218         exit 0
1219     fi
1220 else
1221     if [ "\${consec_off}" -gt 0 ]; then
1222         echo "\[${date -u +%FT%TZ}\] WARP RECOVERED warp=on after \${consec_off} consecutive off readings (
threshold was \${threshold})\" >> \${out_log}"
1223         fi
1224         consec_off=0
1225     fi
1226
1227     if [ "\${state}" = 'off' ]; then
1228         sleep 5
1229     else
1230         sleep 30
1231     fi
1232 done
1233 " >> output.log 2>&1 &
1234 echo $! > "$WARP_WATCHER_PID_FILE"
1235 }
1236
1237 stop_warp_watcher() {
1238     if [ -f "$WARP_WATCHER_PID_FILE" ]; then
1239         local wp
1240         wp=$(cat "$WARP_WATCHER_PID_FILE")
1241         if [ -n "$wp" ] && kill -0 "$wp" 2>/dev/null; then
1242             echo "Stopping background WARP Watcher (PID $wp)..."
1243             kill "$wp" 2>/dev/null || true
1244         fi
1245         rm -f "$WARP_WATCHER_PID_FILE"
1246     fi
1247 }
1248
1249
1250
1251
1252 # Cleanup handler to restore DNS to 1.1.1.1 on error or user interrupt (Ctrl+C / SIGTERM / SIGHUP)
1253 cleanup_handler() {
1254     local exit_code=$?
1255     local trigger_type="$1"
1256     local line_no="$2"
1257
1258     if [ "$SUCCESS_RUN" = "false" ]; then
1259         echo -e "\n[Self-Correction] Script interrupted or failed ($trigger_type). Restoring host DNS to 1.1.1.1..."
1260
1261         if [ -n "$ACTIVE_SERVICE" ]; then
1262             reset_dns
1263         fi
1264
1265         if [ -n "$CURRENT_BG_PID" ]; then
1266             echo "Terminating active command (PID $CURRENT_BG_PID)..."
1267             kill -15 "$CURRENT_BG_PID" 2>> output.log || true
1268         fi
1269
1270         # Disconnect host Tailscale and close the GUI application on failure
1271         disconnect_tailscale_host
1272
1273         # Stop local DNS proxy if running
1274         stop_local_dns_proxy
1275
1276         # Stop sleep prevention
1277         stop_pidfile_daemon "/tmp/nullexit-cafeinate.pid" "system sleep prevention"
1278         stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
1279         stop_warp_watcher

```

```

1280     clear_gateway_active_marker
1281
1282     # Capture warp logs on failure for debugging before teardown
1283     if [ "$trigger_type" = "ERR" ]; then
1284         echo -e "\n--- WARP FAILURE LOGS ---" >> output.log
1285         docker compose logs warp --tail=100 >> output.log 2>&1 || true
1286         echo "-----" >> output.log
1287     fi
1288
1289     # Best-effort container teardown on failure
1290     docker compose down --remove-orphans -t 30 2>> output.log || true
1291
1292     # Nuke leftover network state (proxies, routes, DNS cache, Wi-Fi)
1293     if [ -n "$ACTIVE_SERVICE" ]; then
1294         cleanup_network_state
1295     fi
1296
1297
1298     if [ "$trigger_type" = "ERR" ]; then
1299         echo -e "\n=====
1300         echo "ERROR: Script failed at line $line_no with exit code $exit_code."
1301         echo "=====
1302         echo "Troubleshooting steps:"
1303         echo "1. Run 'colima status' to verify the VM is active."
1304         echo "2. Run 'docker compose ps' to check container health."
1305         echo "3. Run 'docker compose logs' to view application logs."
1306         echo "4. Run 'tailscale status' to check host Tailscale status."
1307         echo "=====
1308     fi
1309     rm -f "${LOCK_FILE:-/tmp/nullexit-toggle.lock}"
1310 fi
1311 }
1312
1313 trap 'cleanup_handler ERR $LINENO' ERR
1314 trap 'cleanup_handler INT' INT
1315 trap 'cleanup_handler TERM' TERM
1316 trap 'cleanup_handler HUP' HUP
1317
1318 # NOPASSWD sudo
1319 # The user has NOPASSWD in /etc/sudoers.d/nullexit for the specific commands
1320 # needed (networksetup, pfctl, route, ifconfig, dscacheutil, killall, pkill,
1321 # and python3 -I scripts/dns-proxy.py for the fallback DNS proxy).
1322 # All privileged calls below use `sudo -n` which will run without prompting.
1323 # No credential caching loop is needed.
1324
1325 # Add Homebrew and standard paths
1326 setup_standard_path
1327
1328 # Check for local DNS proxy tools (used when tailnet data plane is unavailable)
1329 # Uses a Python DNS proxy (handles TCP wire format correctly).
1330 # Python3 is built into macOS no external dependencies.
1331 DNS_PROXY_BIN=""
1332 if command -v python3 >> output.log 2>&1; then
1333     DNS_PROXY_BIN="python3 -I"
1334 elif command -v python >> output.log 2>&1; then
1335     DNS_PROXY_BIN="python -I"
1336 fi
1337
1338 # Path to the DNS proxy script (sibling of this script)
1339 DNS_PROXY_SCRIPT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/scripts/dns-proxy.py"
1340
1341 # Resolve Tailscale CLI on host. We rely on the standalone Homebrew formula
1342 # (`brew install tailscale` + `brew services start tailscale`) NOT the
1343 # Tailscale.app GUI so there is no .app bundle or Network Extension sandbox
1344 # to launch, no menu-bar click to perform, and no scutil Network Service to
1345 # start manually. tailscaled runs as a per-user LaunchAgent (or LaunchDaemon
1346 # if you ran the install with sudo) and the control socket is always available.
1347 TS_BIN=""
1348 if command -v tailscale >> output.log 2>&1; then
1349     TS_BIN="tailscale"
1350 fi
1351
1352 # Baseline: disable Tailscale DNS management at the daemon level.
1353 # `tailscale set` writes a persistent preference. However, `tailscale up --reset`
1354 # (used in steps 7 and 9) RESETS all unspecified flags to defaults, which would
1355 # re-enable `--accept-dns=true` and nullify this `set`.
1356 # Therefore, EVERY `tailscale up --reset` call in this file MUST also pass
1357 # `--accept-dns=false` explicitly. See steps 7 and 9 for the fix.
1358 #
1359 # This initial `set` is still useful as a baseline for non-reset operations
1360 # (e.g. if the user runs `tailscale up` manually without --reset) and covers

```

```

1361 # the brief window between this line and the first `--reset` call.
1362 #
1363 # Runs once at script start while tailscaled is still connected (if it was
1364 # left running from a previous toggle), so the pref takes effect before any
1365 # disconnection or reconnection logic.
1366 if [ -n "$TS_BIN" ]; then
1367     $TS_BIN set --accept-dns=false >> output.log 2>&1 || true
1368 fi
1369
1370 # Disconnect host Tailscale from the mesh. With the standalone daemon, this is the
1371 # *entire* teardown no scutil tunnel stop, no Tailscale.app GUI quit, no pkill.
1372 # tailscaled keeps running (as a LaunchAgent / LaunchDaemon), which is intentional:
1373 # the next `tailscale up` then reconnects in seconds rather than waiting for a
1374 # slow .app launch + Network Extension activation. The `tailscale down` call also
1375 # retains the user's auth state, so we don't force a browser re-login every cycle.
1376 #
1377 # Defined early (before the if/else branches and referenced by cleanup_handler's
1378 # ERR trap) so that any failure before the main logic can still tear down Tailscale.
1379
1380
1381 # Wait for Network Readiness
1382 # On --restart, the gateway tears down while the host network interface may
1383 # still be reconnecting (Wi-Fi, Ethernet, USB tethering, etc.). If we proceed
1384 # too early, get_active_service returns nothing, routing targets the wrong
1385 # interface, and DNS/WARP setup silently fails.
1386 #
1387 # We use `route get default` to detect a valid default gateway rather than
1388 # checking a specific interface (e.g. en0) this generalizes across all
1389 # connection types: Wi-Fi, Ethernet, USB-C adapters, iPhone tethering, etc.
1390 # The moment the OS has any routable default gateway, we proceed.
1391 _net_wait_secs=0
1392 _net_timeout=60
1393 echo "Waiting for network connectivity before starting..."
1394 while true; do
1395     if route get default 2>/dev/null | grep -q 'gateway: '; then
1396         _gw=$(route get default 2>/dev/null | awk '/gateway:/{print $2}')
1397         echo "    Network ready (default gateway: $_gw)."
1398         break
1399     fi
1400     if [ "$_net_wait_secs" -ge "$_net_timeout" ]; then
1401         echo "    [Warning] No default gateway after ${_net_timeout}s proceeding anyway."
1402         break
1403     fi
1404     sleep 2
1405     _net_wait_secs=$(( _net_wait_secs + 2 ))
1406 done
1407
1408 ACTIVE_SERVICE=$(get_active_service)
1409
1410 # Resolve the service name for en0 (usually "Wi-Fi") macOS scutil DNS resolver
1411 # is commonly scoped to en0, so per-service DNS changes on other interfaces are
1412 # ignored unless en0's service is also updated.
1413 ENO_SERVICE=$(get_en0_service)
1414
1415 # Helper to add host-side bypass routes for the WARP WireGuard endpoints.
1416 # This prevents an infinite recursive routing loop (tunnel loop) where
1417 # the container's WireGuard packets exit the VM, reach the host, match the
1418 # default route (the exit node), and get routed back into the tunnel.
1419 # We route via the gateway IP (not interface) to ensure packets are routed
1420 # properly even when the host's default route changes to the Tailscale interface.
1421
1422 # Helper to manually override the host's default route to point through the
1423 # Tailscale utun* interface. Standalone tailscaled on macOS (Homebrew version)
1424 # lacks the platform-specific integration to override the default gateway
1425 # automatically when --exit-node is used.
1426 setup_exit_node_routing() {
1427     local ts_iface
1428     ts_iface=$(ifconfig | awk '/^[a-z0-9]+:/{iface=$1} /inet 100\./{print iface; exit}' | tr -d ':')
1429
1430     if [ -n "$ts_iface" ]; then
1431         echo "Re-routing default gateway to $ts_iface using 0.0.0.0/1 split..."
1432         local host_mtu
1433         host_mtu=$(read_env_var HOST_MTU)
1434         host_mtu=${host_mtu:-1200}
1435
1436         # Lower the host Tailscale MTU (defaults to 1200) to prevent fragmentation when passing
1437         # through the container's WARP tunnel (which also has an MTU of 1280).
1438         sudo -n ifconfig "$ts_iface" mtu "$host_mtu" >> output.log 2>&1 || true
1439
1440         # DO NOT delete the physical default route! It crashes Colima.
1441         # Use the /1 trick to mathematically override the default route.

```

```

1442 sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
1443 sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
1444 sudo -n route add -net 0.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
1445 sudo -n route add -net 128.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
1446
1447 if netstat -nr | grep -E -q "^(0\.\0\.\0\.\0/1|0/1)[[:space:]]+.*$ts_iface"; then
1448     echo " [] Default route successfully overridden via $ts_iface."
1449 else
1450     echo " [!] Failed to verify exit node routing on $ts_iface."
1451 fi
1452 else
1453     echo "[Warning] Could not detect Tailscale utun interface for host routing."
1454 fi
1455 }
1456
1457 # Helper to nuke leftover network state after teardown (proxy settings, routing
1458 # table, DNS cache, Wi-Fi). Prevents the "had to write a whole recovery script"
1459 # problem where stale state kills internet after the gateway stops.
1460 cleanup_network_state() {
1461     echo -e "\nCleaning up network state (proxies, routes, DNS cache, Wi-Fi)..."
1462
1463     # 1. Disable any leftover proxy settings (needs root on many macOS versions)
1464     for svc in "$ACTIVE_SERVICE" "$ENO_SERVICE"; do
1465         disable_all_proxies "$svc"
1466     done
1467     echo " Proxies disabled."
1468
1469     # 2. Flush DNS cache
1470     if command -v dscacheutil >> output.log 2>&1; then
1471         sudo -n dscacheutil -flushcache 2>> output.log || true
1472     fi
1473     sudo -n killall -HUP mDNSResponder 2>> output.log || true
1474     echo " DNS cache flushed."
1475
1476     # 3. Flush stale routing table entries
1477     if command -v route >> output.log 2>&1; then
1478         sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
1479         sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
1480     fi
1481     remove_warp_bypass_routes
1482     disable_killswitch
1483     echo " Routing table and firewall flushed."
1484
1485     # 4. Power-cycle Wi-Fi to clear any lingering interface state
1486     WIFI_PORT=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: Wi-Fi/{getline; print $2
1487 }')
1488 if [ -n "$WIFI_PORT" ]; then
1489     sudo -n networksetup -setairportpower "$WIFI_PORT" off 2>> output.log || true
1490     sleep 2
1491     sudo -n networksetup -setairportpower "$WIFI_PORT" on 2>> output.log || true
1492     echo " Wi-Fi power-cycled (interface $WIFI_PORT)."
1493     sleep 3
1494 else
1495     echo " Wi-Fi interface not detected; bouncing en0..."
1496     sudo -n ifconfig en0 down 2>> output.log || true
1497     sudo -n ifconfig en0 up 2>> output.log || true
1498 fi
1499
1500 # Force restart sharing services to prevent AirDrop / AirPlay discovery freezes
1501 # after network interface/DNS changes.
1502 echo " Resetting macOS sharing services (AirDrop/AirPlay)..."
1503 reset_sharing_services
1504
1505 echo "Network state cleanup complete."
1506 }
1507
1508 SOCKS_PROXY_PORT=${SOCKS_PROXY_PORT}
1509
1510 # Ensure DNS is set to 1.1.1.1 immediately at script start to prevent any DNS deadlocks/hangs
1511 # during status checks, container startup, VM boot, or teardown. We *also* clear any leftover
1512 # `ts.net` search domain from a previous `tailscale up --accept-dns=true` run, otherwise macOS
1513 # would prepend it to every lookup and most public DNS queries would resolve to NXDOMAIN.
1514
1515 echo "Initializing DNS to 1.1.1.1 to ensure reliable internet access..."
1516 reset_dns
1517
1518
1519
1520
1521 echo "Checking Gateway Status..."

```



```

1522 HIJACK_HOST=$(read_env_var GATEWAY_HIJACK_HOST | tr '[:upper:]' '[:lower:]')
1523
1524 # Decide STOP vs START from persisted intent (marker), not a live Docker probe.
1525 # This makes *disable* instant and robust even when Colima/Docker is down.
1526 #
1527 # CRITICAL (set -e + set -x on bash 3.2): gateway_state ECHOES "running"/"stopped"
1528 # and always returns 0, so we branch on the STRING never on a function's exit
1529 # code. A helper that `return 1`'s trips a spurious `set -e` abort under `set -x`
1530 # (DEBUG_TRACE=true) even when guarded, hence the echo contract. See devref 15.11.8.
1531 if [ "$(gateway_state)" = "running" ]; then
1532     TOGGLE_PHASE="stop"
1533     echo -e "\n=====
1534     echo "Gateway is RUNNING. Stopping it now..."
1535     echo -e "=====
1536
1537 # 1. Disconnect host Tailscale cleanly first before stopping the exit-node container
1538 if [[ "$HIJACK_HOST" != "false" ]]; then
1539     disconnect_tailscale_host
1540     echo ""
1541 fi
1542
1543 echo "Stopping Docker containers..."
1544 # `|| true`: a disable must always complete its teardown even if the Docker
1545 # daemon / Colima VM is already down (compose down would exit non-zero and,
1546 # under set -e, abort the STOP). The START path intentionally does NOT guard
1547 # its `docker compose up` a failed start SHOULD trigger cleanup.
1548 run_logged docker compose down --remove-orphans -t 30 || true
1549
1550 # Only stop Colima if the user explicitly opted in (false by default) to prevent breaking their other Docker
1551 # dev projects
1552 STOP_COLIMA=$(read_env_var STOP_COLIMA_ON_EXIT | tr '[:upper:]' '[:lower:]')
1553 if [[ "$STOP_COLIMA" == "true" ]]; then
1554     echo -e "\nStopping Colima VM to free up host RAM and battery..."
1555     run_with_timeout 30 colima stop >> output.log 2>&1 || echo "Warning: Failed to stop Colima gracefully."
1556 else
1557     echo -e "\nLeaving Colima running. (Set STOP_COLIMA_ON_EXIT=true in .env to change this)"
1558 fi
1559
1560 if [[ "$HIJACK_HOST" != "false" ]]; then
1561     # The host's DNS was hijacked to the gateway IP during ENABLE; now that the
1562     # gateway is down, restore DNS to 1.1.1.1 immediately so subsequent lookups
1563     # don't stall for the macOS DNS timeout before the 1.1.1.1 fallback engages.
1564     echo -e "\nRestoring host DNS to 1.1.1.1 (gateway is gone)... "
1565     reset_dns
1566
1567 # 3. Stop local DNS proxy if running
1568 stop_local_dns_proxy
1569
1570 # Stop background daemons
1571 stop_pidfile_daemon "/tmp/nullexit-cafeinate.pid" "system sleep prevention"
1572 stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
1573 stop_warp_watcher
1574 clear_gateway_active_marker
1575
1576 # 4. Nuke leftover network state so internet actually works after teardown
1577 cleanup_network_state
1578 fi
1579
1580 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
1581 echo -e "\nGateway has been successfully STOPPED in ${ELAPSED} seconds."
1582 else
1583     TOGGLE_PHASE="start"
1584     START_GATEWAY=true
1585     echo "[$(date '+%Y-%m-%d %H:%M:%S')] Action: STARTUP GATEWAY" >> "$LOG_FILE"
1586     echo -e "\n=====
1587     echo "Gateway is STOPPED. Starting it now..."
1588     echo -e "=====
1589
1590 # 1. Prevent host exit-node deadlock during VM / Container startup
1591 if [[ "$HIJACK_HOST" != "false" ]]; then
1592     disconnect_tailscale_host
1593 fi
1594
1595 # 2. Wait for physical DHCP lease to settle (crucial for restarts/wake-up/roaming)
1596 wait_for_dhcp_settle
1597
1598 # 3. Boot Colima VM if it is not already running
1599 echo -e "\nChecking Colima VM status..."
1600 if ! run_with_timeout 15 colima status >> output.log 2>&1; then
1601     # Colima networking is gated on the SAME LAN-P2P signal routing-fix uses
1602     # (.lan_p2p_detected overrides TAILSCALE_ALLOW_LAN_P2P in .env). A LAN-reachable

```

```

1602 # VM (--network-address + bridged/shared) needs the socket_vmnet helper and is
1603 # only worthwhile on a trusted home network, so we request it only when P2P is
1604 # allowed; otherwise AP-isolated / enterprise, the common case we boot plain
1605 # user-mode networking. Either way host<->container P2P is UNAFFECTED: it rides
1606 # routing-fix's Docker-gateway / .host_ips RETURN rules, not the VM network mode
1607 # (see devref 15.3.5). colima_start_until_ready returns as soon as Docker answers
1608 # (~15s) instead of blocking on colima's ~2min post-provision hang (see 15.8.7);
1609 # that hang is colima-internal and mode-independent (it also occurs in user mode).
1610 allow_p2p=$(read_env_var "TAILSCALE_ALLOW_LAN_P2P")
1611 [ -z "$allow_p2p" ] && allow_p2p="false"
1612 if [ -f "$SCRIPT_DIR/.lan_p2p_detected" ]; then
1613     allow_p2p=$(tr -d '[:space:]' < "$SCRIPT_DIR/.lan_p2p_detected" 2>/dev/null || echo "false")
1614 fi
1615
1616 if [ "$allow_p2p" = "true" ]; then
1617     colima_network_mode="bridged"
1618     if [ [ "$OSTYPE" == "darwin*" ] ]; then
1619         security_type=$(system_profiler SPAirPortDataType 2>/dev/null | awk '/Current Network Information:/{
1620             found=1} found && /Security:/{print $NF; exit}')
1621         if echo "$security_type" | grep -qi "Enterprise"; then
1622             echo -e "\n [!] WPA2-Enterprise Wi-Fi detected! Bridged mode will cause MAC-spoofing
1623             deauthentication."
1624             echo "      Falling back to '--network-mode shared' for Colima."
1625             colima_network_mode="shared"
1626         fi
1627     fi
1628     echo "Colima is not running. Starting Colima (600MB RAM, vz VM, LAN P2P on: --network-address
1629     $colima_network_mode)..."
1630     colima_start_until_ready --memory 0.6 --vm-type vz --network-address --network-mode "$colima_network_mode
1631     " \
1632     || echo " [!] Colima Docker not confirmed ready; continuing (downstream docker-ps auto-heal will retry
1633     )."
1634 else
1635     colima_network_mode="user"
1636     echo "Colima is not running. Starting Colima (600MB RAM, vz VM, LAN P2P off: fast user-mode networking)
1637     ..."
1638     colima_start_until_ready --memory 0.6 --vm-type vz \
1639     || echo " [!] Colima Docker not confirmed ready; continuing (downstream docker-ps auto-heal will retry
1640     )."
1641 fi
1642 echo "$colima_network_mode" > /tmp/nullexit_colima_mode.txt
1643 else
1644     echo "Colima is already running."
1645 fi
1646
1647 # 3b. Auto-detect and fix Docker Subnet Collisions (e.g. if the user travels to a 172.17.x.x Wi-Fi)
1648 if [ -f "scripts/fix-docker-bridge-collision.sh" ]; then
1649     if bash scripts/fix-docker-bridge-collision.sh --dry | grep -q "collision detected"; then
1650         echo -e "\n[!] WARNING: Docker Subnet Collision Detected with your current Wi-Fi!"
1651         echo -e "      Auto-fixing Colima config to prevent internet jitter..."
1652         bash scripts/fix-docker-bridge-collision.sh --skip-toggle || true
1653     fi
1654 fi
1655
1656 # 3c. Configure swap file inside the VM to prevent OOM on low-memory limits
1657 # We also set vm.swappiness=10 so the Linux kernel strictly prefers physical RAM
1658 # and avoids unnecessarily wearing out the SSD with proactive background swapping.
1659 if ! run_with_timeout 15 colima ssh -- grep -q 'swapfile' /proc/swaps >> output.log 2>&1; then
1660     echo "Configuring 400MB swap file inside the VM to prevent OOM..."
1661     run_with_timeout 30 colima ssh -- sudo sh -c "if [ ! -f /swapfile ]; then dd if=/dev/zero of=/swapfile bs=1
1662     M count=400 status=none && mkswap /swapfile; fi && swapon /swapfile && sysctl vm.swappiness=10" >> output.
1663     log 2>&1 || echo "Warning: Failed to enable swap file inside the VM."
1664 fi
1665
1666 # 4. Clean up corrupted AdGuardHome configurations
1667 if [ -f "adguard/conf/AdGuardHome.yaml" ] && [ ! -s "adguard/conf/AdGuardHome.yaml" ]; then
1668     rm -f "adguard/conf/AdGuardHome.yaml"
1669     echo "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop."
1670 fi
1671
1672 # 4b. Auto-heal stale Docker socket from hard reboots
1673 if ! run_with_timeout 15 docker ps >/dev/null 2>&1; then
1674     echo "Docker daemon is unresponsive (likely a stale socket from a crash). Auto-healing Colima..."
1675     run_with_timeout 120 colima restart >> output.log 2>&1
1676 fi
1677
1678 # 4c. Inject TCP MSS clamp from .env into post-rules.txt before starting
1679 if [ -f .env ] && grep -q "^GATEWAY_MSS=" .env; then
1680     GATEWAY_MSS=$(read_env_var GATEWAY_MSS)
1681     if [ [ "$GATEWAY_MSS" == "[0-9]+" ] ]; then
1682         # macOS sed syntax

```

```

1674     sed -i 's/--set-mss [0-9]*/--set-mss ${GATEWAY_MSS}/g' post-rules.txt 2>/dev/null || \
1675     # Linux sed fallback
1676     sed -i "s/--set-mss [0-9]*/--set-mss ${GATEWAY_MSS}/g" post-rules.txt 2>/dev/null
1677 fi
1678 fi
1679
1680 # 5. Start compose services
1681 write_host_ips
1682
1683 # Procedurally generated ports have already been exported at the top of the script.
1684 echo " [Security] Procedurally randomized proxy ports generated to evade fingerprinting."
1685
1686 # --- HONEY-PORT TRIPWIRE ---
1687 # Generate a random trap port. If any malware does a sequential port scan on localhost,
1688 # it will inevitably hit this port. When it does, we instantly fire a macOS alert.
1689 HONEY_PORT=$(get_free_port)
1690 (
1691     if nc -l localhost "$HONEY_PORT" >/dev/null 2>&1; then
1692         echo "[$(date '+%Y-%m-%d %H:%M:%S')] [CRITICAL SECURITY ALERT] PORT SCAN DETECTED! An unknown application
just pinged the local Honey-Port ($HONEY_PORT)!" >> "$LOG_FILE"
1693         osascript -e 'display notification "An unknown application is aggressively scanning your local ports.
Malware port-scan suspected." with title "nullexit OPSEC Alert" sound name "Basso" 2>/dev/null || true
1694     fi
1695 ) &
1696 disown %1 2>/dev/null || true
1697 echo " [Security] Honey-Port Tripwire armed on random port to detect malware port-scans."
1698
1699 echo -e "\nStarting Docker containers..."
1700 run_logged docker compose up -d --build
1701
1702 # Log the output of the rule compiler for debugging before removing it
1703 docker compose logs rule-compiler >> output.log 2>&1
1704 # Clean up the one-off rule compiler container so it doesn't clutter the Docker UI
1705 docker compose rm -s -f rule-compiler >> output.log 2>&1
1706
1707 RULE_COUNT=$(grep "! Total Block Rules:" adguard/work/userfilters/compiled_rules.txt 2>/dev/null | awk -F': ' '{print $2}' || echo "0")
1708 NATIVE_COUNT=$(grep "! Native AdGuard Rules:" adguard/work/userfilters/compiled_rules.txt 2>/dev/null | awk -F': ' '{print $2}' || echo "0")
1709
1710 if [ "$RULE_COUNT" != "0" ] && [ "$RULE_COUNT" != "" ]; then
1711     if [ "$NATIVE_COUNT" != "0" ] && [ "$NATIVE_COUNT" != "" ]; then
1712         TOTAL_COUNT=$((RULE_COUNT + NATIVE_COUNT))
1713         # Format with commas for readability (e.g. 441,578)
1714         if command -v printf &> /dev/null; then
1715             FORMATTED_RULE=$(printf "%'d" "$RULE_COUNT")
1716             FORMATTED_TOTAL=$(printf "%'d" "$TOTAL_COUNT")
1717             echo " DNS: Compiled $FORMATTED_RULE unique custom rules (Total active in AdGuard: ~$FORMATTED_TOTAL
rules)."
1718         else
1719             echo " DNS: Compiled $RULE_COUNT unique custom rules (Total active in AdGuard: ~$TOTAL_COUNT rules)."
1720         fi
1721     else
1722         echo " DNS: Compiled and loaded $RULE_COUNT optimized rules."
1723     fi
1724 fi
1725
1726 # Report IP blocklist compilation results
1727 IP_COUNT=$(grep "^# Entries:" adguard/work/userfilters/ip_blocklist.ipset 2>/dev/null | awk -F': ' '{print $2}' || echo "0")
1728 if [ "$IP_COUNT" != "0" ] && [ "$IP_COUNT" != "" ]; then
1729     if command -v printf &> /dev/null; then
1730         FORMATTED_IP=$(printf "%'d" "$IP_COUNT")
1731         echo " IP: Loaded $FORMATTED_IP threat intelligence IPs/CIDRs into kernel firewall."
1732     else
1733         echo " IP: Loaded $IP_COUNT threat intelligence IPs/CIDRs into kernel firewall."
1734     fi
1735 fi
1736
1737 # 6. Wait for the gateway container's Tailscale connection to be ready
1738 BYPASS_PING=$(read_env_var GATEWAY_BYPASS_PING | tr '[:upper:]' '[:lower:]')
1739 USE_EXIT_NODE=$(read_env_var GATEWAY_USE_EXIT_NODE | tr '[:upper:]' '[:lower:]')
1740
1741 echo "Waiting for gateway container's Tailscale to connect to the tailnet..."
1742 echo " (This can take 30-60s Tailscale goes through: NoState Starting Running Online)"
1743 connected=false
1744 consecutive_failures=0
1745 for i in {1..60}; do
1746     # Run tailscale status and capture its output so the user can see what's happening.
1747     # Previously this was hidden behind 2>> output.log, making it impossible to debug hangs.
1748     ts_output=$(run_with_timeout 5 docker compose exec -T tailscale tailscale status 2>&1 || true)

```

```

1749
1750 if echo "$ts_output" | grep -q "offers exit node"; then
1751     connected=true
1752     break
1753 fi
1754
1755 # Track consecutive failures to detect a stuck state.
1756 # If we see 40+ consecutive 'NoState' responses, give up early
1757 # the daemon is alive but can't connect to the coordination server.
1758 if echo "$ts_output" | grep -q "NoState"; then
1759     consecutive_failures=$((consecutive_failures + 1))
1760     if [ "$consecutive_failures" -ge 40 ]; then
1761         echo ""
1762         echo " [attempt $i/60] Stuck in 'NoState' for $consecutive_failures checks."
1763         echo " Tailscale daemon is running but can't reach the coordination server."
1764         break
1765     fi
1766 else
1767     consecutive_failures=0
1768 fi
1769
1770 # Print a condensed status every 10 seconds so the user can see progress
1771 if [ $((i % 10)) -eq 0 ]; then
1772     ts_short=$(echo "$ts_output" | head -1 | tr -d '\r\n')
1773     echo " [attempt $i/60] $ts_short"
1774 elif [ $((i % 5)) -eq 0 ]; then
1775     echo -n "[i]"
1776 else
1777     echo -n "."
1778 fi
1779 sleep 1
1780 done
1781 echo ""
1782
1783 if [ "$connected" = "true" ]; then
1784     echo "Gateway container is online on the Tailscale mesh."
1785 elif [ "$BYPASS_PING" = "true" ]; then
1786     echo "[Warning] Gateway container check timed out. Proceeding anyway (GATEWAY_BYPASS_PING is true)..."
1787 else
1788     echo "ERROR: Gateway container failed to initialize Tailscale (timed out after 60s)." >&2
1789     echo " The container was seen in the following states during the wait:" >&2
1790     echo "$ts_output" | sed 's/^/ /' >&2
1791     echo "" >&2
1792     echo " This could mean:" >&2
1793     echo " 1. Tailscale auth key has expired check TS_AUTHKEY in .env" >&2
1794     echo " 2. Network issue inside the container check: docker compose logs tailscale" >&2
1795     echo " 3. gluetun VPN tunnel not connecting check: docker compose logs warp | tail -20" >&2
1796     echo "" >&2
1797     echo " Diagnose manually:" >&2
1798     echo " docker compose exec -T tailscale tailscale status" >&2
1799     echo " docker compose exec -T tailscale tailscale netcheck" >&2
1800 cleanup_handler ERR $LINENO
1801 exit 1
1802 fi
1803
1804 # 6b. Resolve the gateway's Tailscale IP.
1805 # Dynamically query the container for the IP (handles IP changes after re-auth)
1806 # and cache it in .gateway_ip for fast retrieval by recover.sh during Wi-Fi roaming.
1807 #
1808 # CRITICAL: `docker compose exec -T` on macOS/Colima injects carriage
1809 # returns (\r) into captured output. If $TS_IP ends with \r, then
1810 # `networksetup -setdnsservers` silently rejects the malformed IP and
1811 # DNS stays at 1.1.1.1 the primary bug this project was hitting.
1812 # `tr -d '\r'` strips the CR; `awk 'NR==1{print $1; exit}'` takes only
1813 # the first field of the first line.
1814 TS_IP=""
1815 if [ "$connected" = "true" ]; then
1816     TS_IP=$(run_with_timeout 10 docker compose exec -T tailscale tailscale ip -4 2>> output.log | tr -d '\r' |
1817         awk 'NR==1{print $1; exit}' || true)
1818     if [ -n "$TS_IP" ]; then
1819         echo "Resolved gateway Tailscale IP dynamically: $TS_IP"
1820         echo "$TS_IP" > .gateway_ip
1821     else
1822         echo " [!] Failed to resolve gateway IP dynamically. Trying fallback cache..." >&2
1823         TS_IP=$(read_adguard_ip || true)
1824     fi
1825 else
1826     TS_IP=$(read_adguard_ip || true)
1827 fi
1828 # 7. Connect host Mac to Tailscale mesh.

```

```

1829 # With the standalone tailscaled daemon (registered as a per-user LaunchAgent or
1830 # system LaunchDaemon via 'brew services start tailscale'), the previous five-
1831 # phase flow collapses into two trivial CLI calls. There is no .app GUI to launch,
1832 # no menu bar item to click, and no Accessibility permission required.
1833 #
1834 # CRITICAL: If tailscaled is not running, `tailscale up` will silently fail.
1835 # We auto-start it before proceeding, and SKIP the rest of the Tailscale setup
1836 # if it can't be started (DNS hijack to a 100.x.x.x IP and exit-node routing
1837 # are impossible without the host on the mesh).
1838 #
1839 # We connect in two phases:
1840 # Phase A Join mesh WITHOUT exit node (so pre-flight checks can run)
1841 # Phase B Set exit node only after pre-flight checks confirm the gateway works
1842 HOST_ON_MESH=false
1843 SKIP_EXIT_NODE=false
1844 if [ "$HIJACK_HOST" = "false" ]; then
1845     echo -e "\n[Info] HIJACK_HOST is false (Headless Mode). Host networking will remain untouched."
1846     SKIP_EXIT_NODE=true
1847 elif [ -n "$TS_BIN" ]; then
1848     echo -n "Verifying tailscaled is reachable"
1849     daemon_ready=false
1850     status_exit=0
1851     for i in {1..10}; do
1852         # Test if the daemon responds to status check.
1853         # If status returns non-zero, check if it was a normal "stopped/disconnected" state
1854         # (exit code 1 is normal for disconnected). If the command exited quickly (not killed by timeout),
1855         # and the error is just "Tailscale is stopped", then the daemon is alive and ready.
1856         # But if it timed out (exit code 143) or is completely unresponsive, it's wedged.
1857         status_exit=0
1858         if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
1859             daemon_ready=true
1860             break
1861         else
1862             status_exit=$?
1863             if [ "$status_exit" -ne 143 ] && [ -S /var/run/tailscaled.socket ]; then
1864                 daemon_ready=true
1865                 break
1866             fi
1867         fi
1868         echo -n "."
1869         sleep 1
1870     done
1871     echo ""
1872
1873     # If the daemon was unresponsive or socket check failed, attempt auto-restart
1874     if [ "$daemon_ready" != "true" ]; then
1875         if [ "$status_exit" -eq 143 ]; then
1876             warn "tailscaled daemon is wedged/unresponsive. Restarting it..."
1877         else
1878             warn "tailscaled daemon is not running. Attempting to start it..."
1879         fi
1880         restart_tailscaled_daemon
1881
1882         # Re-verify
1883         if run_with_timeout 5 $TS_BIN status >> output.log 2>&1 || [ -S /var/run/tailscaled.socket ]; then
1884             daemon_ready=true
1885         fi
1886     fi
1887
1888     if [ "$daemon_ready" = "true" ]; then
1889         echo "tailscaled is responsive (running as a system service)."
1890
1891         # Phase A: Join mesh without exit node
1892         echo "Connecting host to Tailscale mesh (no exit node yet)..."
1893         if $TS_BIN up; then
1894             $TS_BIN set --ssh=true --accept-dns=false --accept-routes=true --exit-node= || true
1895             HOST_ON_MESH=true
1896             echo "Host is on Tailscale mesh."
1897         else
1898             warn "tailscale up failed even though the daemon is running."
1899             echo " This usually means the host hasn't authenticated with your tailnet yet."
1900             echo " Run this command in a terminal to authenticate:"
1901             echo ""
1902             echo " sudo tailscale up"
1903             echo ""
1904             echo " A browser will open. Log in to your Tailscale account, then re-run toggle.sh."
1905         fi
1906     else
1907         fail "tailscaled could NOT be started automatically."
1908         echo " Run this in a terminal to start and authenticate Tailscale:"
1909         echo ""

```

```

1910     echo "          sudo brew services start tailscale"
1911     echo "          sudo tailscale up"
1912     echo ""
1913     echo "    Then re-run toggle.sh."
1914 fi
1915 else
1916     echo -e "\n[Warning] tailscale CLI not found on PATH. Skipping host Tailscale configuration..."
1917 fi
1918
1919 # Phase B: Pre-flight checks + enable exit node only if safe
1920 if [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" != "false" ] && [ -n "$TS_IP" ]; then
1921     echo -e "\n"
1922     echo "Pre-flight connectivity check"
1923     echo ""
1924
1925     # Check 1: Can we reach the gateway via Tailscale (uses tailscale ping, not ICMP)?
1926     # NOTE: tailscale ping exits 1 when the connection goes through a DERP relay
1927     # ("direct connection not established") even though the pong was received.
1928     # We check for 'pong' in the output (stderr discarded) to accept relayed connections.
1929     echo -n "    [1/3] Gateway reachable via Tailscale... "
1930     ping_ok=false
1931     # The host tailscaled needs a few seconds to propagate the new network map and
1932     # establish a connection route after 'tailscale up --reset'. We retry up to 45 times.
1933     for attempt in {1..45}; do
1934         if $TS_BIN ping --until-direct=false -c 2 --timeout 2s "$TS_IP" 2>/dev/null | grep -q "pong"; then
1935             ping_ok=true
1936             break
1937         fi
1938         sleep 1
1939     done
1940     if [ "$ping_ok" = "true" ]; then
1941         echo "PASS"
1942     else
1943         echo "FAIL"
1944         SKIP_EXIT_NODE=true
1945     fi
1946
1947     # Check 2: Can we reach AdGuard via localhost (exposed port ${DNS_PROXY_PORT})?
1948     # Colima's SSH tunnel only forwards TCP (not UDP), so we use +tcp.
1949     echo -n "    [2/3] AdGuard DNS via localhost... "
1950     if command -v dig &>/dev/null; then
1951         if dig +tcp @127.0.0.1 -p ${DNS_PROXY_PORT} google.com +short +timeout=5 &>/dev/null; then
1952             echo "PASS"
1953         else
1954             echo "FAIL"
1955             SKIP_EXIT_NODE=true
1956         fi
1957     elif command -v nslookup &>/dev/null; then
1958         if nslookup -port=${DNS_PROXY_PORT} google.com 127.0.0.1 &>/dev/null; then
1959             echo "PASS"
1960         else
1961             echo "FAIL"
1962             SKIP_EXIT_NODE=true
1963         fi
1964     else
1965         echo "SKIP (dig/nslookup not available)"
1966     fi
1967
1968     # Check 3: Does the WARP container have external internet?
1969     echo -n "    [3/3] WARP container internet... "
1970     tmp_err=$(mktemp)
1971     if docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace >/dev/null 2>
1972         "$tmp_err"; then
1973         echo "PASS"
1974     else
1975         echo "FAIL"
1976         if [ -s "$tmp_err" ]; then
1977             err_msg=$(cat "$tmp_err" | tr -d '\r')
1978             echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Check 3] WARP container check failed: $err_msg" >> output.log
1979         fi
1980         SKIP_EXIT_NODE=true
1981     fi
1982     rm -f "$tmp_err"
1983
1984     # Act based on check results
1985     if [ "$SKIP_EXIT_NODE" != "true" ]; then
1986         echo -e "\nAll checks passed. Enabling exit node $TS_IP..."
1987         $TS_BIN up || true
1988         if $TS_BIN set --ssh=true --accept-dns=false --accept-routes=true --exit-node="$TS_IP" --exit-node-allow-
1989             lan-access=true; then
1990             echo "Exit node enabled."

```



```

1989     add_warp_bypass_routes
1990     setup_exit_node_routing
1991     enable_killswitch
1992 else
1993     echo "[Warning] Failed to set exit node."
1994     SKIP_EXIT_NODE=true
1995 fi
1996 fi
1997
1998 if [ "$SKIP_EXIT_NODE" = "true" ]; then
1999     warn "Pre-flight checks failed  EXIT NODE + DNS HIJACK SKIPPED."
2000     echo "  Host stays on Tailscale mesh but your internet routes through your normal connection."
2001     echo "  Troubleshoot with:"
2002     echo "    docker compose logs warp | tail -20"
2003     echo "    docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace"
2004     echo ""
2005     echo "  Falling back to SOCKS5 proxy for traffic routing..."
2006 fi
2007 elif [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" = "false" ]; then
2008     echo "USE_EXIT_NODE is false. Skipping exit node and DNS hijack."
2009     SKIP_EXIT_NODE=true
2010 fi
2011
2012 # 8. Apply DNS Hijacking.
2013 # If exit node is enabled: hijack DNS to gateway's Tailscale IP (routes through tailnet).
2014 # Otherwise, leave DNS at 1.1.1.1 (already set by reset at the top).
2015 # AdGuard is also available at localhost:${DNS_PROXY_PORT} for manual configuration in apps.
2016 if [ -n "$TS_IP" ] && [ "$HOST_ON_MESH" = "true" ] && [ "$SKIP_EXIT_NODE" != "true" ]; then
2017     echo -e "\nHijacking host DNS to point ONLY at AdGuard Home ($TS_IP)..."
2018     echo "Setting MagicDNS search domain 'ts.net' so tailnet hostnames resolve..."
2019     if force_dns_to_gateway "$TS_IP"; then
2020         echo "DNS hijack verified: single server = $TS_IP."
2021     else
2022         echo "[Warning] DNS hijack could not be verified."
2023         echo "  If DNS is broken, run manually:"
2024         echo "    networksetup -setdnsservers \"\$ACTIVE_SERVICE\" \"$TS_IP"
2025         echo "    networksetup -setsearchdomains \"\$ACTIVE_SERVICE\" ts.net"
2026     fi
2027 elif [ -n "$TS_IP" ] && [ "$HIJACK_HOST" != "false" ]; then
2028     echo -e "\n[Info] Exit node not enabled (pre-flight checks failed)."
2029     echo "  Trying local DNS proxy for ad-blocking..."
2030     if start_local_dns_proxy; then
2031         echo "  Ad-blocking active via local DNS proxy through AdGuard."
2032     else
2033         echo "  Local DNS proxy unavailable. DNS stays at 1.1.1.1."
2034         echo "  AdGuard available at http://localhost:80 (port ${DNS_PROXY_PORT} for DNS via TCP)."
2035     fi
2036
2037 # Enable SOCKS5 proxy for traffic routing through WARP
2038 # (enabled whenever exit node is skipped, regardless of HOST_ON_MESH)
2039 echo -e "\n  Enabling SOCKS5 proxy for TCP traffic routing through gateway WARP tunnel..."
2040 echo "  (SOCKS5 proxy runs inside gateway container, routes through tun0 -> WARP)"
2041 enable_socks_proxy "$ACTIVE_SERVICE"
2042
2043 # Verify the SOCKS5 proxy works through WARP
2044 echo -n "  Verifying SOCKS5 proxy routes through WARP... "
2045 if curl --socks5-hostname 127.0.0.1:${SOCKS_PROXY_PORT} --max-time 10 -s https://www.cloudflare.com/cdn-cgi/
2046     trace 2>/dev/null | grep -q "warp=on"; then
2047     echo "ok (warp=on)."
2048     echo "  All TCP traffic now encrypted through Cloudflare WARP tunnel."
2049 else
2050     echo "FAILED (proxy not routing through WARP)."
2051     echo "  Disabling SOCKS5 proxy  internet stays on direct connection."
2052     disable_socks_proxy
2053     echo "  SOCKS proxy available at localhost:${SOCKS_PROXY_PORT} for manual configuration."
2054 fi
2055 else
2056     echo -e "\n[Warning] Could not resolve gateway Tailscale IP."
2057 fi
2058
2059 # 9. Verify host connectivity
2060 if [ "$HOST_ON_MESH" = "true" ] && [ -n "$TS_BIN" ]; then
2061     if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
2062         echo "Host is online on the Tailscale mesh."
2063     else
2064         echo "[Warning] Host is not responding on Tailscale."
2065     fi
2066 fi
2067
2068 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
2069 echo -e "\nGateway has been successfully STARTED in ${ELAPSED} seconds."

```

```

2069
2070 # Prevent system from going to sleep while gateway is running
2071 start_sleep_prevention
2072
2073 if [[ "$HIJACK_HOST" != "false" ]]; then
2074     if [ -n "$TS_IP" ] && [ "$TS_IP" != "1.1.1.1" ]; then
2075         start_dns_watcher "$TS_IP"
2076     fi
2077
2078     # Start WARP liveness monitor (logs to output.log on warp flip)
2079     start_warp_watcher
2080 fi
2081
2082 # Reset sharing services after network configuration to prevent AirDrop freezes
2083 echo "Resetting macOS sharing services (AirDrop/AirPlay)..."
2084 reset_sharing_services
2085
2086 # Tell external watchers (post-wake / network-change) the gateway is up.
2087 write_gateway_active_marker
2088
2089 # Local Network Surveillance Check
2090 echo -e "\n"
2091 echo "Local Network Surveillance Check"
2092 echo ""
2093 # Count populated ARP cache entries to detect network scanning or high congestion
2094 # Using '-an' avoids hanging on DNS reverse resolution when the network state is in transition.
2095 ARP_COUNT=$(arp -an 2>/dev/null | grep -iv 'incomplete' | wc -l | tr -d ' ')
2096 if [ "$ARP_COUNT" -gt 15 ]; then
2097     warn "High ARP activity detected ($ARP_COUNT devices in cache)."
2098     warn "This Wi-Fi network may be heavily congested or actively scanned (e.g., arp-scan)."
2099     echo -e " [] Your traffic remains fully encrypted and invisible.\n"
2100 else
2101     echo -e " [] Local network looks quiet ($ARP_COUNT devices). No aggressive scanning detected.\n"
2102 fi
2103 fi
2104
2105 SUCCESS_RUN=true
2106
2107 # Final DNS state summary
2108 if [ -n "$ACTIVE_SERVICE" ]; then
2109     FINAL_DNS=$(networksetup -getdnsservers "$ACTIVE_SERVICE" 2>> output.log || true)
2110     echo -e "\n"
2111     echo -e "DNS STATE: $FINAL_DNS"
2112     echo -e ""
2113 fi
2114
2115 if [ "$START_GATEWAY" = "true" ] && command -v docker >/dev/null 2>&1; then
2116     if docker compose logs rule-compiler 2>/dev/null | grep -q "Warning: Failed to fetch"; then
2117         echo -e "\n[Warning] One or more blocklist URLs failed to download (404/Offline)."
2118         echo " The gateway gracefully fell back to yesterday's cached blocklist."
2119         echo " (Run 'docker logs rule-compiler' later to investigate which link died.)"
2120     fi
2121 fi
2122
2123 rm -f "${LOCK_FILE:-/tmp/nullexit-toggle.lock}"
2124 if [ -t 0 ]; then
2125     read -rp "Press [r] and Enter to instantly reverse state, or just press Enter to exit: " USER_CHOICE
2126     if [[ "${USER_CHOICE}" == "r" || "${USER_CHOICE}" == "R" ]]; then
2127         echo "Reversing gateway state..."
2128         exec bash "$0"
2129     fi
2130 fi
2131
2132 echo -e "\nYou can close this terminal window now."

```

7 recover.sh

```

1 #!/bin/bash
2 # recover.sh nullexit KILL-SWITCH recovery script (macOS + Linux)
3 # Fixes internet when toggle.sh leaves you stranded.
4 #
5 # This is a NUCLEAR option: it undoes EVERYTHING toggle.sh does by:
6 # 1. Disconnecting host Tailscale from the mesh (exit-node off)
7 # 2. Resetting DNS to default on ALL network services
8 # 3. Disabling rogue proxy settings (SOCKS, web, secure web)
9 # 4. Flushing the DNS cache

```

```

10 # 5. Flushing stale routing table entries
11 # 6. Stopping gateway Docker containers
12 # 7. Power-cycling Wi-Fi
13 # 8. Verifying internet is working
14 #
15 # One file, both OSes: the dispatch below runs the self-contained Linux
16 # recovery and exits; on macOS it falls through to the richer dual-mode
17 # implementation (default teardown / --post-wake refresh).
18 #
19 # Usage: double-click Recover-Gateway.applescript
20 # OR ./recover.sh
21 # OR ./recover.sh --post-wake (macOS only lighter refresh, keeps gateway live)
22
23 set -e
24
25 # Resolve script directory (used for path-relative refs)
26 # recover.sh lives at the repo root; SCRIPT_DIR lets us launch toggle.sh
27 # (and reference any other repo-root file) from the same directory no
28 # matter where the user invoked recover.sh from.
29 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
30 cd "$SCRIPT_DIR" || exit 1
31
32 #
33 # LINUX RECOVERY self-contained; runs the full teardown then exits before the
34 # macOS body below. Native Linux tooling (resolvectl / nmcli / ip) throughout.
35 #
36 if [[ "$OSTYPE" != "darwin"* ]]; then
37 # Source common formatting and helper functions
38 source "$SCRIPT_DIR/scripts/common.sh"
39
40 # Always add Homebrew path
41 export PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:$PATH"
42
43 echo ""
44 echo -e "${RED}${BOLD}${NC}"
45 echo -e "${RED}${BOLD} Nullexit Gateway RECOVERY MODE ${NC}"
46 echo -e "${RED}${BOLD}${NC}"
47 echo ""
48 echo "This script will tear down the gateway and restore normal internet."
49 echo ""
50
51 # Cache sudo credentials upfront
52 # Prevents the script from hanging halfway through when it needs to run
53 # privileged commands (route flush, Wi-Fi power cycle, etc.).
54 echo -e "${YELLOW}${BOLD}Authentication required for network recovery...${NC}"
55 sudo -v
56 (
57 while true; do
58 sudo -n true 2>/dev/null
59 sleep 60
60 done
61 ) &
62 SUDO_KEEPER_PID=$!
63 trap 'kill $SUDO_KEEPER_PID 2>/dev/null' EXIT
64
65 # 1. Disconnect host Tailscale
66 step "Disconnecting host Tailscale from mesh"
67 if command -v tailscale >> output.log 2>&1; then
68 # Check if actually connected first (with timeout tailscaled could be wedged)
69 if run_with_timeout 5 tailscale status >> output.log 2>&1; then
70 # Also explicitly reset any exit-node so it doesn't linger.
71 if run_with_timeout 10 tailscale up --reset --ssh=true --accept-dns=false --exit-node= >> output.log
72 2>&1; then
73 ok "Exit-node preference cleared"
74 else
75 warn "tailscale up --reset didn't respond (tailscaled may be wedged)"
76 fi
77 ts_args=""
78 if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
79 if run_with_timeout 10 tailscale down $ts_args >> output.log 2>&1; then
80 ok "Tailscale disconnected"
81 else
82 warn "tailscale down didn't respond"
83 fi
84 else
85 warn "tailscaled not reachable skipping (timeout after 5s)"
86 fi
87 else
88 warn "tailscale CLI not found skipping"
89 fi

```

```

90 # 2. Reset DNS to default on ALL network services
91 step "Resetting DNS to default on active interface"
92
93 ACTIVE_SERVICE=$(ip route get 1.1.1.1 2>/dev/null | awk '{print $5; exit}')
94 if [ -n "$ACTIVE_SERVICE" ]; then
95     sudo resolvectl revert "$ACTIVE_SERVICE" 2>> output.log || true
96     ok "DNS reset on $ACTIVE_SERVICE"
97 else
98     warn "Could not determine active network interface"
99 fi
100
101 # 3. Disable rogue proxy settings
102 step "Disabling proxy settings (SOCKS, web, secure web)"
103 echo " (Linux global SOCKS proxy configuration skipped.)"
104 ok "Proxy settings cleared"
105
106
107 # 4. Flush DNS cache
108 step "Flushing DNS cache"
109 if command -v resolvectl >> output.log 2>&1; then
110     sudo resolvectl flush-caches 2>> output.log || true
111     ok "DNS cache flushed"
112 else
113     warn "resolvectl not found"
114 fi
115
116 # 5. Clear stale routing table
117 step "Flushing stale routing table entries"
118 sudo ip route flush cache >> output.log 2>&1 || true
119 ok "Routing table flushed"
120
121 # 6. Stop gateway Docker containers
122 step "Stopping gateway Docker containers"
123 if command -v docker >> output.log 2>&1 && docker info >> output.log 2>&1; then
124     if [ -f "docker-compose.yml" ]; then
125         docker compose down -t 5 2>> output.log && ok "Containers stopped" || warn "No containers were running"
126     else
127         warn "docker-compose.yml not found in current directory"
128     fi
129 else
130     warn "Docker not available skipping"
131 fi
132
133 # 6b. Stopping sleep prevention (systemd-inhibit)
134 step "Stopping sleep prevention"
135 PID_FILE="/tmp/nullexit-cafeinate.pid"
136 if [ -f "$PID_FILE" ]; then
137     INHIBIT_PID=$(cat "$PID_FILE")
138     if [ -n "$INHIBIT_PID" ] && kill -0 "$INHIBIT_PID" 2>/dev/null; then
139         kill "$INHIBIT_PID" 2>/dev/null || true
140         ok "Sleep prevention stopped (PID $INHIBIT_PID)"
141     else
142         warn "Sleep prevention process not running, cleaning up stale PID file"
143     fi
144     rm -f "$PID_FILE"
145 else
146     ok "Sleep prevention was not active"
147 fi
148
149 # 7. Power-cycle Wi-Fi
150 step "Power-cycling Wi-Fi"
151 if command -v nmcli &>/dev/null; then
152     sudo nmcli radio wifi off 2>> output.log || true
153     sleep 2
154     sudo nmcli radio wifi on 2>> output.log || true
155     ok "Wi-Fi bounced via nmcli"
156     sleep 3
157 else
158     warn "nmcli not found. Falling back to ip link..."
159     WIFI_IFACE=$(ip link | grep -E '[0-9]+: (wl[a-zA-Z0-9]+|wlan[0-9]+):' | awk -F': ' '{print $2}')
160     if [ -n "$WIFI_IFACE" ]; then
161         sudo ip link set "$WIFI_IFACE" down 2>> output.log || true
162         sleep 2
163         sudo ip link set "$WIFI_IFACE" up 2>> output.log || true
164         ok "Wi-Fi bounced via ip link (interface $WIFI_IFACE)"
165         sleep 3
166     else
167         warn "No Wi-Fi interface detected."
168     fi
169 fi
170

```

```

171 # 8. Verify internet connectivity
172 verify_internet_connectivity
173
174 # Summary
175 echo ""
176 echo -e "${BOLD}${NC}"
177 echo -e "${BOLD}RECOVERY SUMMARY${NC}"
178 echo -e "${BOLD}${NC}"
179
180 # Show final DNS state
181 FINAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>> output.log | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | tr
'\n' ' ' || echo "N/A")
182 echo " DNS ($ACTIVE_SERVICE): $FINAL_DNS"
183
184 echo ""
185
186 if [ "$INTERNET_OK" = "true" ]; then
187     echo -e " ${GREEN}${BOLD} Internet is working.${NC}"
188 else
189     echo -e " ${YELLOW}${BOLD} Could not verify internet connectivity.${NC}"
190     echo " This might mean:"
191     echo " - Your Wi-Fi is disconnected from the router"
192     echo " - Tailscale is still interfering (try restarting tailscaled:"
193     echo "     systemctl restart tailscaled)"
194     echo " - You need to re-connect to Wi-Fi in the menu bar"
195     echo " - Try toggling Wi-Fi off/on in the menu bar"
196     echo ""
197     echo " Or try manually:"
198     echo "     resolvectl revert $ACTIVE_SERVICE"
199     extra_args=""
200     if is_kill_switch_enabled; then extra_args="--accept-risk=lose-ssh"; fi
201     echo "     tailscale down $extra_args"
202     echo "     sudo ip route flush cache"
203     echo "     systemctl restart tailscaled"
204 fi
205
206 echo ""
207 echo -e "${GREEN}${BOLD}Recovery complete.${NC}"
208 echo ""
209
210 exit 0
211 fi
212
213 #
214 # macOS RECOVERY (dual-mode: default teardown / --post-wake refresh)
215 #
216
217 # Enforce Cryptographic Script Integrity
218 if [ -f "scripts/crypto.sh" ]; then
219     if ! bash scripts/crypto.sh --verify; then
220         exit 1
221     fi
222 fi
223
224 # Argument parsing
225 # --post-wake: lightweight refresh after sleep/wake or Wi-Fi roam.
226 # Keeps the gateway live while HUPing DNS caches, refreshing the
227 # Tailscale exit-node leak, re-hijacking host DNS, and force-
228 # recreating the warp container if its gluetun healthcheck failed.
229 # Does NOT tear down Docker, Tailscale, sleep prevention, or Wi-Fi.
230 # Called from scripts/watcher.sh on every wake / network-change event
231 # while /tmp/nullexit-gateway-active.marker is present (i.e. the
232 # gateway is currently up). See devref.md 10.29 for the why.
233 POST_WAKE=false
234 AUTO_MODE=false
235 for arg in "$@"; do
236     if [ "$arg" = "--post-wake" ]; then
237         POST_WAKE=true
238     elif [ "$arg" = "--auto" ] || [ "$arg" = "--non-interactive" ]; then
239         AUTO_MODE=true
240     fi
241 done
242
243 rm -f "$SCRIPT_DIR/TUNNEL_FAILED_CLOSED.marker"
244
245 # Prevent concurrent execution of lifecycle scripts (toggle.sh / recover.sh)
246 LOCK_FILE="/tmp/nullexit-toggle.lock"
247 if [ -f "$LOCK_FILE" ]; then
248     LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
249     if [ -n "$LOCK_PID" ] && [ "$LOCK_PID" != "$$" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
250         if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -q -E "toggle\.sh|recover\.sh"; then

```

```

251     if [ "$POST_WAKE" = "true" ]; then
252         echo "[$(date -u +%FT%TZ)] POST-WAKE skipped: another lifecycle script (PID $LOCK_PID) is running." >>
output.log
253         exit 0
254     else
255         echo "Error: Another instance of toggle.sh or recover.sh (PID $LOCK_PID) is already running." >&2
256         exit 1
257     fi
258 fi
259 fi
260 fi
261 echo "$$" > "$LOCK_FILE"
262
263 # Clear the active-state marker in nuclear recovery mode since the gateway is being
264 # completely torn down. This prevents scripts/watcher.sh from triggering post-wake
265 # recoveries in response to network changes caused by the teardown itself.
266 if [ "$POST_WAKE" = "false" ]; then
267     rm -f "/tmp/nullexit-gateway-active.marker"
268 fi
269
270 # WARP Watcher inhibit marker: written immediately in --post-wake mode so the
271 # background WARP Watcher (in toggle.sh) skips failure counting while we are
272 # intentionally tearing down / recreating the warp container. Without this, the
273 # watcher accumulates 6 consecutive warp=off readings during force-recreate (~35s)
274 # and fires nuclear recover.sh, killing a gateway that would have been healthy.
275 WARP_INHIBIT_FILE="/tmp/nullexit-warp-inhibit.marker"
276 if [ "$POST_WAKE" = "true" ]; then
277     touch "$WARP_INHIBIT_FILE"
278     echo "[$(date -u +%FT%TZ)] POST-WAKE started WARP Watcher inhibited to prevent spurious shutdown during warp
recreate." >> output.log
279 fi
280
281 # Ensure the inhibit marker and lock file are always removed when the script exits (success or error)
282 cleanup_recover() {
283     rm -f "$WARP_INHIBIT_FILE"
284     if [ -f "$LOCK_FILE" ] && [ "$(cat "$LOCK_FILE" 2>/dev/null)" = "$$" ]; then
285         rm -f "$LOCK_FILE"
286     fi
287 }
288 trap cleanup_recover EXIT
289
290 # Source common formatting and helper functions
291 source "$SCRIPT_DIR/scripts/common.sh"
292
293 # Always add Homebrew path
294 setup_standard_path
295
296 echo ""
297 if [ "$POST_WAKE" = "true" ]; then
298     echo -e "${YELLOW}${BOLD}${NC}"
299     echo -e "${YELLOW}${BOLD} Nullexit POST-WAKE / POST-ROAM LIGHT ${NC}"
300     echo -e "${YELLOW}${BOLD} (keeps the gateway live, refreshes) ${NC}"
301     echo -e "${YELLOW}${BOLD}${NC}"
302     echo ""
303     echo "Re-hijacking DNS, refreshing Tailscale exit-node, force-recreating"
304     echo "warp if unhealthy, and resetting sharing services. The gateway stays"
305     echo "up. docker compose / Wi-Fi / caffeinate are NOT touched."
306     echo ""
307 else
308     echo -e "${RED}${BOLD}${NC}"
309     echo -e "${RED}${BOLD} Nullexit Gateway RECOVERY MODE ${NC}"
310     echo -e "${RED}${BOLD}${NC}"
311     echo ""
312     echo "This script will tear down the gateway and restore normal internet."
313     echo ""
314 fi
315
316 # Cache sudo credentials upfront (default mode ONLY)
317 # Skip in --post-wake: the LaunchAgent-launched watcher has no TTY, so `sudo -v`
318 # would block forever waiting for a password. All post-wake commands use `sudo -n`
319 # which is non-blocking and relies on /etc/sudoers.d/nullexit NOPASSWD entries.
320 SUDO_KEEPER_PID=""
321 if [ "$POST_WAKE" = "false" ] && [ "$AUTO_MODE" = "false" ] && [ -t 0 ]; then
322     echo -e "${YELLOW}${BOLD}Authentication required for network recovery...${NC}"
323     sudo -v
324     (
325         while true; do
326             sudo -n true 2>/dev/null
327             sleep 60
328         done
329     ) &

```



```

330 SUDO_KEEPER_PID=$!
331 trap 'kill $SUDO_KEEPER_PID 2>/dev/null' EXIT
332 fi
333
334 # Resolve physical default interface and gateway upfront
335 PHYSICAL_IFACE=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: (Wi-Fi|Ethernet)/{
    getline; print $2; exit}')
336 [ -z "$PHYSICAL_IFACE" ] && PHYSICAL_IFACE="en0"
337
338 if [ "$POST_WAKE" = "true" ]; then
339     wait_for_dhcp_settle
340 else
341     # Quick resolution in non-post-wake mode (non-blocking)
342     PHYSICAL_GW=$(ipconfig getpacket "$PHYSICAL_IFACE" 2>> output.log | awk -F'[]{}' '/router/{print $2}')
343 fi
344
345 # 1. Disconnect host Tailscale (default) / Refresh exit-node (post-wake)
346
347 if [ "$POST_WAKE" = "true" ]; then
348     step "Refreshing host Tailscale exit-node routing (post-wake)"
349     if command -v tailscale >> output.log 2>&1; then
350         # Re-issue the SAME exit-node up command toggle.sh START uses. This refreshes
351         # the DERP relay mapping and re-asserts the exit-node preference without
352         # dropping the host's mesh connection (which `tailscale down` would).
353         TS_IP=""
354         for src in .gateway_ip; do
355             [ -f "$src" ] || continue
356             TS_IP=$(cat "$src" 2>> output.log | tr -d '\r' | awk 'NR==1{print $1; exit}' || true)
357             [ -n "$TS_IP" ] && break
358         done
359
360         if [ -n "$TS_IP" ]; then
361             step "Re-establishing exit node $TS_IP via 'tailscale set --exit-node'"
362             # `tailscale set` only mutates the named preference (exit-node here).
363             # Crucially it does NOT call --reset, which would re-apply ALL flags and
364             # trigger a sub-second route-table re-evaluation cycle each post-wake.
365             # On already-connected nodes, the lighter `set` keeps the existing tunnel
366             # alive while only changing the exit-node preference.
367             if run_with_timeout 15 tailscale set --exit-node="$TS_IP" \
368                 --exit-node-allow-lan-access=true \
369                 >> output.log 2>&1; then
370                 ok "Exit-node $TS_IP re-asserted via 'tailscale set'"
371             else
372                 warn "tailscale set --exit-node=$TS_IP didn't respond; falling back to mesh-only"
373                 run_with_timeout 10 tailscale set --exit-node= \
374                     >> output.log 2>&1 || true
375             fi
376         else
377             warn ".gateway_ip missing cannot re-assert exit node"
378             run_with_timeout 10 tailscale up --reset --ssh=true --accept-dns=false --exit-node= \
379                 >> output.log 2>&1 || true
380         fi
381     else
382         warn "tailscale CLI not found"
383     fi
384 else
385     step "Disconnecting host Tailscale from mesh"
386     disconnect_tailscale_host "tailscale"
387 fi
388
389 # 2. Reset DNS to default on ALL network services (default) / Re-hijack gateway DNS (post-wake)
390 if [ "$POST_WAKE" = "true" ]; then
391     step "Re-hijacking host DNS to gateway IP (post-wake / post-roam)"
392     TS_IP=""
393     for src in .gateway_ip; do
394         [ -f "$src" ] || continue
395         TS_IP=$(cat "$src" 2>> output.log | tr -d '\r' | awk 'NR==1{print $1; exit}' || true)
396         [ -n "$TS_IP" ] && break
397     done
398
399     if [ -n "$TS_IP" ]; then
400         # Detect the active network service (using helper in common.sh)
401         ACTIVE_SVC=$(get_active_service)
402         ENO_SVC=$(get_en0_service)
403
404         networksetup -setsearchdomains "$ACTIVE_SVC" "ts.net" 2>> output.log || true
405         networksetup -setdnsservers "$ACTIVE_SVC" "$TS_IP" 2>> output.log || true
406         networksetup -setsearchdomains "$ENO_SVC" "ts.net" 2>> output.log || true
407         networksetup -setdnsservers "$ENO_SVC" "$TS_IP" 2>> output.log || true
408
409         HITS=$(networksetup -getdnsservers "$ACTIVE_SVC" 2>> output.log | grep -Fx "$TS_IP" || true)

```

```

410     if [ -n "$HITS" ]; then
411         ok "DNS re-hijacked to $TS_IP on services: $ACTIVE_SVC, $ENO_SVC"
412     else
413         warn "networksetup accepted but DNS read-back didn't include $TS_IP"
414     fi
415 else
416     warn ".gateway_ip missing cannot re-hijack DNS (user must run toggle.sh START)"
417 fi
418 else
419     step "Resetting DNS to default on all network services (empty = DHCP)"
420
421     # Get ALL network services (handles spaces in names, skips disabled ones)
422     SERVICES=$(networksetup -listallnetworkservices 2>> output.log | grep -v "^An asterisk" | grep -v "^$" ||
423         true)
424     if [ -z "$SERVICES" ]; then
425         # Fallback to common names
426         SERVICES="Wi-Fi Ethernet Thunderbolt Ethernet USB 10/100 LAN"
427     fi
428
429     RESET_COUNT=0
430     while IFS= read -r service; do
431         # Strip leading asterisk (disabled services)
432         service_clean=$(echo "$service" | sed 's/^\*//')
433         [ -z "$service_clean" ] && continue
434
435         # Check if this service has a hardware port (skip VPN/service-only entries)
436         if networksetup -listallhardwareports 2>> output.log | grep -A1 "Port: $service_clean$" | grep -q "Device:"
437         || [ "$service_clean" = "Wi-Fi" ]; then
438             (
439                 networksetup -setsearchdomains "$service_clean" "Empty" 2>> output.log
440                 # Set to "empty" so macOS uses DHCP-assigned DNS (not hardcoded 1.1.1.1)
441                 sudo networksetup -setdnsservers "$service_clean" "empty" 2>> output.log
442             ) && RESET_COUNT=$((RESET_COUNT + 1)) || true
443         fi
444     done <<< "$SERVICES"
445
446     ok "DNS reset on $RESET_COUNT service(s)"
447 fi
448
449 # 3. Disable rogue proxy settings (default only gateway uses proxy in non-exit-node path)
450 if [ "$POST_WAKE" = "false" ]; then
451     step "Disabling proxy settings (SOCKS, web, secure web)"
452     for svc in $(networksetup -listallnetworkservices 2>> output.log | grep -v "^An asterisk" | grep -v "^$" ||
453         echo "Wi-Fi"); do
454         svc_clean=$(echo "$svc" | sed 's/^\*//')
455         [ -z "$svc_clean" ] && continue
456         disable_all_proxies "$svc_clean"
457     done
458     ok "Proxy settings cleared"
459 else
460     step "Proxy settings: leaving untouched (post-wake keeps gateway SOCKS wiring alive)"
461 fi
462
463 # 4. Flush macOS DNS cache (always safe and useful in both modes)
464 step "Flushing DNS cache"
465 if command -v dscacheutil >> output.log 2>&1; then
466     flush_dns_cache
467     ok "DNS cache flushed (mDNSResponder HUP)"
468 else
469     warn "dscacheutil not found"
470 fi
471
472 # 5. Clear stale routing table (always safe in both modes)
473 step "Flushing stale routing table entries"
474 # Resolve physical default gateway IP before flushing
475 # We cannot trust `route get default` here because Tailscale may have already hijacked
476 # the default route to utunX. We must read the actual DHCP router assignment from the hardware.
477 # PHYSICAL_IFACE and PHYSICAL_GW resolved upfront
478
479 # Instead of full route -n flush (which destroys macOS loopback/multicast and wedges the OS until reboot),
480 # we cleanly delete the Tailscale overrides and Cloudflare WARP bypass routes in ALL modes.
481 # This ensures tailscaled isn't blocked from reaching its control plane when waking from sleep.
482 sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
483 sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
484 remove_warp_bypass_routes
485 ok "Routing overrides cleared"
486
487 if [ "$POST_WAKE" = "false" ]; then
488     disable_killswitch
489     ok "Routing table flush completed via targeted deletes and firewall disabled"

```

```

488 else
489   ok "Routing overrides cleared (post-wake mode to preserve Colima bridge100)"
490 fi
491
492 # In post-wake mode we don't bounce Wi-Fi, so we MUST manually restore the physical default route
493 if [ "$POST_WAKE" = "true" ] && [ -n "$PHYSICAL_GW" ]; then
494   # Ensure physical gateway is set just in case it was dropped
495   sudo -n route add default "$PHYSICAL_GW" >> output.log 2>&1 || true
496 fi
497
498 # CONDITIONAL BYPASS ROUTE RESTORATION:
499 # If this was a post-wake recovery and the exit node is active, the route flush
500 # just wiped the static bypass routes for the WARP endpoints. We must re-add
501 # them immediately to prevent a routing loop when Tailscale starts sending traffic.
502 if [ "$POST_WAKE" = "true" ]; then
503   if [ -z "${ACTIVE_IF:-}" ]; then
504     ACTIVE_IF=$(route get default 2>> output.log | awk '/interface:/{print $2; exit}')
505     if [ -z "$ACTIVE_IF" ] || [[ "$ACTIVE_IF" =~ ^tun ]] || [[ "$ACTIVE_IF" =~ ^tun ]]; then
506       for i in en0 en1 en2 en3; do
507         if ifconfig "$i" 2>> output.log | grep -q 'status: active'; then
508           ACTIVE_IF="$i"; break
509         fi
510       done
511     fi
512     [ -z "$ACTIVE_IF" ] && ACTIVE_IF="en0"
513   fi
514
515   echo "Re-adding host bypass routes for Cloudflare WARP endpoints..."
516   remove_warp_bypass_routes
517   if [ -n "${PHYSICAL_GW:-}" ]; then
518     add_warp_bypass_routes "$PHYSICAL_GW"
519   else
520     add_warp_bypass_routes "$ACTIVE_IF"
521   fi
522
523 # Also re-apply the default route pointing to the Tailscale interface
524 ts_iface=$(ifconfig 2>> output.log | grep -B4 "inet 100." | grep -E '[a-z0-9]+' | cut -d: -f1 | head -n 1)
525 if [ -n "$ts_iface" ]; then
526   echo "Re-routing default gateway to $ts_iface using 0.0.0.0/1 split..."
527   # DO NOT delete the physical default route! It crashes Colima.
528   # Use the /1 trick to mathematically override the default route.
529   sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
530   sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
531   sudo -n route add -net 0.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
532   sudo -n route add -net 128.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
533   enable_killswitch
534 fi
535 fi
536
537 # 6. Stop gateway Docker containers (default) / Force-recreate warp if unhealthy (post-wake)
538 if [ "$POST_WAKE" = "true" ]; then
539   # 6a. Check for Roaming Network Mismatch (Bridged vs Enterprise Wi-Fi)
540   if [ -f "/tmp/nullexit_colima_mode.txt" ] && [ -f "$SCRIPT_DIR/../../lan_p2p_detected" ]; then
541     current_colima_mode=$(cat /tmp/nullexit_colima_mode.txt)
542     p2p_allowed=$(cat "$SCRIPT_DIR/../../lan_p2p_detected")
543     required_colima_mode="bridged"
544     [ "$p2p_allowed" == "false" ] && required_colima_mode="shared"
545
546     if [ "$current_colima_mode" != "$required_colima_mode" ]; then
547       warn "Network roaming mismatch detected!"
548       warn "Colima is running in '$current_colima_mode' mode but new network requires '$required_colima_mode'."
549       warn "Executing full gateway restart to protect against MAC-spoofing deauthentication..."
550       echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Roam] Mismatch ($current_colima_mode vs $required_colima_mode).
551       Triggering toggle.sh --restart." >> output.log
552       nohup bash "$SCRIPT_DIR/../../toggle.sh" --restart >/dev/null 2>&1 &
553       exit 0
554     fi
555   fi
556
557 step "Checking Colima VM state"
558 colima_status=$(colima status 2>&1)
559 if echo "$colima_status" | grep -qi "running"; then
560   ok "Colima VM is running"
561   echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Colima status check passed." >> output.log
562 else
563   warn "Colima VM is NOT running or unresponsive"
564   echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Colima is unhealthy or dead! Output: $colima_status"
565   >> output.log
566   echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Attempting to restart Colima..." >> output.log
567   colima restart >> output.log 2>&1 || warn "Failed to restart Colima"
568 fi

```

```

567 step "Checking warp container health (gluetun UDP tunnel)"
568 echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Inspecting warp container state..." >> output.log
569 # The warp container is the most failure-prone link in the chain at wake/roam:
570 # its UDP NAT binding to Cloudflare's edge (162.159.192.1:2408) silently dies
571 # during a Wi-Fi roam. We verify the tunnel is actually passing traffic via curl.
572 WARP_STATE=$(docker inspect --format '{{.State.Health.Status}}' warp 2>> output.log || echo "missing")
573 echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container status: $WARP_STATE" >> output.log
574
575 if [ "$WARP_STATE" = "missing" ]; then
576     warn "warp container is missing leaving gateway containers alone (full relaunch requires \"$SCRIPT_DIR/
577     toggle.sh\")"
578     echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container missing from Docker engine. Requires
579     full toggle.sh launch." >> output.log
580 else
581     tmp_err=$(mktemp)
582     if docker compose exec -T warp wget -qO- --timeout=3 https://www.cloudflare.com/cdn-cgi/trace 2>"$tmp_err"
583     | grep -q "warp=on"; then
584         ok "warp container is healthy and actively tunneling traffic (no recreate needed)"
585         echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container passed live traffic healthcheck (
586         Cloudflare trace successful)." >> output.log
587     else
588         warn "warp container tunnel is dead (Docker status: $WARP_STATE) force-recreating all gateway containers
589         to restore network namespace"
590         echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container is DEAD or STUCK. Forcing recreation
591         of gateway stack..." >> output.log
592         if [ -s "$tmp_err" ]; then
593             err_msg=$(cat "$tmp_err" | tr -d '\r')
594             echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Healthcheck failure details: $err_msg" >> output.
595             log
596             fi
597             docker compose up -d --force-recreate >> output.log 2>&1 || warn "force-recreate failed"
598
599             NEW_STATE=$(docker inspect --format '{{.State.Health.Status}}' warp 2>> output.log || echo "missing")
600             ok "warp container health after recreate: $NEW_STATE"
601             echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container health after recreation: $NEW_STATE"
602             >> output.log
603             fi
604             rm -f "$tmp_err"
605         fi
606     else
607         step "Stopping gateway Docker containers"
608         if command -v docker >> output.log 2>&1 && docker info >> output.log 2>&1; then
609             if [ -f "docker-compose.yml" ]; then
610                 docker compose down -t 5 2>> output.log && ok "Containers stopped" || warn "No containers were running"
611             else
612                 warn "docker-compose.yml not found in current directory"
613             fi
614         else
615             warn "Docker not available skipping"
616         fi
617     fi
618
619 # 6b. Stopping sleep prevention (caffeinate) default only
620 if [ "$POST_WAKE" = "false" ]; then
621     step "Stopping sleep prevention"
622     stop_pidfile_daemon "$PID_CAFFEINATE" "system sleep prevention"
623 else
624     step "Sleep prevention: leaving untouched (caffeinate already re-acquired by parent shell)"
625 fi
626
627 # 6c. Stopping DNS Watcher default only
628 if [ "$POST_WAKE" = "false" ]; then
629     step "Stopping DNS Watcher"
630     stop_pidfile_daemon "$PID_DNS_WATCHER" "background DNS Watcher"
631 else
632     step "DNS Watcher: leaving untouched (it keeps re-hijacking DNS on every roam)"
633 fi
634
635 # 7. Power-cycle Wi-Fi default only
636 if [ "$POST_WAKE" = "false" ]; then
637     step "Power-cycling Wi-Fi"
638     bounce_wifi_interfaces
639 else
640     step "Wi-Fi: leaving untouched (post-wake keeps the current Wi-Fi association)"
641 fi
642
643 # 8. Verify (default checks direct internet; post-wake verifies the gateway)
644 if [ "$POST_WAKE" = "true" ]; then
645     step "Verifying gateway is healthy after post-wake refresh"
646

```

```

640 # Wait a moment for tailscale + warp healthcheck to settle
641 sleep 3
642
643 GATEWAY_OK=false
644 if command -v dig >> output.log 2>&1; then
645     # Source procedural ports if available
646     if [ -f "/tmp/nullexit-ports.env" ]; then
647         source "/tmp/nullexit-ports.env"
648     fi
649     DNS_PORT=${DNS_PROXY_PORT:-5354}
650
651     # AdGuard is exposed at localhost:${DNS_PORT}. If this resolves, the warp tunnel
652     # is functioning properly because AdGuard routes upstream to warp.
653     if dig +tcp @127.0.0.1 -p "$DNS_PORT" google.com +short +timeout=5 >> output.log 2>&1; then
654         ok "Gateway DNS (AdGuard via localhost:${DNS_PORT}) works"
655         GATEWAY_OK=true
656     else
657         warn "Gateway DNS not responding yet"
658     fi
659 fi
660
661 # Direct host gateway check via Tailscale ping (relayed pong counts as success)
662 if command -v tailscale >> output.log 2>&1; then
663     TS_IP=""
664     [ -f .gateway_ip ] && TS_IP=$(read_adguard_ip || true)
665     if [ -n "$TS_IP" ] && tailscale ping --until-direct=false -c 1 --timeout 4s "$TS_IP" 2>> output.log | grep
666         -q pong; then
667         ok "Gateway $TS_IP reachable via Tailscale"
668         GATEWAY_OK=true
669     else
670         warn "Tailscale ping to gateway did not pong exit-node may still be re-establishing"
671     fi
672 fi
673
674 if [ "$GATEWAY_OK" = "true" ]; then
675     echo -e " ${GREEN}${BOLD} Gateway is healthy after post-wake refresh.${NC}"
676 else
677     echo -e " ${YELLOW}${BOLD} Gateway recovery is in-progress.${NC}"
678     echo " The route may need another 30s before queries succeed."
679     echo " If it stays broken for >60s, run: bash \"${SCRIPT_DIR}/toggle.sh\" "
680 fi
681
682 else
683     verify_internet_connectivity
684 fi
685
686 # Summary
687 echo ""
688 echo -e "${BOLD}${NC}"
689 if [ "$POST_WAKE" = "true" ]; then
690     echo -e "${BOLD}POST-WAKE SUMMARY${NC}"
691 else
692     echo -e "${BOLD}RECOVERY SUMMARY${NC}"
693 fi
694 echo -e "${BOLD}${NC}"
695
696 # Show final DNS state
697 FINAL_DNS=$(networksetup -getdnsservers "Wi-Fi" 2>/dev/null || echo "N/A")
698 echo " DNS (Wi-Fi): $FINAL_DNS"
699
700 FINAL_DNS2=$(networksetup -getdnsservers "Ethernet" 2>/dev/null || true)
701 if [[ -z "$FINAL_DNS2" || "$FINAL_DNS2" == *"not a recognized"* || "$FINAL_DNS2" == *"Error"* ]]; then
702     FINAL_DNS2="N/A (Not configured)"
703 fi
704 echo " DNS (Ethernet): $FINAL_DNS2"
705
706 echo ""
707
708 if [ "$POST_WAKE" = "false" ]; then
709     if [ "$INTERNET_OK" = "true" ]; then
710         echo -e " ${GREEN}${BOLD} Internet is working.${NC}"
711     else
712         echo -e " ${YELLOW}${BOLD} Could not verify internet connectivity.${NC}"
713         echo " This might mean:"
714         echo " - Your Wi-Fi is disconnected from the router"
715         echo " - Tailscale is still interfering (try restarting tailscaled:"
716         echo "     brew services restart tailscale)"
717         echo " - You need to re-connect to Wi-Fi in the menu bar"
718         echo " - Try toggling Wi-Fi off/on in the menu bar"
719         echo ""
720         echo " Or try manually:"
721         echo "     networksetup -setdnsservers Wi-Fi empty"

```

```

720     echo "          tailscale down"
721     echo "          sudo route delete -net 0.0.0.0/1"
722     echo "          sudo route delete -net 128.0.0.0/1"
723     echo "          brew services restart tailscale"
724 fi
725 fi
726
727 # Reset sharing services to prevent AirDrop freezes after interface changes
728 # This is the SAME step in BOTH modes (post-wake inherits the previous fix from 10.27).
729 echo -e "    Resetting macOS sharing services (AirDrop/AirPlay)..."
730 reset_sharing_services
731
732 echo ""
733 if [ "${POST_WAKE}" = "true" ]; then
734     echo -e "${YELLOW}${BOLD}Post-wake refresh complete.${NC}"
735     # Remove the WARP Watcher inhibit marker now that the gateway is fully healthy.
736     # The watcher will resume normal liveness monitoring on its next 5s poll.
737     rm -f "$WARP_INHIBIT_FILE"
738     echo "[$(date -u +%FT%TZ)] POST-WAKE complete  WARP Watcher inhibit released." >> output.log
739 else
740     echo -e "${GREEN}${BOLD}Recovery complete.${NC}"
741 fi
742 echo ""

```

8 setup.sh

```

1  #!/bin/bash
2  # setup.sh nullexit one-shot setup script (macOS)
3  # Configures Cloudflare WARP + Tailscale + AdGuard Home from scratch.
4  set -euo pipefail
5
6  # Source common formatting and helper functions
7  source "$(dirname "$0")/scripts/common.sh"
8
9  # Run common installation logic
10 source "$(dirname "$0")/scripts/setup-common.sh"
11
12 # Compile macOS Shortcuts
13 echo -e "\n${BOLD}Compiling macOS Desktop Shortcuts...${NC}"
14 if command -v osacompile &> /dev/null; then
15     osacompile -o "Toggle Gateway.app" "Toggle-Gateway.applescript" >/dev/null 2>&1
16     osacompile -o "Recover Gateway.app" "Recover-Gateway.applescript" >/dev/null 2>&1
17     echo "    Created 'Toggle Gateway.app' and 'Recover Gateway.app'"
18 fi
19
20 # Done
21 echo ""
22 echo -e "${GREEN}${BOLD}${NC}"
23 echo -e "${GREEN}${BOLD}          Setup complete!          ${NC}"
24 echo -e "${GREEN}${BOLD}${NC}"
25 echo ""
26 echo -e "${BOLD}One manual step remaining  approve the exit node:${NC}"
27 echo "    https://login.tailscale.com/admin/machines"
28 echo "    Find your gateway  '...'  'Edit route settings'  enable Exit Node"
29 echo ""
30
31 echo -e "${YELLOW}${BOLD}Sudoers / Background Execution Requirements:${NC}"
32 echo "    To allow silent, non-interactive execution of the firewall and network routes (especially during sleep/
wake recovery), you must configure passwordless sudo."
33 echo "    Run this exact command in your terminal (grants are scoped to least privilege):"
34 echo "    echo \"\$USER ALL=(root) NOPASSWD: /sbin/pfctl, /usr/sbin/networksetup, /sbin/route, /sbin/ifconfig,
/usr/bin/dsccacheutil -flushcache, /usr/bin/killall -HUP mDNSResponder, /usr/bin/killall sharingd rapportd,
/usr/bin/pkill -9 tailscaled, /usr/bin/pkill -f dns-proxy.py, /bin/kill, /usr/bin/true, /opt/homebrew/bin/
brew services * tailscale, /opt/homebrew/bin/brew uninstall tailscale, /usr/local/bin/brew services *
tailscale, /usr/local/bin/brew uninstall tailscale, /usr/bin/python3 -I \${PWD}/scripts/dns-proxy.py, /opt/
homebrew/bin/python3 -I \${PWD}/scripts/dns-proxy.py, /usr/local/bin/python3 -I \${PWD}/scripts/dns-proxy.py\"
| sudo tee /etc/sudoers.d/nullexit >/dev/null && sudo visudo -cf /etc/sudoers.d/nullexit"
35 echo ""
36
37 if [[ -n "${TS_IP:-}" ]]; then
38     echo -e "${BOLD}AdGuard Home dashboard:${NC}  http://${TS_IP}:3000  (username: admin)"
39     echo ""
40 fi
41
42 echo -e "${BOLD}To toggle the gateway on/off:${NC}"
43 echo "    macOS:  double-click the 'Toggle Gateway' app icon!"

```



```

44 echo "   Windows: double-click Toggle-Gateway.bat"
45 echo ""
46
47 if [[ "$OSTYPE" == "darwin"* ]]; then
48     # LuLu localhost filtering check
49     if [ -d "/Applications/LuLu.app" ] || systemextensionsctl list 2>/dev/null | grep -q "com.objective-see.
      lulu"; then
50         echo -e "${YELLOW}${BOLD}LuLu Firewall Detected:${NC}"
51         echo "   LuLu's localhost (loopback) filtering is OFF by default. To properly protect the"
52         echo "   local nullexit proxy ports from malicious local processes, you must manually enable"
53         echo "   'Allow Loopback' (or block it selectively) in LuLu's Preferences -> Rules."
54         echo ""
55     fi
56
57     echo -e "${YELLOW}${BOLD>Note on macOS Tailscale Sandboxing:${NC}"
58     echo "   The Tailscale GUI app (tailscale-app) installed on macOS is sandboxed."
59     echo "   While you cannot run a Tailscale SSH server on this Mac host, you can"
60     echo "   still SSH outbound to other devices on the mesh using their MagicDNS"
61     echo "   addresses directly (e.g. ssh user@device.tailnet-name.ts.net)."
62     echo ""
63 fi

```

9 scripts/setup-linux.sh

```

1  #!/bin/bash
2  # scripts/setup-linux.sh  nullexit one-shot setup script (Linux)
3  # Configures Cloudflare WARP + Tailscale + AdGuard Home from scratch.
4  set -euo pipefail
5
6  # Source common formatting and helper functions
7  source "$(dirname "$0")/common.sh"
8
9  # Run common installation logic
10 source "$(dirname "$0")/setup-common.sh"
11
12 # Compile Linux Desktop Shortcuts
13 echo -e "\n${BOLD}Creating Linux Desktop Shortcuts...${NC}"
14 DIR="$(pwd)"
15 cat <<EOF > "Toggle Gateway.desktop"
16 [Desktop Entry]
17 Version=1.0
18 Name=Toggle Gateway
19 Comment=Start or Stop Nullexit Gateway
20 Exec=bash -c "cd '$DIR' && ./toggle.sh; read -p 'Press Enter to close...' dummy"
21 Icon=utilities-terminal
22 Terminal=true
23 Type=Application
24 Categories=Network;Security;
25 EOF
26
27 cat <<EOF > "Recover Gateway.desktop"
28 [Desktop Entry]
29 Version=1.0
30 Name=Recover Gateway
31 Comment=Emergency DNS and Network Recovery
32 Exec=bash -c "cd '$DIR' && ./recover.sh; read -p 'Press Enter to close...' dummy"
33 Icon=utilities-terminal
34 Terminal=true
35 Type=Application
36 Categories=Network;Security;
37 EOF
38 echo "   Created 'Toggle Gateway.desktop' and 'Recover Gateway.desktop'"
39
40 # Done
41 echo ""
42 echo -e "${GREEN}${BOLD}${NC}"
43 echo -e "${GREEN}${BOLD}                Setup complete!                ${NC}"
44 echo -e "${GREEN}${BOLD}${NC}"
45 echo ""
46 echo -e "${BOLD}One manual step remaining  approve the exit node:${NC}"
47 echo "   https://login.tailscale.com/admin/machines"
48 echo "   Find your gateway '...' 'Edit route settings'  enable Exit Node"
49 echo ""
50
51 if [[ -n "${TS_IP:-}" ]]; then
52     echo -e "${BOLD}AdGuard Home dashboard:${NC}  http://${TS_IP}:3000  (username: admin)"

```

```

53     echo ""
54 fi
55
56 echo -e "${BOLD}To toggle the gateway on/off:${NC}"
57 echo "  Linux: double-click the 'Toggle Gateway' desktop icon!"
58 echo "  (or run ./toggle.sh in terminal)"
59 echo ""

```

10 docker-compose.yml

```

1 services:
2   warp:
3     image: qmcgaw/glutun:v3.41.1
4     container_name: warp
5     hostname: nullexit-gateway
6     cap_add:
7       - NET_ADMIN
8     devices:
9       - /dev/net/tun:/dev/net/tun
10    ports:
11      - "127.0.0.1:${DNS_PROXY_PORT:-5354}:5335/udp" # AdGuard DNS: host binds 5354 (the mapped DNS-proxy
12        port); 5335 is AdGuard's in-container port
13      - "127.0.0.1:${DNS_PROXY_PORT:-5354}:5335/tcp" # AdGuard DNS (TCP)
14      - "41642:41642/udp" # Tailscale WireGuard (direct connection to host)
15      - "127.0.0.1:${SOCKS_PROXY_PORT:-1080}:1080/tcp" # SOCKS5 proxy (routed through WARP tunnel)
16      - "127.0.0.1:${TOR SOCKS_PORT:-9050}:9050/tcp" # Tor SOCKS5 proxy (procedurally generated)
17      - "127.0.0.1:${TOR_CONTROL_PORT:-9051}:9051/tcp" # Tor Control port (procedurally generated)
18    environment:
19      - TZ=America/New_York
20      - VPN_SERVICE_PROVIDER=custom
21      - VPN_TYPE=wireguard
22      - WIREGUARD_PRIVATE_KEY=${WIREGUARD_PRIVATE_KEY}
23      - WIREGUARD_PUBLIC_KEY=${WIREGUARD_PUBLIC_KEY}
24      - WIREGUARD_ADDRESSES=${WIREGUARD_ADDRESSES}
25      - WIREGUARD_ENDPOINT_IP=${WARP_ENDPOINT_1:-162.159.192.1}
26      - WIREGUARD_ENDPOINT_PORT=2408
27      # Set to 25s (WireGuard default) to beat standard 30s hardware NAT/firewall UDP timeouts
28      - WIREGUARD_PERSISTENT_KEEPALIVE_INTERVAL=25s
29      # MTU Fix (Critical): Prevent packet fragmentation from double-encryption
30      - WIREGUARD_MTU=1280
31      # Give WARP more time to establish the connection before internal restart (default is 6s which is too
32      short)
33      - HEALTH_VPN_DURATION_INITIAL=30s
34      # Use plaintext DNS resolvers instead of DNS-over-TLS (DoT) to prevent startup failures on networks where
35      port 853 is blocked
36      - DNS_UPSTREAM_RESOLVER_TYPE=plain
37      - DNS_UPSTREAM_PLAIN_ADDRESSES=1.1.1.1:53,8.8.8.8:53
38      # Allow local network traffic to bypass WARP so Tailscale can establish fast, direct peer-to-peer
39      connections (no DERP relays) on the LAN
40      - FIREWALL_OUTBOUND_SUBNETS=192.168.0.0/16,172.16.0.0/12,10.0.0.0/8
41      # Built-in DNS blocking for ads, malicious domains, and tracking
42      # NOTE: This is the massive "Hagezi-style" built-in blocklist.
43      # You can turn these to 'on' for maximum security if you have spare RAM
44      # or are running this on a cloud exit node with adequate autonomous resources (requires approx 1-2GB of
45      RAM).
46      # Disabled intentionally: AdGuard Home provides the DNS filtering layer instead.
47      # If AdGuard fails, there is no fallback filtering, but the healthcheck
48      # dependency chain ensures Tailscale won't come up if AdGuard is down.
49      - BLOCK_MALICIOUS=off
50      - BLOCK_SURVEILLANCE=off
51      - BLOCK_ADS=off
52    dns:
53      - 1.1.1.1
54    sysctls:
55      - net.ipv4.ip_forward=1
56    volumes:
57      - ./post-rules.txt:/iptables/post-rules.txt:ro
58    restart: unless-stopped
59    healthcheck:
60      test: ["CMD-SHELL", "/glutun-entrypoint healthcheck"]
61      interval: 2s
62      timeout: 5s
63      retries: 15
64      start-period: 60s
65    logging:
66      driver: "json-file"

```

```

62     options:
63         max-size: "10m"
64         max-file: "3"
65
66 tailscale:
67     image: tailscale/tailscale:v1.98.4
68     container_name: tailscale
69     network_mode: service:warp
70     depends_on:
71         warp:
72             condition: service_healthy
73     adguardhome:
74         condition: service_healthy
75     cap_add:
76         - NET_ADMIN
77         - NET_RAW
78         - SYS_MODULE
79     sysctls:
80         - net.ipv4.ip_forward=1
81         - net.ipv6.conf.all.forwarding=1
82     environment:
83         - TZ=America/New_York
84         - TS_EXTRA_ARGS=--advertise-exit-node
85         - PORT=41642
86         - TS_USERSPACE=false
87         - TS_AUTHKEY=${TS_AUTHKEY}
88         - TS_STATE_DIR=/var/lib/tailscale
89         # (Note: For headless/cloud deployments, you can bypass this footgun by
90         # using Tailscale Ephemeral Keys which auto-clean themselves).
91         - TS_AUTH_ONCE=true
92         # MTU Optimization: Force Tailscale packets to be smaller than WARP's 1280 MTU
93         # to prevent WireGuard-in-WireGuard fragmentation, completely eliminating bufferbloat
94         - TS_DEBUG_MTU=1200
95         # Enable Tailscale's built-in SOCKS5 proxy to route out of the WARP tunnel
96         - TS_SOCKS5_SERVER=:1080
97     devices:
98         - /dev/net/tun:/dev/net/tun
99     volumes:
100         - tailscale_state:/var/lib/tailscale
101     restart: unless-stopped
102     logging:
103         driver: "json-file"
104         options:
105             max-size: "10m"
106             max-file: "3"
107
108 routing-fix:
109     image: nullexit-routing-fix:v1.0.0
110     build:
111         context: ./scripts
112         dockerfile: Dockerfile.routing-fix
113     container_name: routing-fix
114     network_mode: service:warp
115     # NOTE: Deliberately NOT sharing tailscale's PID namespace the shared
116     # namespace caused routing-fix to be killed whenever tailscale restarted
117     # (e.g. during gluetun health-check flaps), which dropped all routing
118     # table changes and left traffic leaking through eth0 instead of tun0.
119     cap_add:
120         - NET_ADMIN
121         - SYSLOG
122     depends_on:
123         warp:
124             condition: service_healthy
125         rule-compiler:
126             condition: service_completed_successfully
127     environment:
128         - TZ=America/New_York
129         - WARP_ENDPOINT=${WARP_ENDPOINT_1:-162.159.192.1}
130         - BLOCKED_COUNTRIES=${BLOCKED_COUNTRIES}
131         - TAILSCALE_ALLOW_LAN_P2P=${TAILSCALE_ALLOW_LAN_P2P:-false}
132     volumes:
133         - ./scripts/routing-fix.sh:/routing-fix.sh:ro
134         # SECURITY: ./adguard/work/ is bind-mounted into this container.
135         # Never write to it from the host or another container. The
136         # single allowed writer is `rule-compiler` container manual
137         # edits corrupt the AdGuard cache + BoltDB session store.
138         - ./adguard/work/userfilters:/userfilters:ro
139         - ./adguard/work:/adguard_work:ro
140         - ./app
141     restart: unless-stopped
142     stop_grace_period: 1s

```

```

143     logging:
144         driver: "json-file"
145         options:
146             max-size: "1m"
147             max-file: "1"
148
149     adguardhome:
150         # WARNING: AdGuard is pre-configured with hardcoded credentials (admin/nullexit).
151         # This is fine for a local Mac running behind a NAT, but if you are deploying
152         # this on a public cloud VPS with port 80/3000 exposed, change this immediately!
153         image: adguard/adguardhome:v0.107.77
154         container_name: adguardhome
155         network_mode: service:warp
156         depends_on:
157             warp:
158                 condition: service_healthy
159             rule-compiler:
160                 condition: service_completed_successfully
161         environment:
162             - TZ=America/New_York
163         healthcheck:
164             test: ["CMD-SHELL", "nc -z 127.0.0.1 5335 || exit 1"]
165             interval: 2s
166             timeout: 5s
167             retries: 10
168         volumes:
169             # SECURITY: ./adguard/work/ is bind-mounted into this container.
170             # Never write to it from the host or another container. The
171             # single allowed writer is `rule-compiler` container manual
172             # edits corrupt the AdGuard cache + BoltDB session store.
173             - ./adguard/work:/opt/adguardhome/work
174             - ./adguard/conf:/opt/adguardhome/conf
175         restart: unless-stopped
176         stop_grace_period: 2s
177         logging:
178             driver: "json-file"
179             options:
180                 max-size: "10m"
181                 max-file: "3"
182
183     rule-compiler:
184         image: nullexit-rule-compiler:v1.0.0
185         build:
186             context: ./scripts
187             dockerfile: Dockerfile.rule-compiler
188         container_name: rule-compiler
189         volumes:
190             - ./app
191
192     tor:
193         build: ./docker/tor
194         image: nullexit-tor:v1.0.0
195         container_name: tor
196         network_mode: service:warp
197         depends_on:
198             warp:
199                 condition: service_healthy
200         environment:
201             - TZ=America/New_York
202             - TOR_USE_BRIDGES=${TOR_USE_BRIDGES:-false}
203             - TOR_BRIDGE_LINES_FILE=${TOR_BRIDGE_LINES_FILE:-./tor-bridges.txt}
204             - TOR_PASSWORD=${ADGUARD_PASSWORD:-nullexit}
205             - TOR_EXCLUDE_EXIT_NODES=${TOR_EXCLUDE_EXIT_NODES:-{us}}
206         working_dir: /nullexit
207         volumes:
208             - ./:/nullexit:ro
209             - ./output.log:/output.log:rw
210         restart: unless-stopped
211         logging:
212             driver: "json-file"
213             options:
214                 max-size: "10m"
215                 max-file: "3"
216
217     volumes:
218         tailscale_state:
219
220     networks:
221         default:
222             ipam:
223                 config:

```

11 scripts/common.sh

```

1  #!/bin/bash
2  # common.sh shared bash utilities for nullexit scripts
3
4  # Formatting
5  RED='\033[0;31m'
6  GREEN='\033[0;32m'
7  YELLOW='\033[1;33m'
8  BLUE='\033[0;34m'
9  BOLD='\033[1m'
10 NC='\033[0m'
11
12 # Constants & Environment
13 export MARKER_FILE="/tmp/nullexit-gateway-active.marker"
14 export PID_CAFFEINATE="/tmp/nullexit-cafeinate.pid"
15 export PID_DNS_WATCHER="/tmp/nullexit-dns-watcher.pid"
16 export DEFAULT_WARP_ENDPOINT_1="162.159.192.1"
17 export DEFAULT_WARP_ENDPOINT_2="162.159.193.1"
18
19 setup_standard_path() {
20     export PATH="/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH"
21 }
22
23 step() { echo -e "\n[ $(date '+%Y-%m-%d %H:%M:%S') ] ${BLUE}${BOLD} ${*}${NC}"; }
24 ok() { echo -e "[ $(date '+%Y-%m-%d %H:%M:%S') ] ${GREEN} ${*}${NC}"; }
25 warn() { echo -e "[ $(date '+%Y-%m-%d %H:%M:%S') ] ${YELLOW} ${*}${NC}"; }
26 fail() { echo -e "[ $(date '+%Y-%m-%d %H:%M:%S') ] ${RED} ${*}${NC}"; }
27 die() { echo -e "\n[ $(date '+%Y-%m-%d %H:%M:%S') ] ${RED} ${*}${NC}\n"; exit 1; }
28
29 # Helper Functions
30
31 # Append a timestamped line to output.log (best-effort; never fails the caller).
32 # Used to record lifecycle breadcrumbs (EXEC/EXIT markers, phase changes) so a
33 # START/STOP that is killed or window-closed still leaves a retraceable trail.
34 log_line() {
35     echo "[ $(date '+%Y-%m-%d %H:%M:%S') ] ${*}" >> "${LOG_FILE:-output.log}" 2>/dev/null || true
36 }
37
38 # Run a lifecycle command with FULL observability: log the command, stream its
39 # combined stdout+stderr to the terminal AND append it to output.log via tee,
40 # then log the exit code explicitly. Returns the command's real exit code (not
41 # tee's). This closes the blind spot where `docker compose up` / `colima start`
42 # printed only to the terminal so when they fail (or the run is interrupted),
43 # output.log shows exactly what the last command was and whether it succeeded.
44 #
45 # Usage: run_logged docker compose up -d --build
46 run_logged() {
47     local logf="${LOG_FILE:-output.log}"
48     log_line "EXEC: ${*}"
49     # PIPESTATUS[0] is the command's exit code; tee (PIPESTATUS[1]) is ignored.
50     "$@" 2>&1 | tee -a "$logf"
51     local rc=${PIPESTATUS[0]}
52     log_line "EXIT $rc: ${*}"
53     return "$rc"
54 }
55
56 # Pure-bash timeout (no dependency on GNU coreutils' timeout)
57 # Runs a command with a safety cutoff so a wedged daemon can't hang the script.
58 run_with_timeout() {
59     local timeout_sec="$1"
60     shift
61     if [ $# -eq 0 ]; then return 1; fi
62
63     "$@" &
64     local cmd_pid=$!
65     CURRENT_BG_PID="$cmd_pid"
66
67     (
68         sleep "$timeout_sec"
69         if kill -0 "$cmd_pid" 2>> output.log; then
70             echo -e "\n[Timeout] Command '${*}' exceeded $timeout_sec seconds. Terminating..." >&2
71             kill -15 "$cmd_pid" 2>> output.log || true
72             sleep 2

```

```

73     if kill -0 "$cmd_pid" 2>> output.log; then
74         kill -9 "$cmd_pid" 2>> output.log || true
75     fi
76 fi
77 ) &
78 local watcher_pid=$!
79
80 # Disable set -e temporarily to capture exit status of wait
81 set +e
82 wait "$cmd_pid" 2>> output.log
83 local exit_status=$?
84 set -e
85
86 # Kill the watcher since the command finished
87 kill "$watcher_pid" 2>> output.log && wait "$watcher_pid" 2>> output.log || true
88
89 CURRENT_BG_PID=""
90 return "$exit_status"
91 }
92
93 # Start Colima and return the moment Docker is actually reachable, instead of
94 # blocking on colima 0.10.3's post-provision hang. Observed on macOS/vz: the VM
95 # reaches READY and creates the `colima` docker context in ~10-15s, then
96 # `colima start` frequently fails to return for ~110s (until a watchdog kills it)
97 # even though Docker is fully usable the entire time. We poll the colima docker
98 # context and proceed as soon as it answers, then reap the (usually hung) starter.
99 # The VM keeps running and `colima status` stays accurate. $@ = colima start args.
100 # Returns 0 once Docker responds, 1 if it never does within the cap.
101 colima_start_until_ready() {
102     local max_wait=90 waited=0 ready=false start_pid
103     echo "[$(date '+%Y-%m-%d %H:%M:%S')] EXEC(bg): colima start $*" >> output.log
104     colima start "$@" >> output.log 2>&1 &
105     start_pid=$!
106     while [ "$waited" -lt "$max_wait" ]; do
107         if run_with_timeout 5 docker --context colima info >/dev/null 2>&1; then
108             ready=true
109             break
110         fi
111         # If colima start exited on its own (genuine finish or hard failure), stop waiting.
112         if ! kill -0 "$start_pid" 2>/dev/null; then
113             run_with_timeout 5 docker --context colima info >/dev/null 2>&1 && ready=true
114             break
115         fi
116         sleep 3
117         waited=$((waited + 3))
118     done
119     # Reap the starter (harmless if it already exited); this does NOT stop the VM.
120     kill "$start_pid" 2>/dev/null || true
121     wait "$start_pid" 2>/dev/null || true
122     if [ "$ready" = "true" ]; then
123         docker context use colima >/dev/null 2>&1 || true
124         echo "[$(date '+%Y-%m-%d %H:%M:%S')] Colima Docker ready after ${waited}s (starter reaped)." >> output.log
125         return 0
126     fi
127     echo "[$(date '+%Y-%m-%d %H:%M:%S')] Colima Docker NOT ready after ${max_wait}s." >> output.log
128     return 1
129 }
130
131 # Check if the gateway is active (either containers are running, or host DNS is hijacked)
132 is_gateway_active() {
133     # 1. Check if containers are running (suppress stderr to avoid errors if docker is down)
134     if run_with_timeout 15 docker compose ps --status running 2>/dev/null | grep -q 'warp'; then
135         echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (warp container is running)" >> "$LOG_FILE"
136         return 0
137     fi
138
139     # 2. Check if host DNS was hijacked (not 1.1.1.1 and not default/empty)
140     local current_dns=""
141     local active_svc=""
142     if [[ "$OSTYPE" == "darwin"* ]]; then
143         active_svc=$(get_active_service)
144         current_dns=$(networksetup -getdnsservers "$active_svc" 2>> output.log || true)
145     else
146         current_dns=$(resolvectl dns 2>/dev/null | awk '/Global|Link/ {for(i=4;i<NF;i++) print $i}' | head -n1)
147     fi
148
149     if [[ -n "$current_dns" && "$current_dns" != "1.1.1.1" && ! "$current_dns" =~ "There aren't any DNS Servers"
150         ]]; then
151         echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (DNS was hijacked: $current_dns)" >> "
152         $LOG_FILE"
153         return 0
154     fi
155 }

```



```

152 fi
153
154 # 3. Check if SOCKS proxy is enabled (macOS only)
155 if [[ "$OSTYPE" == "darwin"* ]]; then
156     local socks_proxy
157     socks_proxy=$(networksetup -getsocksfirewallproxy "$active_svc" 2>> output.log || true)
158     if echo "$socks_proxy" | grep -q "Enabled: Yes"; then
159         echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (SOCKS proxy enabled)" >> "$LOG_FILE"
160         return 0
161     fi
162 fi
163
164 echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: FALSE (Containers down, DNS clean, SOCKS disabled)"
165 >> "$LOG_FILE"
166 return 1
167 }
168
169 # Decide whether the gateway SHOULD be running, from persisted *intent* rather
170 # than a live health probe. This is the correct signal for toggle.sh's
171 # STOP-vs-START decision: a disable should tear down whatever the last START
172 # left behind it should NOT depend on whether Docker/Colima happens to be
173 # reachable right now.
174 #
175 # Why this exists: using is_gateway_active() for the decision means every toggle
176 # (including *disable*) runs `docker compose ps` first. When Colima is down the
177 # socket is absent, that call blocks for the full run_with_timeout window, and
178 # under `set -e` + the ERR trap a non-zero result at the `if then else fi`
179 # boundary could abort the script BEFORE the START body runs so the very path
180 # that would boot Colima never executed (observed as `EXIT: code=1 phase=start`).
181 #
182 # Signal source: MARKER_FILE is written at the end of a successful START
183 # (write_gateway_active_marker) and cleared at the top of STOP and in
184 # cleanup_handler. Reading it is instant and cannot hang or abort.
185 #
186 # ECHOES "running" or "stopped" to stdout (and ALWAYS returns 0). Callers read
187 # the string: [ "$(gateway_state)" = "running" ] &&
188 #
189 # CRITICAL why this echoes instead of using a 0/1 return code: on macOS
190 # /bin/bash 3.2, with `set -e` + an `ERR` trap + `set -x` (DEBUG_TRACE=true) all
191 # active, a function that does `return 1` triggers a SPURIOUS `set -e` abort in
192 # the caller even when the call is guarded (`if f; then`, or even `f || rc=$?`).
193 # The `return N` from the function, traced by `set -x`, trips the ERR trap. This
194 # is a genuine bash-3.2 interaction (see devref 15.11.8). Echoing a result and
195 # always returning 0 sidesteps `return`/`set -e`/`set -x` entirely the function
196 # can never contribute a non-zero status, so no caller can be aborted by it.
197 gateway_state() {
198     # Primary: persisted intent. Marker present => a START completed and no STOP
199     # has cleared it yet.
200     #
201     # NOTE: toggle.sh runs a "defensive stale-marker clear" near the top of every
202     # invocation (before this decision) to stop the wake-watcher firing against a
203     # crashed gateway. That would wipe the marker before we read it, so toggle.sh
204     # captures the marker's existence into GATEWAY_MARKER_AT_STARTUP *before* that
205     # clear and we honor it here. If the variable is unset (other callers, or the
206     # --restart pre-check which runs before the defensive clear), fall back to the
207     # live marker file.
208     if [ -n "${GATEWAY_MARKER_AT_STARTUP:-}" ]; then
209         if [ "$GATEWAY_MARKER_AT_STARTUP" = "yes" ]; then echo "running"; return 0; fi
210     elif [ -f "$MARKER_FILE" ]; then
211         echo "running"; return 0
212     fi
213
214     # Marker absent. Normally this means "stopped". But cover the edge case where
215     # the marker was lost (e.g. /tmp cleared, or gateway started by an older build)
216     # yet containers are genuinely up reconcile via is_gateway_active, but ONLY
217     # when Docker is actually reachable, so a dead-socket probe can never hang. On
218     # macOS Docker runs in the Colima VM; if that socket is absent, the gateway
219     # definitively is not running.
220     local docker_up="no"
221     if [[ "$OSTYPE" == "darwin"* ]]; then
222         if [ -S /var/run/docker.sock ] || [ -S "$HOME/.colima/default/docker.sock" ]; then docker_up="yes"; fi
223     else
224         if [ -S /var/run/docker.sock ]; then docker_up="yes"; fi
225     fi
226
227     if [ "$docker_up" = "yes" ] && is_gateway_active; then
228         echo "running"; return 0
229     fi
230     echo "stopped"
231     return 0
232 }

```

```

232
233 # Reads an environment variable from a file (default: .env), strips quotes and whitespace
234 read_env_var() {
235     local var_name="$1"
236     local env_file="${2:-.env}"
237     grep -E "$var_name" "$env_file" 2>/dev/null | cut -d '=' -f2- | tr -d "\"\`" | sed -e 's/^[[:space:]]*//'
238     -e 's/[[:space:]]*$//' | echo ""
239 }
240
241 # Load KILL_SWITCH variable from .env (defaults to false if not found/empty)
242 export KILL_SWITCH
243 KILL_SWITCH=$(read_env_var "KILL_SWITCH" 2>/dev/null | tr '[:upper:]' '[:lower:]')
244 if [ -z "$KILL_SWITCH" ]; then
245     KILL_SWITCH="false"
246 fi
247
248 # Function to get the active network service name (e.g., "Wi-Fi" or "USB 10/100 LAN")
249 get_active_service() {
250     if [[ "$OSTYPE" == "darwin*" ]]; then
251         local iface
252         iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')
253
254         # If the default interface is empty or a VPN tunnel (like utunX), fallback to the active physical interface
255         if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
256             for i in en0 en1 en2 en3; do
257                 if ifconfig "$i" 2>> output.log | grep -q "status: active" && ifconfig "$i" 2>> output.log | grep -q "
258                     inet "; then
259                     iface="$i"
260                     break
261                 fi
262             done
263         fi
264
265         # Default fallback if still empty or not enX
266         if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
267             iface="en0"
268         fi
269
270         # Map interface (e.g., en0) to service name (e.g., Wi-Fi)
271         local service
272         service=$(networksetup -listnetworkserviceorder 2>> output.log | grep -B 1 "Device: $iface" | head -n 1 |
273             sed -E 's/^\([0-9\*]+\)/ /')
274
275         if [ -n "$service" ]; then
276             echo "$service"
277         else
278             # Get the interface used for default internet routing
279             local iface
280             iface=$(ip route get 1.1.1.1 2>/dev/null | awk '{print $5; exit}')
281             if [ -z "$iface" ]; then
282                 echo ""
283                 return 1
284             fi
285             echo "$iface"
286         fi
287     }
288
289 get_en0_service() {
290     local service
291     service=$(networksetup -listnetworkserviceorder 2>> output.log | grep -B1 "Device: en0" | head -1 | sed -E 's
292         /\([0-9\*]+\)/ /' || true)
293     if [ -z "$service" ]; then
294         echo "Wi-Fi"
295     else
296         echo "$service"
297     fi
298 }
299
300 get_warp_endpoint_1() { read_env_var "WARP_ENDPOINT_1" ".env" | grep -Eo '[0-9\.]+' || echo "
301     $DEFAULT_WARP_ENDPOINT_1"; }
302 get_warp_endpoint_2() { read_env_var "WARP_ENDPOINT_2" ".env" | grep -Eo '[0-9\.]+' || echo "
303     $DEFAULT_WARP_ENDPOINT_2"; }
304
305 restart_tailscaled_daemon() {
306     echo " [Tailscale Recovery] tailscaled daemon appears to be wedged/unresponsive."
307     echo " [Tailscale Recovery] Attempting to restart tailscaled service..."
308
309     if run_with_timeout 15 brew services restart tailscale >> output.log 2>&1; then

```

```

307     echo " [Tailscale Recovery] Successfully restarted tailscaled (user service)."
308 elif run_with_timeout 15 sudo -n brew services restart tailscale >> output.log 2>&1; then
309     echo " [Tailscale Recovery] Successfully restarted tailscaled (system service)."
310 else
311     echo " [Tailscale Recovery] WARNING: Failed to restart tailscaled. Manual intervention may be needed."
312 fi
313 sleep 3
314 }
315
316 disconnect_tailscale_host() {
317     local ts_bin="${1:-tailscale}"
318     if command -v "$ts_bin" >> output.log 2>&1; then
319         echo " Disconnecting host Tailscale from mesh..."
320
321         # Check if actually connected first (with timeout tailscaled could be wedged)
322         local status_ok=false
323         if run_with_timeout 5 "$ts_bin" status >> output.log 2>&1; then
324             status_ok=true
325         else
326             local status_exit=$?
327             # If it was a quick exit code 1 (disconnected), it is still reachable
328             if [ "$status_exit" -ne 143 ] && [ -S /var/run/tailscaled.socket ]; then
329                 status_ok=true
330             fi
331         fi
332
333         if [ "$status_ok" = "false" ]; then
334             restart_tailscaled_daemon
335         fi
336
337         # Explicitly reset any exit-node so it doesn't linger.
338         "$ts_bin" up >> output.log 2>&1 || true
339         if run_with_timeout 10 "$ts_bin" set --ssh=true --accept-dns=false --exit-node= >> output.log 2>&1; then
340             echo " [ ] Exit-node preference cleared."
341         else
342             echo " [!] tailscale up didn't respond (tailscaled may be wedged)"
343         fi
344
345         local ts_args=""
346         if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
347         if run_with_timeout 10 "$ts_bin" down $ts_args >> output.log 2>&1; then
348             echo " [ ] Tailscale disconnected."
349         else
350             echo " [!] tailscale down didn't respond"
351         fi
352     else
353         echo " [!] tailscale CLI not found skipping"
354     fi
355 }
356
357 stop_pidfile_daemon() {
358     local pid_file="$1"
359     local label="$2"
360     if [ -f "$pid_file" ]; then
361         local pid
362         pid=$(cat "$pid_file")
363         if kill -0 "$pid" 2>/dev/null; then
364             echo " Stopping $label (PID $pid)..."
365             sudo -n kill "$pid" 2>> output.log || kill "$pid" 2>> output.log || true
366             sleep 0.5
367             if kill -0 "$pid" 2>/dev/null; then
368                 sudo -n kill -9 "$pid" 2>> output.log || kill -9 "$pid" 2>> output.log || true
369             fi
370         fi
371         rm -f "$pid_file"
372     fi
373 }
374
375 disable_all_proxies() {
376     local svc="$1"
377     if [ -n "$svc" ]; then
378         sudo -n networksetup -setsocksfirewallproxystate "$svc" off 2>> output.log || true
379         sudo -n networksetup -setwebproxystate "$svc" off 2>> output.log || true
380         sudo -n networksetup -setsecurewebproxystate "$svc" off 2>> output.log || true
381     fi
382 }
383
384 flush_dns_cache() {
385     if [[ "$OSTYPE" == "darwin"* ]]; then
386         sudo -n dscacheutil -flushcache 2>> output.log || true
387         sudo -n killall -HUP mDNSResponder 2>> output.log || true

```

```

388 else
389     resolvectl flush-caches 2>> output.log || true
390 fi
391 }
392
393 wait_for_dhcp_settle() {
394     # Resolve physical interface (Wi-Fi or Ethernet)
395     local physical_iface
396     physical_iface=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: (Wi-Fi|Ethernet)/{
397         getline; print $2; exit}')
398     [ -z "$physical_iface" ] && physical_iface="en0"
399
400     echo -n "Waiting for physical network interface ($physical_iface) DHCP lease to settle..."
401     local settled=false
402     for attempt in {1..60}; do
403         PHYSICAL_GW=$(ipconfig getpacket "$physical_iface" 2>> output.log | awk -F'[{}]' '/router/{print $2}')
404         local local_ip
405         local_ip=$(ifconfig "$physical_iface" 2>/dev/null | awk '/inet/{print $2}')
406
407         if [ -n "$PHYSICAL_GW" ] && [[ "$PHYSICAL_GW" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
408             [ -n "$local_ip" ] && [[ "$local_ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
409             [[ "$local_ip" != "169.254.*" ]]; then
410             echo "settled (Interface IP: $local_ip, Router IP: $PHYSICAL_GW)."
411             settled=true
412             break
413         fi
414
415         if [ "$attempt" -gt 15 ] && [ -n "$local_ip" ] && [[ "$local_ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
416             [[ "$local_ip" != "169.254.*" ]]; then
417             local fallback_gw
418             fallback_gw=$(route get default 2>> output.log | awk '/gateway:/{print $2}')
419             if [ -n "$fallback_gw" ] && [[ "$fallback_gw" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
420                 PHYSICAL_GW="$fallback_gw"
421                 fi
422                 echo "static/settled detected (Interface IP: $local_ip, Gateway IP: ${PHYSICAL_GW:-unknown})."
423                 settled=true
424                 break
425             fi
426             echo -n "."
427             sleep 0.5
428         done
429         if [ "$settled" = "false" ]; then
430             echo "timed out or no active link. Proceeding anyway."
431         fi
432     }
433
434 reset_sharing_services() {
435     if [[ "$OSTYPE" == "darwin"* ]]; then
436         sudo -n killall sharingd rapportd 2>> output.log || true
437     fi
438 }
439
440 read_adguard_ip() {
441     if [ -f ".gateway_ip" ]; then
442         tr -d '\r' < ".gateway_ip" | awk 'NR==1{print $1;exit}'
443     fi
444 }
445
446 is_kill_switch_enabled() {
447     [ "$KILL_SWITCH" = "true" ]
448 }
449
450 add_warp_bypass_routes() {
451     local target="${1:-}"
452     local ep1
453     ep1=$(get_warp_endpoint_1)
454     local ep2
455     ep2=$(get_warp_endpoint_2)
456
457     if [[ "$OSTYPE" == "darwin"* ]]; then
458         local routing_arg=""
459         local msg_via=""
460         if [ -n "$target" ]; then
461             if [[ "$target" =~ ^en[0-9]+$ || "$target" == "bridge"* ]]; then
462                 routing_arg="-interface $target"
463                 msg_via="interface $target"
464             else
465                 routing_arg="$target"
466                 msg_via="gateway $target"
467             fi
468         fi
469     else

```

```

467     local gateway_ip
468     gateway_ip=$(route get default 2>> output.log | awk '/gateway:/ {print $2}')
469     if [ -z "$gateway_ip" ]; then
470         local iface
471         iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')
472         if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
473             iface="en0"
474         fi
475         routing_arg="-interface $iface"
476         msg_via="interface $iface"
477     else
478         routing_arg="$gateway_ip"
479         msg_via="gateway $gateway_ip"
480     fi
481 fi
482
483 echo -e "\nAdding host bypass routes for Cloudflare WARP endpoints via $msg_via..."
484 sudo -n route delete -host "$sep1" 2>/dev/null || true
485 sudo -n route delete -host "$sep2" 2>/dev/null || true
486 sudo -n route add -host "$sep1" $routing_arg >> output.log 2>&1 || true
487 sudo -n route add -host "$sep2" $routing_arg >> output.log 2>&1 || true
488
489 echo -e "Adding host bypass routes for Tailscale control plane via $msg_via..."
490 sudo -n route delete -net 192.200.0.0/24 2>/dev/null || true
491 sudo -n route add -net 192.200.0.0/24 $routing_arg >> output.log 2>&1 || true
492
493 echo -e "Adding host bypass routes for Tailscale DERP relays via $msg_via..."
494 local derp_ips
495 derp_ips=$(curl -s --connect-timeout 5 https://login.tailscale.com/derpmap/default | grep -oE '"IPv4"[[:space:]]*:[[:space:]]*[0-9.]+' | cut -d'"' -f4 | grep -E '^([0-9]+\.[0-9]+\.[0-9]+\.[0-9])+$' || true)
496 if [ -n "$derp_ips" ]; then
497     echo "$derp_ips" > /tmp/nullexit-derp-ips.txt
498     for ip in $derp_ips; do
499         sudo -n route delete -host "$ip" 2>/dev/null || true
500         sudo -n route add -host "$ip" $routing_arg >> output.log 2>&1 || true
501     done
502 fi
503 else
504     local gateway_ip="${target}"
505     if [ -z "$gateway_ip" ]; then
506         gateway_ip=$(ip route show default 2>/dev/null | awk '/default/ {print $3}')
507     fi
508     if [ -n "$gateway_ip" ]; then
509         sudo ip route add "$sep1" via "$gateway_ip" >> output.log 2>&1 || true
510         sudo ip route add "$sep2" via "$gateway_ip" >> output.log 2>&1 || true
511     fi
512 fi
513 }
514
515 remove_warp_bypass_routes() {
516     local ep1
517     ep1=$(get_warp_endpoint_1)
518     local ep2
519     ep2=$(get_warp_endpoint_2)
520
521     if [[ "$OSTYPE" == "darwin*" ]]; then
522         echo -e "\nRemoving host bypass routes for Cloudflare WARP endpoints..."
523         sudo -n route delete -host "$sep1" >> output.log 2>&1 || true
524         sudo -n route delete -host "$sep2" >> output.log 2>&1 || true
525         echo -e "Removing host bypass routes for Tailscale control plane..."
526         sudo -n route delete -net 192.200.0.0/24 >> output.log 2>&1 || true
527         if [ -f /tmp/nullexit-derp-ips.txt ]; then
528             echo -e "Removing host bypass routes for Tailscale DERP relays..."
529             while read -r ip; do
530                 sudo -n route delete -host "$ip" >> output.log 2>&1 || true
531             done < /tmp/nullexit-derp-ips.txt
532             rm -f /tmp/nullexit-derp-ips.txt
533         fi
534     else
535         sudo ip route del "$sep1" >> output.log 2>&1 || true
536         sudo ip route del "$sep2" >> output.log 2>&1 || true
537     fi
538 }
539
540 bounce_wifi_interfaces() {
541     if [[ "$OSTYPE" == "darwin*" ]]; then
542         local wifi_port
543         wifi_port=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: Wi-Fi/{getline; print $2}')
544         if [ -n "$wifi_port" ]; then
545             sudo -n networksetup -setairportpower "$wifi_port" off 2>> output.log || true

```

```

546     sleep 2
547     sudo -n networksetup -setairportpower "$wifi_port" on 2>> output.log || true
548     echo "Wi-Fi bounced (interface $wifi_port)"
549     sleep 3
550 else
551     echo "Could not detect Wi-Fi interface. Bouncing fallback interfaces..."
552     set +e
553     for iface in en0 en1 en2 en3 en4 en5; do
554         if ifconfig "$iface" >> output.log 2>&1; then
555             sudo -n ifconfig "$iface" down 2>> output.log || true
556             sudo -n ifconfig "$iface" up 2>> output.log || true
557         fi
558     done
559     set -e
560     echo "Fallback interfaces bounced"
561 fi
562 fi
563 }
564
565 get_active_interface() {
566     local iface
567     iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')
568
569     # If the default interface is empty or a VPN tunnel (like utunX), fallback to the active physical interface
570     if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
571         for i in en0 en1 en2 en3; do
572             if ifconfig "$i" 2>> output.log | grep -q "status: active" && ifconfig "$i" 2>> output.log | grep -q "
inet "; then
573                 iface="$i"
574                 break
575             fi
576         done
577     fi
578
579     # Default fallback if still empty or not enX
580     if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
581         iface="en0"
582     fi
583
584     echo "$iface"
585 }
586
587 enable_killswitch() {
588     if [[ "$OSTYPE" == "darwin*" ]]; then
589         if ! is_kill_switch_enabled; then
590             return 0
591         fi
592
593         local ext_if
594         ext_if=$(get_active_interface)
595         local ep1
596         ep1=$(get_warp_endpoint_1)
597         local ep2
598         ep2=$(get_warp_endpoint_2)
599         local pf_conf_path
600         pf_conf_path="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/pf.conf"
601
602         echo -e "\nEnabling macOS Packet Filter (PF) Kill-Switch on $ext_if..."
603
604         # Enable PF and capture the reference token
605         local token_out
606         token_out=$(sudo -n pfctl -E 2>> output.log || true)
607         local token
608         token=$(echo "$token_out" | awk '/Token/{print $NF}')
609         if [ -n "$token" ]; then
610             echo "$token" > /tmp/nullexit-pf-token.txt
611             echo " [] PF enabled with token $token."
612         else
613             echo " [!] WARNING: Could not capture PF reference token (may already be enabled or sudo failed)."
614         fi
615
616         # Load the rules into the anchor
617         if sudo -n pfctl -D "ext_if=$ext_if" -a com.apple/nullexit -f "$pf_conf_path" >> output.log 2>&1; then
618             echo " [] Ruleset loaded into anchor com.apple/nullexit."
619         else
620             echo " [!] ERROR: Failed to load ruleset into anchor."
621         fi
622
623         # Populate tables in the anchor
624         sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T flush >> output.log 2>&1 || true
625         sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T add "$ep1" >> output.log 2>&1 || true

```

```

626 sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T add "$ep2" >> output.log 2>&1 || true
627
628 if [ -f /tmp/nullexit-derp-ips.txt ]; then
629     local derp_ips
630     derp_ips=$(tr '\n' ' ' < /tmp/nullexit-derp-ips.txt)
631     if [ -n "$derp_ips" ]; then
632         sudo -n pfctl -a com.apple/nullexit -t derp_relays -T flush >> output.log 2>&1 || true
633         sudo -n pfctl -a com.apple/nullexit -t derp_relays -T add $derp_ips >> output.log 2>&1 || true
634     fi
635 fi
636 echo " [] Kill-switch tables populated."
637 fi
638 }
639
640 disable_killswitch() {
641     if [[ "$OSTYPE" == "darwin"* ]]; then
642         if ! is_kill_switch_enabled; then
643             return 0
644         fi
645         echo -e "\nDisabling macOS PF Kill-Switch..."
646
647         # Flush rules from anchor com.apple/nullexit
648         sudo -n pfctl -a com.apple/nullexit -F all >> output.log 2>&1 || true
649
650         # Release the enable reference if token is present
651         if [ -f /tmp/nullexit-pf-token.txt ]; then
652             local token
653             token=$(cat /tmp/nullexit-pf-token.txt)
654             if [ -n "$token" ]; then
655                 sudo -n pfctl -X "$token" >> output.log 2>&1 || true
656                 echo " [] Released PF enable reference token $token."
657             fi
658             rm -f /tmp/nullexit-pf-token.txt
659         else
660             echo " [] Rules flushed from anchor com.apple/nullexit."
661         fi
662     fi
663 }
664
665 write_host_ips() {
666     # Gather all active IPv4 addresses on the host and write to .host_ips
667     # routing-fix.sh will read this file to explicitly whitelist them for direct Tailscale P2P
668     local repo_dir
669     repo_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
670
671     if [[ "$OSTYPE" == "darwin"* ]]; then
672         ifconfig | awk '/inet /{print $2}' | grep -v '127.0.0.1' > "$repo_dir/.host_ips" 2>/dev/null || true
673     else
674         ip -4 addr show 2>/dev/null | awk '/inet /{print $2}' | cut -d/ -f1 | grep -v '127.0.0.1' > "$repo_dir/.
675         host_ips" 2>/dev/null || true
676     fi
677 }
678
679 start_dns_watcher() {
680     if [[ "$OSTYPE" == "darwin"* ]]; then
681         local target_ip=$1
682         stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
683         echo " Starting background DNS Watcher for seamless Wi-Fi roaming..."
684         nohup bash -c "
685             source \"$SCRIPT_DIR/scripts/common.sh\"
686             trap 'exit 0' SIGTERM SIGINT SIGHUP
687             while true; do
688                 ACTIVE_IF=$(get_active_service)
689                 if [ -n \"$ACTIVE_IF\" ]; then
690                     CURRENT_DNS=$(networksetup -getdnsservers \"$ACTIVE_IF\" 2>/dev/null)
691                     if [ \"$CURRENT_DNS\" != \"$target_ip\" ]; then
692                         networksetup -setdnsservers \"$ACTIVE_IF\" \"$target_ip\" >/dev/null 2>&1
693                     fi
694                 fi
695                 sleep 30
696             done
697         " >> output.log 2>&1 &
698         echo $! > "$DNS_WATCHER_PID_FILE"
699     else
700         local target_ip=$1
701         stop_dns_watcher
702         echo " Starting background DNS Watcher for seamless Wi-Fi roaming..."
703         nohup bash -c "
704             trap 'exit 0' SIGTERM SIGINT SIGHUP
705             while true; do

```



```

706         if [ -n "\$ACTIVE_SERVICE" ]; then
707             CURRENT_DNS=\$(resolvectl dns "\$ACTIVE_SERVICE" 2>/dev/null | grep -oE
              '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | head -1)
708             if [ "\$CURRENT_DNS" != "\$target_ip" ]; then
709                 resolvectl dns "\$ACTIVE_SERVICE" "\$target_ip" >/dev/null 2>&1 || true
710             fi
711         fi
712         sleep 30
713     done
714     " >> output.log 2>&1 &
715     echo \$! > "\$DNS_WATCHER_PID_FILE"
716 fi
717 }
718
719 # SOCKS5 Proxy (WARP Tunnel)
720 # When Tailscale's exit node can't route through WARP (due to userspace
721 # forwarding), we use a SOCKS5 proxy running inside the warp container's
722 # network namespace. Connections created by this proxy go through the
723 # kernel routing table (table 200 -> tun0 -> WARP), unlike Tailscale's
724 # userspace exit node which bypasses it.
725 #
726 # The SOCKS5 proxy is exposed on localhost:\${SOCKS_PROXY_PORT} via Docker port mapping.
727 # `networksetup -setsocksfirewallproxy` on macOS routes system TCP traffic
728 # through the proxy, which then goes through WARP.
729 #
730 # IMPORTANT: CLI tools like curl do NOT respect macOS system proxy settings.
731 # They must use --socks5-hostname or the ALL_PROXY env variable explicitly.
732
733 enable_socks_proxy() {
734     if [[ "$OSTYPE" == "darwin"* ]]; then
735         local svc="$1"
736         if [ -z "$svc" ]; then
737             svc="\$ACTIVE_SERVICE"
738         fi
739
740         echo -n " Enabling system-wide SOCKS5 proxy on $svc (localhost:\$SOCKS_PROXY_PORT)... "
741         sudo -n networksetup -setsocksfirewallproxy "$svc" 127.0.0.1 \$SOCKS_PROXY_PORT 2>> output.log || true
742         sudo -n networksetup -setsocksfirewallproxystate "$svc" on 2>> output.log || true
743
744         # Also set on en0 service for macOS resolver consistency
745         if [ "$svc" != "\$ENO_SERVICE" ]; then
746             sudo -n networksetup -setsocksfirewallproxy "\$ENO_SERVICE" 127.0.0.1 \$SOCKS_PROXY_PORT 2>> output.log ||
              true
747             sudo -n networksetup -setsocksfirewallproxystate "\$ENO_SERVICE" on 2>> output.log || true
748         fi
749
750         # IMPORTANT: Exclude local LAN and mDNS traffic from the proxy so P2P works natively
751         # without source-IP masking from the Colima VM bridge.
752         sudo -n networksetup -setproxybypassdomains "$svc" 127.0.0.1 localhost 192.168.0.0/16 10.0.0.0/8
              172.16.0.0/12 "*.local" 2>> output.log || true
753         if [ "$svc" != "\$ENO_SERVICE" ]; then
754             sudo -n networksetup -setproxybypassdomains "\$ENO_SERVICE" 127.0.0.1 localhost 192.168.0.0/16 10.0.0.0/8
              172.16.0.0/12 "*.local" 2>> output.log || true
755         fi
756
757         echo "done."
758         echo " All TCP traffic now routed through gateway -> WARP tunnel -> internet."
759         echo " (CLI tools: export ALL_PROXY=socks5://127.0.0.1:\$SOCKS_PROXY_PORT)"
760     else
761         local svc="$1"
762         echo " (Linux global SOCKS proxy configuration is desktop-environment specific, skipping.)"
763         echo " SOCKS5 proxy is active and listening on 127.0.0.1:\$SOCKS_PROXY_PORT"
764     fi
765 }
766
767 disable_socks_proxy() {
768     if [[ "$OSTYPE" == "darwin"* ]]; then
769         for svc in "\$ACTIVE_SERVICE" "\$ENO_SERVICE"; do
770             sudo -n networksetup -setsocksfirewallproxystate "$svc" off 2>> output.log || true
771             done
772             echo " SOCKS5 proxy disabled."
773         else
774             local svc="$1"
775             echo " (Linux global SOCKS proxy configuration skipped.)"
776         fi
777     }
778
779 # Helper to restore host DNS to a clean state (used on script start, on failure,
780 # and after a successful STOP). Without this, a successful toggle-off leaves the
781 # host's resolver pinned to a now-dead gateway IP every lookup stalls for macOS's
782 # DNS timeout (~5s) before falling through to whatever next server is configured.

```

```

783 # With it, we always leave the host in `1.1.1.1 / no search domain`.
784 #
785 # We set DNS on BOTH the active service AND the en0 service because macOS's scutil DNS
786 # resolver is commonly scoped to en0 (Wi-Fi). A per-service change on e.g.
787 # "USB 10/100 LAN" is ignored by the system resolver unless the en0 service is also updated.
788 reset_dns() {
789     if [[ "$OSTYPE" == "darwin"* ]]; then
790         if [ -n "$ACTIVE_SERVICE" ]; then
791             for dns_svc in "$ACTIVE_SERVICE" "$ENO_SERVICE"; do
792                 networksetup -setsearchdomains "$dns_svc" "Empty" 2>> output.log || true
793                 networksetup -setdnsservers "$dns_svc" 1.1.1.1 2>> output.log || true
794             done
795         fi
796     else
797         if [ -n "$ACTIVE_SERVICE" ]; then
798             resolvectl domain "$ACTIVE_SERVICE" "" 2>/dev/null || true
799             resolvectl revert "$ACTIVE_SERVICE" 2>/dev/null || true
800         fi
801     fi
802 }
803
804 # Verify basic direct-internet connectivity after a recovery/teardown: DNS via
805 # 1.1.1.1, then HTTP (curl) or ping to 1.1.1.1. Sets the global INTERNET_OK to
806 # "true" on any successful check (callers read $INTERNET_OK in their summary).
807 # Always returns 0 so a bare call can't trip `set -e`. Used by recover.sh's macOS
808 # and Linux recovery paths (extracted from a formerly-duplicated block).
809 verify_internet_connectivity() {
810     step "Verifying internet connectivity"
811
812     # Wait a moment for the network to settle
813     sleep 2
814
815     INTERNET_OK=false
816
817     # Try DNS resolution first
818     if host -W 3 google.com 1.1.1.1 >> output.log 2>&1; then
819         ok "DNS resolution works (google.com via 1.1.1.1)"
820         INTERNET_OK=true
821     elif nslookup google.com 1.1.1.1 >> output.log 2>&1; then
822         ok "DNS resolution works (nslookup google.com via 1.1.1.1)"
823         INTERNET_OK=true
824     else
825         warn "DNS resolution check failed will still try ping/curl"
826     fi
827
828     # Try actual HTTP connectivity
829     if command -v curl >> output.log 2>&1; then
830         if curl -sf --max-time 5 https://1.1.1.1 >> output.log 2>&1; then
831             ok "Internet reachable via HTTP"
832             INTERNET_OK=true
833         elif curl -sf --max-time 5 http://1.1.1.1 >> output.log 2>&1; then
834             ok "Internet reachable via HTTP (plain)"
835             INTERNET_OK=true
836         else
837             warn "HTTP check failed"
838         fi
839     elif command -v ping >> output.log 2>&1; then
840         if ping -c 1 -W 3 1.1.1.1 >> output.log 2>&1; then
841             ok "Internet reachable via ping"
842             INTERNET_OK=true
843         else
844             warn "Ping check failed"
845         fi
846     fi
847
848     return 0
849 }
850
851 # Local DNS Proxy (Python)
852 # When the tailnet data plane cannot establish, we use a Python DNS proxy.
853 # It listens on UDP:53, receives DNS queries, forwards them over TCP to
854 # AdGuard at 127.0.0.1:${DNS_PROXY_PORT} (Docker port mapping), and returns the response.
855 # The Python script handles the DNS-over-TCP wire format (2-byte length prefix)
856 # correctly unlike socat which just forwards raw bytes.
857 #
858 # Python3 is built into macOS no external dependencies.
859 DNS_PROXY_PID=""
860
861 # Kill any leftover DNS proxy process
862 stop_local_dns_proxy() {
863     if [ -n "$DNS_PROXY_PID" ]; then

```

```

864 kill "$DNS_PROXY_PID" 2>/dev/null || true
865 wait "$DNS_PROXY_PID" 2>/dev/null || true
866 DNS_PROXY_PID=""
867 fi
868 # Clean up any stale Python DNS proxy processes
869 sudo -n pkill -f "dns-proxy.py" 2>/dev/null || true
870 }
871
872 # Start local DNS proxy (Python)
873 start_local_dns_proxy() {
874     if [ -z "$DNS_PROXY_BIN" ]; then
875         echo "Python not found. Python3 is built into macOS this shouldn't happen."
876         return 1
877     fi
878
879     if [ ! -f "$DNS_PROXY_SCRIPT" ]; then
880         echo "DNS proxy script not found at $DNS_PROXY_SCRIPT"
881         return 1
882     fi
883
884     # Kill any leftover process first
885     stop_local_dns_proxy
886
887     echo -n " Starting local DNS proxy via Python (UDP:53 TCP:${DNS_PROXY_PORT})... "
888     sudo -n bash -c "TARGET_PORT=$DNS_PROXY_PORT \"$DNS_PROXY_BIN\" \"$DNS_PROXY_SCRIPT\" \" &
889     DNS_PROXY_PID=$!
890     disown "$DNS_PROXY_PID" 2>/dev/null || true
891
892     # Give it a moment to bind
893     sleep 0.5
894
895     if kill -0 "$DNS_PROXY_PID" 2>/dev/null; then
896         echo "started (PID $DNS_PROXY_PID)."
897
898         # Hijack host DNS to localhost
899         echo -n " Hijacking host DNS to 127.0.0.1 for ad-blocking... "
900         if [[ "$OSTYPE" == "darwin"* ]]; then
901             networksetup -setsearchdomains "$ACTIVE_SERVICE" "Empty" 2>/dev/null || true
902             networksetup -setsearchdomains "$ENO_SERVICE" "Empty" 2>/dev/null || true
903             if ! networksetup -setdnsservers "$ACTIVE_SERVICE" 127.0.0.1; then
904                 echo "FAILED (networksetup error check permissions)."
905                 stop_local_dns_proxy
906                 return 1
907             fi
908             networksetup -setdnsservers "$ENO_SERVICE" 127.0.0.1 2>/dev/null || true
909             sudo -n dscacheutil -flushcache 2>/dev/null || true
910             sudo -n killall -HUP mDNSResponder 2>/dev/null || true
911         else
912             resolvectl domain "$ACTIVE_SERVICE" "" 2>/dev/null || true
913             resolvectl domain "$ACTIVE_SERVICE" "" 2>/dev/null || true
914             if ! sudo -n resolvectl dns "$ACTIVE_SERVICE" 127.0.0.1; then
915                 echo "FAILED (resolvectl error check permissions)."
916                 stop_local_dns_proxy
917                 return 1
918             fi
919             resolvectl dns "$ACTIVE_SERVICE" 127.0.0.1 2>/dev/null || true
920             sudo -n resolvectl flush-caches 2>/dev/null || true
921         fi
922
923         # Verify DNS actually works through the proxy
924         echo -n " Verifying DNS resolution... "
925         if dig @127.0.0.1 google.com +short +timeout=5 &>/dev/null; then
926             echo "ok."
927             return 0
928         else
929             echo "FAILED (DNS queries not reaching AdGuard)."
930             echo " Restoring DNS to 1.1.1.1..."
931             reset_dns
932             stop_local_dns_proxy
933             return 1
934         fi
935     else
936         echo "FAILED (could not bind port 53 is it in use?)."
937         DNS_PROXY_PID=""
938         return 1
939     fi
940 }
941
942 # Helper to FORCIBLY hijack host DNS to a single server (the gateway's AdGuard IP)
943 # and VERIFY via `networksetup -getdnsservers` that the only name server is exactly
944 # $1. Returns 0 iff the live state matches; non-zero if it can't be confirmed.

```

```

945 #
946 # This deliberately does NOT append a 1.1.1.1 fallback: macOS's resolver queries
947 # the list in order and falls back to the next entry on timeout, so on a
948 # misbehaving AdGuard (filter compile crash, OOM kill, etc.) the resolver still
949 # leaks to 1.1.1.1 and silently bypasses ad-blocking. With no fallback, a broken
950 # gateway manifests as a *visible* DNS outage that prompts the user to
951 # investigate which is the correct failure mode.
952 force_dns_to_gateway() {
953     if [[ "$OSTYPE" == "darwin"* ]]; then
954         local target_ip="$1"
955         if [ -z "$target_ip" ] || [ -z "$ACTIVE_SERVICE" ]; then
956             echo " [force_dns] ERROR: target_ip or ACTIVE_SERVICE is empty" >&2
957             return 1
958         fi
959
960         local attempt
961         for attempt in 1 2 3; do
962             echo -n " [force_dns] Attempt $attempt/3: setting DNS to $target_ip on \"\$ACTIVE_SERVICE\" + \"\$ENO_SERVICE\"... "
963
964             # Apply to BOTH the active service AND the en0 service the scutil DNS resolver
965             # is scoped to en0 (Wi-Fi) and ignores per-service changes that don't include it.
966             # Fail fast on ACTIVE_SERVICE; best-effort on ENO_SERVICE (it may not exist).
967             networksetup -setsearchdomains "$ACTIVE_SERVICE" "ts.net" || true
968             if ! networksetup -setdnsservers "$ACTIVE_SERVICE" "$target_ip"; then
969                 echo "FAILED (networksetup error check permissions)"
970                 sleep 1
971                 continue
972             fi
973             networksetup -setsearchdomains "$ENO_SERVICE" "ts.net" || true
974             networksetup -setdnsservers "$ENO_SERVICE" "$target_ip" || true
975
976             local entries
977             entries=$(networksetup -getdnsservers "$ACTIVE_SERVICE" 2>>> output.log \
978                 | awk '/^(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$/ {print}')
979             if echo "$entries" | grep -qFx "$target_ip"; then
980                 echo "VERIFIED"
981                 return 0
982             fi
983
984             local current
985             current=$(echo "$entries" | tr '\n' ' ' | sed 's/ $//')
986             echo "NOT YET (current DNS: [${current:-none}])"
987
988             if [ "$attempt" = "3" ]; then sleep 2; else sleep 1; fi
989             done
990
991             echo " [force_dns] FAILED after 3 attempts"
992             return 1
993         else
994             local target_ip="$1"
995
996             for attempt in {1..3}; do
997                 echo -n " [force_dns] Attempt $attempt/3: setting DNS to $target_ip on $ACTIVE_SERVICE... "
998                 sudo -n resolvectl domain "$ACTIVE_SERVICE" "-ts.net" 2>/dev/null || true
999                 if ! sudo -n resolvectl dns "$ACTIVE_SERVICE" "$target_ip"; then
1000                     echo "FAILED (resolvectl error)"
1001                 else
1002                     # Verify
1003                     entries=$(resolvectl dns "$ACTIVE_SERVICE" 2>>> output.log | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+')
1004                     if [ "$entries" == "$target_ip" ]; then
1005                         echo "VERIFIED"
1006                         return 0
1007                     else
1008                         echo "MISMATCH (Resolver refused lock)"
1009                     fi
1010                 fi
1011                 sleep 2
1012             done
1013             return 1
1014         fi
1015     }

```

12 scripts/watcher.sh

```

1 #!/bin/bash
2 # scripts/watcher.sh nullexit post-wake / post-roam recovery watcher
3 #
4 # Long-running daemon. Launched by launchd as a LaunchAgent at
5 # ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
6 # (label com.nullexit.wake-recovery).
7 #
8 # Two independent listeners run as backgrounded pipelines:
9 #
10 # (1) WAKE listener `log stream` live-follows the macOS unified log for
11 # power-management events (lid closewake, manual wake, DarkWake from
12 # clamshell). Each event triggers run_recover (with a global debounce so
13 # a single wake that emits 2-3 log lines only causes ONE post-wake run).
14 #
15 # (2) NETWORK listener `scutil n.watch` on `State:/Network/Global/IPv4`
16 # fires on every Wi-Fi roam, ethernet swap, hotspot join, captive-portal
17 # re-bind, VPN up/down, etc. Same debounce.
18 #
19 # Both listeners call run_recover, which:
20 # - checks /tmp/nullexit-gateway-active.marker (only act when toggle.sh left
21 # the gateway up)
22 # - debounces (10s default) so multiple events don't pile up
23 # - shells out to `bash <repo>/recover.sh --post-wake` directly (no GUI auth prompt needed due to NOPASSWD
24 # sudoers)
25 #
26 # See devref.md 10.29 for the full Apple power management primer and the
27 # rationale behind using `log stream` + `scutil n.watch` from a shell daemon.
28 #
29 # NOTE: errexit (`set -e`) is intentionally NOT enabled. This is a long-running
30 # daemon, not a discrete-step script, and errexit actively breaks the reconnect
31 # loops below: any failed pipe (log stream on rotation, scutil on state churn)
32 # would kill the outer subshell before `echo "reconnecting..."; sleep 5;` runs,
33 # AND the parent's final `wait` would propagate a non-zero child exit and kill
34 # the entire watcher. Every error path is already wrapped in `|| true` /
35 # `2>/dev/null` fallbacks.
36 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
37 source "$SCRIPT_DIR/common.sh"
38 setup_standard_path
39 LOG="$SCRIPT_DIR/../output.log"
40 # Resolve our own directory; recover.sh lives one level up at the repo root.
41 # This makes the daemon install-agnostic no need to template the path
42 # in launchd, no need to keep a deploy-specific hardcoded location in sync.
43 RECOVER="$SCRIPT_DIR/../recover.sh"
44 MARKER="$MARKER_FILE"
45 DEBOUNCE_FILE="/tmp/nullexit-watcher.last-recovery"
46 DEBOUNCE_SECONDS="${NULLEXIT_DEBOUNCE_SECONDS:-10}"
47
48 exec >> "$LOG" 2>&1
49
50 # Single-instance lock
51 # Without this, repeated lid-closewake cycles under macOS Sonoma+ accumulate
52 # orphan watcher.sh processes: launchd suspends on sleep and SIGCONT-resumes
53 # on wake, and KeepAlive.Crashed=true caused relaunch on any non-zero exit;
54 # the listener grandchildren (log stream | while ... and (echo n.add ...;
55 # sleep 86400) | scutil | while ...) stay alive because pipe producers don't
56 # reliably respond to plain SIGTERM if the consumer is in a half-closed state.
57 #
58 # We solve it with a PID-file lock (POSIX-portable; macOS does NOT ship `flock`).
59 # The trap below removes the lock file on every exit so a stale lock never
60 # outlives a crashed watcher. On launch: read the existing PID, check it is
61 # alive AND is actually a `scripts/watcher.sh` process; if so, exit 0; else
62 # overwrite with our PID and proceed. The check-then-write window has a TOCTOU
63 # race but is microseconds wide and only matters for hand-launched duplicate
64 # test invocations; production is single-launch via launchctl load -w.
65 LOCK_FILE="/tmp/nullexit-watcher.lock"
66 LOCK_PID=""
67 if [ -e "$LOCK_FILE" ]; then
68     LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
69     if [ -n "$LOCK_PID" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
70         # Verify the holder is actually a scripts/watcher.sh process (not some
71         # unrelated process that got the recycled PID). The exact-launchd pattern
72         # `bash <abs-path>/scripts/watcher.sh` is hard to accidentally match with
73         # a `grep scripts/watcher.sh` invocation because we anchor on `^bash`
74         # followed by whitespace and end-of-line on the script path.
75         if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -qE "(^|[:space:])bash[:space:]+.*scripts/watcher\\.sh$"; then
76             echo "[$(date -u +%FT%TZ)] watcher.sh: another instance holds $LOCK_FILE (pid=$LOCK_PID), exiting"
77             exit 0
78         fi
79     fi

```

```

80 fi
81 echo $$ > "$LOCK_FILE"
82
83 echo "[$(date -u +%FT%TZ)] watcher.sh start (pid=$$ ppid=$PPID user=$(id -un) lock=acquired)"
84
85 # Treat launchd-resume as a wake signal. Apple's launchd SUSPENDS LaunchAgents
86 # during system sleep and restores them on wake; during that suspended gap,
87 # macOS's `log stream` cannot catch the wake event that prompted the resume.
88 # The next thing that happens after wake IS our own startup, so we always
89 # fire one post-wake run unconditionally here. The marker check + debounce
90 # in run_recover() make this idempotent: if the gateway isn't up or we just
91 # debounced, it no-ops. Without this, the FIRST wake-after-install goes
92 # unseen and the watcher appears broken until the SECOND wake.
93 # Helper: run recover.sh --post-wake if the gateway is active
94 run_recover() {
95     local why="$1"
96     if [ ! -f "$MARKER" ]; then
97         echo "[$(date -u +%FT%TZ)] $why no $MARKER, skip"
98         return 0
99     fi
100     local now last
101     now=$(date +%s)
102     last=$(cat "$DEBOUNCE_FILE" 2>/dev/null || echo 0)
103     if [ "$(now - last)" -lt "$DEBOUNCE_SECONDS" ]; then
104         echo "[$(date -u +%FT%TZ)] $why debounced ($(now - last)s since last, threshold ${DEBOUNCE_SECONDS}s)"
105         return 0
106     fi
107     echo "$now" > "$DEBOUNCE_FILE"
108     echo "[$(date -u +%FT%TZ)] $why bash $RECOVER --post-wake"
109     bash "$RECOVER" --post-wake
110     local exit_code=$?
111     echo "$(date +%s)" > "$DEBOUNCE_FILE"
112     echo "[$(date -u +%FT%TZ)] $why exit=$exit_code (now=$(date +%s))"
113     return $exit_code
114 }
115
116 if [ -f "$MARKER" ]; then
117     run_recover "initial post-wake (launchd resume = wake signal)"
118 fi
119
120 # LAN P2P Auto-Detection
121 # Detects whether the current network allows safe Tailscale LAN P2P by:
122 # 1. Checking WPA2-Enterprise (802.1x) via system_profiler SPAirPortDataType always forces P2P=false
123 # because enterprise networks almost universally have AP Client Isolation.
124 # 2. AP Isolation probe sends a broadcast ping and checks if ANY LAN host responds.
125 # If no LAN hosts respond on a non-empty network, AP Isolation is active.
126 # Writes 'true' or 'false' to .lan_p2p_detected in the repo root.
127 # routing-fix.sh reads this file dynamically every 30s loop iteration.
128 P2P_OVERRIDE_FILE="$SCRIPT_DIR/../../lan_p2p_detected"
129
130
131 detect_lan_p2p_mode() {
132     local reason="unknown" allow="false"
133
134     # Step 1: Detect Wi-Fi security type via system_profiler (airport binary removed in macOS 14.4+)
135     # SPAirPortDataType lists all known networks; the first "Security:" line is the current association.
136     local security
137     security=$(system_profiler SPAirPortDataType 2>/dev/null \
138         | awk '/Current Network Information:/{found=1} found && /Security:/{print $NF; exit}')
139
140     if [ -z "$security" ]; then
141         # Not on Wi-Fi or system_profiler unavailable fail safe
142         reason="no-wifi" allow="false"
143     elif echo "$security" | grep -qi "Enterprise"; then
144         # WPA2/WPA3 Enterprise = 802.1x always AP isolated
145         reason="802.1x-enterprise" allow="false"
146     elif echo "$security" | grep -qi "Personal\|None\|Open"; then
147         # WPA2/WPA3 Personal or open probe for AP isolation
148         local gw
149         gw=$(route -n get 1.1.1.1 2>/dev/null | awk '/gateway:/{print $2}')
150         if [ -z "$gw" ]; then
151             reason="no-default-route" allow="false"
152         else
153             local responses
154             responses=$(ping -c 3 -t 1 -q "$gw" 2>/dev/null | awk '/packets transmitted/{print $4}')
155             if [ "${responses:-0}" -gt 0 ]; then
156                 reason="wpa-personal-lan-reachable" allow="true"
157             else
158                 reason="wpa-personal-ap-isolated" allow="false"
159             fi
160         fi
161     fi

```

```

161 else
162     # Unknown security type fail safe
163     reason="unknown-security-type" allow="false"
164 fi
165
166 echo "[$(date -u +%FT%TZ)] P2P detect: security='${security:-none}' allow=${allow} (reason: ${reason})"
167 echo "$allow" > "$P2P_OVERRIDE_FILE"
168 write_host_ips
169 }
170
171 # Run once on startup
172 detect_lan_p2p_mode
173
174 # Listener 1: WAKE events via unified log
175 # Predicate picks up:
176 # * "didWake"           regular kernel wake events (lid open, RTC alarm)
177 # * "Wake from Sleep"   powerd-driven normal wake
178 # * "DarkWake"          clamshell-display wake that didn't fully wake the
179 #                       Mac (relevant when lid opens during display sleep)
180 # * "Waking from"       generic catch-all for variations in newer macOS
181 # `[c]` makes the match case-insensitive (the unified log uses mixed-case
182 # phrases). --info reduces noise by dropping debug/trace log levels.
183 # Subsystem-based predicate catches every powermanagement emission (willSleep,
184 # didWake, didDim, didUndim, DarkWake, etc.) regardless of message phrasing
185 # across kernel versions. Phrasing-based predicates ('didWake', 'Wake from Sleep')
186 # are fragile across macOS releases. We accept the extra log noise because the
187 # run_recover() debounce + marker check filter out anything that isn't a real
188 # gateway-relevant event.
189 (
190     # Reconnect loop: macOS rotates the unified log buffer periodically; when
191     # that happens, this `log stream` subscriber's pipe EOFs, the inner
192     # `while read` consumer terminates, and without this wrapper the listener
193     # would be silently dead for the rest of the watcher's lifetime.
194     while ;; do
195         log stream --predicate \
196             'subsystem == "com.apple.powermanagement"' \
197             --style compact --info 2>/dev/null \
198             | while IFS= read -r line; do
199                 [ -z "$line" ] && continue
200                 run_recover "WAKE: $(echo "$line" | head -c 100)"
201             done
202         echo "[$(date -u +%FT%TZ)] log stream subshell exited; reconnecting in 5s"
203         sleep 5
204     done
205 ) &
206
207 # Listener 2: NETWORK state changes via scutil
208 # `scutil n.watch` on a state key prints SCEventUpdate lines whenever the key
209 # changes. We watch Global/IPv4 because that's the umbrella that catches link,
210 # address, route, and DNS changes for ALL interfaces in one namespace.
211 # The pipe is kept alive by an `sleep 86400` loop so scutil never sees EOF
212 # from us (which would silently terminate the watch).
213 (
214     # Reconnect loop: scutil can exit noisily on certain state changes (rare,
215     # but observed). Without the wrapper, the network listener's pipe EOFs and
216     # every subsequent roam goes unseen until launchd restarts us.
217     while ;; do
218         (
219             # Watch both IPv4 and IPv6 umbrella state IPv6-only or dual-stack
220             # networks never fire on the IPv4 key alone, and the user's first roam
221             # in a hotel / airport / on cellular hotspot is often IPv6-first under
222             # NAT64.
223             echo "n.add State:/Network/Global/IPv4"
224             echo "n.add State:/Network/Global/IPv6"
225             echo "n.watch"
226             while true; do sleep 86400; done
227         ) | script -q /dev/null scutil 2>/dev/null \
228             | while IFS= read -r line; do
229                 case "$line" in
230                     *changedKey*|*State:/Network/Global/IPv*|*n.state*|*SCEventUpdate*)
231                         detect_lan_p2p_mode
232                         run_recover "NET: $(echo "$line" | head -c 100)"
233                     ;;
234                     *) ;;
235                 esac
236             done
237         echo "[$(date -u +%FT%TZ)] scutil pipe exited; reconnecting in 5s"
238         sleep 5
239     done
240 ) &
241 # Listener 3: TUNNEL_FAILED_CLOSED.marker Watcher

```



```

242 # Polls for the fail-closed marker dropped by the routing-fix container.
243 (
244     last_notified=0
245     while ;; do
246         if [ -f "$SCRIPT_DIR/../TUNNEL_FAILED_CLOSED.marker" ]; then
247             now=$(date +%s)
248             # Notify once every 5 minutes (300s) if it stays failed
249             if [ "$((now - last_notified))" -ge 300 ]; then
250                 osascript -e 'display notification "VPN Tunnel Health Check Failed. Internet traffic is blackholed to prevent IP leak." with title "NullExit Alert" || true
251                 last_notified=$now
252             fi
253         else
254             last_notified=0
255         fi
256         sleep 3
257     done
258 ) &
259
260 # Lifetime
261 # `wait` blocks until both background pipelines exit (which they normally
262 # never do). launchctl `bootout`/SIGTERM triggers the trap, which kills the
263 # whole process group so the sleep-forever in the scutil pipe also dies.
264 # We catch TERM/INT/HUP for explicit signals and EXIT for any normal-exit path
265 # (so listeners always get cleaned up even if `exit 0` runs somewhere).
266 # `rm -f $LOCK_FILE` first so a stale lock can never block the next launch.
267 # `pkill -TERM -P $$` then targets direct listener-children, then `kill 0`
268 # takes care of any backgrounded group-mates not reachable via the parent.
269 trap 'echo "[(date -u +%FT%TZ)] watcher.sh terminating (signal/exit=$?)"; rm -f "$LOCK_FILE" 2>/dev/null ||
true; pkill -TERM -P $$ 2>/dev/null || true; kill 0 2>/dev/null || true; exit 0' TERM INT HUP EXIT
270 wait

```

13 scripts/crypto.sh

```

1 #!/usr/bin/env bash
2 # crypto.sh - Cryptographic Integrity Check
3
4 set -euo pipefail
5 cd "$(dirname "$0")/.."
6
7 if [ "$#" -ne 1 ] || { [ "$1" != "--sign" ] && [ "$1" != "--verify" ]; }; then
8     echo "Usage: $0 --sign | --verify"
9     exit 1
10 fi
11
12 if [ ! -f .env ]; then
13     echo "Error: .env not found."
14     exit 1
15 fi
16
17 SEED=$(grep "^NULLEXIT_SEED=" .env | cut -d '=' -f2)
18 if [ -z "$SEED" ]; then
19     echo "Error: NULLEXIT_SEED not found in .env."
20     exit 1
21 fi
22
23 if [ "$1" == "--sign" ]; then
24     echo "Generating cryptographic signatures using seed..."
25     rm -f .signatures
26     for file in toggle.sh recover.sh scripts/common.sh scripts/routing-fix.sh scripts/diagnose-host-leak.sh
scripts/watcher.sh scripts/pf.conf post-rules.txt docker-compose.yml; do
27         if [ -f "$file" ]; then
28             hash=$(openssl dgst -sha256 -hmac "$SEED" "$file" | awk '{print $NF}')
29             echo "$file:$hash" >> .signatures
30             echo " [Signed] $file"
31         else
32             echo " [Skipped] $file not found"
33         fi
34     done
35     echo "Signatures saved to .signatures"
36     exit 0
37 fi
38
39 if [ "$1" == "--verify" ]; then
40     if [ ! -f .signatures ]; then
41         echo "Error: .signatures file missing. Run $0 --sign first."

```

```

42     exit 1
43 fi
44 FAIL=0
45 while IFS=: read -r file expected_hash; do
46     if [ -z "$file" ]; then continue; fi
47     if [ ! -f "$file" ]; then
48         echo "[FAIL] $file is missing!"
49         FAIL=1
50         continue
51     fi
52     actual_hash=$(openssl dgst -sha256 -hmac "$SEED" "$file" | awk '{print $NF}')
53     if [ "$actual_hash" != "$expected_hash" ]; then
54         echo "[FAIL] $file has been modified! Hash mismatch."
55         FAIL=1
56     fi
57 done < .signatures
58
59 if [ "$FAIL" -eq 1 ]; then
60     echo ""
61     echo "CRITICAL: Script integrity verification failed!"
62     echo "One or more core files have been modified without authorization."
63     echo "To authorize these changes, run: ./scripts/crypto.sh --sign"
64     echo ""
65     exit 1
66 fi
67 exit 0
68 fi

```

14 .aiignore

```

1 # Environment variables and secrets
2 .env
3 tor-bridges.txt

```

15 .claude/settings.local.json

```

1 {
2   "permissions": {
3     "allow": [
4       "Bash(grep *)"
5     ]
6   }
7 }

```

16 .gitignore

```

1 # Environment variables and secrets
2 .env
3
4 # wgcf generated files containing keys and account info
5 wgcf-account.toml
6 wgcf-profile.conf
7
8 # Tailscale persistent state containing node identity
9 tailscale-state/
10
11 # Binaries
12 wgcf
13
14 # macOS App bundles
15 *.app/
16 *.app
17
18 # AdGuard Home persistent state
19 adguard/
20 .gateway_ip
21 logs.txt
22

```

```

23 #
24 stability_incident_report.md
25 output.log
26 .DS_Store
27 *.desktop
28 blocked.log
29 wtf.md
30 host-leak-diagnostic-*.txt
31 nullexit_review.md
32 nullexit_unified.tex
33 nullexit_unified.txt
34
35 nullexit_unified.synctex.gz
36 tor-bridges.txt
37
38 # Runtime state files
39 .host_ips
40 .lan_p2p_detected
41 .signatures
42 TUNNEL_FAILED_CLOSED.marker
43
44 # Python cache
45 __pycache__/_
46 *.py[cod]
47
48 # LaTeX build artifacts
49 *.aux
50 *.log
51 *.out
52 *.toc
53 *.xdv
54 *.fdb_latexmk
55 *.fls
56 *.synctex*
57 refactor.md

```

17 Recover-Gateway.applescript

```

1 tell application "Finder" to set scriptDir to POSIX path of (container of (path to me) as alias)
2 try
3     do shell script "true" with administrator privileges
4 on error
5     return
6 end try
7 tell application "Terminal"
8     activate
9     do script "cd " & quoted form of scriptDir & "; bash recover.sh"
10 end tell

```

18 Toggle-Gateway.applescript

```

1 tell application "Finder" to set scriptDir to POSIX path of (container of (path to me) as alias)
2 try
3     do shell script "true" with administrator privileges
4 on error
5     return
6 end try
7 tell application "Terminal"
8     activate
9     do script "cd " & quoted form of scriptDir & "; ./toggle.sh"
10 end tell

```

19 benchmarks/benchmark.sh

```

1 #!/usr/bin/env bash
2
3 # benchmark.sh
4 # Automates the full benchmarking suite (both Python download test and Lighthouse HTML test)

```

```

5 # Runs the suite with Gateway ON, toggles OFF, runs again, then toggles back ON.
6
7 set -e
8
9 # Change to project root so we can access toggle.sh reliably
10 cd "$(dirname "$0")/.."
11
12 # Ensure jq is installed
13 if ! command -v jq &> /dev/null; then
14     echo "Error: 'jq' is not installed. Please run 'brew install jq'"
15     exit 1
16 fi
17
18 run_lighthouse() {
19     local url=$1
20     local file_prefix=$2
21     echo " [Lighthouse] Testing $url ..."
22
23     # Run lighthouse headlessly, silencing standard output
24     npx -y lighthouse "$url" --chrome-flags="--headless" --only-categories=performance --output=json --output-
25     path="${file_prefix}.json" >/dev/null 2>&1
26
27     # Parse the results
28     local score=$(jq -r '.categories.performance.score' "${file_prefix}.json")
29     local tti=$(jq -r '.audits["interactive"].displayValue' "${file_prefix}.json")
30     local si=$(jq -r '.audits["speed-index"].displayValue' "${file_prefix}.json")
31
32     # Convert score to percentage
33     local score_pct=$(awk "BEGIN {print $score*100}")
34
35     echo "    -> Performance Score: ${score_pct}%"
36     echo "    -> Time to Interactive: $tti"
37     echo "    -> Speed Index: $si"
38
39     # Cleanup
40     rm -f "${file_prefix}.json"
41 }
42
43 echo "=====
44 echo " PHASE 1: Testing with Gateway ON"
45 echo "=====
46 # 1. Run raw Python download test
47 python3 benchmarks/test_load.py
48
49 # 2. Run Lighthouse HTML render test
50 echo ""
51 echo "Starting Lighthouse HTML Tests (this takes ~30-60s per site)..."
52 run_lighthouse "https://www.cnet.com/" "on_cnet"
53 run_lighthouse "https://www.independent.co.uk/" "on_ind"
54
55 echo ""
56 echo "=====
57 echo " PHASE 2: Toggling Gateway OFF"
58 echo "=====
59 ./toggle.sh
60 echo "Waiting 10 seconds for macOS Wi-Fi to stabilize after DNS reset..."
61 sleep 10
62
63
64 echo ""
65 echo "=====
66 echo " PHASE 3: Testing with Gateway OFF"
67 echo "=====
68 # 1. Run raw Python download test
69 python3 benchmarks/test_load.py
70
71 # 2. Run Lighthouse HTML render test
72 echo ""
73 echo "Starting Lighthouse HTML Tests (this takes ~30-60s per site)..."
74 run_lighthouse "https://www.cnet.com/" "off_cnet"
75 run_lighthouse "https://www.independent.co.uk/" "off_ind"
76
77
78 echo ""
79 echo "=====
80 echo " PHASE 4: Restoring Gateway ON"
81 echo "=====
82 ./toggle.sh
83
84 echo ""

```

```

85 echo "=====
86 echo " BENCHMARKS COMPLETE!"
87 echo "You can scroll up to compare Phase 1 (AdGuard ON) vs Phase 3 (AdGuard OFF)."
```

20 benchmarks/test_load.py

```

1 import urllib.request
2 import urllib.error
3 import time
4 from html.parser import HTMLParser
5 import concurrent.futures
6
7 class ResourceParser(HTMLParser):
8     def __init__(self):
9         super().__init__()
10        self.urls = set()
11
12    def handle_starttag(self, tag, attrs):
13        attrs = dict(attrs)
14        url = None
15        if tag in ['img', 'script']:
16            url = attrs.get('src')
17        elif tag == 'link' and attrs.get('rel') == 'stylesheet':
18            url = attrs.get('href')
19
20        if url and url.startswith('http'):
21            self.urls.add(url)
22
23 def fetch_url(url):
24     try:
25         req = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
26         urllib.request.urlopen(req, timeout=3)
27         return True
28     except Exception:
29         return False
30
31 def measure_url(target_url):
32     print(f"\nTesting {target_url} Load...")
33     start_total = time.time()
34
35     # Fetch main HTML
36     try:
37         req = urllib.request.Request(target_url, headers={'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36'})
38         response = urllib.request.urlopen(req, timeout=5)
39         html = response.read().decode('utf-8', errors='ignore')
40     except Exception as e:
41         print(f"Failed to fetch {target_url}: {e}")
42         return
43
44     parser = ResourceParser()
45     parser.feed(html)
46     urls = list(parser.urls)
47     print(f"Found {len(urls)} subresources to fetch (scripts, images, css)...")
48
49     # Concurrently fetch subresources
50     success_count = 0
51     fail_count = 0
52     max_threads = min(64, max(1, len(urls)))
53     with concurrent.futures.ThreadPoolExecutor(max_workers=max_threads) as executor:
54         results = list(executor.map(fetch_url, urls))
55
56     success_count = sum(1 for r in results if r)
57     fail_count = len(results) - success_count
58
59     total_time = time.time() - start_total
60     print(f"Time taken: {total_time:.2f} seconds")
61     print(f"Successfully fetched: {success_count}, Failed/Blocked: {fail_count}")
62
63 urls_to_test = [
64     "https://www.dailymail.co.uk/home/index.html",
65     "https://www.cnet.com/",
66     "https://www.independent.co.uk/"
67 ]
68
```

```
69 for u in urls_to_test:
70     measure_url(u)
```

21 black_list.txt

```
1 ad.doubleclick.net
2 adclick.g.doubleclick.net
3 adeventtracker.spotify.com
4 adfstat.yandex.ru
5 ads-api.tiktok.com
6 ads-api.twitter.com
7 ads-fa.spotify.com
8 ads-sg.tiktok.com
9 ads.tiktok.com
10 adserver.unityads.unity3d.com
11 adsfs.oppomobile.com
12 an.facebook.com$important
13 connect.facebook.net$important
14 pixel.facebook.com$important
15 analytics.query.yahoo.com
16 analytics.s3.amazonaws.com
17 analytics.spotify.com
18 analyticsengine.s3.amazonaws.com
19 appmetrica.yandex.ru
20 audio2.spotify.com
21 bdapi-ads.realmemobile.com
22 bdapi-in-ads.realmemobile.com
23 bounceexchange.com
24 bs.serving-sys.com
25 business-api.tiktok.com
26 ck.ads.oppomobile.com
27 crashdump.spotify.com
28 data.ads.oppomobile.com
29 data.mistat.india.xiaomi.com
30 data.mistat.rus.xiaomi.com
31 data.mistat.xiaomi.com
32 desktop.spotify.com
33 doubleclick.net
34 ds.metric.spotify.com
35 gemini.yahoo.com
36 googleads.g.doubleclick.net
37 googleadservices.com
38 googlesyndication.com
39 grs.hicloud.com
40 gtm.spotify.com
41 iot-eu-logser.realm.com
42 iot-logser.realm.com
43 log.byteoversea.com
44 log.fc.yahoo.com
45 log.spotify.com
46 m.doubleclick.net
47 mediavisor.doubleclick.net
48 metrics.spotify.com
49 metrika.yandex.ru
50 omaze.com
51 pagead2.googlesyndication.com
52 securepubads.g.doubleclick.net
53 static.doubleclick.net
54 stats.g.doubleclick.net
55 tracking.rus.miui.com
56 udc.m.yahoo.com
57 v.jwpcdn.com
58 webblb-wg.gslb.spotify.com
59 webview.unityads.unity3d.com
60 diagassets.apple.com
61 iphonesubmissions.apple.com
62 radarsubmissions.apple.com
63 metrics.apple.com
64 xp.apple.com
65 heads-fa.spotify.com
66 heads4.spotify.com
67 pixel.spotify.com
68 segs.spotify.com
69 audio-fa.spotify.com
70 events.data.microsoft.com
71 datarouter-prod.ak.epicgames.com
```

```

72 nel.cloudflare.com
73 eu-mobile.events.data.microsoft.com
74 stocks-analytics-events.apple.com

```

22 cloud-init.yaml

```

1 #cloud-config
2 # Generic Cloud-Init Bootstrap Script for Remote Exit Nodes (Ubuntu/Debian)
3 # Paste this into the "User Data" or "Initialization Script" section when creating a VPS.
4
5 package_update: true
6 package_upgrade: true
7
8 packages:
9   - apt-transport-https
10  - ca-certificates
11  - curl
12  - gnupg
13  - lsb-release
14  - git
15  - python3
16  - python3-pip
17
18 runcmd:
19   # 1. Install Docker natively
20   - curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-
      archive-keyring.gpg
21   - echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
      https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/
      docker.list > /dev/null
22   - sudo apt-get update
23   - sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
24
25   # 2. Add default user (ubuntu) to docker group
26   - usermod -aG docker ubuntu || true
27   - usermod -aG docker debian || true
28
29   # 3. Create the installation directory
30   - mkdir -p /opt/nullexit
31   - chown -R ubuntu:ubuntu /opt/nullexit || chown -R debian:debian /opt/nullexit
32
33   # 4. Enable IPv4 and IPv6 forwarding for VPN routing
34   - echo "net.ipv4.ip_forward=1" >> /etc/sysctl.d/99-tailscale.conf
35   - echo "net.ipv6.conf.all.forwarding=1" >> /etc/sysctl.d/99-tailscale.conf
36   - sysctl -p /etc/sysctl.d/99-tailscale.conf
37
38 final_message: "The remote exit node has been fully bootstrapped. SSH in, clone the repo to /opt/nullexit, add
      your .env, and run ./setup-linux.sh!"

```

23 docker/tor/Dockerfile

```

1 FROM debian:bookworm-slim
2
3 RUN apt-get update && \
4     apt-get install -y tor obfs4proxy gosu bash && \
5     rm -rf /var/lib/apt/lists/*
6
7 COPY entrypoint.sh /entrypoint.sh
8 RUN chmod +x /entrypoint.sh
9
10 ENTRYPOINT ["/entrypoint.sh"]

```

24 docker/tor/entrypoint.sh

```

1 #!/bin/bash
2 set -e
3
4 TORRC="/etc/tor/torrc"

```



```

5 mkdir -p /etc/tor /var/lib/tor
6
7 # Base configuration: SOCKS 9050, Control 9051, TransPort 9040, DNSPort 5353 all bound to
8 # 0.0.0.0 (required for Docker/Colima host port-mapping). The ControlPort is secured via a
9 # dynamic HashedControlPassword (added below when TOR_PASSWORD is set); see devref.md 15.10.2.
10 cat <<EOF > $TORRC
11 SocksPort 0.0.0.0:9050
12 ControlPort 0.0.0.0:9051
13 CookieAuthentication 0
14 Log notice stdout
15 DataDirectory /var/lib/tor
16
17 # Transparent proxying & DNS resolution for .onion mapping
18 TransPort 0.0.0.0:9040
19 DNSPort 0.0.0.0:5353
20 AutomapHostsOnResolve 1
21 VirtualAddrNetworkIPv4 198.18.0.0/15
22 EOF
23
24 if [ -n "$TOR_PASSWORD" ]; then
25     HASH=$(tor --hash-password "$TOR_PASSWORD" | tail -n 1)
26     echo "HashedControlPassword $HASH" >> $TORRC
27 fi
28
29 if [ -n "$TOR_EXCLUDE_EXIT_NODES" ]; then
30     echo "ExcludeExitNodes $TOR_EXCLUDE_EXIT_NODES" >> $TORRC
31     echo "StrictNodes 1" >> $TORRC
32 fi
33
34 if [ "${TOR_USE_BRIDGES:-false}" = "true" ]; then
35     BRIDGE_FILE="${TOR_BRIDGE_LINES_FILE:-/tor-bridges.txt}"
36
37     if ! grep -vE "^[[:space:]]*#" "$BRIDGE_FILE" | grep -q '^[[:space:]]'; then
38         MSG="[$(date '+%Y-%m-%d %H:%M:%S')] [CRITICAL] [Tor Proxy] TOR_USE_BRIDGES is enabled, but no bridge
39         lines found at $BRIDGE_FILE. Proxy startup ABORTED. Get bridges from https://bridges.torproject.org."
40         echo "$MSG"
41         if [ -w "/output.log" ]; then
42             echo "$MSG" >> /output.log
43         fi
44         # Sleep indefinitely to prevent docker-compose 'unless-stopped' from triggering a crash loop
45         exec sleep infinity
46     fi
47
48     echo "UseBridges 1" >> $TORRC
49     echo "ClientTransportPlugin obfs4 exec /usr/bin/obfs4proxy" >> $TORRC
50
51     while IFS= read -r line; do
52         # Ignore comments and empty lines
53         [[ -z "$line" || "$line" == \#* ]] && continue
54         echo "Bridge $line" >> $TORRC
55     done < "$BRIDGE_FILE"
56 fi
57
58 # Debian tor package uses 'debian-tor' user
59 chown -R debian-tor:debian-tor /var/lib/tor /etc/tor
60
61 exec gosu debian-tor /usr/bin/tor -f $TORRC

```

25 launchd/com.nullexit.wake-recovery.plist

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
3 <plist version="1.0">
4 <dict>
5     <key>Label</key>
6     <string>com.nullexit.wake-recovery</string>
7
8     <!-- WorkingDirectory + relative program arg keeps this plist install-agnostic.
9          The source-of-truth for the install path is __NULLEXIT_HOME__, which
10         setup.sh or the user must substitute with their absolute nullexit
11         install root at install time. Program arg is `scripts/watcher.sh`
12         RELATIVE to WorkingDirectory, so launchd's evaluated cwd IS the
13         install root and BASH_SOURCE inside scripts/watcher.sh still
14         resolves to the real <install>/scripts/watcher.sh, preserving the
15         SCRIPT_DIR pattern. After install-time `sed -i '' 's|__NULLEXIT_HOME__|<abs_path>|'`,
16         the plist is portable to any clone location. -->

```

```

17 <key>WorkingDirectory</key>
18 <string>__NULLEXIT_HOME__</string>
19
20 <key>ProgramArguments</key>
21 <array>
22 <string>/bin/bash</string>
23 <string>scripts/watcher.sh</string>
24 </array>
25
26 <!-- Start the daemon immediately on launchctl load (instead of waiting
27 for the next user login). Combined with keepAlive below this means
28 once installed: load once, runs forever across logouts/reboots. -->
29 <key>RunAtLoad</key>
30 <true/>
31
32 <!-- Restart policy:
33 SuccessfulExit=false SIGTERM / clean exit does NOT relaunch (so
34 `launchctl bootout` cleanly stops, not loops).
35 Crashed=false HARD crashes ALSO do NOT relaunch. We've seen
36 that repeated sleep/wake cycles on macOS
37 Sonoma+ accumulate orphan watcher.sh PIDs
38 because SIGCONT/resume doesn't reliably kill
39 the prior instance. The watcher.sh script
40 now enforces single-instance via a manual
41 PID-file and process-identity check at the
42 top, so a re-launch on crash is
43 actively dangerous (it would skip the lock
44 and exit; better to surface the crash in
45 logs than to silently 0-process). -->
46 <key>KeepAlive</key>
47 <dict>
48 <key>SuccessfulExit</key>
49 <false/>
50 <key>Crashed</key>
51 <false/>
52 </dict>
53
54 <!-- Background hint to App Nap so it doesn't get suspended when
55 the user is inactive. We're the entire reason this LaunchAgent
56 needs to NOT nap. -->
57 <key>ProcessType</key>
58 <string>Background</string>
59
60 <!-- launchd-level stdout/stderr (the bash launcher shell). The script
61 itself writes to /tmp/nullexit-watcher.log via exec >>. These
62 separate channels help us tell "launchd crashed" from "the bash
63 wrapper piped to recover.sh errored". -->
64 <key>StandardOutPath</key>
65 <string>/tmp/nullexit-watcher.out.log</string>
66 <key>StandardErrorPath</key>
67 <string>/tmp/nullexit-watcher.err.log</string>
68 </dict>
69 </plist>

```

26 post-rules.txt

```

1 iptables -A FORWARD -i tailscale0 -o tun0 -j ACCEPT
2 iptables -A FORWARD -i tun0 -o tailscale0 -j ACCEPT
3 iptables -A INPUT -i tailscale0 -j ACCEPT
4 iptables -A INPUT -i eth0 -p udp --dport 41642 -j ACCEPT
5 iptables -t nat -A POSTROUTING -o tun0 -j MASQUERADE
6 # Clamp MSS to 1120. With Tailscale overhead on top of WARP (each WireGuard encapsulation adds ~60 bytes),
7 # the safe MSS for double-tunneled traffic is 1120 to prevent strict-MSS endpoints from stalling.
8 # NOTE: this 1120 integer is rewritten at boot by toggle.sh from GATEWAY_MSS in .env
9 # (Gluetun's parser can't interpolate variables) change GATEWAY_MSS in .env, not this line.
10 iptables -t mangle -A POSTROUTING -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --set-mss 1120
11
12 iptables -t nat -A PREROUTING -i tailscale0 -p udp --dport 53 -j REDIRECT --to-port 5335
13 iptables -t nat -A PREROUTING -i tailscale0 -p tcp --dport 53 -j REDIRECT --to-port 5335
14
15 # Also redirect DNS from Docker bridge interface so the host can reach AdGuard via localhost:53
16 iptables -t nat -A PREROUTING -i eth0 -p udp --dport 53 -j REDIRECT --to-port 5335
17 iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 53 -j REDIRECT --to-port 5335
18
19 # Block IPv6 exit-node traffic to prevent leakage (Gluetun/WARP is IPv4-only)
20 ip6tables -A FORWARD -i tailscale0 -j DROP

```

```

21
22 # Block DNS-over-TLS (Port 853) to force Android Private DNS to fallback to Port 53
23 iptables -A FORWARD -i tailscale0 -p tcp --dport 853 -j REJECT --reject-with tcp-reset
24 iptables -A FORWARD -i tailscale0 -p udp --dport 853 -j REJECT
25
26 # Block QUIC (UDP 443) to force Facebook/Google to fallback to TCP.
27 # UDP bypasses TCP MSS clamping, causing fragmentation. Over weak Wi-Fi (far from gateway),
28 # dropping a single fragment destroys the entire packet, stalling image downloads.
29 iptables -A FORWARD -i tailscale0 -p udp --dport 443 -j REJECT

```

27 scripts/Dockerfile.routing-fix

```

1 FROM golang:1.22-alpine AS builder
2 WORKDIR /app
3 COPY logger/ ./logger/
4 RUN cd logger && go build -o logger main.go
5
6 FROM alpine:3.20
7 RUN apk add --no-cache iptables iproute2 curl ipset
8 COPY --from=builder /app/logger/logger /usr/local/bin/nullexit-logger
9 COPY routing-fix.sh /routing-fix.sh
10 CMD ["sh", "/routing-fix.sh"]

```

28 scripts/Dockerfile.rule-compiler

```

1 FROM golang:1.22-alpine AS builder
2 WORKDIR /app
3 COPY rule-compiler/ ./rule-compiler/
4 RUN cd rule-compiler && go build -o sync-rules main.go
5
6 FROM alpine:3.20
7 COPY --from=builder /app/rule-compiler/sync-rules /usr/local/bin/sync-rules
8 WORKDIR /app
9 CMD ["sync-rules"]

```

29 scripts/Toggle-Gateway.bat

```

1 <# :
2 @echo off
3 powershell -NoProfile -ExecutionPolicy Bypass -Command "Invoke-Expression ([System.IO.File]::ReadAllText('%~f0
4 ') -replace '^(?s).*?#\s*>\s*\r?\n','')
5 exit /b
6 #>
7 # Toggle-Gateway.bat (Hybrid Batch-PowerShell Toggler) nullexit Gateway Toggler for Windows
8 # Mirrors the sophisticated error handling, checks, and automation of toggle.sh
9
10 # 1. Administrator Check & Elevation
11 $isAdmin = ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([
12 Security.Principal.WindowsBuiltInRole]::Administrator)
13 if (-not $isAdmin) {
14 Write-Host "Requesting Administrator privileges..." -ForegroundColor Yellow
15 Start-Process PowerShell -ArgumentList "-NoProfile -ExecutionPolicy Bypass -Command `Invoke-Expression ([
16 System.IO.File]::ReadAllText('$PSCommandPath') -replace '^(?s).*?#\s*>\s*\r?\n','')`" -Verb RunAs
17 Exit
18 }
19
20 # Set console output encoding to UTF8 for clean symbols
21 [Console]::OutputEncoding = [System.Text.Encoding]::UTF8
22 Clear-Host
23
24 Write-Host "" -ForegroundColor Green
25 Write-Host " nullexit Windows Toggler " -ForegroundColor Green
26 Write-Host "" -ForegroundColor Green
27 Write-Host ""
28
29 # 2. Helpers
30 function Reset-HostDNS {
31 Write-Host "Restoring default DNS (DHCP/DHCP-assigned DNS)..." -ForegroundColor Cyan

```

```

29     $ActiveAdapters = Get-NetAdapter | Where-Object { $_.Status -eq 'Up' }
30     foreach ($adapter in $ActiveAdapters) {
31         Write-Host "    -> Resetting DNS on adapter: $($adapter.Name)" -ForegroundColor Gray
32         Set-DnsClientServerAddress -InterfaceIndex $adapter.InterfaceIndex -ResetServerAddresses -ErrorAction
33         SilentlyContinue
34     }
35 }
36
37 function Hijack-HostDNS ($ip) {
38     Write-Host "Hijacking DNS to route through AdGuard ($ip)..." -ForegroundColor Yellow
39     $ActiveAdapters = Get-NetAdapter | Where-Object { $_.Status -eq 'Up' }
40     foreach ($adapter in $ActiveAdapters) {
41         Write-Host "    -> Pointing DNS to $ip on adapter: $($adapter.Name)" -ForegroundColor Gray
42         Set-DnsClientServerAddress -InterfaceIndex $adapter.InterfaceIndex -ServerAddresses $ip -ErrorAction
43         SilentlyContinue
44     }
45 }
46
47 function Get-HostTailscalePath {
48     $paths = @(
49         "env:ProgramFiles\Tailscale\tailscale.exe",
50         "env:ProgramFiles (x86)\Tailscale\tailscale.exe",
51         "tailscale.exe"
52     )
53     foreach ($path in $paths) {
54         if (Test-Path $path) { return $path }
55     }
56     return $null
57 }
58
59 # 3. Detect State
60 # Prevent deadlocks by resetting DNS to default temporarily during checks
61 Reset-HostDNS
62
63 Write-Host "Checking Docker daemon status..." -ForegroundColor Cyan
64 & docker info >$null 2>&1
65 if ($LASTEXITCODE -ne 0) {
66     Write-Host "Docker is not running. Starting Docker Desktop..." -ForegroundColor Yellow
67     $dockerPath = "C:\Program Files\Docker\Docker\Docker Desktop.exe"
68     if (Test-Path $dockerPath) {
69         Start-Process $dockerPath
70         Write-Host "Waiting for Docker daemon to become responsive..." -ForegroundColor Gray
71         $timeout = 60
72         while ($timeout -gt 0) {
73             Start-Sleep -Seconds 2
74             & docker info >$null 2>&1
75             if ($LASTEXITCODE -eq 0) {
76                 Write-Host "Docker is ready!" -ForegroundColor Green
77                 break
78             }
79             $timeout -= 2
80         }
81         if ($timeout -le 0) {
82             Write-Error "Docker failed to start in time. Aborting."
83             Exit 1
84         }
85     } else {
86         Write-Error "Docker Desktop executable not found at standard path. Please start Docker manually."
87         Exit 1
88     }
89 }
90
91 # Detect if gateway is already running
92 $runningWarp = docker compose ps --status running --format json | ConvertFrom-Json -ErrorAction
93     SilentlyContinue | Where-Object { $_.Service -eq "warp" }
94 $isGatewayActive = ($null -ne $runningWarp)
95
96 # 4. Execution
97 if ($isGatewayActive) {
98     # STOP PATH
99     Write-Host "Gateway is currently RUNNING. Disabling now..." -ForegroundColor Yellow
100     Write-Host ""
101
102     # Disconnect host Tailscale from exit node if installed
103     $tsCli = Get-HostTailscalePath
104     if ($null -ne $tsCli) {
105         Write-Host "Disconnecting host Tailscale from exit node..." -ForegroundColor Cyan
106         & $tsCli up --exit-node= --accept-dns=true >$null 2>&1
107     }
108
109     Write-Host "Stopping Docker containers..." -ForegroundColor Cyan

```

```

107     docker compose down -t 5
108
109     Reset-HostDNS
110     Write-Host "Gateway has been successfully STOPPED." -ForegroundColor Green
111 } else {
112     # START PATH
113     Write-Host "Gateway is currently STOPPED. Enabling now..." -ForegroundColor Yellow
114     Write-Host ""
115
116     # Clean up corrupted AdGuardHome configurations
117     $confFile = "adguard\conf\AdGuardHome.yaml"
118     if (Test-Path $confFile) {
119         $fileSize = (Get-Item $confFile).Length
120         if ($fileSize -eq 0) {
121             Remove-Item $confFile -Force
122             Write-Host "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop." -
123                 ForegroundColor Yellow
124         }
125     }
126
127     Write-Host "Starting Docker containers..." -ForegroundColor Cyan
128     docker compose up -d
129
130     Write-Host "Waiting for container's Tailscale connection to offer Exit Node..." -ForegroundColor Cyan
131     Write-Host " (This can take up to 60 seconds..." -ForegroundColor Gray
132
133     $elapsed = 0
134     $maxWait = 60
135     $resolvedIP = $null
136
137     while ($elapsed -lt $maxWait) {
138         $statusJson = docker compose exec -T tailscale tailscale status --json 2>$null
139         if ($null -ne $statusJson) {
140             $status = $statusJson | ConvertFrom-Json -ErrorAction SilentlyContinue
141             if ($null -ne $status -and $null -ne $status.Self) {
142                 # Check if it has a valid Tailscale IP and is running
143                 if ($status.Self.Online -and $status.Self.TailscaleIPs.Count -gt 0) {
144                     $resolvedIP = $status.Self.TailscaleIPs[0]
145                     break
146                 }
147             }
148             Start-Sleep -Seconds 2
149             $elapsed += 2
150         }
151     }
152
153     if ($null -ne $resolvedIP) {
154         Write-Host "Tailscale IP resolved: $resolvedIP" -ForegroundColor Green
155         Write-Host ""
156
157         # Configure host Tailscale client to route through this exit node
158         $tsCli = Get-HostTailscalePath
159         if ($null -ne $tsCli) {
160             Write-Host "Configuring host Tailscale to use Exit Node ($resolvedIP)..." -ForegroundColor Cyan
161             & $tsCli up --exit-node=$resolvedIP --accept-dns=false >$null 2>&1
162         }
163
164         # Hijack DNS
165         Hijack-HostDNS $resolvedIP
166         Write-Host ""
167         Write-Host "Gateway has been successfully STARTED." -ForegroundColor Green
168     } else {
169         Write-Host "Warning: Could not resolve gateway Tailscale IP. DNS hijacking skipped." -ForegroundColor
170         Red
171
172         # Fallback to .gateway_ip if it exists
173         if (Test-Path ".gateway_ip") {
174             $fallbackIP = Get-Content ".gateway_ip" -TotalCount 1
175             if (-not [string]::IsNullOrEmpty($fallbackIP)) {
176                 Write-Host "[Fallback] Using static IP from .gateway_ip: $fallbackIP" -ForegroundColor Yellow
177                 Hijack-HostDNS $fallbackIP
178                 Write-Host "Gateway STARTED (with static fallback IP)." -ForegroundColor Green
179             }
180         } else {
181             Write-Host "Gateway started in offline/unauthenticated state. Run setup.sh if this is a fresh setup
182             ." -ForegroundColor Yellow
183         }
184     }
185 }
186
187 Write-Host ""
188 Write-Host "Press any key to exit..."

```

```
185 [void]$Host.UI.RawUI.ReadKey("NoEcho, IncludeKeyDown")
```

30 scripts/check_circuits.py

```
1  #!/usr/bin/env python3
2  import socket
3  import os
4  import re
5
6  def query_tor(port, command, password=""):
7      try:
8          s = socket.socket()
9          s.settimeout(2)
10         s.connect(('127.0.0.1', port))
11         s.sendall(f"AUTHENTICATE \'{password}\'\\r\\n{command}\\r\\nQUIT\\r\\n".encode())
12         response = b""
13         while True:
14             chunk = s.recv(4096)
15             if not chunk:
16                 break
17             response += chunk
18         return response.decode()
19     except Exception as e:
20         return f"Error connecting to Tor Control Port: {e}"
21
22 def get_ports():
23     ports = {}
24     ports_file = "/tmp/nullexit-ports.env"
25     if os.path.exists(ports_file):
26         with open(ports_file, 'r') as f:
27             for line in f:
28                 if line.startswith("export "):
29                     # Split at first '=' to handle potential multiple '=' in value
30                     parts = line.replace("export ", "").strip().split("=", 1)
31                     if len(parts) == 2:
32                         key, val = parts
33                         try:
34                             ports[key] = int(val)
35                         except ValueError:
36                             ports[key] = val
37     return ports
38
39 def get_password(ports):
40     # 1. Try ports.env
41     password = ports.get("TOR_PASSWORD")
42     if password:
43         return password
44
45     # 2. Try .env
46     if os.path.exists(".env"):
47         try:
48             with open(".env", "r") as f:
49                 for line in f:
50                     if line.startswith("ADGUARD_PASSWORD="):
51                         return line.split("=", 1)[1].strip().strip('"\'')
52                     if line.startswith("TOR_PASSWORD="):
53                         return line.split("=", 1)[1].strip().strip('"\'')
54         except Exception:
55             pass
56
57     # 3. Fallback
58     return "nullexit"
59
60 def main():
61     ports = get_ports()
62     control_port = ports.get("TOR_CONTROL_PORT", 9051)
63     password = get_password(ports)
64
65     print(f"Connecting to Tor Control Port on 127.0.0.1:{control_port}...")
66     status = query_tor(control_port, "GETINFO circuit-status", password)
67
68     if "Error" in status:
69         print(status)
70         return
71
72     print("\nActive Tor Circuits:")
```

```

73     circuits = re.findall(r"(\d+)\s+BUILT\s+(.?)\s+PURPOSE", status)
74     if not circuits:
75         print("No active built circuits found yet. Tor might still be bootstrapping.")
76         return
77
78     for circ_id, path_str in circuits:
79         print(f"\nCircuit #{circ_id}:")
80         nodes = path_str.split(",")
81         for idx, node in enumerate(nodes):
82             fingerprint = node.split("~")[0].replace("$", "")
83             nickname = node.split("~")[1] if "~" in node else "unknown"
84
85             # Fetch node IP
86             ns_info = query_tor(control_port, f"GETINFO ns/id/{fingerprint}", password)
87             ip = "unknown"
88             for line in ns_info.split("\n"):
89                 if line.startswith("r "):
90                     parts = line.split(" ")
91                     if len(parts) >= 7:
92                         ip = parts[6]
93                         break
94
95             # Fetch country
96             country = "unknown"
97             if ip != "unknown":
98                 country_info = query_tor(control_port, f"GETINFO ip-to-country/{ip}", password)
99                 for line in country_info.split("\n"):
100                     if line.startswith("250-ip-to-country/"):
101                         country = line.split("=")[1].strip().upper()
102                         break
103
104             role = "Guard" if idx == 0 else ("Exit" if idx == len(nodes) - 1 else "Middle")
105             print(f" [{role}] {nickname} ({ip}) - Country: {country}")
106
107 if __name__ == "__main__":
108     main()

```

31 scripts/diagnose-host-leak.sh

```

1  #!/bin/bash
2  # scripts/diagnose-host-leak.sh  nullexit host-routing diagnostic
3  #
4  # Symptom this addresses: tests like https://www.whatismyip.com on the
5  # *HOST MAC* return the underlying physical ISP/ASN
6  # local ISP / campus network") even though the nullexit gateway is active and remote
7  # tailnet clients correctly egress via Cloudflare WARP. The container and
8  # the gateway itself are healthy  only the host's own routing is leaking.
9  #
10 # This script runs 7 checks in sequence, classifies the result into ONE
11 # of the three known leak scenarios, writes the full report to a
12 # timestamped file in the project root, and prints a verdict + a
13 # ready-to-run fix command on stdout. Pass --fix to apply the matching
14 # remediation in the same invocation and re-verify egress afterwards.
15 # See devref.md 10.30 for the deep-dive rationale.
16 #
17 # Usage:
18 #   bash scripts/diagnose-host-leak.sh           # diagnose + write report
19 #   bash scripts/diagnose-host-leak.sh --fix      # diagnose + fix + re-verify
20 #   bash scripts/diagnose-host-leak.sh --watch   # full baseline then loop checks every 60s
21 #   bash scripts/diagnose-host-leak.sh --watch 30 # same, with custom interval (seconds)
22 #   bash scripts/diagnose-host-leak.sh --help    # this message
23 #
24 # Multiple runs produce timestamped files (one per run) so historical
25 # reports can be diffed. The newest file is the most recent.
26
27 set -uo pipefail
28 # NOTE: errexit (set -e) is intentionally NOT enabled. Several checks
29 # below are EXPECTED to fail and that's diagnostic data (e.g. `tailscale
30 # status` returns non-zero when the daemon is wedged). Every command
31 # pipes to a helper that records the exit code without killing the script.
32
33 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
34 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
35 TIMESTAMP="$(date -u +%Y%m%dT%H%M%SZ)"
36 source "$SCRIPT_DIR/common.sh"
37 LOG_FILE="$PROJECT_ROOT/output.log"

```



```

38 REPORT_FILE="$PROJECT_ROOT/host-leak-diagnostic-$TIMESTAMP.txt"
39
40 # Argument parsing
41 DO_FIX=false
42 DO_WATCH=false
43 WATCH_INTERVAL=60
44 WATCH_LOG="$PROJECT_ROOT/host-leak-watch.log"
45 case "${1:-}" in
46     --fix) DO_FIX=true ;;
47     --watch)
48         DO_WATCH=true
49         # Optional second argument: custom interval in seconds
50         if [ -n "${2:-}" ] && [[ "$2" =~ ^[0-9]+$ ]]; then
51             WATCH_INTERVAL="$2"
52         fi
53         ;;
54     --help|-h)
55         sed -n '2,31p' "$0"
56         exit 0
57         ;;
58     *)
59         : # default: diagnose only
60         ;;
61     *)
62         echo "Unknown argument: $1" >&2
63         echo "Run with --help for usage." >&2
64         exit 2
65         ;;
66 esac
67
68 # Colours (auto-disabled when stdout is not a tty, e.g. piped)
69 if [ -t 1 ]; then
70     RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
71     BOLD='\033[1m'; NC='\033[0m'
72 else
73     RED=''; GREEN=''; YELLOW=''; BOLD=''; NC=''
74 fi
75
76 # Output helpers write to BOTH stdout and the report file
77 exec 3>> "$REPORT_FILE" || {
78     printf '\n[FATAL] cannot open %s for write\n' "$REPORT_FILE" >&2
79     exit 1
80 }
81 section() {
82     local title="$1"
83     printf '\n%b %s %b\n' "$BOLD" "$title" "$NC" >&3
84     printf '\n%b %s %b\n' "$BOLD" "$title" "$NC"
85 }
86 line() { printf ' %b\n' "$1" >&3; printf ' %b\n' "$1"; }
87 ok() { line "${GREEN}${NC} $1"; }
88 warn() { line "${YELLOW}${NC} $1"; }
89 fail() { line "${RED}${NC} $1"; }
90 note() { line "(no ${1})"; }
91
92 # Watch-mode header (only shown when entering watch loop)
93 if [ "$DO_WATCH" = "true" ]; then
94     echo ""
95     echo ""
96     echo " n u l l e x i t   w a t c h   m o d e "
97     echo ""
98     echo " Interval:          ${WATCH_INTERVAL}s"
99     echo " Watch log:         $WATCH_LOG"
100     echo ""
101 else
102     echo ""
103     echo " n u l l e x i t   h o s t - r o u t i n g   d i a g n o s t i c "
104     echo ""
105     echo " Timestamp (UTC): $TIMESTAMP"
106     echo " Project root:    $PROJECT_ROOT"
107     echo " Report file:     $REPORT_FILE"
108     echo " Mode:            ${([ "$DO_FIX" = true ] && echo "diagnose + apply fix" || echo "diagnose only (pass --
109         fix to remediate))}"
109     echo ""
110     echo "" >&3
111     echo "" >&3
112     echo " n u l l e x i t   h o s t - r o u t i n g   d i a g n o s t i c " >&3
113     echo "" >&3
114     echo " Timestamp (UTC): $TIMESTAMP" >&3
115     echo " Mode:            ${([ "$DO_FIX" = true ] && echo "diagnose + apply fix" || echo "diagnose only")}" >&3
116 fi
117

```

```

118 # Always add Homebrew to PATH (same as toggle.sh / recover.sh)
119 export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"
120
121 # Pure-bash timeout (macOS lacks GNU `timeout`)
122 run_with_timeout() {
123     local timeout_sec="$1"
124     shift
125     if [ "$#" -eq 0 ]; then return 1; fi
126     "$@" &
127     local cmd_pid=$!
128     (
129         sleep "$timeout_sec"
130         if kill -0 "$cmd_pid" 2>> "$LOG_FILE"; then
131             kill -9 "$cmd_pid" 2>> "$LOG_FILE" || true
132         fi
133     ) >> "$LOG_FILE" 2>&1 &
134     local watcher_pid=$!
135     set +e
136     wait "$cmd_pid"
137     local exit_status=$?
138     set -uo pipefail
139     kill "$watcher_pid" 2>> "$LOG_FILE" || true
140     return "$exit_status"
141 }
142
143 #
144 # Quick-check functions (used by --watch loop; return simple status strings)
145 # Each echoes its result so the caller can capture and compare.
146 #
147
148 quick_warp() {
149     # Returns: "on", "off", "unknown"
150     local out
151     out=$(run_with_timeout 8 curl -s --max-time 6 \
152         https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null || echo "")
153     local warp
154     warp=$(printf '%s' "$out" | awk -F=' ' '/^warp={print $2; exit}')
155     printf '%s' "${warp:-unknown}"
156 }
157
158 quick_ipv6() {
159     # Returns: the IPv6 address if leaking, or empty string if clean
160     local out
161     out=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>/dev/null || echo "")
162     if [ -n "$out" ] && printf '%s' "$out" | grep -qE '^[0-9a-fA-F:]+$'; then
163         printf '%s' "$out"
164     fi
165 }
166
167 quick_default_route() {
168     # Returns: "utun" if default route is via Tailscale utun*,
169     #           "wifi" if via physical Wi-Fi (192.168.x.x),
170     #           "other" otherwise
171     local route
172     route=$(netstat -rn 2>/dev/null | awk '/^default/ {print $0; exit}')
173     if [ -z "$route" ]; then
174         printf '%s' "none"
175     elif printf '%s' "$route" | grep -qE 'utun[0-9]+'; then
176         printf '%s' "utun"
177     elif printf '%s' "$route" | grep -qE '^(default|0\.\.0\.\.0).*\b192\.\.168\.'; then
178         printf '%s' "wifi"
179     else
180         printf '%s' "other"
181     fi
182 }
183
184 # Watch loop (runs after baseline diagnostic when --watch is passed)
185 run_watch_loop() {
186     local baseline_warp="$1"
187     local baseline_default="$2"
188     local baseline_ipv6_leak="$3"
189     local interval="$4"
190
191     echo ""
192     echo ""
193     echo " Baseline established. Monitoring every ${interval}s..."
194     echo "   warp:      $baseline_warp"
195     echo " default:    $baseline_default"
196     echo ""
197
198     # Safety: warn if interval is very short (avoids hammering APIs)

```

```

199 if [ "$interval" -lt 30 ]; then
200     printf '%b Short interval (%ss) rate-limiting risk on external APIs\b\n' "$YELLOW" "$interval" "$NC"
201     printf ' Consider 30s or longer for production use.\n'
202 fi
203 printf '[%s] WATCH START warp=%s default=%s interval=%ss\n' \
204     "$(date -u +%Y-%m-%dT%H:%M:%SZ)" "$baseline_warp" "$baseline_default" "$interval" \
205     >> "$WATCH_LOG"
206
207 local iteration=0
208 local last_warp="$baseline_warp"
209 local last_default="$baseline_default"
210 local last_ipv6_ok=$( [ "$baseline_ipv6_leak" = "false" ] && echo true || echo false)
211
212 trap 'printf "\n%bWatch stopped by user. Log at: %s\b\n" "$YELLOW" "$WATCH_LOG" "$NC"; exit 0' INT TERM
213
214 while true; do
215     iteration=$((iteration + 1))
216
217     local warp_now ipv6_now default_now
218     warp_now=$(quick_warp)
219     ipv6_now=$(quick_ipv6)
220     default_now=$(quick_default_route)
221
222     local ts
223     ts=$(date -u +%H:%M:%S)
224
225     # WARP check
226     if [ "$warp_now" != "$last_warp" ]; then
227         if [ "$warp_now" = "off" ] || [ "$warp_now" = "unknown" ]; then
228             printf '\n%b ALERT %b\n' "$RED$BOLD" "$NC"
229             printf '%b[%s] %bWARP FLIPPED: %s %s\b YOUR IP IS LEAKING!\n' \
230                 "$RED" "$ts" "$BOLD" "$last_warp" "$warp_now" "$NC"
231             printf '[%s] ALERT warp flipped %s\b\n' "$ts" "$last_warp" "$warp_now" >> "$WATCH_LOG"
232         else
233             printf '\n%b[%s] warp recovered: %s %s\b\n' \
234                 "$GREEN" "$ts" "$last_warp" "$warp_now" "$NC"
235             printf '[%s] OK warp recovered %s\b\n' "$ts" "$last_warp" "$warp_now" >> "$WATCH_LOG"
236         fi
237         last_warp="$warp_now"
238     fi
239
240     # IPv6 check
241     if [ -n "$ipv6_now" ]; then
242         if [ "$last_ipv6_ok" = true ]; then
243             printf '\n%b ALERT %b\n' "$RED$BOLD" "$NC"
244             printf '%b[%s] %bIPv6 LEAK DETECTED %s\b\n' \
245                 "$RED" "$ts" "$BOLD" "$ipv6_now" "$NC"
246             printf '[%s] ALERT IPv6 leak %s\b\n' "$ts" "$ipv6_now" >> "$WATCH_LOG"
247             last_ipv6_ok=false
248         fi
249     else
250         if [ "$last_ipv6_ok" = false ]; then
251             printf '%b[%s] IPv6 leak resolved\b\n' "$GREEN" "$ts" "$NC"
252             printf '[%s] OK IPv6 resolved\n' "$ts" >> "$WATCH_LOG"
253         fi
254         last_ipv6_ok=true
255     fi
256
257     # Default route check
258     if [ "$default_now" != "$last_default" ]; then
259         if [ "$default_now" != "utun" ]; then
260             printf '\n%b ALERT %b\n' "$RED$BOLD" "$NC"
261             printf '%b[%s] %bDEFAULT ROUTE CHANGED: %s %s\b traffic may bypass WARP!\n' \
262                 "$RED" "$ts" "$BOLD" "$last_default" "$default_now" "$NC"
263             printf '[%s] ALERT route changed %s\b\n' "$ts" "$last_default" "$default_now" >> "$WATCH_LOG"
264         else
265             printf '%b[%s] default route recovered: %s %s\b\n' \
266                 "$GREEN" "$ts" "$last_default" "$default_now" "$NC"
267             printf '[%s] OK route recovered %s\b\n' "$ts" "$last_default" "$default_now" >> "$WATCH_LOG"
268         fi
269         last_default="$default_now"
270     fi
271
272     # Heartbeat (every 10th iteration or if all OK)
273     if [ $((iteration % 10)) -eq 0 ]; then
274         printf '[%s] #d warp=%s ipv6=%s route=%s\n' \
275             "$ts" "$iteration" "$warp_now" "${ipv6_now:-none}" "$default_now" >> "$WATCH_LOG"
276         printf '\r%b[%s] #d warp=%s route=%s (Ctrl+C to stop)%b\n' \
277             "$GREEN" "$ts" "$iteration" "$warp_now" "$default_now" "$NC"
278     fi
279

```

```

280     sleep "$interval"
281 done
282 }
283
284 #
285 # Begin main diagnostic (skipped in non-watch modes if we're just watching)
286 #
287
288 ACTIVE_SERVICE=$(get_active_service)
289 ENO_SERVICE=$(networksetup -listnetworkserviceorder 2>> "$LOG_FILE" \
290     | grep -B 1 "Device: en0" | head -1 \
291     | sed -E 's/^\([0-9\*]+\)/ /' || true)
292 [ -z "$ENO_SERVICE" ] && ENO_SERVICE="Wi-Fi"
293
294 # Resolve the gateway's Tailscale IP.
295 GATEWAY_TS_IP=""
296 if [ -f "$PROJECT_ROOT/.gateway_ip" ]; then
297     GATEWAY_TS_IP=$(cat "$PROJECT_ROOT/.gateway_ip" 2>> "$LOG_FILE" \
298         | tr -d '\r' | awk 'NR==1{print $1; exit}')
299 fi
300 if [ -z "$GATEWAY_TS_IP" ] && command -v docker >> "$LOG_FILE" 2>&1 \
301     && docker compose ps --status running 2>/dev/null | grep -q 'tailscale'; then
302     GATEWAY_TS_IP=$(run_with_timeout 10 docker compose exec -T tailscale \
303         tailscale ip -4 2>> "$LOG_FILE" \
304         | tr -d '\r' | awk 'NR==1{print $1; exit}')
305 fi
306
307 # Scenario classification state
308 SCENARIO=""
309 WARP_STATUS=""
310 SOCKS5_ENABLED=false
311 TS_ON_MESH=false
312 DEFAULT_VIA_UTUN=false
313 IPV6_LEAK=false
314
315 #
316 section "1/8 Host Tailscale mesh status"
317 #
318 if ! command -v tailscale >> "$LOG_FILE" 2>&1; then
319     fail "tailscale CLI not on PATH install via 'brew install tailscale' then re-run"
320     echo " (a Tailscale daemon is required for the exit-node path to work)"
321 else
322     set +e
323     TS_OUT=$(run_with_timeout 5 tailscale status 2>&1)
324     TS_EXIT=$?
325     set -uo pipefail
326     TS_HEAD=$(printf '%s\n' "$TS_OUT" | head -5 | tr -d '\r')
327     if [ "$TS_EXIT" -eq 137 ]; then
328         fail "tailscaled daemon is wedged/unresponsive (5s timeout)"
329         echo " Fix path: 'sudo brew services restart tailscale'"
330     elif printf '%s' "$TS_OUT" | grep -qE 'Logged out|not logged in|expired'; then
331         fail "tailscale is logged out / auth expired on this host"
332         echo " Fix path: 'sudo tailscale up' (browser auth once)"
333     elif printf '%s' "$TS_OUT" | grep -q '100\.'; then
334         ok "Host is on tailnet (100.x.x.x visible in status)"
335         TS_ON_MESH=true
336         if printf '%s' "$TS_OUT" | grep -q "$GATEWAY_TS_IP"; then
337             ok "Gateway $GATEWAY_TS_IP visible in host's peer list"
338         elif [ -n "$GATEWAY_TS_IP" ]; then
339             warn "Gateway $GATEWAY_TS_IP NOT in host's peer list possible auth/key mismatch"
340         fi
341         line " first 5 lines of 'tailscale status' "
342         line "$$(printf '%s' "$TS_HEAD" | awk '{print " " "$0}')"
343     else
344         fail "tailscale output unrecognized:"
345         line "$$(printf '%s' "$TS_OUT" | head -3 | awk '{print " " "$0}')"
346     fi
347 fi
348
349 #
350 section "2/8 Active SOCKS5 proxy on Wi-Fi"
351 #
352 SOCKS_OUT=$(networksetup -getsocksfirewallproxy "$ACTIVE_SERVICE" 2>> "$LOG_FILE" \
353     || networksetup -getsocksfirewallproxy Wi-Fi 2>> "$LOG_FILE" \
354     || echo "Could not query SOCKS5 proxy state")
355 if printf '%s' "$SOCKS_OUT" | grep -q "Enabled: Yes"; then
356     fail "SOCKS5 proxy IS enabled on $ACTIVE_SERVICE toggle.sh fell back to SOCKS5 mode last run"
357     SOCKS5_ENABLED=true
358     line " Full state:"
359     line "$$(printf '%s' "$SOCKS_OUT" | awk '{print " " "$0}')"
360     line " Browse on macOS ignores system SOCKS5 by default traffic bypasses WARP."

```

```

361 else
362   ok "SOCKS5 proxy is OFF on $ACTIVE_SERVICE  exit-node path should be active"
363 fi
364
365 #
366 section "3/8 Egress IP + WARP tunnel verification (definitive test)"
367 #
368 CDN_OUT=$(run_with_timeout 8 curl -s --max-time 6 \
369   https://www.cloudflare.com/cdn-cgi/trace 2>&1 || echo "(curl failed)")
370 CDN_WARP=$(printf '%s' "$CDN_OUT" | awk -F=' ' /^warp={print $2; exit}')
371 CDN_IP=$(printf '%s' "$CDN_OUT" | awk -F=' ' /^ip={print $2; exit}')
372 CDN_COLO=$(printf '%s' "$CDN_OUT" | awk -F=' ' /^colo={print $2; exit}')
373 WARP_STATUS="$CDN_WARP"
374 if [ "$CDN_WARP" = "on" ]; then
375   ok "warp=on tunnel is hot, host traffic IS going through Cloudflare WARP"
376   line " Public IP: $CDN_IP"
377   line " Cloudflare: ${CDN_COLO:-unknown colo}"
378   line " whatismyip.com's 'local ISP' result is its ISP-database error, not yours."
379 elif [ "$CDN_WARP" = "off" ]; then
380   fail "warp=off host traffic is NOT going through WARP"
381   line " Public IP: $CDN_IP (NOT Cloudflare)"
382   warn "This is the actual leak."
383 else
384   warn "could not determine warp status (curl failed or returned unexpected shape)"
385   line " Raw output: $(printf '%s' "$CDN_OUT" | head -3 | tr '\n' '|')"
386 fi
387
388 #
389 section "4/8 IPv6 leak probe (bypasses IPv4 exit-node on campus networks)"
390 #
391 IPV6_OUT=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>&1 || echo "")
392 if [ -n "$IPV6_OUT" ] && printf '%s' "$IPV6_OUT" | grep -qE '[0-9a-fA-F:]+$'; then
393   fail "host has live IPv6 egress $IPV6_OUT (A6 record bypasses WARP)"
394   IPV6_LEAK=true
395   line " Tailscale exit-node advertisement is IPv4-only, so IPv6 traffic skips it."
396   line " Quick-fix: 'sudo networksetup -setv6off $ACTIVE_SERVICE'"
397 else
398   ok "no IPv6 egress detected (good IPv6 won't bypass the tunnel)"
399 fi
400
401 #
402 section "5/8 Host egress route for real traffic (route -n get 1.1.1.1)"
403 #
404 # WHY NOT 'netstat -rn | grep default':
405 # On macOS, Tailscale does NOT replace the physical default route installed by
406 # DHCP. Instead it adds a second interface-scoped route bound to the utun
407 # interface that the kernel prefers for real traffic forwarding. The literal
408 # "default" entry (192.168.x.x via en0) is left untouched in the table as a
409 # BSD routing quirk it's not lying, it's just answering a different question.
410 # The correct question is: "where does a real destination actually resolve?"
411 # 'route -n get 1.1.1.1' asks the kernel's forwarding logic directly.
412 ROUTE_GET=$(route -n get 1.1.1.1 2>/dev/null)
413 ROUTE_IFACE=$(echo "$ROUTE_GET" | awk '/interface:{print $2}')
414 ROUTE_GATEWAY=$(echo "$ROUTE_GET" | awk '/gateway:{print $2}')
415 if [ -z "$ROUTE_IFACE" ]; then
416   warn "route -n get 1.1.1.1 returned no interface routing table may be empty"
417 else
418   line " interface: $ROUTE_IFACE gateway: ${ROUTE_GATEWAY:-n/a}"
419   if echo "$ROUTE_IFACE" | grep -qE '^utun[0-9]+'; then
420     ok "1.1.1.1 resolves via $ROUTE_IFACE Tailscale interface-scoped route is winning"
421     DEFAULT_VIA_UTUN=true
422   elif echo "$ROUTE_IFACE" | grep -qE '^en[0-9]+'; then
423     fail "1.1.1.1 resolves via $ROUTE_IFACE (physical Wi-Fi) exit-node interface route not active"
424   else
425     warn "1.1.1.1 resolves via unexpected interface: $ROUTE_IFACE"
426   fi
427 fi
428
429 #
430 section "6/8 Last toggle.sh START-relevant lines in output.log (if any)"
431 #
432 if [ ! -f "$LOG_FILE" ]; then
433   note "output.log"
434   line " toggle.sh was never run from this directory. Run './toggle.sh' first,"
435   line " then re-run this diagnostic."
436 else
437   HITS=$(grep -E 'Exit node enabled|SKIP_EXIT_NODE|Pre-flight|Falling back to SOCKS|Starting|Successfully' \
438     "$LOG_FILE" 2>/dev/null | tail -8 || true)
439   if [ -z "$HITS" ]; then
440     note "matching lines in output.log"
441   else

```

```

442     line "$(printf '%s\n' "$HITS" | awk '{print "    "$0}')"
443 fi
444 fi
445
446 #
447 section "7/8 Gateway IP we should be pointing at"
448 #
449 if [ -n "$GATEWAY_TS_IP" ]; then
450     ok "gateway Tailscale IP: $GATEWAY_TS_IP"
451 else
452     warn "could not resolve gateway Tailscale IP"
453     line " (looked in .gateway_ip and tried docker compose exec tailscale ip -4)"
454 fi
455
456 #
457 section "8/8 Tailscale System Extension (App Store conflict check)"
458 #
459 SE_PENDING_UNINSTALL=false
460 SE_ACTIVE=false
461 if command -v systemextensionsctl >/dev/null 2>&1; then
462     SE_OUT=$(systemextensionsctl list 2>/dev/null | grep -iE '(io|com)\.tailscale')
463     if [ -n "$SE_OUT" ]; then
464         line " Tailscale System Extension(s) detected:"
465         line "$(printf '%s' "$SE_OUT" | awk '{print "    "$0}')"
466         if printf '%s' "$SE_OUT" | grep -q 'waiting to uninstall'; then
467             SE_PENDING_UNINSTALL=true
468             warn "System Extension is pending uninstall on next reboot."
469         else
470             SE_ACTIVE=true
471             warn "Active Tailscale System Extension detected. This will conflict with Homebrew Tailscale."
472         fi
473     else
474         ok "no Tailscale System Extension detected"
475     fi
476 else
477     note "systemextensionsctl not available (not on macOS or permission restricted)"
478 fi
479
480 #
481 section "CLASSIFICATION applies ONE of the four known scenarios"
482 #
483
484 if [ "$WARP_STATUS" = "on" ] && [ "$IPV6_LEAK" = "false" ]; then
485     SCENARIO="OK"
486 elif [ "$SE_PENDING_UNINSTALL" = "true" ]; then
487     SCENARIO="SE_PENDING"
488 elif [ "$SOCKS5_ENABLED" = "true" ]; then
489     SCENARIO="A"
490 elif [ "$IPV6_LEAK" = "true" ]; then
491     SCENARIO="B"
492 elif [ "$DEFAULT_VIA_UTUN" = "false" ] && [ "$TS_ON_MESH" = "true" ]; then
493     SCENARIO="C"
494 else
495     SCENARIO="UNKNOWN"
496 fi
497
498 case "$SCENARIO" in
499     SE_PENDING)
500         echo ""
501         echo -e " ${RED}${BOLD}VERDICT: Scenario SE_PENDING Tailscale System Extension Pending Uninstall${NC}"
502         echo " A prior App Store Tailscale System Extension is pending uninstall."
503         echo " Since System Integrity Protection (SIP) is enabled, macOS blocks manual"
504         echo " uninstallation of terminated system extensions until the next reboot."
505         echo " Brew-managed tailscaled cannot engage the Network Extension slot until this is resolved."
506         echo ""
507         echo "${BOLD}Recommended fix:${NC}"
508         echo " sudo shutdown -r now"
509         echo " # After reboot, run: ./toggle.sh"
510         ;;
511     A)
512         echo ""
513         echo -e " ${RED}${BOLD}VERDICT: Scenario A SOCKS5 fallback lane${NC}"
514         echo " toggle.sh's pre-flight checks failed last run. The script activated"
515         echo " SOCKS5 + local DNS proxy as a fallback. DNS gets hijacked but"
516         echo " browsers on macOS ignore the system SOCKS5 traffic egresses"
517         echo " direct through the local network."
518         echo ""
519         echo "${BOLD}Recommended fix:${NC}"
520         echo " echo 'GATEWAY_BYPASS_PING=true' >> $PROJECT_ROOT/.env"
521         echo " sudo tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \"\"
522         if [ -n "$GATEWAY_TS_IP" ]; then

```

```

523     echo "          --exit-node=$GATEWAY_TS_IP --exit-node-allow-lan-access=true"
524     else
525     echo "          --exit-node=\$GATEWAY_TS_IP --exit-node-allow-lan-access=true # fill in $GATEWAY_TS_IP"
526     fi
527     echo "          sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"
528     echo "          ./toggle.sh"
529     ;;
530 B)
531     echo ""
532     echo -e "  ${RED}${BOLD}VERDICT: Scenario B IPv6 leak over dual-stack campus/local IPv6${NC}"
533     echo "  Tailscale exit-node advertises only IPv4. macOS sends AAAA queries"
534     echo "  straight out en0, which is dual-stack on most campus APs IPv6"
535     echo "  traffic bypasses the WARP tunnel entirely."
536     echo ""
537     echo "  ${BOLD}Recommended fix:${NC}"
538     echo "          sudo networksetup -setv6off $ACTIVE_SERVICE"
539     echo "          sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"
540     echo "          # Re-enable IPv6 later with: sudo networksetup -setv6automatic $ACTIVE_SERVICE"
541     ;;
542 C)
543     echo ""
544     echo -e "  ${RED}${BOLD}VERDICT: Scenario C Tailscale route-freeze${NC}"
545     echo "  Host's default route points at the Wi-Fi gateway instead of the"
546     echo "  Tailscale utun* interface. The exit-node preference is set but the"
547     echo "  routing-table assertion did not take effect (devref 10.26)."

```



```

603 printf '%s\n' "$SECOND_BLOCK" >&3
604
605 #
606 # --fix mode
607 #
608 if [ "$DO_FIX" = "true" ] && [ "$SCENARIO" != "OK" ] && [ "$SCENARIO" != "UNKNOWN" ]; then
609     echo ""
610     echo ""
611     echo " A P P L Y I N G   F I X   (scenario $SCENARIO)"
612     echo ""
613     echo ""
614
615 case "$SCENARIO" in
616     SE_PENDING)
617         warn "System Extension uninstall is pending on next reboot."
618         line "Please run 'sudo shutdown -r now' to reboot the system."
619         ;;
620     A)
621         if [ -n "$GATEWAY_TS_IP" ]; then
622             line " writing GATEWAY_BYPASS_PING=true to .env"
623             ENV="$PROJECT_ROOT/.env"
624             [ -f "$ENV" ] || touch "$ENV"
625             if grep -q '^GATEWAY_BYPASS_PING=' "$ENV" 2>/dev/null; then
626                 sed -i 's/^GATEWAY_BYPASS_PING=.*\/GATEWAY_BYPASS_PING=true\/' "$ENV"
627             else
628                 printf '\nGATEWAY_BYPASS_PING=true\n' >> "$ENV"
629             fi
630             line " re-asserting tailscale exit-node (with --reset)"
631             sudo -n tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \
632                 --exit-node="$GATEWAY_TS_IP" --exit-node-allow-lan-access=true \
633                 >> "$LOG_FILE" 2>&1 \
634                 && ok "tailscale exit-node re-asserted for $GATEWAY_TS_IP" \
635                 || warn "tailscale up did not return success (check $LOG_FILE)"
636         else
637             warn "no gateway Tailscale IP resolved skipping tailscale up"
638             warn "fix manually: 'sudo tailscale up --reset ... --exit-node=<TS_IP>'"
639         fi
640         ;;
641     B)
642         line " disabling IPv6 on $ACTIVE_SERVICE while gateway is up"
643         sudo -n networksetup -setv6off "$ACTIVE_SERVICE" >> "$LOG_FILE" 2>&1 \
644             && ok "IPv6 disabled on $ACTIVE_SERVICE (re-enable with 'setv6automatic')\" \
645             || warn "networksetup -setv6off failed (check $LOG_FILE)"
646         ;;
647     C)
648         line " restarting tailscaled (route-table fix)"
649         run_with_timeout 15 brew services restart tailscale >> "$LOG_FILE" 2>&1 \
650             || run_with_timeout 15 sudo -n brew services restart tailscale >> "$LOG_FILE" 2>&1
651         sleep 3
652         line " flushing stale host routes + DNS cache"
653         sudo -n route delete -net 0.0.0.0/1 >> "$LOG_FILE" 2>&1 || true
654         sudo -n route delete -net 128.0.0.0/1 >> "$LOG_FILE" 2>&1 || true
655         sudo -n dscacheutil -flushcache >> "$LOG_FILE" 2>&1 || true
656         sudo -n killall -HUP mDNSResponder >> "$LOG_FILE" 2>&1 || true
657         if [ -n "$GATEWAY_TS_IP" ]; then
658             line " re-asserting exit-node after restart"
659             sudo -n tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \
660                 --exit-node="$GATEWAY_TS_IP" --exit-node-allow-lan-access=true \
661                 >> "$LOG_FILE" 2>&1 \
662                 && ok "tailscale exit-node re-asserted for $GATEWAY_TS_IP" \
663                 || warn "tailscale up did not return success (check $LOG_FILE)"
664         fi
665         ;;
666     esac
667
668     echo ""
669     echo "Re-verifying after fix..."
670     sleep 3
671     CDN_WARP_AFTER=$(run_with_timeout 8 curl -s --max-time 6 \
672         https://www.cloudflare.com/cdn-cgi/trace 2>&1 \
673         | awk -F=' ' '/warp=/{print $2; exit}')
674     IPV6_AFTER=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>&1 || echo "")
675     printf '\n[AFTER FIX]\n' >&3
676     printf ' warp status: %s\n' "$CDN_WARP_AFTER" >&3
677     printf ' IPv6 egress: %s\n' "${IPV6_AFTER:-none}" >&3
678     line " warp status: ${CDN_WARP_AFTER:-unknown}"
679     line " IPv6 egress: ${IPV6_AFTER:-none}"
680     if [ "$CDN_WARP_AFTER" = "on" ] && [ -z "$IPV6_AFTER" ]; then
681         echo ""
682         ok "Fix verified host is now routing through WARP. Remote clients unaffected."
683     else

```

```

684     echo ""
685     warn "Fix did not fully take effect on first try. Likely causes:"
686     line "    tailscaled needed a longer settling window (re-run diagnostic in 30s)"
687     line "    The classified scenario was wrong; paste this report for triage."
688     line "    Tailscale needs a full auth refresh: 'sudo tailscale up'"
689 fi
690 fi
691
692 echo ""
693 echo ""
694 echo "    Full report written to:"
695 echo "        $REPORT_FILE"
696 echo ""
697 echo ""
698
699 #
700 # --watch mode: enter the continuous monitoring loop after baseline diagnostic
701 #
702 if [ "$DO_WATCH" = "true" ]; then
703     # Build baseline from the full diagnostic just completed
704     BASELINE_WARP="$WARP_STATUS"
705     BASELINE_DEFAULT=$( [ "$DEFAULT_VIA_UTUN" = "true" ] && echo "utun" || echo "wifi/other" )
706
707     if [ "$SCENARIO" != "OK" ]; then
708         warn "Baseline diagnostic found issues (scenario: $SCENARIO)."
709         line "    Watch loop will still run alerts fire on any STATE CHANGE from current baseline."
710         line "    Fix the issue first? Re-run with:  bash scripts/diagnose-host-leak.sh --fix"
711         echo ""
712     fi
713
714     run_watch_loop "$BASELINE_WARP" "$BASELINE_DEFAULT" "$IPV6_LEAK" "$WATCH_INTERVAL"
715 fi
716
717 exit 0

```

32 scripts/dns-proxy.py

```

1  #!/usr/bin/env python3
2  """
3  dns-proxy.py  Local DNS proxy for nullexit gateway.
4
5  Listens on UDP port 53, receives DNS queries, forwards them over TCP
6  to AdGuard at 127.0.0.1:5354 (the Docker port mapping), and returns
7  the response via UDP. Handles the DNS-over-TCP wire format (2-byte
8  length prefix) correctly unlike socat which just forwards raw bytes.
9  """
10
11 import socket
12 import struct
13 import sys
14 import os
15 from concurrent.futures import ThreadPoolExecutor
16
17 LISTEN_ADDR = os.environ.get("LISTEN_ADDR", "127.0.0.1")
18 LISTEN_PORT = int(os.environ.get("LISTEN_PORT", 53))
19 TARGET_HOST = os.environ.get("TARGET_HOST", "127.0.0.1")
20 TARGET_PORT = int(os.environ.get("TARGET_PORT", 5354))
21
22
23 def handle_query(data: bytes, client_addr: tuple, sock: socket.socket) -> None:
24     """Forward a single DNS query over TCP and send the response back via UDP."""
25     try:
26         tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
27         tcp.settimeout(5)
28         tcp.connect((TARGET_HOST, TARGET_PORT))
29
30         # DNS over TCP: prepend 2-byte length (big-endian)
31         tcp.sendall(struct.pack("!H", len(data)) + data)
32
33         # Read the 2-byte response length
34         raw_len = tcp.recv(2)
35         if len(raw_len) < 2:
36             tcp.close()
37             return
38         resp_len = struct.unpack("!H", raw_len)[0]
39

```

```

40     # Read the full response
41     response = b""
42     while len(response) < resp_len:
43         chunk = tcp.recv(resp_len - len(response))
44         if not chunk:
45             break
46         response += chunk
47
48     tcp.close()
49
50     # Send the response back over UDP (strip TCP length prefix)
51     if response:
52         sock.sendto(response, client_addr)
53     except Exception:
54         # Silently drop failed queries (malformed, timeout, etc.)
55         pass
56
57
58 def main() -> None:
59     # Create UDP socket
60     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
61     sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
62
63     try:
64         sock.bind((LISTEN_ADDR, LISTEN_PORT))
65     except PermissionError:
66         print(f"dns-proxy: ERROR need root to bind port {LISTEN_PORT}", flush=True)
67         sys.exit(1)
68     except OSError as e:
69         print(f"dns-proxy: ERROR {e}", flush=True)
70         sys.exit(1)
71
72     print(f"dns-proxy: listening on UDP {LISTEN_ADDR}:{LISTEN_PORT} TCP {TARGET_HOST}:{TARGET_PORT}", flush=True)
73
74     # Use a thread pool to handle concurrent DNS queries efficiently without thread exhaustion
75     executor = ThreadPoolExecutor(max_workers=64)
76
77     while True:
78         try:
79             data, client_addr = sock.recvfrom(4096) # Support EDNS0 (up to 4096 bytes) to prevent truncation
80             executor.submit(handle_query, data, client_addr, sock)
81         except KeyboardInterrupt:
82             break
83         except Exception:
84             pass
85
86     executor.shutdown(wait=False)
87     sock.close()
88
89
90 if __name__ == "__main__":
91     main()

```

33 scripts/fix-docker-bridge-collision.sh

```

1  #!/bin/bash
2  # scripts/fix-docker-bridge-collision.sh nullexit fix for devref 9.13
3  #
4  # Symptom: host default route points at Docker's bridge gateway (172.17.0.1)
5  # instead of the real Wi-Fi gateway. ALL host internet egress fails. Remote
6  # tailnet clients route through the gateway correctly (their tunnel is
7  # independent of the host's broken route table), so they look healthy while
8  # the host itself cannot reach anything.
9  #
10 # Root cause: Docker's default bridge claims 172.17.0.0/16 by default. When the
11 # host's physical Wi-Fi also lands in the same /16 extremely common on
12 # university networks which hand out 172.16.0.0/12 the kernel
13 # installs Docker's bridge as the default route, and every packet bound for
14 # the internet is silently absorbed by docker0 instead of egressing.
15 #
16 # This script:
17 # 1. Detects the actual collision (Docker's 172.17.0.0/16 vs the host's
18 # Wi-Fi subnet, queried live via the ACTIVE interface not hardcoded en0)
19 # 2. Picks a non-colliding Docker bip using /8-family routing (works for
20 # 172.16.0.0/12 campus networks; naive prefix-match would pick 172.26/24

```

```

21 # which still lives INSIDE a /12 of 172.16.0.0/12)
22 # 3. Backs up ~/.colima/default/colima.yaml with an ISO-8601 timestamp
23 # 4. Edits the file in place, idempotent (replace existing docker.bip /
24 # append new docker.bip without disturbing other keys; skip YAML comments)
25 # 5. Restarts Colima so the VM rebuilds with the new bridge subnet
26 # 6. Rebinds Wi-Fi DHCP so the host re-learns its real Wi-Fi gateway
27 # 7. Verifies the new default route is no longer the Docker bridge
28 # 8. Re-runs toggle.sh to bring the gateway back up cleanly
29 # 9. Re-runs scripts/diagnose-host-leak.sh and prints the new verdict
30 #
31 # Usage:
32 # bash scripts/fix-docker-bridge-collision.sh # apply the fix
33 # bash scripts/fix-docker-bridge-collision.sh --dry # show what would change, then exit
34 # bash scripts/fix-docker-bridge-collision.sh --skip-toggle # fix but don't restart gateway
35 # bash scripts/fix-docker-bridge-collision.sh --help # usage
36
37 set -uo pipefail
38 # NOTE: errexit (`set -e`) is intentionally NOT enabled so intermediate
39 # `command || warn` patterns can continue past non-fatal failures.
40 # Every critical failure (colima restart, sed, cp backup, toggle.sh)
41 # uses an explicit `|| die` so a half-applied state never escapes.
42
43 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
44 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
45 LOG_FILE="$PROJECT_ROOT/output.log"
46 source "$SCRIPT_DIR/common.sh"
47 TIMESTAMP="$(date -u +%Y%m%dT%H%M%S)"
48 COLIMA_YAML="$HOME/.colima/default/colima.yaml"
49 PLATFORM="$(uname -s)"
50
51 # Argument parsing
52 DRY_RUN=false
53 SKIP_TOGGLE=false
54 case "${1:-}" in
55     --dry|-n) DRY_RUN=true ;;
56     --skip-toggle) SKIP_TOGGLE=true ;;
57     --help|-h)
58         sed -n '28,37p' "$0"
59         exit 0
60     ;;
61     *)
62         : # default: apply
63     ;;
64 *)
65     printf 'Unknown argument: %s\n' "$1" >&2
66     exit 2
67     ;;
68 esac
69
70
71
72 # Helper: print "[dry-run] $cmd" or actually run $cmd. Wraps the redirect
73 # INSIDE the function so outer `>> $FILE` statements never accidentally
74 # append dry-run noise to the real file. (Earlier draft didn't and the
75 # dry-run polluted $COLIMA_YAML.)
76 run() {
77     if [ "$DRY_RUN" = "true" ]; then
78         printf '[dry-run] %s\n' "$*"
79         return 0
80     fi
81     printf '%s\n' "$*"
82     "$@"
83 }
84
85 # Platform-aware sed in-place. macOS BSD sed needs an empty argument;
86 # GNU sed (Linux) does not. Earlier draft used `-i` everywhere and
87 # would have failed on Linux silently.
88 sed_inplace() {
89     local expr="$1"
90     local file="$2"
91     case "$PLATFORM" in
92         Darwin) run sed -i '' "$expr" "$file" ;;
93         *) run sed -i "$expr" "$file" ;;
94     esac
95 }
96
97 # 0. Resolve the ACTIVE physical interface once (used everywhere)
98 # Replaces the earlier `en0` hardcoding. Same algorithm recover.sh uses:
99 # `route get default` if utun/tun, walk en0..en5 first with `status: active`
100 # + `inet` fall back to en0. This means the script also works on hosts
101 # whose primary NIC is en1/en2 (Thunderbolt Ethernet, USB-LAN, etc.).

```

```

102 resolve_active_iface() {
103     local iface
104     iface=$(route get default 2>> "$LOG_FILE" | awk '/interface:/{print $2; exit}')
105     if [ -z "$iface" ] || [ "${iface#tun}" != "$iface" ] || [ "${iface#tun}" != "$iface" ]; then
106         for i in en0 en1 en2 en3 en4 en5; do
107             if ifconfig "$i" 2>> "$LOG_FILE" | grep -q "status: active" \
108                 && ifconfig "$i" 2>> "$LOG_FILE" | grep -q "inet "; then
109                 iface="$i"; break
110             fi
111         done
112     fi
113     [ -z "$iface" ] && iface="en0"
114     printf '%s\n' "$iface"
115 }
116 ACTIVE_IFACE=$(resolve_active_iface)
117
118 # 1. Detect collision
119 step "1/9 Detecting Docker-bridge vs Wi-Fi subnet collision"
120
121 # Host subnet on the ACTIVE interface (not hardcoded en0).
122 HOST_SUBNET=$(ifconfig "$ACTIVE_IFACE" 2>> "$LOG_FILE" \
123     | awk '/inet /{print $2; exit}')
124 HOST_GATEWAY=$(route get default 2>> "$LOG_FILE" \
125     | awk '/gateway:/{print $2; exit}')
126 DEFAULT_DEV=$(route get default 2>> "$LOG_FILE" \
127     | awk '/interface:/{print $2; exit}')
128
129 printf ' active iface: %s\n' "$ACTIVE_IFACE"
130 printf ' iface inet: %s\n' "${HOST_SUBNET:-none}"
131 printf ' default via: %s\n' "${HOST_GATEWAY:-none}"
132 printf ' default dev: %s\n' "${DEFAULT_DEV:-none}"
133
134 # Collision fires when the host's default-route gateway is 172.17.0.1 (Docker's
135 # bridge) OR when the host subnet is in 172.17.0.0/16. The full-bridge-collision
136 # case (gateway is literally the docker bridge IP) is the smoking gun.
137 COLLISION=false
138 if [ -n "$HOST_GATEWAY" ] && [ "${HOST_GATEWAY#172.17.}" != "$HOST_GATEWAY" ]; then
139     COLLISION=true
140     fail "default gateway $HOST_GATEWAY collides with Docker's default bridge (172.17.0.0/16)"
141 elif [ -n "$HOST_SUBNET" ] && [ "${HOST_SUBNET#172.17.}" != "$HOST_SUBNET" ]; then
142     COLLISION=true
143     fail "host subnet $HOST_SUBNET overlaps Docker's 172.17.0.0/16 bridge"
144 else
145     ok "no collision detected between host Wi-Fi and Docker's default bridge"
146 fi
147
148 if [ "$COLLISION" = "false" ]; then
149     echo ""
150     echo "Nothing to fix. If you still see egress issues, run:"
151     echo "    bash scripts/diagnose-host-leak.sh"
152     exit 0
153 fi
154
155 # 2. Pick non-colliding bip via /8-family routing
156 step "2/9 Picking a non-colliding Docker bip for ~/.colima/default/colima.yaml"
157
158 # Earlier draft tried naive prefix-match: "if candidate's 3-octet prefix is
159 # not a literal prefix of host gateway, pick it". That breaks on /12 campus
160 # networks (172.16.0.0/12 covers 172.16-172.31): a candidate like
161 # 172.26.0.1/24 LIVES INSIDE the host's /12 even though the prefix test
162 # passes. Fix: classify the host's address family FIRST, then choose ALL
163 # candidates from a DIFFERENT RFC1918 family. /8 boundaries are coarse
164 # enough that no /24 picked this way can ever collide.
165 case "$HOST_GATEWAY" in
166     172.16.*|172.17.*|172.18.*|172.19.*|172.20.*|172.21.*|172.22.*|172.23.*|\
167     172.24.*|172.25.*|172.26.*|172.27.*|172.28.*|172.29.*|172.30.*|172.31.*)
168         HOST_FAMILY="172"
169         # Host is in 172.x jump past 172.31 entirely. Pick from 10.x / 192.168.x.
170         CANDIDATES="10.200.0.1/24 10.201.0.1/24 192.168.99.1/24"
171         ;;
172     10.*)
173         HOST_FAMILY="10"
174         # Host is in 10.x pick from 172.x (well outside any commonly-deployed
175         # 10.0.0.0/8 slice) / 192.168.x.
176         CANDIDATES="172.26.0.1/24 172.28.0.1/24 192.168.99.1/24"
177         ;;
178     192.168.*)
179         HOST_FAMILY="192"
180         CANDIDATES="172.26.0.1/24 10.200.0.1/24 10.201.0.1/24"
181         ;;
182     *)

```

```

183     HOST_FAMILY="unknown"
184     warn "host gateway $HOST_GATEWAY is not in any RFC1918 range (/8 family)"
185     warn "falling back to abstract candidate set; verify collision-free manually"
186     CANDIDATES="172.26.0.1/24 10.200.0.1/24 192.168.99.1/24"
187     ;;
188 esac
189
190 printf '  host address family: %s\n' "$HOST_FAMILY"
191 printf '  candidate pool:      %s\n' "$CANDIDATES"
192
193 # Sanity-check the chosen pool has at least one entry that doesn't literally
194 # share a /24 with the host gateway. Naive check (just /24 prefix) is fine
195 # here because we already picked across families.
196 CHOSEN=""
197 for cand in $CANDIDATES; do
198     PREFIX3=$(printf '%s' "$cand" | cut -d. -f1-3)
199     if [ -n "$HOST_GATEWAY" ] && [ "${HOST_GATEWAY#"$PREFIX3".}" != "$HOST_GATEWAY" ]; then
200         warn "skip $cand (literal /24 collision with host gateway $HOST_GATEWAY)"
201         continue
202     fi
203     CHOSEN="$cand"
204     ok "chose bip=$CHOSEN (cross-family, no /24 literal collision)"
205     break
206 done
207
208 if [ -z "$CHOSEN" ]; then
209     CHOSEN="172.26.0.1/24"
210     warn "could not find a clean candidate; defaulting to $CHOSEN"
211     warn "verify with 'route get default' after restart"
212 fi
213
214 # 3. Backup colima.yaml
215 step "3/9 Backing up $COLIMA_YAML"
216 mkdir -p "$(dirname "$COLIMA_YAML")"
217 BACKUP="$HOME/.colima/default/colima.yaml.bak.$TIMESTAMP"
218 if [ -f "$COLIMA_YAML" ]; then
219     # `cp` is safe in dry-run because run() suppresses the inner command
220     # entirely no chance of accidentally writing a real backup.
221     run cp -p "$COLIMA_YAML" "$BACKUP" || die "backup failed"
222     if [ "$DRY_RUN" = "false" ]; then ok "backup: $BACKUP"; fi
223 else
224     warn "no existing $COLIMA_YAML (will create fresh)"
225 fi
226
227 # 4. Edit colima.yaml in place (idempotent)
228 step "4/9 Setting docker.bip=$CHOSEN in colima.yaml (in place)"
229
230 # Resolve the existing docker.bip (if any) without picking up commented-out
231 # YAML keys. awk-based extraction; the negated-comment check (`! /#[[:space:]]*#`)
232 # ensures `# bip: 1.2.3.4/24` is treated as a comment, not the active value.
233 EXISTING_BIP=""
234 if [ -f "$COLIMA_YAML" ]; then
235     EXISTING_BIP=$(awk '
236         /^[[:space:]]*#/ {next}
237         /^[[:space:]]*bip:[[:space:]]*/ {print $2; exit}
238     ' "$COLIMA_YAML" 2>> "$LOG_FILE" || true)
239 fi
240
241 if [ -n "$EXISTING_BIP" ] && [ "$EXISTING_BIP" = "$CHOSEN" ]; then
242     ok "colima.yaml already has docker.bip=$CHOSEN no edit needed"
243 elif [ -n "$EXISTING_BIP" ]; then
244     printf '  (replacing existing docker.bip=%s with %s)\n' "$EXISTING_BIP" "$CHOSEN"
245     # Use sed_inplace so Linux / macOS get the right syntax. Skip YAML comment
246     # lines so existing `# bip:` comments aren't rewired.
247     sed_inplace "s|^\\([[:space:]]*bip:[[:space:]]*\\).*\\|\\1$CHOSEN|" "$COLIMA_YAML" \
248         || die "sed replacement failed (colima.yaml malformed?)"
249     ok "colima.yaml updated"
250 else
251     # No existing `bip:` key. Branch:
252     # - file has `docker:` block append `bip:` under it
253     # - file has no `docker:` block append new docker: section
254     # - file does not exist create fresh
255     # Each branch is dry-run-aware so no branch can accidentally leak noise
256     # into $COLIMA_YAML during a preview.
257     HAS_DOCKER_BLOCK=false
258     if [ -f "$COLIMA_YAML" ] && grep -q '^docker:' "$COLIMA_YAML"; then
259         HAS_DOCKER_BLOCK=true
260     fi
261
262     if [ "$HAS_DOCKER_BLOCK" = "true" ]; then
263         if [ "$DRY_RUN" = "true" ]; then

```

```

264     printf '    [dry-run] would append to %s:\n bip: %s\n' "$COLIMA_YAML" "$CHOSEN"
265 else
266     printf '\n bip: %s\n' "$CHOSEN" >> "$COLIMA_YAML" \
267     || die "append failed"
268     ok "appended bip under existing docker: block"
269 fi
270 elif [ -f "$COLIMA_YAML" ]; then
271     if [ "$DRY_RUN" = "true" ]; then
272         printf '    [dry-run] would append to %s:\ndocker:\n bip: %s\n' "$COLIMA_YAML" "$CHOSEN"
273     else
274         printf '\ndocker:\n bip: %s\n' "$CHOSEN" >> "$COLIMA_YAML" \
275         || die "append failed"
276         ok "appended new docker: block at end of file"
277     fi
278 else
279     if [ "$DRY_RUN" = "true" ]; then
280         printf '    [dry-run] would create %s with:\ndocker:\n bip: %s\n' "$COLIMA_YAML" "$CHOSEN"
281     else
282         printf 'docker:\n bip: %s\n' "$CHOSEN" > "$COLIMA_YAML" \
283         || die "create failed"
284         ok "created fresh $COLIMA_YAML with docker.bip=$CHOSEN"
285     fi
286 fi
287 fi
288
289 # Always show the final colima.yaml docker section for visibility
290 printf '\n%b colima.yaml `docker:` section now reads:%b\n' "$BOLD" "$NC"
291 awk '
292 /~docker:/{p=1}
293 p && /^[^ ]/ && !/~docker:/{exit}
294 p
295 ' "$COLIMA_YAML" 2>/dev/null | sed 's/~/ /' || true
296
297 if [ "$DRY_RUN" = "true" ]; then
298     printf '\n %b(dry-run: exiting before colima restart / dhcp rebind / toggle.sh)%b\n' "$YELLOW" "$NC"
299     exit 0
300 fi
301
302 # 5. Restart Colima
303 step "5/9 Restarting Colima VM with new bip"
304 if ! command -v colima >> "$LOG_FILE" 2>&1; then
305     die "colima not on PATH install with 'brew install colima' first"
306 fi
307 # No timeout wrapper (would mask actual VM startup time). If colima hangs,
308 # the user kills with Ctrl+C and can manually `colima restart` from a fresh
309 # terminal the rest of this script is idempotent so re-running works.
310 colima restart >> "$LOG_FILE" 2>&1 || die "colima restart failed; check $LOG_FILE"
311 ok "colima restarted"
312
313 # 6. Rebind DHCP on the ACTIVE interface (not hardcoded en0)
314 step "6/9 Rebinding DHCP on $ACTIVE_IFACE so host re-learns its real gateway"
315
316 # Map ifname macOS network service name (Wi-Fi, USB 10/100 LAN, etc.).
317 # Same logic as the diagnostic + toggle.sh.
318 ACTIVE_SVC=""
319 if [ "$PLATFORM" = "Darwin" ]; then
320     ACTIVE_SVC=$(get_active_service)
321 fi
322
323 case "$PLATFORM" in
324     Darwin)
325         sudo -n networksetup -setdhcp "$ACTIVE_SVC" renew >> "$LOG_FILE" 2>&1 \
326         && ok "DHCP rebind issued on $ACTIVE_SVC" \
327         || warn "setdhcp renew failed; try 'sudo ipconfig set $ACTIVE_IFACE DHCP' manually"
328         ;;
329     Linux)
330         sudo -n dhclient -r "$ACTIVE_IFACE" >> "$LOG_FILE" 2>&1 || true
331         sudo -n dhclient "$ACTIVE_IFACE" >> "$LOG_FILE" 2>&1 \
332         && ok "dhclient rebind on $ACTIVE_IFACE" \
333         || warn "dhclient rebind failed"
334         ;;
335     *)
336         warn "unsupported platform $PLATFORM; skip DHCP rebind"
337         ;;
338 esac
339
340 # 7. Verify new default route
341 step "7/9 Verifying the new default route is NOT the Docker bridge"
342 sleep 2 # give DHCP + kernel a beat
343 NEW_DEFAULT=$(netstat -rn 2>> "$LOG_FILE" | awk '/^default/ {print $0; exit}')
344 printf ' %s\n' "${NEW_DEFAULT:-no default route detected}"

```



```

345 NEW_GATEWAY=$(printf '%s' "$NEW_DEFAULT" | awk '{print $2; exit}')
346
347 if [ -n "$NEW_GATEWAY" ] && [ "${NEW_GATEWAY#172.17.}" != "$NEW_GATEWAY" ]; then
348     fail "default route STILL points at 172.17.x.x DHCP rebind may need manual touch"
349     warn "try: sudo ipconfig set $ACTIVE_IFACE DHCP (or reconnect Wi-Fi in the menu bar)"
350 elif [ -n "$NEW_GATEWAY" ]; then
351     ok "default route is now via $NEW_GATEWAY (no longer Docker's bridge)"
352 else
353     warn "could not determine new gateway; verify with 'route get default'"
354 fi
355
356 # 8. Re-run toggle.sh to bring the gateway back up
357 if [ "$SKIP_TOGGLE" = "true" ]; then
358     step "8/9 Skipping toggle.sh (--skip-toggle)"
359     warn "gateway is currently down run ./toggle.sh manually when ready"
360 else
361     step "8/9 Re-running toggle.sh to bring the gateway back up"
362     if bash "$PROJECT_ROOT/toggle.sh" 2>&1 | tee -a "$LOG_FILE"; then
363         ok "toggle.sh completed"
364     else
365         warn "toggle.sh returned non-zero see $LOG_FILE for details"
366     fi
367 fi
368
369 # 9. Re-run the diagnostic + show verdict
370 step "9/9 Re-running host-leak diagnostic to confirm fix"
371 if bash "$SCRIPT_DIR/diagnose-host-leak.sh" 2>&1 | tee -a "$LOG_FILE"; then
372     LATEST_REPORT=$(ls -t "$PROJECT_ROOT"/host-leak-diagnostic-*.txt 2>/dev/null | head -1)
373     if [ -n "$LATEST_REPORT" ]; then
374         VERDICT=$(awk '/^ Scenario:/ {print $2; exit}' "$LATEST_REPORT" 2>/dev/null)
375         WARP=$(awk -F': ' '/^ WARP egress:/ {print $2; exit}' "$LATEST_REPORT" 2>/dev/null)
376         printf '\n Most recent verdict: Scenario=%s WARP=%s\n' \
377             "${VERDICT:-?}" "${WARP:-?}"
378         if [ "$VERDICT" = "OK" ]; then
379             ok "fix confirmed by diagnostic host traffic is going through WARP"
380         fi
381     fi
382 else
383     warn "diagnostic exited non-zero; verify manually"
384 fi
385
386 printf '\n%b\b\n' "$BOLD" "$NC"
387 printf '%bDone.%b Full transcript at %s\n' "$BOLD" "$NC" "$LOG_FILE"
388 printf '%b\b\n' "$BOLD" "$NC"
389 exit 0

```

34 scripts/fix-ssh-delay.sh

```

1 #!/bin/bash
2 # scripts/fix-ssh-delay.sh Fix ~20s SSH connection delay caused by UseDNS
3 #
4 # macOS OpenSSH defaults UseDNS to "yes". When Tailscale SSH connects from
5 # a peer (phone/laptop), sshd performs a reverse-DNS (PTR) lookup on the
6 # client's 100.x.x.x Tailscale IP. Since DNS is hijacked to the nullexit
7 # gateway AdGuard Cloudflare WARP, and 100.64.0.0/10 is CGNAT private
8 # space, the PTR query times out (~15-20s) before SSH proceeds.
9 #
10 # This script uncomments "UseDNS no" in /etc/ssh/sshd_config and reloads
11 # sshd. Existing SSH sessions are NOT dropped.
12 #
13 # Usage:
14 #   sudo bash scripts/fix-ssh-delay.sh
15
16 set -euo pipefail
17
18 SSHD_CONFIG="/etc/ssh/sshd_config"
19
20 # Check we're root
21 if [ "$(id -u)" -ne 0 ]; then
22     echo "ERROR: This script must be run with sudo (it edits $SSHD_CONFIG)"
23     echo "Usage: sudo bash scripts/fix-ssh-delay.sh"
24     exit 1
25 fi
26
27 # Apply config (idempotent)
28 if grep -qE '^UseDNS no' "$SSHD_CONFIG" 2>/dev/null; then

```

```

29     echo " UseDNS is already set to 'no' in $SSH_CONFIG"
30 elif grep -qE '^#UseDNS' "$SSH_CONFIG" 2>/dev/null; then
31     echo " Uncommenting 'UseDNS no' in $SSH_CONFIG"
32     sed -i 's/^#UseDNS no/UseDNS no/' "$SSH_CONFIG"
33 elif grep -qEi '^UseDNS' "$SSH_CONFIG" 2>/dev/null; then
34     echo " Changing existing UseDNS line to 'UseDNS no'"
35     sed -i 's/^UseDNS.*/UseDNS no/' "$SSH_CONFIG"
36 else
37     echo " Appending 'UseDNS no' to $SSH_CONFIG"
38     echo "" >> "$SSH_CONFIG"
39     echo "UseDNS no" >> "$SSH_CONFIG"
40 fi
41
42 # Verify
43 if ! grep -qE '^UseDNS no' "$SSH_CONFIG"; then
44     echo " Failed to apply UseDNS no check $SSH_CONFIG manually"
45     exit 1
46 fi
47
48 # Reload sshd (does NOT drop existing connections)
49 echo " Reloading sshd..."
50
51 SSHD_PID=$(pgrep -f /usr/sbin/sshd 2>/dev/null | head -1 || true)
52 if [ -n "$SSHD_PID" ]; then
53     kill -HUP "$SSHD_PID"
54     echo " Sent SIGHUP to sshd (PID $SSHD_PID) config reloaded"
55 else
56     echo " No running sshd found. macOS starts sshd on-demand per connection."
57     echo " The new 'UseDNS no' config will take effect on the next SSH connection."
58 fi
59
60 echo ""
61 echo " Done. SSH connections should now connect instantly."
62 echo " Existing sessions were not interrupted."

```

35 scripts/generate__tex.py

```

1 #!/usr/bin/env python3
2 import os
3
4 PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
5
6 IGNORE_DIRS = {'.git', '.github', 'adguard', '__pycache__', 'Recover Gateway.app', 'Toggle Gateway.app'}
7 IGNORE_FILES = {'.env', 'nullexit_unified.tex', 'nullexit_unified.pdf', 'output.log', 'blocked.log', '.DS_Store',
8                 '.lan_p2p_detected', '.signatures', '.gateway_ip', 'TUNNEL_FAILED_CLOSED.marker', '.host_ips', 'refactor.md'}
9 IGNORE_EXTS = {'.pdf', '.tex', '.png', '.jpg', '.jpeg', '.zip', '.tar', '.gz', '.log', '.aux', '.fls', '.fdb_latexmk',
10                '.out', '.toc', '.xdv', '.synctex(busy)'}
11
12 # Exclude from code block inclusion, but keep in tree
13 SKIP_CONTENT_FILES = {'LICENSE'}
14
15 PRIORITY_FILES = [
16     'README.md',
17     'agent.md',
18     'devref.md',
19     'diagrams.md',
20     'toggle.sh',
21     'recover.sh',
22     'setup.sh',
23     'scripts/setup-linux.sh',
24     'docker-compose.yml',
25     'scripts/common.sh',
26     'scripts/watcher.sh',
27     'scripts/crypto.sh'
28 ]
29
30 def get_language(filename):
31     ext = os.path.splitext(filename)[1].lower()
32     if ext == '.sh' or ext == '.applescript':
33         return 'bash'
34     elif ext == '.py':
35         return 'Python'
36     elif ext == '.go':
37         return 'Go'
38     elif ext in {'.yaml', '.yml'}:

```



```

116 nullexit/
117 """
118
119     tree = generate_tree(PROJECT_ROOT)
120     midamble = r"""\end{verbatim}
121 \newpage
122 """
123
124 all_relpaths = []
125 for root, dirs, files in os.walk(PROJECT_ROOT):
126     dirs[:] = sorted([d for d in dirs if d not in IGNORE_DIRS])
127     for file in sorted(files):
128         if file in IGNORE_FILES or file in SKIP_CONTENT_FILES or os.path.splitext(file)[1].lower() in
IGNORE_EXTS:
129             continue
130
131         filepath = os.path.join(root, file)
132         relpath = os.path.relpath(filepath, PROJECT_ROOT)
133         all_relpaths.append(relpath)
134
135 def sort_key(path):
136     if path in PRIORITY_FILES:
137         return (0, PRIORITY_FILES.index(path))
138     return (1, path)
139
140 all_relpaths.sort(key=sort_key)
141
142 with open(tex_path, 'w', encoding='utf-8') as f:
143     f.write(preamble)
144     f.write(tree)
145     f.write(midamble)
146
147     for relpath in all_relpaths:
148         lang = get_language(relpath)
149         lang_attr = f"[language={lang}]" if lang else ""
150
151         f.write(f"\\section{{{escape_tex(relpath)}}}\n")
152         f.write(f"\\begin{{{lstlisting}}}{lang_attr}\n")
153         try:
154             with open(os.path.join(PROJECT_ROOT, relpath), 'r', encoding='utf-8', errors='ignore') as src:
155                 content = src.read()
156                 import re
157                 # Strip all non-ASCII characters (emojis, box drawings, em dashes, etc)
158                 content = re.sub(r'[\x00-\x7F]+', '', content)
159                 f.write(content)
160                 if not content.endswith('\n'):
161                     f.write('\n')
162             except Exception as e:
163                 f.write(f"Error reading file: {e}\n")
164             f.write("\\end{lstlisting}\n\n")
165
166         f.write("\\end{document}\n")
167
168     print(f"Generated {tex_path}")
169
170 if __name__ == '__main__':
171     main()

```

36 scripts/logger/go.mod

```

1 module nullexit/logger
2
3 go 1.22

```

37 scripts/logger/main.go

```

1 package main
2
3 import (
4     "bufio"
5     "encoding/json"
6     "fmt"

```

```

7  "io"
8  "log"
9  "os"
10 "regexp"
11 "strings"
12 "time"
13 )
14
15 const (
16     queryLogPath = "/adguard_work/data/querylog.json"
17     procKmsgPath = "/proc/kmsg"
18     outputLogPath = "/app/blocked.log"
19 )
20
21 func logMessage(msg string) {
22     timestamp := time.Now().Format("2006-01-02 15:04:05")
23     formattedMsg := fmt.Sprintf("%s %s\n", timestamp, msg)
24     fmt.Print(formattedMsg)
25
26     f, err := os.OpenFile(outputLogPath, os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
27     if err != nil {
28         log.Printf("Error writing to log file: %v\n", err)
29         return
30     }
31     defer f.Close()
32     f.WriteString(formattedMsg)
33 }
34
35 func tailFile(filepath string, out chan<- string) {
36     for {
37         if _, err := os.Stat(filepath); os.IsNotExist(err) {
38             time.Sleep(1 * time.Second)
39             continue
40         }
41         break
42     }
43
44     f, err := os.Open(filepath)
45     if err != nil {
46         log.Printf("Error opening file %s: %v\n", filepath, err)
47         return
48     }
49     defer func() {
50         if f != nil {
51             f.Close()
52         }
53     }()
54
55     // Seek to end
56     f.Seek(0, io.SeekEnd)
57     reader := bufio.NewReader(f)
58
59     for {
60         line, err := reader.ReadString('\n')
61         if err != nil {
62             if err == io.EOF {
63                 stat, err := os.Stat(filepath)
64                 if err == nil {
65                     currentOffset, _ := f.Seek(0, io.SeekCurrent)
66                     if stat.Size() < currentOffset {
67                         // File truncated or rotated
68                         f.Close()
69                         for {
70                             if _, err := os.Stat(filepath); os.IsNotExist(err) {
71                                 time.Sleep(1 * time.Second)
72                                 continue
73                             }
74                             break
75                         }
76                         f, _ = os.Open(filepath)
77                         reader = bufio.NewReader(f)
78                         continue
79                     }
80                 }
81                 time.Sleep(500 * time.Millisecond)
82                 continue
83             }
84             log.Printf("Error reading file %s: %v\n", filepath, err)
85             time.Sleep(1 * time.Second)
86             continue
87         }

```

```

88     out <- line
89 }
90 }
91
92 type adguardLogEntry struct {
93     IP      string `json:"IP"`
94     QH      string `json:"QH"`
95     QT      string `json:"QT"`
96     Result  struct {
97         IsFiltered bool `json:"IsFiltered"`
98         Reason      int  `json:"Reason"`
99         Rules      []struct {
100             Text string `json:"Text"`
101         } `json:"Rules"`
102     } `json:"Result"`
103 }
104
105 func monitorDNS() {
106     logMessage("[System] Starting DNS block logger...")
107     lines := make(chan string)
108     go tailFile(queryLogPath, lines)
109
110     reasonMap := map[int]string{
111         2: "CustomRule",
112         3: "BlockList",
113         4: "SafeBrowsing",
114         5: "ParentalControl",
115         6: "SafeSearch",
116         7: "BlockedService",
117     }
118
119     for line := range lines {
120         var data adguardLogEntry
121         if err := json.Unmarshal([]byte(line), &data); err != nil {
122             continue
123         }
124
125         res := data.Result
126         if res.IsFiltered || (res.Reason != 0 && res.Reason != 1) {
127             qh := data.QH
128             if qh == "" {
129                 qh = "unknown"
130             }
131             qt := data.QT
132             if qt == "" {
133                 qt = "unknown"
134             }
135             clientIP := data.IP
136             if clientIP == "" {
137                 clientIP = "unknown"
138             }
139
140             ruleText := "unknown rule"
141             if len(res.Rules) > 0 && res.Rules[0].Text != "" {
142                 ruleText = res.Rules[0].Text
143             }
144
145             reasonStr, ok := reasonMap[res.Reason]
146             if !ok {
147                 reasonStr = fmt.Sprintf("Reason-%d", res.Reason)
148             }
149
150             logMessage(fmt.Sprintf("[DNS] Blocked %s (Type: %s) for client %s | Reason: %s | Rule: %s", qh, qt,
151                 clientIP, reasonStr, ruleText))
152         }
153     }
154
155     func monitorIPs() {
156         logMessage("[System] Starting IP block logger...")
157
158         prefixRe := regexp.MustCompile(`^IP_BLOCK_[A-Z_]+`)
159         srcRe := regexp.MustCompile(`^SRC=([0-9a-fA-F\.:]+)`)
160         dstRe := regexp.MustCompile(`^DST=([0-9a-fA-F\.:]+)`)
161         protoRe := regexp.MustCompile(`^PROTO=([A-Z0-9]+)`)
162         sptRe := regexp.MustCompile(`^SPT=(\d+)`)
163         dptRe := regexp.MustCompile(`^DPT=(\d+)`)
164         inRe := regexp.MustCompile(`^IN=([a-zA-Z0-9\.\_-]+)`)
165         outRe := regexp.MustCompile(`^OUT=([a-zA-Z0-9\.\_-]+)`)
166
167         f, err := os.Open(procKmsgPath)

```

```

168 if err != nil {
169     if os.IsPermission(err) {
170         logMessage("[System] ERROR: Insufficient permissions to read /proc/kmsg. Enable CAP_SYSLOG.")
171     } else {
172         logMessage(fmt.Sprintf("[System] ERROR in IP logger: %v", err))
173     }
174     return
175 }
176 defer f.Close()
177
178 reader := bufio.NewReader(f)
179 for {
180     line, err := reader.ReadString('\n')
181     if err != nil {
182         if err == io.EOF {
183             time.Sleep(100 * time.Millisecond)
184             continue
185         }
186         logMessage(fmt.Sprintf("[System] ERROR reading kmsg: %v", err))
187         time.Sleep(1 * time.Second)
188         continue
189     }
190
191     if strings.Contains(line, "IP_BLOCK") {
192         match := prefixRe.FindStringSubmatch(line)
193         if len(match) > 1 {
194             prefix := match[1]
195
196             src := "unknown"
197             if m := srcRe.FindStringSubmatch(line); len(m) > 1 {
198                 src = m[1]
199             }
200
201             dst := "unknown"
202             if m := dstRe.FindStringSubmatch(line); len(m) > 1 {
203                 dst = m[1]
204             }
205
206             proto := "unknown"
207             if m := protoRe.FindStringSubmatch(line); len(m) > 1 {
208                 proto = m[1]
209             }
210
211             spt := ""
212             if m := sptRe.FindStringSubmatch(line); len(m) > 1 {
213                 spt = m[1]
214             }
215
216             dpt := ""
217             if m := dptRe.FindStringSubmatch(line); len(m) > 1 {
218                 dpt = m[1]
219             }
220
221             inIf := ""
222             if m := inRe.FindStringSubmatch(line); len(m) > 1 {
223                 inIf = m[1]
224             }
225
226             outIf := ""
227             if m := outRe.FindStringSubmatch(line); len(m) > 1 {
228                 outIf = m[1]
229             }
230
231             direction := "inbound"
232             if strings.HasSuffix(prefix, "DST") {
233                 direction = "outbound"
234             }
235
236             listType := "MALICIOUS"
237             if !strings.Contains(prefix, "MALICIOUS") {
238                 parts := strings.Split(prefix, "_")
239                 if len(parts) >= 3 {
240                     listType = "GEO_" + parts[2]
241                 }
242             }
243
244             portStr := ""
245             if dpt != "" {
246                 portStr = ":" + dpt
247             }
248

```



```

249     srcPortStr := ""
250     if spt != "" {
251         srcPortStr = "." + spt
252     }
253
254     ifStr := ""
255     if inIf != "" && outIf != "" {
256         ifStr = fmt.Sprintf("IF: %s->%s", inIf, outIf)
257     } else if inIf != "" || outIf != "" {
258         ifStr = fmt.Sprintf("IF: %s%s", inIf, outIf)
259     }
260
261     logMessage(fmt.Sprintf("[IP] Blocked %s to %s%s (Proto: %s) from %s%s (%s) | List: %s", direction, dst,
262         portStr, proto, src, srcPortStr, ifStr, listType))
263     }
264 }
265 }
266
267 func main() {
268     logMessage("[System] Nullexit Block Logger starting...")
269
270     go monitorDNS()
271     monitorIPs()
272 }

```

38 scripts/pf.conf

```

1 # scripts/pf.conf - Packet Filter configuration for nullexit killswitch
2 # Dynamically parameterized by pfctl macro override
3
4 # ext_if must be passed via command line, e.g. -D ext_if=en0
5 # If not passed, default to en0
6 ext_if = "en0"
7 ts_if = "utun+"
8
9 # Tables populated dynamically via pfctl
10 table <vpn_endpoints> persist
11 table <derp_relays> persist
12
13 # Core Rules
14 set skip on lo0                # Ensure local loopback is never blocked
15 scrub out all max-mss 1160    # TCP MSS Clamping: Prevents fragmentation on host TCP flows (like Tailscale
16                                # control plane)
17 pass quick on $ts_if all      # Allow all Tailscale traffic (essential to prevent SSH lockout)
18
19 # Default deny outbound WAN traffic
20 block out on $ext_if all
21
22 # Block inbound Screen Sharing (VNC) and SSH from local Wi-Fi, forcing them to use Tailscale
23 block in on $ext_if proto tcp from any to any port { 22, 5900 }
24
25 # Exceptions for tunnel handshakes and coordination
26 pass out quick on $ext_if proto udp to <vpn_endpoints> port 2408
27 pass out quick on $ext_if proto udp to <derp_relays>
28 pass out quick on $ext_if proto tcp to 192.200.0.0/24 port 443
29
30 # Allow local LAN traffic (AirDrop, local Tailscale P2P, etc)
31 pass out quick on $ext_if proto tcp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
32 pass out quick on $ext_if proto udp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
33 pass out quick on $ext_if proto icmp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
34
35 # Allow DHCP outbound (client port 68 to server port 67) so we can obtain/renew IP address
36 pass out quick on $ext_if proto udp from any port 68 to any port 67

```

39 scripts/reinstall-host-tailscale.sh

```

1 #!/usr/bin/env bash
2 #
3 # scripts/reinstall-host-tailscale.sh
4 #
5 # Complete uninstall + reinstall of the host-side Homebrew Tailscale.

```

```

6 #
7 # Use this when the brew LaunchAgent is wedged, Login Items entries are
8 # duplicated, "Tailscale is stopped" persists despite
9 # `brew services restart tailscale`, or you simply want a clean baseline
10 # before re-engaging the gateway as exit node.
11 #
12 # Idempotent. Safe to re-run. Use --dry to preview every step with zero
13 # changes. Use --yes to skip the per-step confirmation prompts.
14 #
15 # This script ONLY touches the HOST-side Tailscale (the one installed by
16 # Homebrew `brew install tailscale` and managed by launchd as
17 # homebrew.mxcl.tailscale). The nullexit gateway container's tailscaled
18 # (which lives inside the Colima VM at 100.100.21.8 and offers the exit
19 # node to this host) is unaffected.
20
21 set -u
22
23 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
24
25 DRY=0
26 YES=0
27
28 for arg in "$@"; do
29     case "$arg" in
30         --dry) DRY=1 ;;
31         --yes) YES=1 ;;
32         --help|-h) # NOTE: heredoc is UNQUOTED so $SCRIPT_DIR expands in the body. If you
33             # add new $-style placeholders here, they'll expand the same way.
34             cat <<USAGE
35 Usage: bash "$SCRIPT_DIR/reinstall-host-tailscale.sh" [--dry] [--yes]
36
37 --dry Print every step, make zero changes. Use to audit before
38 committing.
39 --yes Auto-confirm the prompts at every destructive step. Useful in
40 CI / fully-scripted recovery; do not use until you've read
41 the script.
42
43 After this script completes cleanly, you still must (MANUALLY):
44
45 1. Open System Settings Privacy & Security Local Network and
46 enable Tailscale. macOS will prompt automatically the first time
47 the fresh install of `tailscale` is invoked.
48
49 2. Re-engage the tailnet + exit node:
50     sudo tailscale up --ssh=true --accept-dns=false \
51       --exit-node="$(cat .gateway_ip | tr -d '\r' | awk 'NR==1{print $1; exit}')" \
52       --exit-node-allow-lan-access=true
53
54 3. Re-run ./toggle.sh from the project root.
55
56 4. Final verification:
57     curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)='
58     expected: warp=on, ip=<Cloudflare>, colo=YYZ/YUL
59 USAGE
60     exit 0
61     ;;
62     *)
63     echo "Unknown flag: $arg" >&2
64     exit 2
65     ;;
66     esac
67 done
68
69 # ----- helpers -----
70
71 confirm() {
72     if [ "$YES" = 1 ] || [ "$DRY" = 1 ]; then
73         return 0
74     fi
75     printf "      %s [y/N] " "$1"
76     read -r ans
77     case "$ans" in
78         y|Y|yes|YES) return 0 ;;
79         *)
80             echo "      aborted by user"
81             exit 1
82             ;;
83     esac
84 }
85
86 header() {

```

```

87 printf '\n %s\n' "$1"
88 }
89
90 # with_timeout <seconds> <cmd...>
91 # Run a command in the background and kill it if it exceeds <seconds>.
92 # Returns the command's exit code if it finished in time, 124 (matching
93 # GNU `timeout` convention) if it timed out. macOS doesn't ship GNU
94 # coreutils `timeout`, so this is the bash-3.2-friendly replacement.
95 # stdout + stderr are discarded (we only care about exit code); redirect
96 # or pipe externally if you need the output.
97 with_timeout() {
98     local timeout_s="${1:-30}"
99     shift
100     "$@" >/dev/null 2>&1 &
101     local cmdpid=$!
102     local elapsed=0
103     while kill -0 "$cmdpid" 2>/dev/null; do
104         if [ "$elapsed" -ge "$timeout_s" ]; then
105             kill -9 "$cmdpid" 2>/dev/null
106             wait "$cmdpid" 2>/dev/null
107             return 124
108         fi
109         sleep 1
110         elapsed=$((elapsed+1))
111     done
112     wait "$cmdpid" 2>/dev/null
113     return $?
114 }
115
116 # ----- 0. Pre-flight -----
117 header "Pre-flight current host tailscale state"
118 echo " brew: $(command -v brew >/dev/null 2>&1 && brew --version | head -1 || echo 'not on
119 PATH')"
119 echo " tailscaled running: $(pgrep -lf tailscaled 2>/dev/null | wc -l | tr -d ' ') process(es)"
120 echo " brew service tailscale: $(brew services list 2>/dev/null | awk ' $1=="tailscale"{print $2, $3}' | tr
121 '\n' ' ' | sed 's/ $//')
121 echo " LaunchAgent plists (under ~/Library/LaunchAgents/):"
122 ls -l ~/Library/LaunchAgents/ 2>/dev/null | grep -i 'tail' | sed 's/~/ /' || echo " (none)"
123
124 # Detect Tailscale's macOS Network Extension (residual from prior App Store
125 # Tailscale.app installs). Lives under /Library/Apple/SystemExtensions/ and
126 # is managed by `systemextensionsctl` NOT launchd. If loaded, brew
127 # tailscaled will NOT engage the Network Extension framework slot because
128 # the SE still has it claimed. Symptom: 'sudo tailscale status' returns
129 # "Tailscale is stopped" even when the daemon is alive and listening.
130 se_found=0
131 if command -v systemextensionsctl >/dev/null 2>&1; then
132     # Grab any line matching tailscale and network-extension
133     se_line=$(systemextensionsctl list 2>/dev/null | grep -iE '(io|com)\.tailscale\..*network-extension' | head -
134 n 1)
134 if [ -n "$se_line" ]; then
135     se_found=1
136     # Extract the teamID (10-char uppercase/numeric word)
137     se_teamID=$(echo "$se_line" | tr -s ' \t' '\n' | grep -E '^[A-Z0-9]{10}$' | head -n 1)
138     # Extract the bundleID (starts with io.tailscale or com.tailscale)
139     se_bundleID=$(echo "$se_line" | tr -s ' \t' '\n' | grep -E '^(io|com)\.tailscale\.' | head -n 1 | sed 's
140 /(.*/)/')
141
142     # If teamID is not found, fallback to '-'
143     if [ -z "$se_teamID" ]; then
144         se_teamID="-"
145     fi
146 fi
147
148 if [ "$se_found" = 1 ] && [ -n "$se_bundleID" ]; then
149     echo " Tailscale Network Extension System Extension is loaded (residual from App Store Tailscale.app
150 install)"
151 echo " Team ID: $se_teamID, Bundle ID: $se_bundleID"
152 echo " brew tailscaled cannot engage the NE slot while this SE has it claimed."
153 if [ "$DRY" = 1 ]; then
154     echo " [dry] (would prompt) sudo systemextensionsctl uninstall $se_teamID $se_bundleID"
155 else
156     confirm "Run 'sudo systemextensionsctl uninstall $se_teamID $se_bundleID'? (frees the NE slot for brew
157 tailscale; non-fatal if it errors)"
158 sudo systemextensionsctl uninstall "$se_teamID" "$se_bundleID" 2>&1 \
159 || echo " ! uninstall returned non-zero (may require reboot, non-fatal)"
160 fi
161 else
162     echo " no Tailscale System Extension loaded"
163 fi

```

```

162
163 # Pattern used by the Step-1 bootout loops, in both dry + real branches.
164 # Widened from a bare /tailscale/ so the same loops also pick up any
165 # nullexit-labeled LaunchAgents (e.g., com.nullexit.wake-recovery, the
166 # caffeinate + post-wake recovery hook). The wake-recovery LaunchAgent
167 # is re-installed by ./toggle.sh / setup.sh on re-engage, so wiping it
168 # here is non-fatal and is part of restoring a clean baseline.
169 TAILSCALE_LABEL_RE='tailscale'
170 NULLEXIT_LABEL_RE='nullexit'
171
172 # ----- 1. brew services stop -----
173 header "Step 1 Stop the brew-managed service + unload LaunchAgents"
174 # Detect whether the brew-managed service is registered as $USER (gui/$UID
175 # domain) or as root (system domain). NOPASSWD sudoers can leave a stale
176 # root-owned registration after a prior 'sudo brew services ...' round;
177 # 'brew services stop' without sudo refuses to touch a root-owned plist
178 # (Error: Service 'tailscale' is started as 'root').
179 brew_status_line="$(brew services list 2>/dev/null | awk '1=="tailscale"')
180 brew_status_user="$(echo "$brew_status_line" | awk '{print $3}')"
181 if [ -n "$brew_status_line" ]; then
182     if [ "$DRY" = 1 ]; then
183         echo "    [dry] brew services stop tailscale (user=$brew_status_user)"
184     else
185         if [ "$brew_status_user" = "root" ]; then
186             confirm "Run 'sudo brew services stop tailscale' (service is registered as root)?"
187             sudo brew services stop tailscale 2>&1 || echo "    (sudo brew services stop returned non-zero non-fatal)"
188         else
189             echo "    brew services stop tailscale (user=$brew_status_user, no sudo needed)"
190             brew services stop tailscale 2>&1 || echo "    (brew reported it wasn't actually running)"
191         fi
192     fi
193     echo "    unload any tailscale-related LaunchAgents (prevents launchd from respawning after kill)"
194     if [ "$DRY" = 1 ]; then
195         echo "    [dry] for each 'tailscale|nullexit'-matched label in gui/$UID domain:"
196         while IFS= read -r label; do
197             [ -z "$label" ] && continue
198             echo "    [dry] launchctl bootout gui/$UID/$label"
199             done < <(launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" 'NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
200             echo "    [dry] for each 'tailscale|nullexit'-matched label in system domain (root-owned brew plist):"
201             while IFS= read -r label; do
202                 [ -z "$label" ] && continue
203                 echo "    [dry] (would prompt) sudo launchctl bootout system/$label"
204                 done < <(sudo -n launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" 'NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
205             if launchctl print system/com.tailscale.ipn.macos >/dev/null 2>&1; then
206                 echo "    [dry] (would prompt) sudo launchctl bootout system/com.tailscale.ipn.macos"
207             fi
208             if launchctl print system/com.tailscale.ipn.macsys >/dev/null 2>&1; then
209                 echo "    [dry] (would prompt) sudo launchctl bootout system/com.tailscale.ipn.macsys"
210             fi
211         else
212             # Bootout every gui-domain LaunchAgent whose label matches 'tailscale'
213             # OR 'nullexit'. The nullexit arm catches com.nullexit.wake-recovery,
214             # which holds caffeinate/watcher.sh alive across a daemon reset not
215             # desired while the script is establishing a fresh baseline. Awake-
216             # recovery is re-installed by ./toggle.sh / setup.sh on re-engage, so
217             # wiping here is non-fatal. The pattern stays narrower than a bare
218             # /tail/ or /null/ so unrelated Apple services are not unloaded.
219             while IFS= read -r label; do
220                 [ -z "$label" ] && continue
221                 echo "    launchctl bootout gui/$UID/$label"
222                 launchctl bootout "gui/$UID/$label" 2>/dev/null || true
223             done < <(launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" 'NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
224             # Bootout every system-domain LaunchDaemon/Agent whose label matches
225             # 'tailscale' OR 'nullexit'. This catches the root-owned brew plist left
226             # behind by prior 'sudo brew services ...' invocations that was the
227             # case that left tailscaled PID 47447 57734 respawning between sudo
228             # pkill runs. Defensive nullexit catch in case the user previously
229             # `sudo ln -s`'d the wake-recovery plist into /Library/LaunchDaemons.
230             while IFS= read -r label; do
231                 [ -z "$label" ] && continue
232                 confirm "Run 'sudo launchctl bootout system/$label'?"
233                 sudo launchctl bootout "system/$label" 2>/dev/null || true
234             done < <(sudo -n launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" 'NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
235             # Legacy Tailscale.app system-domain entries. Each one is checked and
236             # confirmed separately so we don't sudo-prompt or sudo-bootout for nothing.
237             if launchctl print system/com.tailscale.ipn.macos >/dev/null 2>&1; then

```

```

238     confirm "Run 'sudo launchctl bootout system/com.tailscale.ipn.macos'"
239     sudo launchctl bootout system/com.tailscale.ipn.macos 2>/dev/null || true
240 fi
241 if launchctl print system/com.tailscale.ipn.macsys >/dev/null 2>&1; then
242     confirm "Run 'sudo launchctl bootout system/com.tailscale.ipn.macsys'"
243     sudo launchctl bootout system/com.tailscale.ipn.macsys 2>/dev/null || true
244 fi
245 fi
246 echo "    waiting up to 10s for tailscaled to exit"
247 for i in 1 2 3 4 5 6 7 8 9 10; do
248     if [ "$DRY" = 1 ]; then
249         echo "    [dry] sleep 1; loop check would terminate here"
250         break
251     fi
252     sleep 1
253     if ! pgrep -lf tailscaled >/dev/null 2>&1; then
254         echo "    tailscaled exited in ${i}s"
255         break
256     fi
257     [ "$i" = 10 ] && echo "    tailscaled still alive after 10s Step 2 will deal with it"
258 done
259 else
260     echo "    brew reports no tailscale service loaded"
261 fi
262
263 # ----- 2. Kill any orphan tailscaled -----
264 header "Step 2 Kill any orphan tailscaled process(es) (may escalate to sudo)"
265 if ! pgrep -lf tailscaled >/dev/null 2>&1; then
266     echo "    no orphans"
267 else
268     echo "    Stray process(es):"
269     pgrep -lf tailscaled | sed 's/^/    /'
270     echo "    no-sudo: pkill tailscaled"
271     if [ "$DRY" = 1 ]; then
272         echo "    [dry] pkill tailscaled on failure: sudo pkill -9 tailscaled"
273     else
274         if pkill tailscaled 2>/dev/null; then
275             echo "    no-sudo pkill succeeded"
276         else
277             echo "    ! no-sudo pkill insufficient; cannot escalate yet"
278         fi
279         sleep 1
280     fi
281 fi
282
283 # Recheck; if still alive, offer sudo escalation
284 if { [ "$DRY" = 0 ] && pgrep -lf tailscaled >/dev/null 2>&1; } || [ "$DRY" = 1 ]; then
285     if [ "$DRY" = 1 ]; then
286         echo "    [dry] sudo pkill -9 tailscaled (escalation would fire here)"
287     else
288         confirm "Run 'sudo pkill -9 tailscaled' to escalate?"
289         sudo pkill -9 tailscaled 2>&1
290         sleep 1
291     fi
292 fi
293
294 if [ "$DRY" = 0 ]; then
295     if pgrep -lf tailscaled >/dev/null 2>&1; then
296         echo "    Failed to kill stragglers bailing out"
297         pgrep -lf tailscaled | sed 's/^/    /'
298         exit 3
299     fi
300     echo "    all tailscaled processes gone"
301 fi
302
303 # ----- 3. brew uninstall -----
304 header "Step 3 brew uninstall tailscale"
305 if ! brew list tailscale >/dev/null 2>&1; then
306     echo "    not currently installed via brew skip"
307 else
308     # Detect whether the keg directory is owner=$USER or owner=root. If the
309     # keg is root-owned (typically a residual of some prior 'sudo brew ...'
310     # cycle that did perms fixups), 'brew uninstall' without sudo refuses
311     # to remove the files and prints: 'Error: Could not remove tailscale
312     # keg! Do so manually: sudo rm -rf /opt/homebrew/Cellar/tailscale/<v>'.
313     # Detect that and escalate to a sudoed uninstall, with a manual rm -rf
314     # fallback if `sudo brew uninstall` itself errors out (e.g., partial
315     # state where the keg dir is gone but brew still thinks it's installed).
316     # Detect keg state. Treat `(dir absent)` AND `(dir present + empty)` as the
317     # same `<unknown>` bucket so both escalate to sudo. The empty-parent case
318     # is what you get after a manual `sudo rm -rf /opt/homebrew/Cellar/tailscale/<v>`

```

```

319 # from inside: brew's parent dir is left present-but-empty and brew still
320 # thinks the package is installed.
321 keg_dir="/opt/homebrew/Cellar/tailscale"
322 if [ -d "$keg_dir" ] && [ -n "$(ls -A "$keg_dir" 2>/dev/null)" ]; then
323     keg_owner="$(stat -f '%Su' "$keg_dir" 2>/dev/null || echo '<unknown>')"
324 else
325     keg_owner="<unknown>"
326 fi
327 if [ "$DRY" = 1 ]; then
328     echo "    [dry] brew uninstall tailscale (keg_owner=$keg_owner)"
329 else
330     # 'root' AND '<unknown>' both require sudo escalation. The '<unknown>'
331     # case is the partial-state path: keg dir was already removed (manually
332     # or by a prior failed uninstall) but brew's internal state still says
333     # installed. Without this branch the script would fall through to a
334     # non-sudo brew uninstall that fails again with the same 'Could not
335     # remove tailscale keg!' error.
336     if [ "$keg_owner" = "root" ] || [ "$keg_owner" = "<unknown>" ]; then
337         confirm "Run 'sudo brew uninstall tailscale' (keg=$keg_owner)?"
338         sudo brew uninstall tailscale 2>&1 || {
339             echo "    ! sudo brew uninstall failed; offering manual rm -rf fallback"
340             # Guard the glob so an empty/absent dir doesn't literal-expand the
341             # '*' and confuse the user with a sudo error about a non-existent path.
342             if [ -d /opt/homebrew/Cellar/tailscale ] && [ -n "$(ls -A /opt/homebrew/Cellar/tailscale 2>/dev/null)"
343 ]; then
344                 confirm "Run 'sudo rm -rf /opt/homebrew/Cellar/tailscale/*'?"
345                 sudo rm -rf /opt/homebrew/Cellar/tailscale/* 2>&1
346             else
347                 echo "    keg dir already empty or absent nothing more to rm"
348             fi
349         }
350     else
351         confirm "Run 'brew uninstall tailscale' (removes LaunchAgent plist + linked symlinks)?"
352         brew uninstall tailscale 2>&1
353     fi
354 fi
355
356 # ----- 4. Wipe state directories -----
357 header "Step 4 Wipe stale state"
358 # User-owned
359 declare -a USER_PATHS
360 USER_PATHS=(
361     "$HOME/.local/share/tailscale"
362     "$HOME/.local/run/tailscaled.socket"
363     "$HOME/.local/run/tailscaled.state"
364     "$HOME/.local/state/tailscale"
365     "$HOME/Library/Application Support/Tailscale"
366     "$HOME/Library/Caches/com.tailscale.ipn.macos"
367     "$HOME/Library/Caches/Tailscale"
368     "$HOME/Library/Logs/Tailscale"
369 )
370 echo "    User-owned paths (no sudo):"
371 for p in "${USER_PATHS[@]}; do
372     if [ -e "$p" ]; then
373         if [ "$DRY" = 1 ]; then
374             echo "    [dry] rm -rf '$p'"
375         else
376             rm -rf "$p"
377             echo "    rm -rf '$p'"
378         fi
379     fi
380 done
381
382 # System
383 declare -a SUDO_PATHS
384 SUDO_PATHS=(
385     "/var/lib/tailscale"
386     "/var/cache/tailscale"
387 )
388 echo "    System paths under /var (will prompt for sudo if any are present):"
389 eligible_sudo=0
390 for p in "${SUDO_PATHS[@]}; do
391     if [ -e "$p" ]; then
392         eligible_sudo=1
393         if [ "$DRY" = 1 ]; then
394             echo "    [dry] sudo rm -rf '$p'"
395         else
396             if confirm "Run 'sudo rm -rf $p'?" ; then
397                 sudo rm -rf "$p" && echo "    rm -rf '$p'"
398             fi
399         fi
400     fi
401 done

```

```

399     fi
400 done
401 [ "$eligible_sudo" = 0 ] && echo "      (none of /var/lib/tailscale, /var/cache/tailscale are present)"
402
403 # ----- 5. Wipe stale LaunchAgent plists -----
404 header "Step 5 Wipe stale LaunchAgent plist files"
405 declare -a PLIST_FILES
406 PLIST_FILES=(
407     "$HOME/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
408     "$HOME/Library/LaunchAgents/com.tailscale.ipn.macos.plist"
409     "$HOME/Library/LaunchAgents/homebrew.mxcl.tailscale.plist.bak"
410     "$HOME/Library/LaunchAgents/com.nullexit.wake-recovery.plist"
411 )
412 for f in "${PLIST_FILES[@]}; do
413     if [ -e "$f" ]; then
414         if [ "$DRY" = 1 ]; then
415             echo "      [dry] rm -f '$f'"
416         else
417             rm -f "$f"
418             echo "      rm -f '$f'"
419         fi
420     fi
421 done
422 echo "      Remaining tailscale|nullexit plists (should be empty):"
423 ls -l ~/Library/LaunchAgents/ 2>/dev/null | grep -i 'tail' | sed 's/^/' || echo "      (none)"
424
425 # ----- 5.5. Drain Background-Items (Login Items) -----
426 header "Step 5.5 Drain Background-Items (Login Items)"
427 # macOS 13+ keeps a parallel "Background Items" database alongside
428 # classic LaunchAgents. Even when we wipe ~/Library/LaunchAgents/homebrew.mxcl.tailscale.plist
429 # in Step 5, the .btm file at
430 # ~/Library/Application Support/com.apple.backgroundtaskmanagementagent.backgrounditems.btm
431 # can still hold a Background-Items entry pointing at the (now-dead) plist
432 # path. Next login: macOS sees the entry, tries to bootstrap, finds the
433 # plist missing, and either silently regenerates it via brew's post-install
434 # hook (causing tailscaled respawn at next login) or refuses outright.
435 # `sfltool reset-login-items` is Apple's canonical API for wiping the
436 # entire .btm file. It DOES wipe all Login Items the user re-adds what
437 # they want via System Settings General Login Items. Acceptable here
438 # because the goal of this script is "fresh baseline for Tailscale".
439 BTM="$HOME/Library/Application Support/com.apple.backgroundtaskmanagementagent/backgrounditems.btm"
440 if [ -f "$BTM" ]; then
441     if [ "$DRY" = 1 ]; then
442         echo "      [dry] found $BTM; would prompt sfltool reset-login-items"
443     else
444         confirm "Run 'sfltool reset-login-items' (wipes ALL Login Items; rebuild via System Settings General
445             Login Items)?"
446         if sfltool reset-login-items 2>&1; then
447             echo "      Login Items drained"
448         else
449             echo "      ! sfltool reset-login-items failed (non-fatal; do manually: System Settings General Login
450                 Items)"
451         fi
452     fi
453 else
454     echo "      no backgrounditems.btm present"
455 fi
456
457 # ----- 6. brew install -----
458 header "Step 6 brew install tailscale (fresh)"
459 if brew list tailscale >/dev/null 2>&1; then
460     echo "      already installed skip"
461 else
462     confirm "Run 'brew install tailscale'?"
463     if [ "$DRY" = 1 ]; then
464         echo "      [dry] brew install tailscale"
465     else
466         brew install tailscale 2>&1
467     fi
468 fi
469
470 # ----- 6.5. Strip quarantine xattr from keg -----
471 header "Step 6.5 Strip quarantine xattr from tailscale install"
472 # brew's freshly poured bottles don't carry com.apple.quarantine (the
473 # bottles are content-addressed and Homebrew-verified), but the upstream
474 # installer (Tailscale.app from App Store) and any manual download do.
475 # Strip the xattr from the entire keg as a defensive safety net so the
476 # freshly-installed tailscaled binary and any helper binaries are not
477 # Gatekeeper-blocked on first launch. Idempotent no-op if no quarantine
478 # xattr is present.

```



```

478 quarantine_count="$(xattr -l /opt/homebrew/Cellar/tailscale 2>/dev/null | grep -c '^[^]* com.apple.quarantine
    ' || echo 0)"
479 if [ "${quarantine_count:-0}" -gt 0 ]; then
480     if [ "$DRY" = 1 ]; then
481         echo "    [dry] found $quarantine_count file(s) with com.apple.quarantine; would prompt xattr -dr"
482     else
483         confirm "Run 'xattr -dr com.apple.quarantine /opt/homebrew/Cellar/tailscale'? (strips quarantine xattr from
            $quarantine_count file(s); non-fatal if it errors)"
484         if xattr -dr com.apple.quarantine /opt/homebrew/Cellar/tailscale 2>&1; then
485             echo "        quarantine xattr stripped"
486         else
487             echo "        ! xattr -dr failed (non-fatal; open /opt/homebrew/Cellar/tailscale in Finder, right-click
            tailscaled Open Anyway)"
488         fi
489     fi
490 else
491     echo "        no com.apple.quarantine xattr on tailscale install"
492 fi
493
494 # ----- 7. brew services start (NO sudo!) -----
495 header "Step 7 Start the service as $USER (NO sudo, with multi-tier self-healing)"
496 if ! command -v tailscale >/dev/null 2>&1; then
497     echo "        tailscale not on PATH after install brew install errored; aborting"
498     exit 4
499 fi
500
501 # Aliveness fan-out: ANY one of these is taken as proof of life. None of
502 # them alone is reliable pgrep misses tailscaled if it does setproctitle
503 # or fork-detach; launchctl's reported PID can be a shim that exited
504 # within 1s; tailscale-cli hangs while macOS Local-Network permission is
505 # unresolved. Combined, they cover each other's blind spots.
506
507 tailscaled_api_up() {
508     # Strategy A: tailscale CLI can talk to the local API. Canonical proof.
509     # Bound at 8s tailscale status itself returns quickly on success OR
510     # failure, but if the post-install state is wedged it can hang.
511     with_timeout 8 /opt/homebrew/bin/tailscale status >/dev/null 2>&1
512 }
513
514 tailscaled_proc_up() {
515     # Strategy B: any process has 'tailscaled' substring in argv.
516     pgrep -lf tailscaled >/dev/null 2>&1
517 }
518
519 tailscaled_lc_up() {
520     # Strategy C: launchctl-managed service has a non-dash, non-zero PID.
521     local lc_pid
522     lc_pid="$(launchctl list 2>/dev/null | awk '$3 == "homebrew.mxcl.tailscale" {print $1; exit;}")"
523     if [ -n "$lc_pid" ] && [ "$lc_pid" != "-" ] && [ "$lc_pid" -gt 0 ] 2>/dev/null; then
524         return 0
525     fi
526     return 1
527 }
528
529 tailscaled_is_alive() {
530     tailscaled_api_up && return 0
531     tailscaled_proc_up && return 0
532     tailscaled_lc_up && return 0
533     return 1
534 }
535
536 confirm "Run 'brew services start tailscale'?"
537 if [ "$DRY" = 1 ]; then
538     echo "    [dry] brew services start tailscale"
539     echo "    [dry] (then poll up to 30s for tailscaled_is_alive)"
540     echo "    [dry] (recovery tier 1: launchctl kickstart)"
541     echo "    [dry] (recovery tier 2: launchctl bootout + bootstrap of explicit plist)"
542     echo "    [dry] (recovery tier 3: brew services restart (stop+start cycle))"
543     echo "    [dry] (recovery tier 4: direct /opt/homebrew/bin/tailscaled spawn (bypasses launchd))"
544 else
545     brew services start tailscale 2>&1
546     # Primary poll tailscaled_is_alive covers all 3 detection strategies.
547     echo "    polling for tailscaled to come up (up to ~30s)"
548     tailscaled_up=0
549     for i in 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15; do
550         if tailscaled_is_alive; then
551             tailscaled_up=1
552             echo "        tailscaled alive after ${i}*2s"
553             break
554         fi
555         sleep 2

```

```

556 done
557
558 # Recovery tier 1: kickstart the launchd-managed service. This is the
559 # cheapest and most-aligned-with-launchd fix when brew has loaded the
560 # LaunchAgent but tailscaled didn't fork. Kickstart tells launchd
561 # "re-spawn if not running" without forcing a teardown.
562 if [ "$tailscaled_up" = 0 ]; then
563     echo "    ! Recovery tier 1: launchctl kickstart"
564     if launchctl kickstart -kp "gui/$UID/homebrew.mxcl.tailscale" 2>&1; then
565         for i in 1 2 3 4 5 6 7 8 9 10; do
566             if tailscaled_is_alive; then
567                 tailscaled_up=1
568                 echo "        tailscaled alive after kickstart (${i}*2s)"
569                 break
570             fi
571             sleep 2
572         done
573     else
574         echo "    ! kickstart returned non-zero (LaunchAgent may not be loaded)"
575     fi
576 fi
577
578 # Recovery tier 2: explicit plist bootout + bootstrap. Forces a fresh
579 # launchd registration cycle for the brew-managed service, which fixes
580 # the wider class of "brew says started but daemon never spawned"
581 # wedge states that kickstart alone can't dig out of.
582 if [ "$tailscaled_up" = 0 ]; then
583     echo "    ! Recovery tier 2: launchctl bootout + bootstrap of explicit plist"
584     launchctl bootout "gui/$UID/homebrew.mxcl.tailscale" 2>/dev/null || true
585     sleep 1
586     # Brew installs the canonical plist template at $HOMEBREW_PREFIX/Library/.
587     # On Apple Silicon that's /opt/homebrew/Library/LaunchAgents/. On Intel
588     # it's /usr/local/Library/LaunchAgents/. Detect which and pick the
589     # right one so this script works across both architectures.
590     if [ -f /opt/homebrew/Library/LaunchAgents/homebrew.mxcl.tailscale.plist ]; then
591         plist_path="/opt/homebrew/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
592     elif [ -f /usr/local/Library/LaunchAgents/homebrew.mxcl.tailscale.plist ]; then
593         plist_path="/usr/local/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
594     else
595         plist_path=""
596     fi
597     if [ -n "$plist_path" ]; then
598         if launchctl bootstrap "gui/$UID" "$plist_path" 2>&1; then
599             for i in 1 2 3 4 5 6 7 8 9 10; do
600                 if tailscaled_is_alive; then
601                     tailscaled_up=1
602                     echo "        tailscaled alive after bootstrap (${i}*2s)"
603                     break
604                 fi
605                 sleep 2
606             done
607         else
608             echo "    ! explicit bootstrap returned non-zero"
609         fi
610     else
611         echo "    ! could not locate brew plist template; skipping bootstrap tier"
612     fi
613 fi
614
615 # Recovery tier 3: brew services restart (forces a stop + start cycle,
616 # which unsticks LaunchAgents with stale 'started' state from prior
617 # half-cycles). This is the canonical brew-doc fix for 'services don't
618 # actually start even though brew says started'.
619 if [ "$tailscaled_up" = 0 ]; then
620     echo "    ! Recovery tier 3: brew services restart (forces stop+start cycle)"
621     if brew services restart tailscale 2>&1; then
622         for i in 1 2 3 4 5 6 7 8 9 10; do
623             if tailscaled_is_alive; then
624                 tailscaled_up=1
625                 echo "        tailscaled alive after brew restart (${i}*2s)"
626                 break
627             fi
628             sleep 2
629         done
630     else
631         echo "    ! brew services restart returned non-zero"
632     fi
633 fi
634
635 # Recovery tier 4: last-resort direct spawn of the daemon binary,
636 # bypassing launchd entirely. If tailscaled crashes here too, we'll

```

```

637 # capture its actual stderr in /tmp for the user to read (NOPASSWD
638 # sudoers often have local-network permission denied the captured
639 # stderr makes it obvious what to fix in System Settings).
640 if [ "$tailscaled_up" = 0 ]; then
641     echo "    ! Recovery tier 4: direct spawn (bypasses launchd; tail stderr to /tmp)"
642     /opt/homebrew/bin/tailscaled >/tmp/nullexit-tailscaled-spawn.log 2>&1 &
643     spawned_pid=$!
644     disown "$spawned_pid" 2>/dev/null || true
645     sleep 3
646     if tailscaled_is_alive; then
647         tailscaled_up=1
648         echo "        tailscaled alive via direct spawn (pid=$spawned_pid)"
649         echo "        (NOTE: not under launchd supervision a reboot, re-run of"
650         echo "        this script, or ./toggle.sh is required to recreate."
651         echo "        Last 5 lines of stderr captured to /tmp/nullexit-tailscaled-spawn.log:)"
652         tail -n 5 /tmp/nullexit-tailscaled-spawn.log 2>/dev/null | sed 's/~/ /' || true
653     fi
654 fi
655
656 if [ "$tailscaled_up" = 0 ]; then
657     echo "    HARD FAIL: tailscaled refuses to stay alive across every recovery tier"
658     echo "    launchctl-managed state (if available):"
659     launchctl print "gui/$UID/homebrew.mxcl.tailscale" 2>/dev/null \
660     | grep -E 'state =|last exit code =|runs =|path =' \
661     | sed 's/~/ /' \
662     || echo "    (no launchctl state available for homebrew.mxcl.tailscale)"
663     echo "    Last 10 lines of /tmp/nullexit-tailscaled-spawn.log (if any):"
664     tail -n 10 /tmp/nullexit-tailscaled-spawn.log 2>/dev/null | sed 's/~/ /' || echo "    (no log)"
665     echo "    Most likely remaining cause: macOS Local Network permission denied."
666     echo "    System Settings Privacy & Security Local Network toggle Tailscale ON."
667     exit 7
668 fi
669 fi
670 sleep 3
671
672 # ----- 8. Verify -----
673 header "Step 8 Verify fresh install"
674 echo "    brew service tailscale: $(brew services list 2>/dev/null | awk '$1=="tailscale"{{print $2, $3}}' | tr
675 '\n' ' ' | sed 's/ $//')
676 echo "    tailscaled process: $(pgrep -lf tailscaled 2>/dev/null | head -1 || echo 'NONE (check brew logs)
677 ')
678 ts_ver=""
679 if [ -x /opt/homebrew/bin/tailscale ]; then
680     ts_ver="$(/opt/homebrew/bin/tailscale version 2>/dev/null | head -1 | tr -d '\n')"
681 elif command -v tailscale >/dev/null 2>&1; then
682     ts_ver="$(tailscale version 2>/dev/null | head -1 | tr -d '\n')"
683 fi
684 echo "    tailscale version: ${ts_ver:-binary missing}"
685 ts_status=""
686 if [ -x /opt/homebrew/bin/tailscale ]; then
687     ts_status="$(/opt/homebrew/bin/tailscale status 2>/dev/null | head -3)"
688 fi
689 if [ -n "$ts_status" ]; then
690     echo "    tailscale status:"
691     printf '%s\n' "$ts_status" | sed 's/~/ /'
692 else
693     echo "    tailscale status: (binary missing or errored)"
694 fi
695
696 if { [ "$DRY" = 0 ] && /opt/homebrew/bin/tailscale status 2>/dev/null | grep -q '^Tailscale is stopped'; };
697 then
698     echo
699     echo "    NOTICE: 'tailscale status' still reports stopped after a fresh install."
700     echo "    Two equally likely causes:"
701     echo "    (a) macOS hasn't yet shown the first-run 'Local Network' prompt for"
702     echo "        Tailscale open System Settings Privacy & Security Local"
703     echo "        Network. Toggle Tailscale ON."
704     echo "    (b) macOS denied Local Network permission at some prior point."
705     echo "    Either way: System Settings Privacy & Security Local Network."
706     echo "    If the entry isn't present, invoking any tailscale command will"
707     echo "    re-trigger the prompt automatically."
708 fi
709
710 # ----- 9. Done -----
711 header "Done"
712 if [ "$DRY" = 1 ]; then
713     echo "    (dry-mode finished; no follow-up commands were issued.)"
714     exit 0
715 fi
716
717 cat <<'DONE'

```

```

715 Manual follow-ups (all of these; the script stops short of running sudo
716 tailscale up so you can verify the brew install is actually live first):
717
718 1. System Settings Privacy & Security Local Network Tailscale ON
719 macOS will normally pop the prompt the first time you run `tailscale`.
720 If it didn't, trigger it via the menu bar check.
721
722 2. Re-engage the tailnet + exit node:
723     sudo tailscale up --ssh=true --accept-dns=false \
724       --exit-node="$(cat .gateway_ip | tr -d '\r' | awk 'NR==1{print $1; exit}')" \
725       --exit-node-allow-lan-access=true
726
727 3. Re-run: ./toggle.sh
728 (re-installs the com.nullexit.wake-recovery LaunchAgent wiped
729 in Step 5 to clear the slate and re-engages the gateway)
730
731 4. Final verification:
732     curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)=|
733     expect: warp=on, ip=<Cloudflare IP>, colo=YYZ/YUL
734
735 Container-side (gateway / 100.100.21.8) Tailscale is unaffected by this
736 script. Other tailnet devices (S24, iPhone, Windows) are unaffected.
737
738 DONE
739

```

40 scripts/routing-fix.sh

```

1 #!/bin/sh
2 # routing-fix: Maintains routing for SOCKS5 proxy traffic through WARP (tun0)
3 # and FORWARD rules for Tailscale exit-node return traffic.
4 #
5 # Runs inside the warp container's network namespace. This script ensures:
6 # 1. Table 200 has default via tun0 (for SOCKS5 proxy traffic from 172.18.0.2)
7 # with the WARP endpoint exception via eth0 (prevents tunnel loop)
8 # 2. The CGNAT ip rule (100.64.0.0/10 table 52 tailscale routes) is maintained for Tailscale
9 # 3. FORWARD RELATED,ESTABLISHED rule allows return traffic for exit-node
10 # forwarded connections (S24 tailscale0 eth0 host internet)
11 #
12 # CRITICAL: Does NOT touch the main table's default route. Gluetun handles that
13 # internally via its own routing rules and WireGuard fwmark (0xca6c table 51820).
14 # Overriding the main table default would create a loop: encrypted WireGuard
15 # packets exiting tun0 would match default dev tun0 and re-enter the tunnel.
16
17 set -e
18
19 trap 'echo "routing-fix: Caught signal, exiting..."; exit 0' TERM INT QUIT
20
21 sleep 15 &
22 wait $! || true
23
24 echo "routing-fix: Setting up routing for SOCKS5 proxy through WARP (tun0)..."
25
26 # Dynamically detect Docker subnet from eth0
27 DOCKER_NET=$(ip route show dev eth0 | grep -v default | head -1 | awk '{print $1}')
28 DOCKER_GW=$(ip route show default | head -1 | awk '{print $3}')
29 WARP_ENDPOINT="${WARP_ENDPOINT}"
30 TAILSCALE_ALLOW_LAN_P2P="${TAILSCALE_ALLOW_LAN_P2P:-false}"
31 IP_BLOCKLIST_FILE="/userfilters/ip_blocklist.ipset"
32 LAST_IP_MTIME=""
33 echo "routing-fix: Docker network: ${DOCKER_NET}, gateway: ${DOCKER_GW}"
34
35 # Table 200: Used by ip rule 100 (from 172.18.0.2) for SOCKS5 proxy traffic
36 # WARP endpoint goes via eth0 to avoid tunnel loop, everything else via tun0
37 ip route add "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || \
38 ip route replace "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || true
39 ip route replace default dev tun0 table 200 2>/dev/null || true
40
41 # Main table: Just ensure the Docker subnet route exists via eth0
42 # (gluetun owns the default route here do NOT touch it)
43 ip route replace "${DOCKER_NET}" dev eth0 2>/dev/null || true
44
45 # FORWARD RELATED,ESTABLISHED: Allow return traffic for exit-node connections.
46 # The container has BOTH iptables (nftables backend) and iptables-legacy
47 # loaded. Gluetun's post-rules.txt uses the nftables backend, while Tailscale
48 # uses the legacy backend. Packets go through BOTH stacks, so we add the rule

```

```

49 # to both to ensure it survives regardless of which backend has policy DROP.
50 #
51 # Packet flow: S24 outgoing (tailscale0->eth0) ACCEPTed by first rule.
52 # Return traffic (eth0->eth0 via table 52 to host) needs RELATED,ESTABLISHED
53 # to match the conntrack entry created by the outgoing flow.
54 add_fwd_related_established() {
55     # nftables backend (used by gluetun's post-rules.txt)
56     if ! iptables -C FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null; then
57         iptables -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null || true
58     fi
59     # legacy backend (used by Tailscale's ts-forward)
60     if command -v iptables-legacy >/dev/null 2>&1; then
61         if ! iptables-legacy -C FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null; then
62             iptables-legacy -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null || true
63         fi
64     fi
65 }
66
67 add_fwd_related_established
68
69 # Policy Routing for Tailscale P2P:
70 # Tailscale sends its encrypted mesh UDP packets from port 41642.
71 # If these go out tun0 (WARP), Cloudflare's strict NAT drops the returning
72 # hole-punch packets, causing Tailscale to fail P2P and fall back to DERP relays (high latency).
73 # We mark packets originating from port 41642 and force them out the main table (eth0).
74 #
75 # LAN P2P is controlled by two sources (in priority order):
76 # 1. /app/.lan_p2p_detected written by watcher.sh on macOS on every network change
77 #    using system_profiler SPAirPortDataType (802.1x detection) + AP isolation probe. Overrides .env.
78 # 2. TAILSCALE_ALLOW_LAN_P2P env var (from .env) static fallback.
79 # When disabled, we explicitly DROP local RFC1918-destined Tailscale UDP so the
80 # phone cleanly falls back to DERP rather than getting poisoned by macOS gvproxy SNAT.
81 add_tailscale_p2p_bypass() {
82     # Dynamic override from watcher.sh (network auto-detection) takes priority over .env
83     local allow_p2p="${TAILSCALE_ALLOW_LAN_P2P:-false}"
84     if [ -f "/app/.lan_p2p_detected" ]; then
85         allow_p2p=$(cat "/app/.lan_p2p_detected" 2>/dev/null | tr -d '[:space:]')
86     fi
87
88     # Always allow P2P directly to the Docker host gateway (the Mac running this container).
89     # The gateway container is always co-located with the host Mac on the same machine,
90     # so direct Tailscale P2P via the Docker bridge is always safe and avoids DERP overhead.
91     # This RETURN rule must be inserted BEFORE the general DROP rules so it takes priority.
92     local default_gw
93     default_gw=$(ip route show default | awk '{print $3}')
94     if [ -n "$default_gw" ]; then
95         if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "${default_gw}/32" -j RETURN 2>/dev/null; then
96             iptables -t mangle -I OUTPUT 1 -p udp --sport 41642 -d "${default_gw}/32" -j RETURN 2>/dev/null || true
97         fi
98     fi
99
100     # Also explicitly whitelist all physical IPs of the host Mac (e.g. 172.17.52.223).
101     # The Tailscale control plane registers the Mac using its physical IP, so the
102     # container will attempt P2P handshakes using that physical IP instead of the
103     # Docker bridge IP. If this physical IP falls within 172.16.0.0/12, it would be dropped.
104     if [ -f "/app/.host_ips" ]; then
105         while IFS= read -r host_ip; do
106             [ -z "$host_ip" ] && continue
107             if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "${host_ip}/32" -j RETURN 2>/dev/null; then
108                 iptables -t mangle -I OUTPUT 1 -p udp --sport 41642 -d "${host_ip}/32" -j RETURN 2>/dev/null || true
109             fi
110         done < "/app/.host_ips"
111     fi
112
113     if [ "${allow_p2p}" != "true" ]; then
114         # Drop Tailscale UDP packets destined for local subnets to prevent macOS gvproxy SNAT
115         # endpoint poisoning. When disabled (campus/enterprise WPA2-Enterprise with AP isolation),
116         # dropping these forces the S24 to use DERP relays reliably.
117         for subnet in "192.168.0.0/16" "10.0.0.0/8" "172.16.0.0/12"; do
118             if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null; then
119                 iptables -t mangle -A OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null || true
120             fi
121         done
122     else
123         # Remove DROP rules if they exist (switching from restricted to allowed)
124         for subnet in "192.168.0.0/16" "10.0.0.0/8" "172.16.0.0/12"; do
125             while iptables -t mangle -D OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null; do :; done
126         done
127     fi
128
129     # Bypass STUN and P2P packets to public IPs (DERP relays and remote peers) out eth0

```

```

130 if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -j MARK --set-mark 0x8888 2>/dev/null; then
131 iptables -t mangle -A OUTPUT -p udp --sport 41642 -j MARK --set-mark 0x8888 2>/dev/null || true
132 fi
133 if ! ip rule show | grep -q 'fwmark 0x8888'; then
134 ip rule add fwmark 0x8888 lookup 254 pref 98 2>/dev/null || true
135 fi
136 }
137
138 add_tailscale_p2p_bypass
139
140 add_onion_transparent_proxy() {
141 for ipt in iptables iptables-legacy; do
142 command -v "$ipt" >/dev/null 2>&1 || continue
143 if ! $ipt -t nat -C PREROUTING -d 198.18.0.0/15 -p tcp -j REDIRECT --to-ports 9040 2>/dev/null; then
144 $ipt -t nat -I PREROUTING -d 198.18.0.0/15 -p tcp -j REDIRECT --to-ports 9040 2>/dev/null || true
145 fi
146 if ! $ipt -C FORWARD -d 198.18.0.0/15 -j ACCEPT 2>/dev/null; then
147 $ipt -I FORWARD -d 198.18.0.0/15 -j ACCEPT 2>/dev/null || true
148 fi
149 done
150 }
151
152 add_tailscale_p2p_bypass
153 add_onion_transparent_proxy
154
155 create_log_drop_chain() {
156 local ipt="$1"
157 local chain="$2"
158 local prefix="$3"
159
160 if ! $ipt -N "$chain" 2>/dev/null; then
161 $ipt -F "$chain" 2>/dev/null || true
162 fi
163 $ipt -A "$chain" -j LOG --log-prefix "$prefix"
164 $ipt -A "$chain" -j DROP 2>/dev/null || true
165 }
166
167 add_country_block() {
168 # To add a country to the blocklist, simply define the 'BLOCKED_COUNTRIES' variable in .env with 2-letter ISO
169 # country codes.
170 # You can find the country code by looking at the filename on http://www.ipdeny.com/ipblocks/data/countries/
171 # For example, to block China (cn) and Russia (ru), set: BLOCKED_COUNTRIES="cn ru" in your .env file
172 for zone in $BLOCKED_COUNTRIES; do
173 set_name="block_${zone}"
174 zone_upper=$(echo "$zone" | tr 'a-z' 'A-Z')
175 chain_name="LOG_DROP_${zone_upper}"
176 log_prefix="IP_BLOCK_${zone_upper}: "
177
178 # Create the ipset if it doesn't exist
179 if ! ipset list "$set_name" >/dev/null 2>&1; then
180 ipset create "$set_name" hash:net 2>/dev/null || true
181 echo "routing-fix: Downloading ${zone_upper} IPs for blocklist..."
182 curl -sS "http://www.ipdeny.com/ipblocks/data/countries/${zone}.zone" | grep -v '^#' | while read -r
183 subnet; do
184 [ -n "$subnet" ] && ipset add "$set_name" "$subnet" 2>/dev/null || true
185 done
186 fi
187
188 # Apply LOG_DROP rule to both backends
189 for ipt in iptables iptables-legacy; do
190 command -v "$ipt" >/dev/null 2>&1 || continue
191
192 create_log_drop_chain "$ipt" "$chain_name" "$log_prefix"
193
194 # Clean up old plain DROP rules to ensure we hit the log-and-drop chains
195 while $ipt -D FORWARD -m set --match-set "$set_name" dst -j DROP 2>/dev/null; do ;; done
196
197 if ! $ipt -C FORWARD -m set --match-set "$set_name" dst -j "$chain_name" 2>/dev/null; then
198 $ipt -I FORWARD -m set --match-set "$set_name" dst -j "$chain_name" 2>/dev/null || true
199 fi
200 done
201 done
202 }
203
204 add_ip_blocklist() {
205 # Only reload when the compiled file has actually changed (mtime check).
206 # This prevents a pointless ipset restore on every 30-second loop tick.
207 if [ ! -f "$IP_BLOCKLIST_FILE" ]; then
208 return 0
209 fi

```

```

209 CURRENT_MTIME=$(stat -c %Y "$IP_BLOCKLIST_FILE" 2>/dev/null || echo "0")
210 if [ "$CURRENT_MTIME" != "$LAST_IP_MTIME" ]; then
211     echo "routing-fix: IP blocklist changed, reloading..."
212     # ipset restore handles the atomic swap internally:
213     # create_new populate swap with live destroy_new
214     if ipset restore < "$IP_BLOCKLIST_FILE" 2>/dev/null; then
215         LAST_IP_MTIME="$CURRENT_MTIME"
216         echo "routing-fix: IP blocklist loaded ($(ipset list block_malicious | grep -c '^[0-9]') entries)."
217     else
218         echo "routing-fix: Warning: ipset restore failed. Retrying next cycle."
219         return 1
220     fi
221 fi
222
223 # Apply FORWARD rules in BOTH iptables backends.
224 for ipt in iptables iptables-legacy; do
225     command -v "$ipt" >/dev/null 2>&1 || continue
226
227     create_log_drop_chain "$ipt" LOG_DROP_MALICIOUS_DST "IP_BLOCK_MALICIOUS_DST: "
228     create_log_drop_chain "$ipt" LOG_DROP_MALICIOUS_SRC "IP_BLOCK_MALICIOUS_SRC: "
229
230     # Clean up old plain DROP rules to ensure we hit the log-and-drop chains
231     while $ipt -D FORWARD -m set --match-set block_malicious dst -j DROP 2>/dev/null; do ;; done
232     while $ipt -D FORWARD -m set --match-set block_malicious src -j DROP 2>/dev/null; do ;; done
233
234     if ! $ipt -C FORWARD -m set --match-set block_malicious dst -j LOG_DROP_MALICIOUS_DST 2>/dev/null; then
235         $ipt -I FORWARD -m set --match-set block_malicious dst -j LOG_DROP_MALICIOUS_DST 2>/dev/null || true
236     fi
237
238     if ! $ipt -C FORWARD -m set --match-set block_malicious src -j LOG_DROP_MALICIOUS_SRC 2>/dev/null; then
239         $ipt -I FORWARD -m set --match-set block_malicious src -j LOG_DROP_MALICIOUS_SRC 2>/dev/null || true
240     fi
241 done
242 }
243
244 # Start background block logger (DNS + IP)
245 if ! pgrep -f "nullexit-logger" >/dev/null; then
246     echo "routing-fix: Starting background block logger..."
247     nullexit-logger &
248 fi
249
250 add_country_block
251 add_ip_blocklist
252
253 echo 'routing-fix: Routes applied.'
254
255 UNHEALTHY_COUNT=0
256
257 # Re-assert loop (every 30 seconds)
258 while true; do
259     # Table 200 routes
260     ip route replace "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || true
261
262     # Tunnel health check
263     if ! curl -m 3 -sS --interface tun0 https://cloudflare.com/cdn-cgi/trace >/dev/null 2>&1; then
264         UNHEALTHY_COUNT=$((UNHEALTHY_COUNT + 1))
265     else
266         UNHEALTHY_COUNT=0
267         if [ -f "/app/TUNNEL_FAILED_CLOSED.marker" ]; then
268             rm -f "/app/TUNNEL_FAILED_CLOSED.marker"
269         fi
270     fi
271
272     if [ "$UNHEALTHY_COUNT" -ge 3 ]; then
273         echo "routing-fix: Tunnel health check failed $UNHEALTHY_COUNT times. Failing closed."
274         ip route del default dev tun0 table 200 2>/dev/null || true
275         ip route replace blackhole default table 200 2>/dev/null || true
276         touch "/app/TUNNEL_FAILED_CLOSED.marker"
277     else
278         ip route replace default dev tun0 table 200 2>/dev/null || true
279     fi
280
281     # Main table: keep Docker subnet route
282     ip route replace "${DOCKER_NET}" dev eth0 2>/dev/null || true
283
284     # CGNAT ip rule for Tailscale
285     if ! ip rule show | grep -q '100.64.0.0/10'; then
286         ip rule add to 100.64.0.0/10 lookup 52 pref 99 2>/dev/null || true
287         echo 'routing-fix: CGNAT rule missing, re-injected'
288     fi
289

```



```

290 # FORWARD RELATED, ESTABLISHED: Allow return traffic in both iptables
291 # backends. Gluetun may reset the nftables ruleset on VPN reconnect,
292 # and Tailscale may reset the legacy ruleset on auth refresh.
293 add_fwd_related_established
294
295 # Ensure P2P packets bypass WARP to maintain low latency
296 add_tailscale_p2p_bypass
297
298 # Ensure transparent proxying for .onion is active
299 add_onion_transparent_proxy
300
301 # Enforce country blocklist
302 add_country_block
303
304 # Enforce IP blocklist (reloads automatically when file changes)
305 add_ip_blocklist
306
307 sleep 30 &
308 wait $! || true
309 done

```

41 scripts/rule-compiler/go.mod

```

1 module nullexit/rule-compiler
2
3 go 1.22

```

42 scripts/rule-compiler/main.go

```

1 package main
2
3 import (
4     "bufio"
5     "bytes"
6     "crypto/md5"
7     "encoding/base64"
8     "encoding/hex"
9     "fmt"
10    "io"
11    "net/http"
12    "net/netip"
13    "os"
14    "os/exec"
15    "path/filepath"
16    "regexp"
17    "sort"
18    "strings"
19    "sync"
20    "time"
21 )
22
23 const (
24     BlackListPath = "black_list.txt"
25     WhiteListPath = "white_list.txt"
26     OutputPath    = "adguard/work/userfilters/compiled_rules.txt"
27     IpOutputPath  = "adguard/work/userfilters/ip_blocklist.ipset"
28     IpCacheDir    = "adguard/work/userfilters/cache/ip"
29     CacheDir      = "adguard/work/userfilters/cache"
30 )
31
32 var CoreLists = []string{
33     "https://raw.githubusercontent.com/anudeepND/blacklist/master/adservers.txt",
34     "https://raw.githubusercontent.com/anudeepND/blacklist/master/facebook.txt",
35     "https://raw.githubusercontent.com/crazy-max/WindowsSpyBlocker/master/data/hosts/spy.txt",
36     "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adbblock/native.samsung.txt",
37     "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adbblock/native.apple.txt",
38     "https://abp.oisd.nl/basic/",
39     "https://adaway.org/hosts.txt",
40     "https://pgl.yoyo.org/adserver/serverlist.php?hostformat=adbblockplus&showintro=1&mimetype=plaintext",
41     "https://raw.githubusercontent.com/nextdns/cname-cloaking-blocklist/master/domains",
42 }
43

```

```

44 var MediumAdditions = []string{
45     "https://raw.githubusercontent.com/lightswitch05/hosts/master/docs/lists/facebook-extended.txt",
46     "https://someonewhocares.org/hosts/zero/hosts",
47     "https://urlhaus.abuse.ch/downloads/hostfile/",
48 }
49
50 var HeavyAdditions = []string{
51     "https://raw.githubusercontent.com/StevenBlack/hosts/master/hosts",
52     "https://raw.githubusercontent.com/Perflyst/PiHoleBlocklist/master/SmartTV-AGH.txt",
53     "https://raw.githubusercontent.com/DandelionSprout/adfilt/master/GameConsoleAdblockList.txt",
54 }
55
56 var IpBlockLists = []string{
57     "https://feodotracker.abuse.ch/downloads/ipblocklist.txt",
58     "https://www.spamhaus.org/drop/drop.txt",
59     "https://rules.emergingthreats.net/fwrules/emerging-Block-IPs.txt",
60     "https://cinsscore.com/list/ci-badguys.txt",
61 }
62
63 func getProfiles() map[string][]string {
64     profiles := make(map[string][]string)
65
66     light := append([]string(nil), CoreLists...)
67     light = append(light, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/light.txt")
68     profiles["light"] = light
69
70     medium := append([]string(nil), CoreLists...)
71     medium = append(medium, MediumAdditions...)
72     medium = append(medium, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/multi.txt")
73     profiles["medium"] = medium
74
75     heavy := append([]string(nil), CoreLists...)
76     heavy = append(heavy, MediumAdditions...)
77     heavy = append(heavy, HeavyAdditions...)
78     heavy = append(heavy, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/pro.txt")
79     profiles["heavy"] = heavy
80
81     return profiles
82 }
83
84 func loadEnvProfile() string {
85     profile := "medium"
86
87     if bytesData, err := os.ReadFile(".env"); err == nil {
88         scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
89         for scanner.Scan() {
90             line := strings.TrimSpace(scanner.Text())
91             if line != "" && !strings.HasPrefix(line, "#") {
92                 parts := strings.SplitN(line, "=", 2)
93                 if len(parts) == 2 {
94                     key := strings.TrimSpace(parts[0])
95                     val := strings.TrimSpace(parts[1])
96                     if key == "GATEWAY_RULE_PROFILE" {
97                         val = strings.Trim(val, "\"'")
98                         profile = strings.ToLower(val)
99                     }
100                 }
101             }
102         }
103     } else if !os.IsNotExist(err) {
104         fmt.Printf("Warning: Failed to read .env file for profile configuration (%v). Using default.\n", err)
105     }
106
107     if envVal := os.Getenv("GATEWAY_RULE_PROFILE"); envVal != "" {
108         profile = strings.ToLower(envVal)
109     }
110
111     profiles := getProfiles()
112     if _, exists := profiles[profile]; !exists {
113         fmt.Printf("Warning: Profile '%s' is invalid. Falling back to 'medium'.\n", profile)
114         profile = "medium"
115     }
116     return profile
117 }
118
119 func loadDomains(filepath string) map[string]bool {
120     domains := make(map[string]bool)
121     if bytesData, err := os.ReadFile(filepath); err == nil {
122         scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
123         for scanner.Scan() {
124             line := strings.TrimSpace(scanner.Text())

```

```

125     if line != "" && !strings.HasPrefix(line, "#") {
126         domains[strings.ToLower(line)] = true
127     }
128 }
129 }
130 return domains
131 }
132
133 func parseDomainsFromContent(content string) map[string]bool {
134     domains := make(map[string]bool)
135     scanner := bufio.NewScanner(strings.NewReader(content))
136     ipRegex := regexp.MustCompile(`^d{1,3}\.d{1,3}\.d{1,3}\.d{1,3}$`)
137     agRegex := regexp.MustCompile(`^\|([a-z0-9.-]+)\`$`)
138     domainRegex := regexp.MustCompile(`^[a-z0-9.-]+$`)
139
140     for scanner.Scan() {
141         line := strings.TrimSpace(scanner.Text())
142         if line == "" || strings.HasPrefix(line, "!") || strings.HasPrefix(line, "[") {
143             continue
144         }
145         parts := strings.SplitN(line, "#", 2)
146         line = strings.TrimSpace(parts[0])
147         if line == "" {
148             continue
149         }
150
151         fields := strings.Fields(line)
152         var rule string
153         if len(fields) >= 2 {
154             if ipRegex.MatchString(fields[0]) {
155                 rule = fields[1]
156             } else {
157                 rule = fields[0]
158             }
159         } else {
160             rule = fields[0]
161         }
162
163         rule = strings.TrimSpace(strings.ToLower(rule))
164
165         var domain string
166         if m := agRegex.FindStringSubmatch(rule); m != nil {
167             domain = m[1]
168         } else if domainRegex.MatchString(rule) {
169             domain = rule
170         } else {
171             domain = rule
172         }
173
174         if domain != "" && domain != "localhost" && domain != "0.0.0.0" && domain != "127.0.0.1" && domain != "
175             broadcasthost" {
176             domains[domain] = true
177         }
178     }
179     return domains
180 }
181
182 // WARNING: Parsing AdGuardHome.yaml using regex instead of a proper YAML parser
183 // is brittle and assumes the exact format currently emitted by AdGuard.
184 // If an AdGuard version bump updates the configuration schema format,
185 // this parser will fail silently or loudly and should be refactored to use gopkg.in/yaml.v3.
186 func getAdguardNativeLists() []string {
187     yamlPath := "adguard/conf/AdGuardHome.yaml"
188     var enabledUrls []string
189     if _, err := os.Stat(yamlPath); os.IsNotExist(err) {
190         return enabledUrls
191     }
192
193     contentBytes, err := os.ReadFile(yamlPath)
194     if err != nil {
195         fmt.Printf("Warning: Failed to parse AdGuardHome.yaml for native lists (%v)\n", err)
196         return enabledUrls
197     }
198     content := string(contentBytes)
199
200     filtersRegex := regexp.MustCompile(`(?ms)^filters:(.)*(?:[a-zA-Z_]+:|z)`)
201     match := filtersRegex.FindStringSubmatch(content)
202     if len(match) > 1 {
203         filtersText := match[1]
204         blocks := strings.Split(filtersText, "\n - ")
205         urlRegex := regexp.MustCompile(`url:s*(S+)`)

```

```

205
206     for _, block := range blocks {
207         if strings.TrimSpace(block) == "" {
208             continue
209         }
210         if strings.Contains(block, "enabled: true") {
211             urlMatch := urlRegex.FindStringSubmatch(block)
212             if len(urlMatch) > 1 {
213                 url := urlMatch[1]
214                 if !strings.Contains(url, "compiled_rules.txt") {
215                     enabledUrls = append(enabledUrls, url)
216                 }
217             }
218         }
219     }
220 }
221 return enabledUrls
222 }
223
224 func getAdguardCompiledRulesCachePath() string {
225     yamlPath := "adguard/conf/AdGuardHome.yaml"
226     if _, err := os.Stat(yamlPath); os.IsNotExist(err) {
227         return ""
228     }
229
230     contentBytes, err := os.ReadFile(yamlPath)
231     if err != nil {
232         fmt.Printf("Warning: Failed to resolve compiled_rules.txt cache path (%v)\n", err)
233         return ""
234     }
235     content := string(contentBytes)
236
237     filtersRegex := regexp.MustCompile(`(?ms)^filters:(.*)"(?![a-zA-Z_]+\|z)`)
238     match := filtersRegex.FindStringSubmatch(content)
239     if len(match) > 1 {
240         blocks := strings.Split(match[1], "\n - ")
241         idRegex := regexp.MustCompile(`id:\s*(\d+)`)
242         for _, block := range blocks {
243             if !strings.Contains(block, "compiled_rules.txt") {
244                 continue
245             }
246             idMatch := idRegex.FindStringSubmatch(block)
247             if len(idMatch) > 1 {
248                 return fmt.Sprintf("adguard/work/data/filters/%s.txt", idMatch[1])
249             }
250         }
251     }
252     return ""
253 }
254
255 func handleFetchError(urlStr string, cacheFile string, err error) map[string]bool {
256     if _, statErr := os.Stat(cacheFile); statErr == nil {
257         fmt.Printf("-> Warning: Failed to fetch %s (%v). Falling back to expired local cache.\n", urlStr, err)
258         content, readErr := os.ReadFile(cacheFile)
259         if readErr == nil {
260             domains := parseDomainsFromContent(string(content))
261             fmt.Printf("-> Loaded from expired cache (%d domains).\n", len(domains))
262             return domains
263         }
264         fmt.Printf("-> Warning: Failed to read expired cache (%v). Using local lists only.\n", readErr)
265     } else {
266         fmt.Printf("-> Warning: Failed to fetch %s (%v). Using local lists only for this source.\n", urlStr, err)
267     }
268     return make(map[string]bool)
269 }
270
271 func fetchRemoteDomains(urlStr string) map[string]bool {
272     os.MkdirAll(CacheDir, 0755)
273
274     hash := md5.Sum([]byte(urlStr))
275     urlHash := hex.EncodeToString(hash[:])
276     cacheFile := filepath.Join(CacheDir, fmt.Sprintf("%s.txt", urlHash))
277
278     if stat, err := os.Stat(cacheFile); err == nil {
279         fileAge := time.Since(stat.ModTime()).Seconds()
280         if fileAge < 86400 {
281             fmt.Printf("Loading remote blacklist from cache: %s\n", urlStr)
282             content, err := os.ReadFile(cacheFile)
283             if err == nil {
284                 domains := parseDomainsFromContent(string(content))

```

```

285     fmt.Printf(" -> Loaded from cache (copy is %.1f hours old, %d domains).\n", fileAge/3600.0, len(domains
286 ))
287     return domains
288 }
289     fmt.Printf(" -> Warning: Failed to read cache file %s (%v). Re-fetching...\n", cacheFile, err)
290 }
291
292     fmt.Printf("Fetching remote blacklist from: %s ...\n", urlStr)
293
294     req, err := http.NewRequest("GET", urlStr, nil)
295     if err != nil {
296         return handleFetchError(urlStr, cacheFile, err)
297     }
298     req.Header.Set("User-Agent", "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)")
299
300     client := &http.Client{Timeout: 15 * time.Second}
301     resp, err := client.Do(req)
302     if err != nil {
303         return handleFetchError(urlStr, cacheFile, err)
304     }
305     defer resp.Body.Close()
306
307     if resp.StatusCode != 200 {
308         return handleFetchError(urlStr, cacheFile, fmt.Errorf("HTTP %d", resp.StatusCode))
309     }
310
311     bodyBytes, err := io.ReadAll(resp.Body)
312     if err != nil {
313         return handleFetchError(urlStr, cacheFile, err)
314     }
315
316     content := string(bodyBytes)
317     domains := parseDomainsFromContent(content)
318
319     if len(domains) < 10 {
320         return handleFetchError(urlStr, cacheFile, fmt.Errorf("Sanity check failed: only found %d domains. Possible
321 404 or bad URL", len(domains)))
322     }
323
324     err = os.WriteFile(cacheFile, bodyBytes, 0644)
325     if err != nil {
326         fmt.Printf(" -> Warning: Failed to save cache file (%v)\n", err)
327     }
328
329     fmt.Printf(" -> Successfully fetched and cached %d domains.\n", len(domains))
330     return domains
331 }
332
333 func optimizeSubdomains(domains map[string]bool, listName string) map[string]bool {
334     fmt.Printf("Optimizing %s by removing redundant subdomains...\n", listName)
335     rawCount := len(domains)
336     if rawCount == 0 {
337         return make(map[string]bool)
338     }
339
340     optimized := make(map[string]bool)
341     for domain := range domains {
342         if strings.ContainsAny(domain, "|^@$/") {
343             optimized[domain] = true
344             continue
345         }
346
347         parts := strings.Split(domain, ".")
348         hasParent := false
349
350         for i := 1; i < len(parts)-1; i++ {
351             parent := strings.Join(parts[i:], ".")
352             if domains[parent] {
353                 hasParent = true
354                 break
355             }
356         }
357
358         if !hasParent {
359             optimized[domain] = true
360         }
361     }
362
363     saved := rawCount - len(optimized)
364     reduction := float64(0)

```

```

364 if rawCount > 0 {
365     reduction = (float64(saved) / float64(rawCount)) * 100
366 }
367 fmt.Printf(" -> Reduced %s from %d to %d domains (-%d / %.1f%% reduction).\n", listName, rawCount, len(
    optimized), saved, reduction)
368 return optimized
369 }
370
371 func parseIpsFromContent(content string) map[string]bool {
372     ips := make(map[string]bool)
373     scanner := bufio.NewScanner(strings.NewReader(content))
374
375     for scanner.Scan() {
376         line := strings.TrimSpace(scanner.Text())
377         if line == "" || strings.HasPrefix(line, "#") || strings.HasPrefix(line, ";") {
378             continue
379         }
380
381         fields := strings.Fields(line)
382         if len(fields) > 0 {
383             token := strings.TrimRight(fields[0], ";,")
384             if _, err := netip.ParsePrefix(token); err == nil {
385                 ips[token] = true
386             } else if _, err := netip.ParseAddr(token); err == nil {
387                 ips[token] = true
388             }
389         }
390     }
391     return ips
392 }
393
394 func handleIpFetchError(urlStr string, cacheFile string, err error) map[string]bool {
395     if _, statErr := os.Stat(cacheFile); statErr == nil {
396         fmt.Printf(" -> Warning: fetch failed (%v). Falling back to stale cache.\n", err)
397         content, readErr := os.ReadFile(cacheFile)
398         if readErr == nil {
399             ips := parseIpsFromContent(string(content))
400             fmt.Printf(" -> %d IPs loaded from stale cache.\n", len(ips))
401             return ips
402         }
403         fmt.Printf(" -> Warning: stale cache also failed (%v).\n", readErr)
404     } else {
405         fmt.Printf(" -> Warning: %s failed (%v). No cache available. Skipping.\n", urlStr, err)
406     }
407     return make(map[string]bool)
408 }
409
410 func fetchRemoteIps(urlStr string) map[string]bool {
411     os.MkdirAll(IpCacheDir, 0755)
412
413     hash := md5.Sum([]byte(urlStr))
414     urlHash := hex.EncodeToString(hash[:])
415     cacheFile := filepath.Join(IpCacheDir, fmt.Sprintf("%s.txt", urlHash))
416
417     if stat, err := os.Stat(cacheFile); err == nil {
418         fileAge := time.Since(stat.ModTime()).Seconds()
419         if fileAge < 86400 {
420             fmt.Printf("Loading IP feed from cache: %s\n", urlStr)
421             content, err := os.ReadFile(cacheFile)
422             if err == nil {
423                 ips := parseIpsFromContent(string(content))
424                 fmt.Printf(" -> %d IPs (cache is %.1fh old).\n", len(ips), fileAge/3600.0)
425                 return ips
426             }
427             fmt.Printf(" -> Warning: cache read failed (%v). Re-fetching...\n", err)
428         }
429     }
430
431     fmt.Printf("Fetching IP feed: %s ...\n", urlStr)
432     req, err := http.NewRequest("GET", urlStr, nil)
433     if err != nil {
434         return handleIpFetchError(urlStr, cacheFile, err)
435     }
436     req.Header.Set("User-Agent", "Mozilla/5.0")
437
438     client := &http.Client{Timeout: 15 * time.Second}
439     resp, err := client.Do(req)
440     if err != nil {
441         return handleIpFetchError(urlStr, cacheFile, err)
442     }
443     defer resp.Body.Close()

```

```

444
445     if resp.StatusCode != 200 {
446         return handleIpFetchError(urlStr, cacheFile, fmt.Errorf("HTTP %d", resp.StatusCode))
447     }
448
449     bodyBytes, err := io.ReadAll(resp.Body)
450     if err != nil {
451         return handleIpFetchError(urlStr, cacheFile, err)
452     }
453
454     content := string(bodyBytes)
455     ips := parseIpsFromContent(content)
456     if len(ips) < 1 {
457         return handleIpFetchError(urlStr, cacheFile, fmt.Errorf("Sanity check failed: only %d IPs found. Possible
458         404", len(ips)))
459     }
460
461     err = os.WriteFile(cacheFile, bodyBytes, 0644)
462     if err != nil {
463         fmt.Printf(" -> Warning: cache write failed (%v)\n", err)
464     }
465
466     fmt.Printf(" -> %d IPs fetched and cached.\n", len(ips))
467     return ips
468 }
469
470 func compileIpBlocklist() int {
471     fmt.Println("\n IP Blocklist Compilation ")
472
473     allIps := make(map[string]bool)
474     var mu sync.Mutex
475     var wg sync.WaitGroup
476
477     maxThreads := 8
478     if len(IpBlockLists) < 8 {
479         maxThreads = len(IpBlockLists)
480     }
481     semaphore := make(chan struct{}, maxThreads)
482
483     for _, urlStr := range IpBlockLists {
484         wg.Add(1)
485         go func(u string) {
486             defer wg.Done()
487             semaphore <- struct{}{}
488             defer func() { <-semaphore }()
489
490             ips := fetchRemoteIps(u)
491             mu.Lock()
492             for ip := range ips {
493                 allIps[ip] = true
494             }
495             mu.Unlock()
496         }(urlStr)
497     }
498     wg.Wait()
499
500     fmt.Printf("Total unique entries before filtering: %d\n", len(allIps))
501
502     reservedStr := []string{
503         "10.0.0.0/8",
504         "172.16.0.0/12",
505         "192.168.0.0/16",
506         "127.0.0.0/8",
507         "169.254.0.0/16",
508         "100.64.0.0/10",
509         "0.0.0.0/8",
510     }
511     var reserved []netip.Prefix
512     for _, s := range reservedStr {
513         p, _ := netip.ParsePrefix(s)
514         reserved = append(reserved, p)
515     }
516
517     cleanIps := make(map[string]bool)
518     for entry := range allIps {
519         var prefix netip.Prefix
520         p, err := netip.ParsePrefix(entry)
521         if err != nil {
522             addr, err2 := netip.ParseAddr(entry)
523             if err2 != nil {
524                 continue

```



```

524     }
525     prefix = netip.PrefixFrom(addr, addr.BitLen())
526 } else {
527     prefix = p
528 }
529
530 overlaps := false
531 for _, r := range reserved {
532     if r.Overlaps(prefix) {
533         overlaps = true
534         break
535     }
536 }
537 if !overlaps {
538     cleanIps[entry] = true
539 }
540 }
541
542 removed := len(allIps) - len(cleanIps)
543 if removed > 0 {
544     fmt.Printf(" -> Removed %d private/reserved entries.\n", removed)
545 }
546 fmt.Printf(" -> Final IP blocklist: %d entries.\n", len(cleanIps))
547
548 os.MkdirAll(filepath.Dir(IpOutputPath), 0755)
549
550 f, err := os.Create(IpOutputPath)
551 if err != nil {
552     fmt.Printf("Error writing IP output file: %v\n", err)
553     return len(cleanIps)
554 }
555 defer f.Close()
556
557 f.WriteString("# nullexit Compiled IP Blocklist\n")
558 f.WriteString("# Sources: Feodo Tracker, Spamhaus DROP/EDROP, Emerging Threats, CINS\n")
559 f.WriteString(fmt.Sprintf("# Entries: %d\n\n", len(cleanIps)))
560
561 f.WriteString("create block_malicious_new hash:net maxelem 200000 -exist\n")
562
563 var sortedIps []string
564 for ip := range cleanIps {
565     sortedIps = append(sortedIps, ip)
566 }
567 sort.Strings(sortedIps)
568
569 for _, ip := range sortedIps {
570     f.WriteString(fmt.Sprintf("add block_malicious_new %s -exist\n", ip))
571 }
572
573 f.WriteString("create block_malicious hash:net maxelem 200000 -exist\n")
574 f.WriteString("swap block_malicious block_malicious_new\n")
575 f.WriteString("destroy block_malicious_new\n")
576
577 fmt.Printf("IP blocklist written to %s\n", IpOutputPath)
578 return len(cleanIps)
579 }
580
581 func main() {
582     profile := loadEnvProfile()
583     fmt.Printf("Active memory profile: '%s'\n", strings.ToUpper(profile))
584
585     localBlackList := loadDomains(BlackListPath)
586     whiteList := loadDomains(WhiteListPath)
587
588     blackList := make(map[string]bool)
589     for k := range localBlackList {
590         blackList[k] = true
591     }
592
593     profiles := getProfiles()
594     urlsToFetch := profiles[profile]
595
596     fmt.Printf("\nStarting concurrent downloads for %d lists...\n", len(urlsToFetch))
597     startTime := time.Now()
598
599     var mu sync.Mutex
600     var wg sync.WaitGroup
601
602     maxThreads := 16
603     if len(urlsToFetch) < 16 {
604         maxThreads = len(urlsToFetch)

```

```

605 }
606 semaphore := make(chan struct{}, maxThreads)
607
608 for _, urlStr := range urlsToFetch {
609     wg.Add(1)
610     go func(u string) {
611         defer wg.Done()
612         semaphore <- struct{}{}
613         defer func() { <-semaphore }()
614
615         domains := fetchRemoteDomains(u)
616         mu.Lock()
617         for d := range domains {
618             blacklist[d] = true
619         }
620         mu.Unlock()
621     }(urlStr)
622 }
623 wg.Wait()
624
625 fmt.Printf("Finished concurrent downloads in %.2f seconds using %d threads.\n", time.Since(startTime).Seconds
626 (), maxThreads)
627
628 adguardNativeUrls := getAdguardNativeLists()
629 var adguardNativeDomains map[string]bool
630 if len(adguardNativeUrls) > 0 {
631     fmt.Printf("\nFetching %d AdGuard native list(s) for deduplication...\n", len(adguardNativeUrls))
632     adguardNativeDomains = make(map[string]bool)
633
634     maxNativeThreads := 4
635     if len(adguardNativeUrls) < 4 {
636         maxNativeThreads = len(adguardNativeUrls)
637     }
638     nativeSemaphore := make(chan struct{}, maxNativeThreads)
639
640     for _, urlStr := range adguardNativeUrls {
641         wg.Add(1)
642         go func(u string) {
643             defer wg.Done()
644             nativeSemaphore <- struct{}{}
645             defer func() { <-nativeSemaphore }()
646
647             domains := fetchRemoteDomains(u)
648             mu.Lock()
649             for d := range domains {
650                 adguardNativeDomains[d] = true
651             }
652             mu.Unlock()
653         }(urlStr)
654     }
655     wg.Wait()
656
657     if len(adguardNativeDomains) > 0 {
658         fmt.Println("Deduplicating compiled rules against AdGuard native lists...")
659         originalSize := len(blacklist)
660         for d := range adguardNativeDomains {
661             delete(blacklist, d)
662         }
663         fmt.Printf(" -> Removed %d redundant rules already covered by AdGuard.\n", originalSize-len(blacklist))
664     }
665 }
666
667 var contradictions []string
668 for d := range whitelist {
669     if blacklist[d] {
670         contradictions = append(contradictions, d)
671     }
672 }
673
674 if len(contradictions) > 0 {
675     fmt.Printf("Detected %d contradictions. Whitelist taking priority.\n", len(contradictions))
676     sort.Strings(contradictions)
677     for _, domain := range contradictions {
678         if localBlacklist[domain] {
679             fmt.Printf(" -> Removed local blacklist domain: %s\n", domain)
680         }
681         delete(blacklist, domain)
682     }
683 }
684
685 blacklist = optimizeSubdomains(blacklist, "blacklist")

```

```

685 whiteList = optimizeSubdomains(whiteList, "whitelist")
686
687 os.MkdirAll(filepath.Dir(OutputPath), 0755)
688 outFile, err := os.Create(OutputPath)
689 if err != nil {
690     fmt.Printf("Error creating output file: %v\n", err)
691     return
692 }
693 defer outFile.Close()
694
695 outFile.WriteString("! Custom Compiled Rules (Auto-Generated)\n")
696 outFile.WriteString(fmt.Sprintf("! Memory Profile: %s\n", strings.ToUpper(profile)))
697 outFile.WriteString(fmt.Sprintf("! Total Block Rules: %d\n", len(blackList)))
698 outFile.WriteString(fmt.Sprintf("! Native AdGuard Rules: %d\n", len(adguardNativeDomains)))
699 outFile.WriteString(fmt.Sprintf("! Total Whitelist Rules: %d\n\n", len(whiteList)))
700
701 outFile.WriteString("! --- Blacklist Rules ---\n")
702
703 var sortedBlacklist []string
704 for d := range blackList {
705     sortedBlacklist = append(sortedBlacklist, d)
706 }
707 sort.Strings(sortedBlacklist)
708
709 for _, domain := range sortedBlacklist {
710     if strings.Contains(domain, "$") {
711         parts := strings.SplitN(domain, "$", 2)
712         dom, mod := parts[0], parts[1]
713         if strings.HasPrefix(dom, "|") {
714             if strings.HasSuffix(dom, "~") {
715                 outFile.WriteString(fmt.Sprintf("%s%s\n", dom, mod))
716             } else {
717                 outFile.WriteString(fmt.Sprintf("%s~%s\n", dom, mod))
718             }
719         } else {
720             outFile.WriteString(fmt.Sprintf("||%s~%s\n", dom, mod))
721         }
722     } else if strings.HasPrefix(domain, "/") || strings.HasPrefix(domain, "|") || strings.HasSuffix(domain, "|") || strings.HasSuffix(domain, "~") {
723         outFile.WriteString(fmt.Sprintf("%s\n", domain))
724     } else {
725         outFile.WriteString(fmt.Sprintf("||%s~\n", domain))
726     }
727 }
728
729 outFile.WriteString("\n! --- Whitelist Rules ---\n")
730 var sortedWhitelist []string
731 for d := range whiteList {
732     sortedWhitelist = append(sortedWhitelist, d)
733 }
734 sort.Strings(sortedWhitelist)
735
736 for _, domain := range sortedWhitelist {
737     if strings.HasPrefix(domain, "/") || strings.HasPrefix(domain, "|") || strings.HasSuffix(domain, "|") || strings.HasSuffix(domain, "~") || strings.HasPrefix(domain, "@@") {
738         rule := domain
739         if !strings.HasPrefix(domain, "@@") {
740             rule = "@@" + domain
741         }
742         outFile.WriteString(fmt.Sprintf("%s\n", rule))
743     } else {
744         outFile.WriteString(fmt.Sprintf("@@||%s~\n", domain))
745     }
746 }
747
748 fmt.Printf("\nSuccessfully compiled %d block rules and %d allow rules to %s\n", len(blackList), len(whiteList), OutputPath)
749
750 cachedFilterPath := getAdguardCompiledRulesCachePath()
751 if cachedFilterPath != "" {
752     input, err := os.ReadFile(OutputPath)
753     if err == nil {
754         err = os.WriteFile(cachedFilterPath, input, 0644)
755         if err == nil {
756             fmt.Printf("Updated AdGuard filter cache: %s\n", cachedFilterPath)
757         } else {
758             fmt.Printf("Warning: Could not update AdGuard filter cache %s (%v)\n", cachedFilterPath, err)
759         }
760     } else {
761         fmt.Printf("Warning: Could not read OutputPath to copy to cache (%v)\n", err)
762     }
763 }

```

```

763 } else {
764     fmt.Println("Warning: Could not determine compiled_rules.txt cache path from AdGuardHome.yaml; skipping
        cache update.")
765 }
766
767 adguardReachable := false
768
769 dockerComposeYamlExists := false
770 if _, err := os.Stat("docker-compose.yml"); err == nil {
771     dockerComposeYamlExists = true
772 }
773 dockerPath, err := exec.LookPath("docker")
774 if err == nil && dockerComposeYamlExists {
775     cmd := exec.Command(dockerPath, "compose", "ps", "--status", "running", "-q", "adguardhome")
776     out, err := cmd.Output()
777     if err == nil && strings.TrimSpace(string(out)) != "" {
778         fmt.Println("Force-recreating AdGuard Home container to drop persisted filter state...")
779         upCmd := exec.Command(dockerPath, "compose", "up", "-d", "--force-recreate", "--no-deps", "adguardhome")
780         err := upCmd.Run()
781         if err == nil {
782             fmt.Println("AdGuard Home force-recreated successfully.")
783             time.Sleep(8 * time.Second)
784             adguardReachable = true
785         } else {
786             fmt.Println("Failed to restart AdGuard Home. Is it running?")
787         }
788     }
789 }
790
791 if adguardReachable {
792     agPass := "nullexit"
793     if bytesData, err := os.ReadFile(".env"); err == nil {
794         scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
795         for scanner.Scan() {
796             line := scanner.Text()
797             if strings.HasPrefix(line, "ADGUARD_PASSWORD=") {
798                 parts := strings.SplitN(line, "=", 2)
799                 if len(parts) == 2 {
800                     agPass = strings.Trim(strings.TrimSpace(parts[1]), "\"'")
801                 }
802                 break
803             }
804         }
805     }
806
807     creds := base64.StdEncoding.EncodeToString([]byte(fmt.Sprintf("admin:%s", agPass)))
808     req, err := http.NewRequest("POST", "http://127.0.0.1:3000/control/filtering/refresh", bytes.NewBuffer([]
        byte("{}")))
809     if err == nil {
810         req.Header.Set("Authorization", fmt.Sprintf("Basic %s", creds))
811         req.Header.Set("Content-Type", "application/json")
812         client := &http.Client{Timeout: 15 * time.Second}
813         resp, err := client.Do(req)
814         if err == nil {
815             fmt.Printf("Triggered AdGuard filter refresh via REST API (HTTP=%d).\n", resp.StatusCode)
816             resp.Body.Close()
817         } else {
818             fmt.Printf("Warning: AdGuard filter refresh API call failed (%v). New whitelist will not load until
                next refresh tick (~24h) or manual refresh.\n", err)
819         }
820     }
821 }
822
823 compileIpBlocklist()
824 }

```

43 scripts/setup-common.sh

```

1 #!/bin/bash
2 # 1. Docker
3 step "Checking Docker"
4
5 if ! command -v docker >> output.log 2>&1; then
6     echo ""
7     if [[ "$OSTYPE" == "darwin"* ]]; then
8         echo " Docker is not installed."

```

```

9      # Note: Homebrew aggressively rolls forward and discourages version pinning.
10     # This setup is confirmed stable on Colima v0.10.3 and Docker v29.6.1.
11     read -rp " Would you like to automatically install Colima & Docker via Homebrew? [y/N]: "
INSTALL_DOCKER
12     if [[ "$INSTALL_DOCKER" == "y" || "$INSTALL_DOCKER" == "Y" ]]; then
13         step "Installing Colima and Docker..."
14         brew install colima docker docker-compose
15         step "Starting Colima VM..."
16         colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged
17     else
18         echo ""
19         echo " Please install Docker manually. Two options:"
20         echo " A) Colima (Recommended): brew install colima docker docker-compose && colima start --memory
0.6 --vm-type vz --network-address --network-mode bridged"
21         echo " B) Docker Desktop: https://www.docker.com/products/docker-desktop/"
22         die "Install Docker, then re-run this script."
23     fi
24 else
25     echo " Docker is not installed."
26     # Note: The Docker convenience script always fetches the latest stable release.
27     # This setup is confirmed stable on Docker Engine v29.6.1.
28     read -rp " Would you like to automatically install Docker? (Requires sudo) [y/N]: " INSTALL_DOCKER
29     if [[ "$INSTALL_DOCKER" == "y" || "$INSTALL_DOCKER" == "Y" ]]; then
30         step "Installing Docker..."
31         curl -fsSL https://get.docker.com | sh
32         sudo usermod -aG docker "$USER"
33         echo -e "\n\033[0;32m[OK] Docker installed successfully!\033[0m"
34         echo -e "\033[0;33m[!] CRITICAL: You must log out of your SSH session and log back in for Docker
permissions to apply.\033[0m"
35         die "Please log out, log back in, and re-run setup.sh."
36     else
37         echo " Please install Docker manually:"
38         echo " curl -fsSL https://get.docker.com | sh"
39         echo " sudo usermod -aG docker \"$USER\" && newgrp docker"
40         die "Install Docker, then re-run this script."
41     fi
42 fi
43 fi
44
45 if ! docker info >> output.log 2>&1; then
46     echo ""
47     if [[ "$OSTYPE" == "darwin"* ]]; then
48         echo " Docker is installed but not running."
49         echo " If using Colima: colima start --memory 0.6 --vm-type vz --network-address --network-mode
bridged"
50         echo " If using Docker Desktop: open the app."
51     else
52         echo " Docker is installed but not running."
53         echo " Run: sudo systemctl start docker"
54     fi
55     die "Start Docker, then re-run this script."
56 fi
57
58 if ! docker compose version >> output.log 2>&1; then
59     die "Docker Compose v2 not found. Update Docker or install the compose plugin:\n https://docs.docker.com/
compose/install/"
60 fi
61
62 ok "Docker $(docker --version | grep -oE '[0-9.]+' | head -1) is running."
63
64 # 1.5 Python 3
65 step "Checking Python 3"
66
67 if ! command -v python3 >> output.log 2>&1; then
68     echo ""
69     if [[ "$OSTYPE" == "darwin"* ]]; then
70         echo " Python 3 is not installed."
71         echo " macOS includes it via Xcode Command Line Tools, or you can install it via Homebrew:"
72         echo " brew install python3"
73         die "Install Python 3, then re-run this script."
74     else
75         echo " Python 3 is not installed."
76         echo " Please install it using your system's package manager. For example:"
77         echo " Debian/Ubuntu: sudo apt install python3"
78         echo " Fedora: sudo dnf install python3"
79         echo " Arch: sudo pacman -S python"
80         die "Install Python 3, then re-run this script."
81     fi
82 fi
83
84 # Note: This setup is confirmed stable on Python 3.9.6 (macOS default).

```

```

85 PYTHON_VER=$(python3 --version 2>&1 | head -n 1)
86 ok "$PYTHON_VER is installed."
87
88 # 2. Tailscale (host)
89 step "Checking Tailscale (host)"
90 # The host needs its own Tailscale installation separate from the Docker
91 # container so Toggle-Gateway can run 'tailscale down' before stopping
92 # containers and 'tailscale up' after starting them. Without this, the
93 # exit-node deadlock that setup.sh is designed to prevent can still occur.
94
95 # Resolve the tailscale binary: CLI (brew/package) or bundled macOS app
96 TAILSCALE_BIN=""
97 if command -v tailscale >> output.log 2>&1; then
98     TAILSCALE_BIN="tailscale"
99 elif [[ -x "/Applications/Tailscale.app/Contents/MacOS/Tailscale" ]]; then
100     TAILSCALE_BIN="/Applications/Tailscale.app/Contents/MacOS/Tailscale"
101 fi
102
103 RECOMMENDED_TS_VERSION="1.98.5"
104
105 if [[ -z "$TAILSCALE_BIN" ]]; then
106     warn "Tailscale not found on host installing..."
107
108     if [[ "$OSTYPE" == "darwin"* ]]; then
109         if command -v brew >> output.log 2>&1; then
110             step "Installing Tailscale CLI formula via Homebrew (standalone, no .app GUI)..."
111             # Note: We install Tailscale via Homebrew, targeting the stable version (recommended: 1.98.5)
112             brew install tailscale -q
113             TAILSCALE_BIN="tailscale"
114
115             step "Starting tailscaled as a per-user LaunchAgent (auto-starts at login)..."
116             if brew services start tailscale 2>&1 | grep -qE 'Successfully|started'; then
117                 ok "tailscaled LaunchAgent registered."
118             else
119                 warn "Could not auto-start tailscaled. Run 'brew services start tailscale' manually."
120                 warn "For a system-wide LaunchDaemon that runs before login, run instead:"
121                 warn "    sudo brew services start tailscale"
122             fi
123             echo ""
124         else
125             die "Homebrew not found. Install Tailscale manually (recommended version: ${RECOMMENDED_TS_VERSION}):
126             https://tailscale.com/download/mac\n Then re-run this script."
127         fi
128     elif [[ "$OSTYPE" == "linux-gnu"* ]]; then
129         curl -fsSL https://tailscale.com/install.sh | sh
130         sudo systemctl enable --now tailscaled 2>> output.log || true
131         TAILSCALE_BIN="tailscale"
132     else
133         die "Unsupported OS. Install Tailscale manually (recommended version: ${RECOMMENDED_TS_VERSION}):
134         https://tailscale.com/download\n Then re-run this script."
135     fi
136
137     TAILSCALE_VER=$(("${TAILSCALE_BIN}" version 2>> output.log | head -n 1)
138     if [[ "$TAILSCALE_VER" != "$RECOMMENDED_TS_VERSION" ]]; then
139         warn "Tailscale installed, but version ($TAILSCALE_VER) differs from recommended version (
140         $RECOMMENDED_TS_VERSION)."
141     else
142         ok "Tailscale installed (version $TAILSCALE_VER)."
143     fi
144 else
145     TAILSCALE_VER=$(("${TAILSCALE_BIN}" version 2>> output.log | head -n 1)
146     if [[ "$TAILSCALE_VER" != "$RECOMMENDED_TS_VERSION" ]]; then
147         warn "Tailscale is already installed ($TAILSCALE_VER), but recommended version is
148         $RECOMMENDED_TS_VERSION for host/container consistency."
149     else
150         ok "Tailscale is already installed at recommended version ($TAILSCALE_VER)."
151     fi
152 fi
153
154 # Authenticate the host machine if it isn't already connected.
155 # This is an interactive step it opens a browser login URL.
156 # We do it now, before containers start, so Toggle-Gateway can
157 # safely run 'tailscale down/up' from day one.
158 if ! "${TAILSCALE_BIN}" status >> output.log 2>&1; then
159     echo ""
160     echo " Your host machine needs to join your Tailscale network."
161     echo " A browser window or login URL will appear authenticate, then"
162     echo " return here. The script will continue automatically."
163     echo ""

```

```

162     if [[ "$OSTYPE" == "linux-gnu"* ]]; then
163         sudo ${TAILSCALE_BIN} up
164     else
165         ${TAILSCALE_BIN} up
166     fi
167     ok "Host machine connected to Tailscale."
168 else
169     ok "Host machine is already connected to Tailscale."
170 fi
171
172 # 3. wgcf
173 step "Checking wgcf (Cloudflare WARP key tool)"
174
175 if ! command -v wgcf >> output.log 2>&1; then
176     warn "wgcf not found installing..."
177
178     if [[ "$OSTYPE" == "darwin"* ]]; then
179         command -v brew >> output.log 2>&1 || die "Homebrew not found.\nInstall wgcf manually: https://github.com/ViRb3/wgcf/releases"
180         brew install wgcf -q
181
182     elif [[ "$OSTYPE" == "linux-gnu"* ]]; then
183         ARCH=$(uname -m)
184         case $ARCH in
185             x86_64) WGCF_ARCH="amd64" ;;
186             aarch64) WGCF_ARCH="arm64" ;;
187             armv7l) WGCF_ARCH="armv7" ;;
188             *) die "Unsupported architecture: $ARCH.\nInstall wgcf manually: https://github.com/ViRb3/wgcf/releases" ;;
189         esac
190
191         echo " Fetching latest release..."
192         WGCF_VERSION=$(curl -sf https://api.github.com/repos/ViRb3/wgcf/releases/latest \
193             | grep 'tag_name' | cut -d'"' -f4)
194         [[ -z "$WGCF_VERSION" ]] && die "Could not fetch wgcf version. Check your internet connection."
195
196         sudo curl -sL \
197             "https://github.com/ViRb3/wgcf/releases/download/${WGCF_VERSION}/wgcf_${WGCF_VERSION#v}_linux_${WGCF_ARCH}" \
198             -o /usr/local/bin/wgcf
199         sudo chmod +x /usr/local/bin/wgcf
200     else
201         die "Unsupported OS. Install wgcf manually: https://github.com/ViRb3/wgcf/releases"
202     fi
203 fi
204
205 ok "wgcf is ready."
206
207 # 4. WARP profile
208 step "Generating Cloudflare WARP profile"
209
210 if [[ -f "wgcf-profile.conf" ]]; then
211     warn "wgcf-profile.conf already exists skipping key generation."
212 else
213     wgcf register --accept-tos 2>> output.log || die "wgcf register failed. Check your internet connection."
214     wgcf generate 2>> output.log || die "wgcf generate failed."
215     ok "WARP profile generated."
216 fi
217
218 # Parse keys (IPv4 address only gluetun doesn't need the IPv6 one)
219 PRIVATE_KEY=$(awk '/PrivateKey/{print $3}' wgcf-profile.conf)
220 PUBLIC_KEY=$(awk '/PublicKey/{print $3}' wgcf-profile.conf)
221 ADDRESSES=$(awk '/Address/{print $3}' wgcf-profile.conf | grep -v ':' | head -1)
222
223 [[ -z "$PRIVATE_KEY" ]] && die "Could not parse PrivateKey from wgcf-profile.conf"
224 [[ -z "$PUBLIC_KEY" ]] && die "Could not parse PublicKey from wgcf-profile.conf"
225 [[ -z "$ADDRESSES" ]] && die "Could not parse IPv4 Address from wgcf-profile.conf"
226
227 ok "WARP keys extracted."
228
229 # 5. Tailscale auth key
230 step "Tailscale authentication"
231
232 TS_AUTHKEY=""
233 if [[ -f ".env" ]]; then
234     EXISTING_TS_KEY=$(read_env_var TS_AUTHKEY)
235     if [[ -n "$EXISTING_TS_KEY" ]]; then
236         TS_AUTHKEY="$EXISTING_TS_KEY"
237         warn "Found existing TS_AUTHKEY in .env. Skipping prompt."
238     fi
239 fi

```



```

240
241 if [[ -z "$TS_AUTHKEY" ]]; then
242     echo ""
243     echo "    Generate a key at: https://login.tailscale.com/admin/settings/keys"
244     echo "    'Generate auth key' enable Reusable if you plan to re-run setup"
245     echo ""
246     read -rp "    Paste your Tailscale auth key: " TS_AUTHKEY
247     echo ""
248 fi
249
250 [[ -z "$TS_AUTHKEY" ]] && die "Tailscale auth key is required."
251 [[ "$TS_AUTHKEY" != tskey-auth-* ]] && warn "Key doesn't look like a Tailscale auth key proceeding anyway."
252 ok "Auth key accepted."
253
254 # 6. AdGuard Home admin password
255 step "AdGuard Home admin account"
256
257 # Hardcoded default credentials to prevent lockout
258 # Username: admin
259 # Password: nullexit
260 if [[ -f ".env" ]]; then
261     EXISTING_AG_PASS=$(read_env_var ADGUARD_PASSWORD)
262     if [[ -n "$EXISTING_AG_PASS" ]]; then
263         ADGUARD_PASSWORD="$EXISTING_AG_PASS"
264         ADGUARD_PWD_ESC="$EXISTING_AG_PASS"
265         ok "Found existing ADGUARD_PASSWORD in .env."
266     else
267         ADGUARD_PASSWORD="nullexit"
268         ADGUARD_PWD_ESC="nullexit"
269         ok "Default password set to 'nullexit'."
270     fi
271 else
272     ADGUARD_PASSWORD="nullexit"
273     ADGUARD_PWD_ESC="nullexit"
274     ok "Default password set to 'nullexit'."
275 fi
276
277 # 7. Write .env
278 step "Writing .env"
279
280
281 if [[ -f ".env" ]]; then
282     read -rp "    .env already exists. Overwrite? [y/N]: " OVERWRITE
283     if [[ "$OVERWRITE" != "y" && "$OVERWRITE" != "Y" ]]; then
284         warn "Keeping existing .env."
285     else
286         WRITE_ENV=1
287     fi
288 else
289     WRITE_ENV=1
290 fi
291
292 if [[ "${WRITE_ENV:-0}" == "1" ]]; then
293     # Preserve existing configuration flags if .env already exists
294     EXISTING_PROFILE="medium"
295     EXISTING_MSS="1120"
296     EXISTING_HIJACK="true"
297     EXISTING_EXIT="true"
298     EXISTING_THRESH="6"
299     EXISTING_KILL="false"
300     EXISTING_BLOCKED="kp il"
301     EXISTING_HOST_MTU="1200"
302
303     if [[ -f ".env" ]]; then
304         [[ -n "$(read_env_var GATEWAY_RULE_PROFILE)" ]] && EXISTING_PROFILE=$(read_env_var GATEWAY_RULE_PROFILE)
305
306         [[ -n "$(read_env_var GATEWAY_MSS)" ]] && EXISTING_MSS=$(read_env_var GATEWAY_MSS)
307         [[ -n "$(read_env_var GATEWAY_HIJACK_HOST)" ]] && EXISTING_HIJACK=$(read_env_var GATEWAY_HIJACK_HOST)
308         [[ -n "$(read_env_var GATEWAY_USE_EXIT_NODE)" ]] && EXISTING_EXIT=$(read_env_var GATEWAY_USE_EXIT_NODE)
309         [[ -n "$(read_env_var WARP_FAIL_THRESHOLD)" ]] && EXISTING_THRESH=$(read_env_var WARP_FAIL_THRESHOLD)
310         [[ -n "$(read_env_var KILL_SWITCH)" ]] && EXISTING_KILL=$(read_env_var KILL_SWITCH)
311         [[ -n "$(read_env_var BLOCKED_COUNTRIES)" ]] && EXISTING_BLOCKED=$(read_env_var BLOCKED_COUNTRIES)
312         [[ -n "$(read_env_var HOST_MTU)" ]] && EXISTING_HOST_MTU=$(read_env_var HOST_MTU)
313     fi
314
315     cat > .env <<EOF
316 WIREGUARD_PRIVATE_KEY=${PRIVATE_KEY}
317 WIREGUARD_PUBLIC_KEY=${PUBLIC_KEY}
318 WIREGUARD_ADDRESSES=${ADDRESSES}
319 TS_AUTHKEY=${TS_AUTHKEY}
320 GATEWAY_RULE_PROFILE=${EXISTING_PROFILE}

```

```

320 # Safe MSS for double-tunneled traffic (Tailscale + WARP). Change to 1180 if you experience slow speeds and
    have a healthy path.
321 GATEWAY_MSS=${EXISTING_MSS}
322 # Lower the host Tailscale MTU to 1200 to prevent fragmentation when passing through the 1280-byte WARP tunnel.
323 HOST_MTU=${EXISTING_HOST_MTU}
324 # Set to false to run as a 'Headless Server' (Docker acts as an exit node, but the host Mac/Linux's own
    internet is NOT hijacked).
325 GATEWAY_HIJACK_HOST=${EXISTING_HIJACK}
326 GATEWAY_USE_EXIT_NODE=${EXISTING_EXIT}
327 WARP_FAIL_THRESHOLD=${EXISTING_THRESH}
328 KILL_SWITCH=${EXISTING_KILL}
329 ADGUARD_PASSWORD=${ADGUARD_PASSWORD}
330 BLOCKED_COUNTRIES="${EXISTING_BLOCKED}"
331 EOF
332     ok ".env written."
333 fi
334
335 # 8. Prepare directories
336 step "Preparing AdGuard directories"
337
338 mkdir -p adguard/conf adguard/work/userfilters
339
340 # Remove empty AdGuardHome.yaml that would cause a crash loop (same logic as Toggle-Gateway scripts)
341 if [[ -f "adguard/conf/AdGuardHome.yaml" && ! -s "adguard/conf/AdGuardHome.yaml" ]]; then
342     rm "adguard/conf/AdGuardHome.yaml"
343     warn "Removed empty AdGuardHome.yaml to prevent crash loop."
344 fi
345
346 ok "Directories ready."
347
348 # 9. Compile DNS filter rules
349 step "Compiling DNS filter rules (this may take a minute)"
350
351
352 # 10. Start containers
353 step "Starting containers"
354 # Note: tailscale depends on adguardhome being healthy (port 5335 up),
355 # so Docker Compose will hold it back automatically until the wizard is done.
356 docker compose up -d
357 ok "Containers starting."
358
359 # 11. Wait for AdGuard Home setup wizard
360 step "Waiting for AdGuard Home setup wizard"
361 echo " (up to 60 seconds...)"
362
363 MAX=60; ELAPSED=0
364 until docker exec routing-fix curl -sf http://127.0.0.1:3000 >> output.log 2>&1; do
365     sleep 2; ELAPSED=$((ELAPSED + 2))
366     [[ $ELAPSED -ge $MAX ]] && die "AdGuard Home didn't come up.\nDebug with: docker compose logs adguardhome"
367 done
368
369 ok "AdGuard Home is up."
370
371 # 12. Configure AdGuard Home via API
372 step "Configuring AdGuard Home"
373
374 # Run all API calls in a single docker exec sh -c so the session cookie file
375 # is shared across calls without leaving the container.
376 docker exec routing-fix sh -c "
377     set -e
378
379     # Step 1: Complete the setup wizard (sets admin creds + DNS port to 5335)
380     curl -sf -X POST http://127.0.0.1:3000/control/install/configure \
381         -H 'Content-Type: application/json' \
382         -d '{"web":{"ip":"'0.0.0.0','port':3000},'dns":{"ip":"'0.0.0.0','port':5335},'username':'\
383             admin','password':'${ADGUARD_PWD_ESC}'}' \
384         >/dev/null || true
385
386     # Step 2: Wait for AdGuard to restart after wizard
387     sleep 4
388     WAITED=0
389     until curl -sf http://127.0.0.1:3000 >/dev/null 2>&1; do
390         sleep 2; WAITED=$((WAITED + 2))
391         [ $WAITED -ge 30 ] && { echo 'AdGuard did not restart in time'; exit 1; }
392     done
393
394     # Step 3: Login to get session cookie
395     curl -sf -X POST http://127.0.0.1:3000/control/login \
396         -H 'Content-Type: application/json' \
397         -d '{"name':'admin','password':'${ADGUARD_PWD_ESC}'}' \
398         -c /tmp/agh.cookie >/dev/null

```

```

398
399 # Step 4: Point upstream DNS back through the WARP tunnel
400 curl -sf -X POST http://127.0.0.1:3000/control/dns_config \
401   -H 'Content-Type: application/json' \
402   -b /tmp/agh.cookie \
403   -d '{"upstream_dns":["127.0.0.1:53","[/onion/]127.0.0.1:5353"],"bootstrap_dns":["1.1.1.1","8.8.8.8"],"upstream_mode":{"load_balance"}}' \
404   >/dev/null
405
406 # Step 5: Register the compiled rules file as a filter list
407 curl -sf -X POST http://127.0.0.1:3000/control/filtering/add_url \
408   -H 'Content-Type: application/json' \
409   -b /tmp/agh.cookie \
410   -d '{"name":"Custom Compiled Rules","url":"/opt/adguardhome/work/userfilters/compiled_rules.txt"}' \
411   >/dev/null || true
412
413 # Step 6: Force a filter refresh to load the rules immediately
414 curl -sf -X POST http://127.0.0.1:3000/control/filtering/refresh \
415   -H 'Content-Type: application/json' \
416   -b /tmp/agh.cookie \
417   -d '{"whitelist":false}' \
418   >/dev/null || true
419
420 rm -f /tmp/agh.cookie
421
422
423 ok "AdGuard Home configured."
424
425 # 13. Wait for Tailscale
426 step "Waiting for Tailscale to authenticate"
427 echo " (Tailscale only starts once AdGuard's healthcheck passes up to 90s)"
428
429 MAX=90; ELAPSED=0; TS_IP=""
430 until [[ -n "$TS_IP" ]]; do
431     TS_IP=$(docker exec tailscale tailscale ip -4 2>> output.log || true)
432     if [[ -z "$TS_IP" ]]; then
433         sleep 3; ELAPSED=$((ELAPSED + 3))
434         if [[ $ELAPSED -ge $MAX ]]; then
435             warn "Tailscale hasn't authenticated yet."
436             warn "Once it does, re-run the toggle script it will resolve the IP dynamically."
437             break
438         fi
439     fi
440 done
441
442 if [[ -n "$TS_IP" ]]; then
443     ok "Tailscale IP: ${TS_IP} (resolved dynamically by the toggle script on each startup)"
444 fi

```

44 scripts/unlock-files.sh

```

1 #!/bin/bash
2 # Resolve project root relative to this script's location, so the script
3 # works regardless of the caller's CWD.
4 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
6
7 echo "Unlocking .env..."
8 if [ -f "$PROJECT_ROOT/.env" ]; then
9     sudo cat "$PROJECT_ROOT/.env" > "$PROJECT_ROOT/.env.tmp" && mv "$PROJECT_ROOT/.env.tmp" "$PROJECT_ROOT/.env"
10    echo ".env unlocked"
11 fi
12
13 echo "Unlocking AdGuard cache files..."
14 for f in "$PROJECT_ROOT"/adguard/work/userfilters/compiled_rules.txt \
15         "$PROJECT_ROOT"/adguard/work/userfilters/ip_blocklist.ipset \
16         "$PROJECT_ROOT"/adguard/work/userfilters/cache/*.txt \
17         "$PROJECT_ROOT"/adguard/work/userfilters/cache/ip/*.txt \
18         "$PROJECT_ROOT"/adguard/work/data/filters/*.txt; do
19     if [ -f "$f" ]; then
20         cat "$f" > "$f.tmp" && mv "$f.tmp" "$f"
21         echo "Unlocked $f"
22     fi
23 done
24

```

```
25 echo ""
26 echo "All locked files have been successfully bypassed via atomic rename!"
```

45 tor-bridges.txt

46 white_list.txt

```
1 0.client-channel.google.com
2 1drv.com
3 2.android.pool.ntp.org
4 akamaihd.net
5 akamai technologies.com
6 akamaized.net
7 amazonaws.com
8 android.clients.google.com
9 api-adservices.apple.com
10 api.ipify.org
11 api.rlje.net
12 app-api.ted.com
13 appleid.apple.com
14 apps.skype.com
15 appsbackup-pa.clients6.google.com
16 appsbackup-pa.googleapis.com
17 appspot-preview.l.google.com
18 apt.sonarr.tv
19 aspnetscdn.com
20 attestation.xboxlive.com
21 ax.phobos.apple.com.edgesuite.net
22 books-analytics-events.apple.com
23 brightcove.net
24 c.s-microsoft.com
25 cdn.cloudflare.net
26 cdn.embedly.com
27 cdn.optimizely.com
28 cdn.vidible.tv
29 cdn2.optimizely.com
30 cdn3.optimizely.com
31 cdnjs.cloudflare.com
32 cert.mgt.xboxlive.com
33 clientconfig.passport.net
34 clients1.google.com
35 clients2.google.com
36 clients3.google.com
37 clients4.google.com
38 clients5.google.com
39 clients6.google.com
40 connectivitycheck.android.com
41 connectivitycheck.gstatic.com
42 continuum.dds.microsoft.com
43 cpms.spop10.ams.plex.bz
44 cpms35.spop10.ams.plex.bz
45 cse.google.com
46 ctldl.windowsupdate.com
47 cws.conviva.com
48 d2c8v52ll5s99u.cloudfront.net
49 d2gatte9o95jao.cloudfront.net
50 dashboard.plex.tv
51 dataplicity.com
52 def-vef.xboxlive.com
53 delivery.vidible.tv
54 dev.virtualearth.net
55 device.auth.xboxlive.com
56 display.ugc.bazaarvoice.com
57 displaycatalog.mp.microsoft.com
58 dl.delivery.mp.microsoft.com
59 dl.dropbox.com
60 dl.dropboxusercontent.com
61 dns.msftncsi.com
62 download.sonarr.tv
63 drift.com
64 driftt.com
```

65 dynupdate.no-ip.com
66 ecn.dev.virtualearth.net
67 edge.api.brightcove.com
68 eds.xboxlive.com
69 fonts.gstatic.com
70 forums.sonarr.tv
71 g.live.com
72 geo-prod.do.dsp.mp.microsoft.com
73 geo3.ggpht.com
74 gfws1.geforce.com
75 giphy.com
76 github.com
77 github.io
78 googleapis.com
79 gravatar.com
80.gstatic.com
81 help.ui.xboxlive.com
82 hls.ted.com
83 i.ytimg.com
84 i1.ytimg.com
85 iadsdk.apple.com
86 imagesak.secureserver.net
87 img.vidible.tv
88 imgix.net
89 imgs.xkcd.com
90 instantmessaging-pa.googleapis.com
91 intercom.io
92 jquery.com
93 jsdelivr.net
94 keystone.mwbsys.com
95 lastfm-img2.akamaized.net
96 licensing.xboxlive.com
97 live.com
98 livepassdl.conviva.com
99 login.live.com
100 login.microsoftonline.com
101 manifest.googlevideo.com
102 meta-db-worker02.pop.ric.plex.bz
103 meta.plex.bz
104 meta.plex.tv
105 microsoftonline.com
106 msftncsi.com
107 my.plexapp.com
108 nexusrules.officeapps.live.com
109 nine.plugins.plexapp.com
110 no-ip.com
111 node.plexapp.com
112 notes-analytics-events.apple.com
113 notify.xboxlive.com
114 npr-news.streaming.adswizz.com
115 ns1.dropbox.com
116 ns2.dropbox.com
117 o1.email.plex.tv
118 o2.sg0.plex.tv
119 ocsp.apple.com
120 office.com
121 office.net
122 office365.com
123 officeclient.microsoft.com
124 om.cbsi.com
125 onedrive.live.com
126 outlook.live.com
127 outlook.office365.com
128 pings.conviva.com
129 placeholder.it
130 placeholderimgix.net
131 players.brightcove.net
132 pricelist.skype.com
133 products.office.com
134 proxy.plex.bz
135 proxy.plex.tv
136 proxy02.pop.ord.plex.bz
137 pubsub.plex.bz
138 pubsub.plex.tv
139 raw.githubusercontent.com
140 redirector.googlevideo.com
141 res.cloudinary.com
142 s.gateway.messenger.live.com
143 s.marketwatch.com
144 s.youtube.com
145 s.ytimg.com

```

146 s1.wp.com
147 s2.youtube.com
148 s3.amazonaws.com
149 sa.symcb.com
150 secure.avangate.com
151 secure.brightcove.com
152 secure.surveymonkey.com
153 services.sonarr.tv
154 skyhook.sonarr.tv
155 spclient.wg.spotify.com
156 ssl.p.jwpcdn.com
157 staging.plex.tv
158 status.plex.tv
159 t.co
160 t0.ssl.ak.dynamic.tiles.virtualearth.net
161 t0.ssl.ak.tiles.virtualearth.net
162 tawk.to
163 tedcdn.com
164 themoviedb.com
165 thetvdb.com
166 tinyurl.com
167 title.auth.xboxlive.com
168 title.mgt.xboxlive.com
169 traffic.libsyn.com
170 tvdb2.plex.tv
171 tvthemes.plexapp.com
172 twimg.com
173 ui.skype.com
174 video-stats.l.google.com
175 videos.vidible.tv
176 vidtech.cbsinteractive.com
177 weather-analytics-events.apple.com
178 widget-cdn.rpxnow.com
179 win10.ipv6.microsoft.com
180 wp.com
181 ws.audioscrobbler.com
182 www.dataplicity.com
183 www.googleapis.com
184 www.msftconnecttest.com
185 www.msftncsi.com
186 www.no-ip.com
187 www.youtube-nocookie.com
188 xbox.ipv6.microsoft.com
189 xboxexperiencesprod.experimentation.xboxlive.com
190 xflight.xboxlive.com
191 xkms.xboxlive.com
192 xsts.auth.xboxlive.com
193youtu.be
194 youtube-nocookie.com
195 yt3.ggpht.com
196 zee.cws.conviva.com
197
198 # YouTube family base domains so every subdomain (video stream servers on
199 # *.googlevideo.com, image hosts on *.ytimg.com, profile thumbnails on
200 # *.ggpht.com, etc.) is automatically allowlisted via the AdGuard '||domain^'
201 # wildcard. The specific s.youtube.com / manifest.googlevideo.com entries
202 # above are kept for clarity but will be optimized away by rule-compiler
203 # because their parents are now in this list.
204 youtube.com
205 googlevideo.com
206 ytimg.com
207 ggpht.com
208 gvt1.com
209 gvt2.com
210 gvt3.com
211 gvt1.net
212 gvt2.net
213 gvt3.net
214
215 edge-mqtt.facebook.com
216 gateway.instagram.com
217 i.instagram.com
218
219 facebook.com
220 fb.com
221 fbcdn.net
222 fbsbx.com
223 messenger.com
224 whatsapp.com
225 web.whatsapp.com
226 instagram.com

```

```
227 cdninstagram.com
228 static.whatsapp.net
229 media-bos5-1.cdn.whatsapp.net
230 media-yyz1-1.cdn.whatsapp.net
231 webtp.whatsapp.net
232 mmg.whatsapp.net
```